Top 200 Most-Asked SQL Questions

SQL Interview Prep — 2026 Edition · Learn & Excel

If your interview is 2-3 weeks out, this is the file. Where the Top 50 gives you the headline questions, this file widens the surface to the 200 questions that account for the bulk of every SQL round across Indian MNCs, product companies, GCCs, and startups. It is organized by **frequency tier**, not by topic — so you can spend most of your prep budget on the Tier S questions that show up in 80%+ of interviews, and treat Tier B as the polish layer.

How to use it: read Tier S first and don't move on until you can recite the 30-second answers cold. Tier A is where mid-level rounds live — work through it over a weekend. Tier B is depth-padding for senior rounds and the "tell me more" follow-ups; skim it the week of your interview.

Front matter: Frequency Heatmap

Topic / Tier	Service MNCs	Indian Product Co	Startup / GCC
Joins (INNER/OUTER)	100%	95%	90%
Subqueries + EXISTS/IN	95%	90%	85%
Window functions	60%	95%	90%
Indexes (basics)	75%	95%	85%
Indexes (advanced)	30%	80%	70%
Transactions + ACID	80%	90%	80%
Isolation levels	35%	85%	75%
MVCC + locking	25%	80%	70%
Aggregations + GROUP BY	90%	90%	85%
Modern SQL (CTE, MERGE, JSON)	40%	80%	70%
Normalization (1NF-3NF)	80%	60%	50%
Schema design at scale	25%	80%	70%
Snowflake / BigQuery	5%	60%	55%
Performance tuning (EXPLAIN)	30%	80%	70%
Stored procs / triggers	60%	30%	30%

Use the heatmap to bias your prep: a TCS L1 round will hit Joins + Aggregations + Normalization; a Razorpay L3 will skip basics and grill you on Windows + Indexes + Transactions + Modern features.

Tier S — Asked in 80%+ of SQL interviews (50 Qs)

These are the same 50 questions from `01 Top 50 Most-Asked`. Reproduced here so this file stands alone. If you have already worked through Top 50, skim Tier S and move directly to Tier A.

Q1 · INNER vs LEFT vs RIGHT vs FULL OUTER JOIN

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Accenture, Wipro · Topic: Joins

The question (interviewer's verbatim phrasing):

> Can you walk me through the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN and FULL OUTER JOIN? Give me a real example where each one would return different rows.

The 30-second answer: INNER keeps only matched rows from both tables; LEFT keeps every row from the left table plus matches from the right (NULLs where no match); RIGHT is the mirror of LEFT; FULL OUTER keeps everything from both sides and pads with NULLs wherever a match is missing.

Step-by-step thought process: 1. Open with the matched-vs-unmatched framing — that's what the interviewer is testing. 2. Use a concrete example: `orders` and `payments`. INNER gives you only orders that have a payment row. LEFT gives you all orders, including those without payments (useful for finding payment failures). RIGHT gives you all payments, including orphan payment rows. FULL OUTER gives you both sets of orphans in one go. 3. Mention that RIGHT JOIN is rarely used in production — most people flip the table order and use LEFT JOIN for readability. 4. Be ready for the follow-up: "How would you use LEFT JOIN to find orders without payments?" Answer: `LEFT JOIN payments p ON o.id = p.order_id WHERE p.order_id IS NULL` — this is the anti-join pattern.

Edge cases to mention: - If the join condition matches multiple rows on the right, INNER and LEFT both duplicate the left row — junior candidates forget this. - FULL OUTER JOIN is not supported in MySQL at all (still missing in 8.0 and 8.4) — you simulate it with `UNION ALL` of a LEFT JOIN and an anti-joined LEFT JOIN. - NULLs in the join column never match, even to other NULLs — that's standard SQL three-valued logic.

What NOT to say: - Do not say "LEFT JOIN includes all rows from both tables" — that's wrong, it only includes all rows from the left. - Do not draw Venn diagrams as your only explanation; interviewers want row-level reasoning, not pictures.

Common follow-up: *"If a candidate writes a LEFT JOIN but puts the right-table filter in the WHERE clause, what happens?"* — It silently converts to an INNER JOIN because the NULLs get filtered out. Put the filter in the ON clause instead.

```
-- Find orders that never got paid
SELECT o.id, o.user_id, o.amount
FROM orders o
LEFT JOIN payments p ON o.id = p.order_id
WHERE p.order_id IS NULL;
```

Q2 · Self-Join — Manager-Employee Pattern

Tags: Difficulty: Easy · Asked at: Infosys, Cognizant, TCS, Wells Fargo · Topic: Joins

The question (interviewer's verbatim phrasing):

> You have an employees table with `id`, `name` and `manager_id` where `manager_id` is a self-reference. Write a query to list every employee with their manager's name.

The 30-second answer: Join the table to itself with two aliases — one for the employee row, one for the manager row — matching `e.manager_id = m.id`, and use LEFT JOIN so the CEO (no manager) doesn't disappear.

Step-by-step thought process: 1. State the core idea: a self-join is just a join where both sides reference the same table with different aliases. 2. Write the LEFT JOIN — emphasise LEFT because the top-level manager has a NULL `manager_id` and would otherwise vanish. 3. Mention common variations: finding pairs of employees with the same manager, or hierarchical levels (which leads into recursive CTEs). 4. Anticipate the follow-up about multi-level hierarchy — that requires a recursive CTE, not a single self-join.

Edge cases to mention: - Cycles: if `A` manages `B` and `B` somehow manages `A`, a recursive query loops forever — use a depth limit. - The CEO row will show `NULL` as manager name with LEFT JOIN — use `COALESCE(m.name, 'No manager')` to display cleanly. - If the same employee can have multiple managers (matrix orgs), one self-join row per manager is correct — explain the cardinality.

What NOT to say: - Do not use the same alias for both sides; the query will not parse and you'll look careless. - Do not skip the LEFT JOIN and then claim "the top manager is also in the result" — INNER JOIN drops them.

Common follow-up: "*Now show me each employee, their manager and their manager's manager — three levels.*" — Either chain self-joins or, cleaner, use a recursive CTE.

```
SELECT e.name AS employee, COALESCE(m.name, 'No manager') AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id;
```

Q3 · Anti-Join and the NOT IN NULL Trap

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Goldman, JP Morgan · Topic: Joins

The question (interviewer's verbatim phrasing):

> I want all users who have never placed an order. Show me three different ways to write it and tell me which one is safest.

The 30-second answer: Three options — `LEFT JOIN ... WHERE order_id IS NULL`, `NOT EXISTS (...)`, and `NOT IN (...)`. The first two are safe; `NOT IN` is dangerous because if the subquery returns even a single NULL, the entire result becomes empty.

Step-by-step thought process: 1. Lead with the three forms — interviewer wants to know you've seen all of them. 2. Explain why `NOT IN` breaks: `x NOT IN (1, 2, NULL)` evaluates as `x != 1 AND x != 2 AND x != NULL`, and `x != NULL` is UNKNOWN, so the whole expression is UNKNOWN, which is not TRUE, so the row is filtered out. 3. State your preference: `NOT EXISTS` — it handles NULLs correctly and the optimizer treats it as an anti-semi-join. 4. Be ready to explain when `LEFT JOIN ... IS NULL` is preferred: when you also need columns from the left table efficiently.

Edge cases to mention: - If the column is declared `NOT NULL`, `NOT IN` is safe — but don't rely on schema you can't see. - `NOT EXISTS` short-circuits on the first match — usually faster on large right-side tables. - LEFT JOIN with IS NULL can be slower if the right table is huge and there are many matches that you then throw away.

What NOT to say: - Do not say "all three are equivalent." They are not equivalent in the presence of NULLs. - Do not assert one is always fastest — it depends on cardinality and indexes.

Common follow-up: "*What does the planner actually do with NOT EXISTS?*" — Postgres and modern SQL Server convert it to an anti-join operator, which is essentially a join that emits the left row only when no right match exists.

```
SELECT u.id, u.name
FROM users u
WHERE NOT EXISTS (
  SELECT 1 FROM orders o WHERE o.user_id = u.id
);
```

Q4 · EXISTS vs IN vs JOIN

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Walmart Labs, Cred · Topic: Joins

The question (interviewer's verbatim phrasing):

> When would you use EXISTS versus IN versus a JOIN? Are they always interchangeable?

The 30-second answer: They are semantically different — JOIN can multiply rows when the right side has duplicates, while EXISTS and IN return one row per left match. Pick EXISTS when you just need a check, JOIN when you need columns from the other table, IN when the list is small or comes from a constant set.

Step-by-step thought process: 1. Start with the cardinality point — that's the trap. A JOIN to `orders` from `users` where a user has 10 orders gives you 10 user rows; EXISTS gives you 1. 2. EXISTS is a semi-join: it stops scanning the inner table as soon as it finds the first match. 3. IN with a subquery is logically the same as EXISTS in most modern optimizers (Postgres, SQL Server, MySQL 8), but EXISTS reads more clearly when the inner query is correlated. 4. Anticipate: "Which is faster?" Answer: usually the same after the planner normalizes them — but EXISTS wins when the right table is huge and you have a covering index on the join column.

Edge cases to mention: - `IN` with a NULL in the list behaves differently from EXISTS — same NULL trap as anti-join. - If you JOIN and then use `DISTINCT` to dedupe, that's almost always slower than rewriting as EXISTS. - In MySQL versions before 5.6, `IN (subquery)` was notoriously slow — modern versions fixed this.

What NOT to say: - Do not say "EXISTS is always faster than IN." Modern optimizers usually generate the same plan. - Do not blindly use JOIN when you only need a boolean check — you'll get duplicate rows.

Common follow-up: *"Rewrite this JOIN+DISTINCT query using EXISTS."* — Practice this; it's a 30-second test of whether you actually understand the cardinality issue.

```
-- Users who have at least one successful payment
SELECT u.id, u.name
FROM users u
WHERE EXISTS (
  SELECT 1 FROM payments p
  WHERE p.user_id = u.id AND p.status = 'SUCCESS'
);
```

Q5 · Correlated Subquery — What and Why

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wells Fargo, Target · Topic: Subqueries

The question (interviewer's verbatim phrasing):

> What's a correlated subquery? Give me an example and tell me about its performance.

The 30-second answer: A correlated subquery references a column from the outer query, so it has to be re-evaluated for every outer row — conceptually it's a row-by-row loop. Modern optimizers often rewrite

correlated EXISTS/IN into joins, but a correlated scalar subquery in the SELECT list usually stays as a per-row evaluation.

Step-by-step thought process: 1. Define it clearly: the inner query depends on a value from the outer query — that's the "correlation." 2. Show the classic example: "for each user, get the timestamp of their latest order." A correlated scalar subquery in the SELECT clause does this. 3. Explain the cost: if you have 1 million users and the inner subquery is not optimized, you do 1 million inner scans — O(N*M) in the worst case. 4. Offer the fix: rewrite as a window function (`ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY created_at DESC)`) or a join to a derived table that pre-aggregates.

Edge cases to mention: - A correlated subquery returning more than one row in a scalar context throws an error — guard with `LIMIT 1` or aggregation. - Postgres can sometimes decorrelate; SQL Server's plan will show a "Nested Loops" operator with an Apply. - Inside EXISTS, a correlated subquery is the idiomatic way and the planner handles it well.

What NOT to say: - Do not say "correlated subqueries are always slow" — for small outer sets or indexed inner queries they're fine. - Do not confuse "correlated" with "nested." All correlated subqueries are nested; not all nested subqueries are correlated.

Common follow-up: "*Rewrite this correlated scalar subquery using a window function.*" — Be fluent at this rewrite; it's a very common refactor in real systems.

```
-- Correlated: one inner scan per user
SELECT u.id,
       (SELECT MAX(created_at) FROM orders o WHERE o.user_id = u.id) AS last_order
FROM users u;
```

Q6 · Semi-Join

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Razorpay, Zerodha · Topic: Joins

The question (interviewer's verbatim phrasing):

> What's a semi-join? Why is EXISTS the natural way to write one?

The 30-second answer: A semi-join returns rows from the left table where at least one match exists in the right table, with no duplication and no right-side columns — essentially the "do any matching rows exist?" pattern, which EXISTS expresses directly.

Step-by-step thought process: 1. Contrast with INNER JOIN: INNER returns a row for every match (10 orders → 10 rows); semi-join returns one row per left match (10 orders → 1 row). 2. SQL has no `SEMI JOIN` keyword in most dialects — you express it via EXISTS, IN, or sometimes via JOIN+DISTINCT (worse). 3. Mention that the optimizer literally calls this operator a "semi-join" in EXPLAIN output —

useful for reading query plans. 4. Anti-semi-join is the mirror — "no match exists" — expressed via NOT EXISTS.

Edge cases to mention: - DISTINCT after a JOIN works but is usually slower because the join materialises duplicates first, then dedupes. - Some warehouses (Snowflake, BigQuery) expose `LEFT SEMI JOIN` syntax directly — handy for ETL. - A semi-join through EXISTS short-circuits — the inner query stops at the first match per outer row.

What NOT to say: - Do not say "semi-join is a fancy term for INNER JOIN" — that misses the no-duplication point. - Do not write `JOIN ... GROUP BY` to dedupe when EXISTS does it cleaner.

Common follow-up: "*How would you write an anti-semi-join in standard SQL?*" — NOT EXISTS, or LEFT JOIN with IS NULL.

```
-- Semi-join via EXISTS
SELECT m.id, m.business_name
FROM merchants m
WHERE EXISTS (
  SELECT 1 FROM transactions t WHERE t.merchant_id = m.id AND t.amount > 100000
);
```

Q7 · LATERAL Join / CROSS APPLY

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Walmart Labs, Goldman · Topic: Joins

The question (interviewer's verbatim phrasing):

> Have you used LATERAL joins? When would you reach for one?

The 30-second answer: LATERAL (CROSS APPLY in SQL Server) lets the right side of a join reference columns from the left side — useful for per-row subqueries like "top 3 orders for each user" where a regular subquery cannot see the outer row.

Step-by-step thought process: 1. State the core capability: in a regular subquery in FROM, you cannot reference outer columns. LATERAL flips that — the right side is evaluated once per left row. 2. The textbook use case: top-N per group. `SELECT u.*, o.* FROM users u CROSS JOIN LATERAL (SELECT * FROM orders WHERE user_id = u.id ORDER BY created_at DESC LIMIT 3) o`. 3. Mention it's available in Postgres (`LATERAL`), SQL Server (`CROSS APPLY` and `OUTER APPLY`), and Oracle (12c+). MySQL added it in 8.0.14. 4. Contrast with window functions: for top-N, `ROW_NUMBER()` works for fixed N; LATERAL is cleaner when the inner query is complex or N varies.

Edge cases to mention: - `OUTER APPLY` / `LEFT JOIN LATERAL` keeps left rows even when the inner query returns zero rows. - Performance: planner typically uses a nested loop — fine for small left side, painful for huge ones. - The inner LIMIT is the killer feature — window functions cannot directly limit per partition without an outer filter.

What NOT to say: - Do not claim it's "just a subquery" — the correlation in FROM is the whole point. - Do not avoid it out of fear; it's standard SQL and shows seniority.

Common follow-up: *"Could you have written this with ROW_NUMBER instead?"* — Yes for simple top-N, but LATERAL is cleaner when the inner query has multiple joins or aggregates.

```
SELECT u.id, o.id AS order_id, o.amount
FROM users u
CROSS JOIN LATERAL (
  SELECT * FROM orders WHERE user_id = u.id ORDER BY created_at DESC LIMIT 3
) o;
```

Q8 · CROSS JOIN — Intentional vs Accidental

Tags: Difficulty: Easy · Asked at: TCS, Cognizant, Accenture, Infosys · Topic: Joins

The question (interviewer's verbatim phrasing):

> What's a CROSS JOIN and when would you actually want one?

The 30-second answer: A CROSS JOIN produces the Cartesian product — every row of A combined with every row of B, so N*M rows. Intentional use: generating combinations (dates × stores, sizes × colours). Accidental use: forgetting the join condition, which blows up your result set.

Step-by-step thought process: 1. Define Cartesian product crisply — N × M rows, no condition. 2. Give a real intentional example: a reporting query that needs every (store, date) pair so missing days appear as zero — you CROSS JOIN a stores table with a calendar table, then LEFT JOIN sales. 3. Mention the accidental form: writing `FROM orders, payments` without a WHERE join condition is an implicit CROSS JOIN — a classic junior mistake that takes down production. 4. Anticipate the follow-up about explicit syntax — always prefer `CROSS JOIN` keyword over comma-separated tables; it documents intent.

Edge cases to mention: - CROSS JOIN with a WHERE clause is equivalent to an INNER JOIN — but the explicit syntax is clearer. - 1000 × 1000 = 1 million rows. 100k × 100k = 10 billion. Cardinality goes up fast. - Some optimizers warn or refuse without explicit `CROSS JOIN` keyword — treat that as a feature, not annoyance.

What NOT to say: - Do not say "CROSS JOIN is always a mistake." Calendar joins, combination generation, and density tables all need it. - Do not use the comma syntax in modern code; it hides intent.

Common follow-up: "*How would you generate a row for every (date, store) pair in the last 30 days, even when there are no sales?*" — CROSS JOIN a date series with stores, then LEFT JOIN sales.

```
SELECT d.day, s.id, COALESCE(SUM(o.amount), 0) AS revenue
FROM generate_series(CURRENT_DATE - 29, CURRENT_DATE, '1 day') AS d(day)
CROSS JOIN stores s
LEFT JOIN orders o ON o.store_id = s.id AND o.order_date = d.day
GROUP BY d.day, s.id;
```

Q9 · Multi-Table Join Order

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Wells Fargo, AmEx · Topic: Joins

The question (interviewer's verbatim phrasing):

> Does the order in which I write joins in a query matter for performance?

The 30-second answer: For standard inner joins the optimizer is free to reorder them, so the order you write usually doesn't matter logically. But the join *shape* — star vs chain vs bushy — affects cost, and outer joins (LEFT/RIGHT/FULL) restrict reordering, so written order can matter there.

Step-by-step thought process: 1. Start with the cost-based optimizer point: Postgres, SQL Server, Oracle all reorder INNER joins using statistics. You write what reads well; the planner picks the order. 2. Outer joins are not freely reorderable because they change which rows survive. The planner respects the written nesting. 3. Mention the planner's join-order search limit: Postgres uses exhaustive dynamic programming until the join list exceeds `geqo_threshold` (default 12 items), at which point it switches to genetic optimization (GEQO). Separately, `join_collapse_limit` (default 8) controls how many explicit JOIN expressions get flattened into a single FROM list for reordering. 4. Real-world tip: when joining 10+ tables, break into CTEs or temp tables to constrain the planner and stabilise execution time.

Edge cases to mention: - `STRAIGHT_JOIN` in MySQL forces the written order — last-resort hint when the optimizer gets it wrong. - Statistics being stale is the most common reason a planner picks a bad join order — run `ANALYZE` after large data changes. - In Postgres, `join_collapse_limit = 1` disables reordering entirely — useful for debugging but not production.

What NOT to say: - Do not say "always join the smallest table first." The planner already knows table sizes; that advice is from a different era. - Do not claim outer-join order doesn't matter — it does, semantically.

Common follow-up: "*You have a 10-table query that's slow. How do you debug the join order?*" — Run EXPLAIN ANALYZE, check estimated vs actual row counts, refresh statistics, and consider breaking the query into stages.

Q10 · Subquery in SELECT vs FROM vs WHERE

Tags: Difficulty: Medium · Asked at: TCS, Razorpay, Swiggy, Target · Topic: Subqueries

The question (interviewer's verbatim phrasing):

> A subquery can sit in the SELECT list, in the FROM clause, or in the WHERE clause. What's the difference and when do you use which?

The 30-second answer: SELECT subquery returns one scalar value per row (correlated, often slow); FROM subquery (derived table) materialises a temporary result set you can join to; WHERE subquery is for filtering (EXISTS, IN, scalar comparison).

Step-by-step thought process: 1. Walk through each position with a concrete example. SELECT subquery: `SELECT u.id, (SELECT COUNT(*) FROM orders o WHERE o.user_id = u.id) AS order_count FROM users u`. FROM subquery: `FROM (SELECT user_id, SUM(amount) AS total FROM orders GROUP BY user_id) o`. WHERE subquery: `WHERE user_id IN (SELECT id FROM users WHERE country = 'IN')`. 2. State the performance angle: SELECT subqueries are usually correlated and run once per row. FROM subqueries (derived tables / CTEs) are evaluated once. WHERE subqueries via EXISTS/IN are usually optimized into semi-joins. 3. Mention CTEs as the modern alternative to FROM subqueries — same execution but far more readable for multi-step logic. 4. Anticipate: "When would you choose a window function instead of a SELECT subquery?" — When you need the value per row and the inner query depends on the outer row, window functions are usually faster and clearer.

Edge cases to mention: - SELECT subqueries must return exactly one row and one column — otherwise runtime error. - A derived table in FROM needs an alias in standard SQL (Postgres enforces this strictly; MySQL is lenient). - Postgres materialises CTEs by default before version 12; from 12 onwards it inlines them like derived tables.

What NOT to say: - Do not say "they're all the same, just different syntax." The semantics and performance differ meaningfully. - Do not nest 5 levels of subqueries when a CTE chain would read cleanly.

Common follow-up: "*Rewrite this scalar subquery in SELECT as a LEFT JOIN to a derived table.*" — Practise this; it's the most common interview refactor.

Section B — Window Functions

Q11 · ROW_NUMBER vs RANK vs DENSE_RANK

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Walmart Labs, Cred · Topic: Windows

The question (interviewer's verbatim phrasing):

> What's the difference between ROW_NUMBER, RANK and DENSE_RANK? Give me a case where picking the wrong one breaks your query.

The 30-second answer: ROW_NUMBER gives a unique sequence with no ties (1, 2, 3, 4); RANK gives ties the same number but skips the next value (1, 1, 3, 4); DENSE_RANK gives ties the same number with no gaps (1, 1, 2, 3). Picking ROW_NUMBER for "top 3 by revenue" when there are ties silently drops valid tied rows.

Step-by-step thought process: 1. Open with the tie behaviour — that's the whole point of the question. 2. Use the canonical example: ordering students by marks. If two students tie at 95, ROW_NUMBER assigns them 1 and 2 arbitrarily; RANK gives both 1 and the next gets 3; DENSE_RANK gives both 1 and the next gets 2. 3. State the practical rule: ROW_NUMBER for pagination and "give me one row per group"; DENSE_RANK for "top N including ties"; RANK when the gap is semantically meaningful (Olympic medals). 4. Anticipate the trap: `WHERE rn = 1` with ROW_NUMBER for top-per-group works, but `WHERE rnk <= 3` with RANK can return more than 3 rows because of skipped numbers — usually you want DENSE_RANK here.

Edge cases to mention: - ROW_NUMBER with a non-unique ORDER BY is non-deterministic — tied rows can come back in any order. Add a tiebreaker column. - RANK skipping numbers can confuse downstream code that assumes sequential ranks. - All three respect PARTITION BY — counting resets per partition.

What NOT to say: - Do not say "they're basically the same." Tied data exposes the difference instantly. - Do not use ROW_NUMBER for "top 3 by score" interview questions when the interviewer probably wants ties included.

Common follow-up: "*Top 3 highest-paid employees per department, including ties.*" — DENSE_RANK with `rnk <= 3` is the canonical answer.

```
SELECT *, DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk
FROM employees
QUALIFY rnk <= 3;  -- Snowflake/BigQuery; else wrap in subquery
```

Q12 · PARTITION BY vs GROUP BY

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, AmEx · Topic: Windows

The question (interviewer's verbatim phrasing):

> What's the conceptual difference between PARTITION BY in a window function and GROUP BY?

The 30-second answer: GROUP BY collapses rows — you get one row per group. PARTITION BY divides rows into groups for the window function to operate on, but every input row is preserved in the output.

Step-by-step thought process: 1. Lead with row preservation — that's the headline difference. 2. Concrete example: `SELECT user_id, amount, AVG(amount) OVER (PARTITION BY user_id) FROM orders`. You see every order plus that user's average alongside. With GROUP BY you'd see one row per user with the average only. 3. Mention they can coexist: you can GROUP BY one column and use window functions over the grouped result for cross-group comparison (running totals over weekly aggregates, for example). 4. Anticipate: "When would you use PARTITION BY without ORDER BY?" Answer: for partition-wide aggregates like average, count, sum where you don't need ordering.

Edge cases to mention: - A window function with no PARTITION BY treats the whole result set as a single partition — useful for "share of total" calculations. - Window functions execute after WHERE/GROUP BY/HAVING but before ORDER BY/LIMIT — this ordering matters for the filter-window trap (Q18). - Performance-wise, PARTITION BY usually requires a sort or hash — heavy on huge tables.

What NOT to say: - Do not say "PARTITION BY is the same as GROUP BY" — losing the row-preservation distinction is a red flag. - Do not GROUP BY when you wanted a window function — you'll lose row-level detail and have to re-join.

Common follow-up: "*Show me each order, the user's total spend, and the order's share of that total — in one query.*" — Window function with PARTITION BY plus arithmetic.

Q13 · Frame Clauses — ROWS vs RANGE

Tags: Difficulty: Hard · Asked at: Goldman, Razorpay, Walmart Labs, JP Morgan · Topic: Windows

The question (interviewer's verbatim phrasing):

> Walk me through frame clauses. What's the difference between ROWS BETWEEN and RANGE BETWEEN, and what does UNBOUNDED PRECEDING mean?

The 30-second answer: ROWS counts physical rows; RANGE groups rows with the same ORDER BY value into a logical band. UNBOUNDED PRECEDING means "from the start of the partition." Default frame for most aggregates is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which silently includes ties — the source of many bugs.

Step-by-step thought process: 1. Define the frame: a sliding window of rows over which the function aggregates, anchored at the current row. 2. Show ROWS vs RANGE with an example. Three rows with ORDER BY `day` where two rows share the same day: ROWS BETWEEN 1 PRECEDING AND CURRENT ROW gives the previous row and current; RANGE includes all rows tied with the current row's day. 3. UNBOUNDED PRECEDING = from start. UNBOUNDED FOLLOWING = to end. CURRENT ROW = the current row. These combine to define the window. 4. The trap: the default frame for SUM/AVG with ORDER BY is `RANGE UNBOUNDED PRECEDING TO CURRENT ROW`, so duplicate ORDER BY values get summed together as one step — usually not what you want. Specify `ROWS` explicitly.

Edge cases to mention: - Without ORDER BY, the default frame is the entire partition — useful for "share of partition total." - With ORDER BY, the default frame becomes "start to current row" — running total semantics with the RANGE tie behaviour. - `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` is the standard "7-day moving sum" pattern.

What NOT to say: - Do not use RANGE for moving averages — ties will lump rows together and corrupt the average. - Do not omit the frame and assume ROWS — the default is RANGE-based in most engines.

Common follow-up: "*Write a 7-day moving average of daily transaction volume.*" — `AVG(amount) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)`.

```
SELECT day, amount,
       SUM(amount) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS rolling_7d
FROM transactions;
```

Q14 · LAG and LEAD

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Zerodha, Cred · Topic: Windows

The question (interviewer's verbatim phrasing):

> Explain LAG and LEAD. Show me how you'd compute month-over-month revenue change.

The 30-second answer: LAG returns the value from a previous row in the partition; LEAD returns the value from a following row. Both accept an offset and a default value for when the row doesn't exist — crucial for the first/last row in the partition.

Step-by-step thought process: 1. Define the signature: `LAG(column, offset, default) OVER (PARTITION BY ... ORDER BY ...)`. Offset defaults to 1, default value defaults to NULL. 2. Show the MoM example: `LAG(revenue, 1, 0) OVER (ORDER BY month)` then subtract to get the change. 3. Mention LEAD is the mirror — useful for "time to next event" calculations like session timeouts or delivery latency. 4. Anticipate the gotcha: NULL default vs 0 default. If you subtract from NULL, you get NULL, not the first row's full value. Specify the default explicitly.

Edge cases to mention: - For the first row in a partition, LAG returns the default (NULL unless you specify). - Without PARTITION BY, LAG operates over the whole result set — useful for time-series with one entity. - LAG/LEAD don't accept the frame clause — they reference specific rows, not a range.

What NOT to say: - Do not write a self-join with `row_number - 1` to simulate LAG — slow and ugly. - Do not forget the default value when downstream code chokes on NULL.

Common follow-up: *"Now compute the percentage change, not absolute."* — `(current - prev) / NULLIF(prev, 0)` — guard against division by zero.

```
SELECT month, revenue,
       revenue - LAG(revenue, 1, 0) OVER (ORDER BY month) AS mom_change
FROM monthly_revenue;
```

Q15 · FIRST_VALUE / LAST_VALUE Frame Trap

Tags: Difficulty: Hard · Asked at: Goldman, Walmart Labs, JP Morgan, AmEx · Topic: Windows

The question (interviewer's verbatim phrasing):

> If I write `LAST_VALUE(amount) OVER (PARTITION BY user_id ORDER BY created_at)`, what value do I actually get?

The 30-second answer: You get the amount of the current row, not the last row in the partition — because the default frame with ORDER BY is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, so "last" means last-seen-so-far. To get the true partition last, specify `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.

Step-by-step thought process: 1. State the trap up front — interviewers love this question because most candidates get it wrong. 2. Explain the why: window frames default to "start to current row" with ORDER BY, so LAST_VALUE returns the current row's value, not the partition's last value. 3. Give the fix: explicit frame `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`. Now you actually get the last value. 4. Mention FIRST_VALUE works correctly with the default frame because the first row of "start to current row" is always the partition's first row.

Edge cases to mention: - An alternative for "last value" is `FIRST_VALUE` with a reversed ORDER BY — sometimes cleaner. - NTH_VALUE has the same frame dependency — explicit frame essential. - This trap doesn't exist in some warehouse dialects that default differently — but trust the standard behaviour.

What NOT to say: - Do not say "LAST_VALUE returns the partition's last row" without the frame caveat — that's the wrong answer. - Do not omit the frame "to keep the query short" — correctness first.

Common follow-up: "*Get the first and last transaction amount per user, in one query.*" — Two window expressions with explicit frames.

```
SELECT user_id, created_at, amount,
       FIRST_VALUE(amount) OVER (PARTITION BY user_id ORDER BY created_at) AS first_amt,
       LAST_VALUE(amount) OVER (PARTITION BY user_id ORDER BY created_at
                                ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS last_
FROM transactions;
```

Q16 · Running Totals

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy · Topic: Windows

The question (interviewer's verbatim phrasing):

> Compute the running total of daily revenue.

The 30-second answer: `SUM(revenue) OVER (ORDER BY day)` — with ORDER BY, the default frame accumulates from the start of the partition to the current row, which is the running total semantics.

Step-by-step thought process: 1. State the one-liner immediately — it's a short answer question, don't over-engineer. 2. Mention that without PARTITION BY, the running total runs across the whole result. Add `PARTITION BY user_id` for per-user running totals. 3. Make the default frame explicit if asked — `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` for safety, since RANGE with ties can lump days together (Q13). 4. Anticipate the variant: "running total of the last 7 days" — change frame to `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW`.

Edge cases to mention: - Duplicate dates with RANGE-based default frame will share the same running total — usually wrong; use ROWS. - Missing days are silently skipped — if you need contiguous days, CROSS JOIN with a date series first. - Performance: a single window sort is usually faster than self-joining the table to itself.

What NOT to say: - Do not write a correlated subquery `(SELECT SUM(...) FROM t2 WHERE t2.day <= t1.day)` — that's O(N^2) and the wrong answer in 2026. - Do not omit ORDER BY — without it, SUM returns the partition total on every row, not a running total.

Common follow-up: "*What if I want running total to reset every month?*" — `PARTITION BY DATE_TRUNC('month', day) ORDER BY day`.

```
SELECT day, revenue,
       SUM(revenue) OVER (ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS runnin
FROM daily_revenue;
```

Q17 · NTILE

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target, AmEx, Wells Fargo · Topic: Windows

The question (interviewer's verbatim phrasing):

> What does NTILE do? Will it always give you equal-sized buckets?

The 30-second answer: NTILE(n) divides the partition into n approximately-equal buckets and tags each row with its bucket number. Buckets are equal-sized when the row count divides evenly; otherwise, the first few buckets each get one extra row.

Step-by-step thought process: 1. Define the function: `NTILE(4) OVER (ORDER BY amount)` puts every row into one of 4 buckets (quartiles). 2. Show the unequal case: 10 rows into 4 buckets — buckets 1 and 2 get 3 rows each, buckets 3 and 4 get 2 rows each. The "spillover" rows go to the front. 3. Common use cases: assigning customers to deciles by spend, splitting test/control groups, A/B bucketing. 4. Anticipate the trap: NTILE does not group by value — two rows with the same ORDER BY value can land in different buckets. Use PERCENT_RANK or manual bucketing for value-based bucketing.

Edge cases to mention: - With PARTITION BY, NTILE buckets reset per partition. - For percentile-style bucketing where ties must share a bucket, use `WIDTH_BUCKET` or arithmetic on PERCENT_RANK instead. - NTILE on a small partition (fewer rows than buckets) leaves trailing buckets empty.

What NOT to say: - Do not say "NTILE gives exactly equal buckets" — wrong for non-divisible row counts. - Do not confuse NTILE buckets with percentiles — they're row-based, not value-based.

Common follow-up: "*How would you assign users to deciles by lifetime spend?*" — `NTILE(10) OVER (ORDER BY total_spend)`.

Q18 · The Filter-Window Trap

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, Flipkart · Topic: Windows

The question (interviewer's verbatim phrasing):

> Why can't I write `WHERE ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY created_at DESC) = 1` ? And what's the right way?

The 30-second answer: WHERE runs before window functions are evaluated, so the window function isn't visible at WHERE time. Wrap the query in a subquery or CTE and filter on the window output in the outer query — or use QUALIFY (Snowflake, BigQuery, Teradata).

Step-by-step thought process: 1. Explain the logical execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → window functions → DISTINCT → ORDER BY → LIMIT. Window functions evaluate after WHERE. 2. Show the two fixes. First, the portable one — wrap in a CTE: `WITH ranked AS (SELECT *, ROW_NUMBER() OVER (...) AS rn FROM t) SELECT * FROM ranked WHERE rn = 1`. Second, the warehouse-only one: `SELECT * FROM t QUALIFY ROW_NUMBER() OVER (...) = 1`. 3. Mention QUALIFY is supported in Snowflake, BigQuery, Teradata, Databricks SQL — not in Postgres, MySQL, or SQL Server yet. 4. Anticipate the follow-up about HAVING: HAVING also runs before window functions, so it has the same restriction.

Edge cases to mention: - Subquery vs CTE: in modern Postgres, both inline the same way. In older Postgres (<12), CTEs were optimization fences. - QUALIFY is cleaner but locks you to specific dialects — call this out in code reviews. - You can use a window function inside an ORDER BY clause directly — that one position works because ORDER BY runs after window evaluation.

What NOT to say: - Do not propose "just add it to WHERE" without acknowledging the order problem — interviewers want to hear you understand why it fails. - Do not assume QUALIFY exists everywhere; ask which warehouse the team uses before writing it.

Common follow-up: "*What's the difference between filtering with WHERE vs HAVING vs QUALIFY?*" — WHERE filters rows before grouping, HAVING filters after grouping but before windows, QUALIFY filters after windows.

```
-- Latest order per user (portable form)
WITH ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY created_at DESC) AS rn
  FROM orders
)
SELECT * FROM ranked WHERE rn = 1;
```

Q19 · PERCENT_RANK and CUME_DIST

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Walmart Labs, Target · Topic: Windows

The question (interviewer's verbatim phrasing):

> What's the difference between PERCENT_RANK and CUME_DIST?

The 30-second answer: Both return a value between 0 and 1 reflecting position in the partition. PERCENT_RANK is `(rank - 1) / (total_rows - 1)` — the relative rank, starts at 0. CUME_DIST is `count_of_rows_with_value_<=_current / total_rows` — the cumulative distribution, never returns 0.

Step-by-step thought process: 1. State the formulas — interviewers want to see you actually know the maths, not just hand-wave. 2. PERCENT_RANK is "what fraction of rows are strictly below me?" — useful for percentile-style rankings. 3. CUME_DIST is "what fraction of rows are at or below me?" — used in statistical analysis and CDF computations. 4. Example: 4 rows with values 10, 20, 20, 30. For value 20: PERCENT_RANK = (2-1)/(4-1) = 0.333; CUME_DIST = 3/4 = 0.75.

Edge cases to mention: - Ties get the same value for both functions — they're not based on row position alone. - PERCENT_RANK on a single-row partition is 0 (no other rows to be ranked against), not NULL. - CUME_DIST is monotonically non-decreasing as you walk the ORDER BY — useful sanity check.

What NOT to say: - Do not confuse with NTILE — NTILE gives bucket numbers, not fractions. - Do not say they're "basically the same" — the inclusion-of-current-row difference matters in statistics.

Common follow-up: "*When would you use PERCENT_RANK in a real analytics query?*" — Computing percentile-rank salaries, exam scores, or merchant performance bands.

Q20 · Top-N-Per-Group via Window Functions

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, PhonePe · Topic: Windows

The question (interviewer's verbatim phrasing):

> Get the top 3 highest-amount orders for each customer. Why is a window function the canonical answer?

The 30-second answer: Number rows within each customer partition using `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY amount DESC)`, then keep where the row number is ≤ 3. A pre-window GROUP BY can't do this because GROUP BY collapses rows; you'd need an awkward correlated subquery instead.

Step-by-step thought process: 1. State the pattern. Number rows per partition, then filter. Three steps. 2. Pick the right ranking function: ROW_NUMBER if you want exactly 3 even with ties, DENSE_RANK if "top 3 including ties" is what they mean. 3. Show the CTE wrapper (filter trap from Q18) — `WITH`

`ranked AS (...) SELECT * FROM ranked WHERE rn <= 3`. 4. Mention the alternative — LATERAL join (Q7) — which is cleaner when the inner logic is complex but slower when N is small and the left side is huge.

Edge cases to mention: - Ties in amount with ROW_NUMBER will arbitrarily pick a winner — add a tiebreaker like `id` in the ORDER BY. - For "top 3 including ties" use DENSE_RANK ≤ 3 — can return more than 3 rows per partition. - Performance: window functions sort once per partition — much faster than correlated subqueries.

What NOT to say: - Do not use a correlated subquery `(SELECT COUNT(*) FROM o2 WHERE o2.customer_id = o1.customer_id AND o2.amount > o1.amount) < 3` — works but slow. - Do not LIMIT in the inner query — LIMIT is per-query, not per-partition. Window function or LATERAL is the right tool.

Common follow-up: *"What if I want top 3 per customer, but only for the last 90 days?"* — Add `WHERE created_at >= CURRENT_DATE - 90` in the CTE before windowing.

```
WITH ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY amount DESC, id) AS rn
  FROM orders
)
SELECT * FROM ranked WHERE rn <= 3;
```

Section C — Indexes & Performance

Q21 · B-tree Index Internals

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, Walmart Labs · Topic: Indexes

The question (interviewer's verbatim phrasing):

> Explain how a B-tree index works internally. Why is lookup O(log n)?

The 30-second answer: A B-tree stores keys sorted in a balanced tree of pages — each internal page holds key ranges that point to child pages; leaf pages hold the actual keys plus row pointers (or, in clustered indexes, the row itself). Lookup walks from root to leaf, one page read per level, and tree depth grows logarithmically with row count.

Step-by-step thought process: 1. Sketch the structure: root page → internal pages → leaf pages, all balanced and sorted. 2. Explain the height. With page fan-out of ~100-500, a 1 billion row index is only 4-5 levels deep — so a lookup is 4-5 page reads, hence O(log n). 3. Mention what's at the leaf: in Postgres, a heap row pointer (CTID); in MySQL InnoDB, the primary key index leaf contains the actual

row, and secondary index leaves contain the primary key (the "clustered index" model). 4. Anticipate the follow-up: range scans walk leaves left-to-right via sibling pointers — O(log n + k) where k is the matched range size.

Edge cases to mention: - B-tree is best for equality and range; hash indexes win on pure equality (Q25). - Insertions can cause page splits — high write rates fragment the index over time, hence periodic REINDEX/OPTIMIZE. - Leaf pages typically hold dozens to hundreds of keys per 8KB page; depth stays small even for huge tables.

What NOT to say: - Do not say "B-tree is a binary tree." It's not — it's a high-fan-out balanced tree (the B is for "balanced," not "binary"). - Do not claim O(1) lookup — that's hash territory, not B-tree.

Common follow-up: *"Why does a B-tree work well for ORDER BY queries?"* — Because the leaves are stored in sorted order, the engine can stream them out without an extra sort.

Q22 · Composite Index Column Order

Tags: Difficulty: Medium · Asked at: TCS, Razorpay, Swiggy, Flipkart · Topic: Indexes

The question (interviewer's verbatim phrasing):

> If I create an index on `(user_id, created_at, status)`, can the database use it for a query that filters only on `created_at`?

The 30-second answer: No — composite indexes follow the leftmost-prefix rule. The index can serve queries that filter on `user_id`, or `user_id + created_at`, or all three; it cannot efficiently serve a query that filters only on `created_at` or only on `status`, because the leading column is required.

Step-by-step thought process: 1. Explain the physical layout: a composite index sorts rows by the first column, then by the second within ties, then the third. Without the first-column filter, the index is just a giant unsorted list with respect to the columns you care about. 2. State the leftmost-prefix rule clearly. Filter on `(user_id)`, `(user_id, created_at)`, or `(user_id, created_at, status)` — yes. Filter on `(created_at)` alone or `(status)` alone — no. 3. Mention the "index skip scan" feature (Oracle, MySQL 8) — it can sometimes use a non-leading column on low-cardinality leading columns, but it's slow and usually a sign you need a different index. 4. Anticipate: "What's the right column order?" Equality columns first, then range columns, then often-ORDER-BY columns last.

Edge cases to mention: - Postgres can sometimes do a "bitmap index scan" combining two separate single-column indexes — but it's almost always slower than a properly designed composite index. - For ORDER BY to use the index, the ORDER BY columns must match the index column order (and direction, unless the engine supports backward scans). - An index on `(a, b)` covers queries that filter on `a` and order by `b` — that's a key composite-index use case.

What NOT to say: - Do not say "the index covers any column listed in it." That's the trap. - Do not create one composite index per query — index maintenance cost adds up (Q29).

Common follow-up: "*I have queries that filter on `(user_id, created_at)` and queries that filter only on `created_at`. What do I do?*" — Either create both `(user_id, created_at)` and `(created_at)` indexes, or reverse the composite order if the `created_at` queries are more frequent.

Q23 · Covering Index and Index-Only Scans

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Walmart Labs, JP Morgan · Topic: Indexes

The question (interviewer's verbatim phrasing):

> What's a covering index? What's the INCLUDE clause in Postgres or SQL Server?

The 30-second answer: A covering index includes all columns needed by the query — both the filter/sort columns and the SELECT-list columns — so the engine can answer the query entirely from the index without touching the table. INCLUDE adds non-key columns to the index leaves so they're available without being part of the sort key.

Step-by-step thought process: 1. Define index-only scan. EXPLAIN will literally say "Index Only Scan" in Postgres or "Index Seek with no Key Lookup" in SQL Server — the engine never reads the heap. 2. Show the value of INCLUDE. `CREATE INDEX ... ON orders (user_id) INCLUDE (amount, status)` — the index is still sorted by `user_id`, but the leaf pages also store `amount` and `status` so `SELECT amount, status FROM orders WHERE user_id = ?` is fully covered. 3. Why not just put everything in the key? Wider keys mean fewer entries per page, deeper trees, more write cost. INCLUDE keeps the key narrow but the leaf rich. 4. Anticipate: "When is index-only scan not actually free?" — In Postgres, visibility map must be up to date; otherwise the engine still has to verify rows in the heap. Run VACUUM regularly.

Edge cases to mention: - MySQL doesn't have INCLUDE syntax; you add extra columns to the index key — wider but works. - Covering indexes on wide tables can be much larger than the indexed columns alone — measure storage. - Postgres index-only scan can degrade if VACUUM hasn't updated the visibility map after heavy writes.

What NOT to say: - Do not say "covering index means SELECT *" — that's an anti-pattern; only cover what you actually need. - Do not assume covering always wins — the extra write/storage cost can hurt write-heavy tables.

Common follow-up: "*How do you tell if you got an index-only scan?*" — EXPLAIN ANALYZE will say "Index Only Scan" plus "Heap Fetches: 0" (Postgres) or no Key Lookup operator (SQL Server).

Q24 · When the Index Isn't Used

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Razorpay, AmEx · Topic: Indexes

The question (interviewer's verbatim phrasing):

> I have an index on `email` but the query `WHERE LOWER(email) = 'x@y.com'` is doing a full table scan. Why?

The 30-second answer: Wrapping the column in a function (LOWER) prevents the engine from using the index, because the index is sorted by `email`, not by `LOWER(email)`. Fix it by storing emails normalized, by creating a functional index on `LOWER(email)`, or by ensuring the comparison is direct.

Step-by-step thought process: 1. State the principle: any operation that transforms the indexed column on the left side of the comparison (function, arithmetic, cast) disables the index. 2. List common offenders. `LOWER(email) = ?`, `email || '_' = ?`, `CAST(created_at AS DATE) = ?`, `created_at + INTERVAL '1 day' = ?`. Also implicit casts — comparing a `VARCHAR` column to an integer literal in MySQL casts every row. 3. Give fixes for each. Functional index in Postgres: `CREATE INDEX ON users (LOWER(email))`. For dates, rewrite `WHERE created_at >= '2026-01-01' AND created_at < '2026-01-02'` instead of `WHERE DATE(created_at) = '2026-01-01'`. 4. Other reasons indexes get skipped: low cardinality where a full scan is cheaper, `LIKE '%foo'` where the leading wildcard prevents B-tree use, OR conditions that the planner can't decompose, very small tables where index lookup overhead exceeds full scan cost.

Edge cases to mention: - LIKE 'foo%' can use a B-tree index (anchored prefix); LIKE '%foo%' cannot — use trigram or full-text indexes. - Implicit casts often happen with `WHERE phone_number = 9876543210` against a VARCHAR column — quote the literal. - The planner makes cost-based decisions — sometimes it's right to skip the index. Always check EXPLAIN with actual data sizes.

What NOT to say: - Do not say "the index is broken" or "the optimizer is wrong" without checking — usually it's a wrap, cast, or wildcard issue. - Do not force the index with hints as your first move; understand why it's skipped first.

Common follow-up: "*Show me how you'd debug an index-not-being-used issue end to end.*" — EXPLAIN ANALYZE, check column transformations, check statistics with ANALYZE, check selectivity.

Q25 · Hash Index vs B-tree

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Walmart Labs, Cred · Topic: Indexes

The question (interviewer's verbatim phrasing):

> When would you choose a hash index over a B-tree?

The 30-second answer: Hash indexes give O(1) lookup for equality only — they cannot do ranges, ORDER BY, or prefix matches. So pick hash when you exclusively do equality lookups on a column and the table is huge enough that the constant-factor improvement matters; pick B-tree for everything else.

Step-by-step thought process: 1. Explain the data structures briefly. Hash index = hash table on disk: hash the key, get the bucket, find the row. B-tree = sorted balanced tree. 2. State the equality vs range distinction. Hash wins on equality alone. B-tree handles equality, range (`>` , `<` , `BETWEEN`), prefix LIKE, ORDER BY, all from the same structure. 3. Practical reality. In Postgres, hash indexes were unsafe before version 10 (no WAL logging); now they're production-ready but rarely chosen because B-tree is almost as fast on equality and far more flexible. In MySQL InnoDB, hash indexes exist as the "adaptive hash index" built automatically. 4. Anticipate: "Why don't we always use hash for primary keys?" — Because primary keys are often used in range scans and ORDER BY (pagination), where hash is useless.

Edge cases to mention: - Hash collisions degrade lookup; high-cardinality keys help. - Hash indexes don't support multi-column indexes meaningfully (you'd hash a tuple but lose the leftmost-prefix benefit). - In-memory hash tables (Redis-style) are different from disk-based hash indexes — don't confuse the two.

What NOT to say: - Do not say "hash is always faster than B-tree." For low-cardinality range queries B-tree wins; for tiny tables both are wasted. - Do not use hash for a column you'll later need to range-query — you'll have to rebuild.

Common follow-up: "*What is the InnoDB adaptive hash index?*" — InnoDB watches access patterns on B-tree pages; when it sees the same pages hit repeatedly, it builds an in-memory hash overlay automatically.

Q26 · Index Scan vs Index Seek vs Table Scan

Tags: Difficulty: Medium · Asked at: TCS, Wells Fargo, AmEx, Target · Topic: Indexes

The question (interviewer's verbatim phrasing):

> In EXPLAIN output, what's the difference between an index scan, an index seek, and a table scan?

The 30-second answer: Index seek navigates the B-tree to a specific key range — best case. Index scan reads the whole index sequentially — used when many rows match or no useful key range exists. Table scan (or sequential scan in Postgres) reads every heap row — used when no index helps or the table is small.

Step-by-step thought process: 1. Define each operator precisely. Seek = targeted descent into the tree. Scan = full traversal of the index leaves. Table/sequential scan = read heap pages directly. 2. State the cost ordering: seek (fastest, few pages) → index scan (medium, all index pages) → table scan (slowest for selective queries, but fastest for full-table aggregates). 3. Mention the SQL Server terminology specifically — interviewers from MS-stack shops use these exact terms. Postgres EXPLAIN uses "Index Scan," "Index Only Scan," "Bitmap Index Scan," "Seq Scan." 4. Anticipate: "When is a table scan actually the right plan?" — When the query touches a large fraction of the table (say >10-20% of rows), the random I/O of index lookups exceeds the cost of sequential reads. Planner makes this call from statistics.

Edge cases to mention: - Index scan can be "Index Only" if all needed columns are in the index — never touches the heap. - Bitmap index scan combines multiple indexes by ORing/ANDing bitmaps — useful when no single composite index fits. - A "Seek + Key Lookup" plan (SQL Server) means the index seek found rows but had to fetch additional columns from the table — fix with covering index.

What NOT to say: - Do not say "table scans are always bad." For small tables or large fractions of the table, they're the right choice. - Do not assume "Index Scan" means seek — in Postgres EXPLAIN, "Index Scan" can mean full index traversal.

Common follow-up: "*Walk me through reading a Postgres EXPLAIN ANALYZE plan.*" — Lead with rows/loops, compare estimated vs actual, identify the most expensive operator.

Q27 · Reading EXPLAIN ANALYZE

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Goldman, Cred · Topic: Performance

The question (interviewer's verbatim phrasing):

> Walk me through how you'd read an EXPLAIN ANALYZE output. What does "cost" mean?

The 30-second answer: EXPLAIN shows the planner's predicted plan with estimated costs; EXPLAIN ANALYZE actually runs the query and adds real timing and row counts. "Cost" is an arbitrary internal unit — Postgres roughly calibrates 1.0 to a single sequential page read. The real signal is the gap between estimated and actual rows.

Step-by-step thought process: 1. Distinguish EXPLAIN (cheap, planning only) vs EXPLAIN ANALYZE (executes the query — beware of side effects on UPDATE/DELETE; wrap in BEGIN/ROLLBACK). 2. Explain the cost numbers. Cost is `startup_cost..total_cost` — startup is what's done before the first row is returned. The number itself is not seconds; it's a relative internal unit. 3. The most useful diagnostic: actual rows vs estimated rows. A 1000x mismatch means stats are stale (run ANALYZE) or the planner has bad correlation estimates — both lead to bad join orders. 4. Look for: full table scans on big tables, nested loops with high outer-row counts, hash joins spilling to disk (`Buffers: temp written=...`), and the order of operations (innermost = run first).

Edge cases to mention: - Always read EXPLAIN bottom-up — the deepest nested operator runs first. - Multiply Actual Rows × Loops for true row count in nested loop inner sides. - BUFFERS option (`EXPLAIN (ANALYZE, BUFFERS)`) shows page hits — useful for spotting cache effectiveness.

What NOT to say: - Do not say "the cost is in milliseconds." It isn't. Actual time is in EXPLAIN ANALYZE output, not in the cost. - Do not optimize without reading actual numbers — guessing usually makes things worse.

Common follow-up: *"A query is slow only in production. How would you diagnose?"* — Capture EXPLAIN ANALYZE in prod (with BUFFERS), compare against staging stats, check for plan flips due to changed data distribution.

Q28 · Low-Cardinality Index and Partial Indexes

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Walmart Labs, AmEx · Topic: Indexes

The question (interviewer's verbatim phrasing):

> I have an index on `status` which has only two values, ACTIVE and INACTIVE. The query planner ignores it. Why? And when would a partial index help?

The 30-second answer: Low-cardinality indexes get skipped because the planner estimates a large fraction of the table matches — at that selectivity, a sequential scan beats jumping through the index. A partial index `WHERE status = 'ACTIVE'` indexes only the rare rows, making it small and worth using.

Step-by-step thought process: 1. Explain selectivity. The planner estimates how many rows match. If matches are >5-10% of the table, the random I/O cost of index lookups exceeds sequential scan; planner picks seq scan. 2. With only two distinct values, status is splitting the table 50/50 or similar — index lookups would re-read most of the table anyway. 3. Partial index: `CREATE INDEX ON orders (created_at) WHERE status = 'ACTIVE'` — only indexes ACTIVE rows. Smaller index, faster lookups, and queries filtering for ACTIVE can use it. 4. Mention this is heavily used in soft-delete patterns: index only `WHERE deleted_at IS NULL`. Saves storage and improves lookup speed.

Edge cases to mention: - Partial indexes need the WHERE condition to be present in the query — exact match required. - MySQL doesn't have partial indexes; you can simulate with generated columns + index. - For low-cardinality columns where most queries filter the rare value, partial is the right answer; if both values are queried equally, neither index helps much.

What NOT to say: - Do not say "always index every column." Bad indexes hurt writes and waste cache (Q29). - Do not blame the planner for "ignoring" a useless index — it's making the right call.

Common follow-up: "*What about a multi-column index starting with status?*" — Generally still bad because leading low-cardinality columns force the index scan to read many pages; lead with the high-cardinality column instead.

```
-- Partial index for the common "active" filter
CREATE INDEX idx_active_orders_created
  ON orders (created_at)
  WHERE status = 'ACTIVE';
```

Q29 · Index Maintenance Cost

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Goldman · Topic: Performance

The question (interviewer's verbatim phrasing):

> What's the downside of adding indexes? Why can't you just index every column?

The 30-second answer: Every INSERT, UPDATE on an indexed column, and DELETE has to update each affected index — write amplification. Indexes also consume disk and buffer cache memory that could be used for hotter data. So more indexes = slower writes, larger storage, and sometimes worse read cache hit rates.

Step-by-step thought process: 1. State the write cost. An INSERT into a table with 5 indexes does 6 writes (1 heap + 5 indexes). UPDATE costs: if the updated column is indexed, the index entry has to move; if not, the index is untouched (HOT updates in Postgres). 2. Mention buffer cache competition. Indexes take memory; if indexes don't fit in RAM, you do random disk I/O for every lookup. Better to have a few well-targeted indexes that fit in cache. 3. Index bloat. Postgres MVCC keeps old tuples until VACUUM; indexes bloat with churn and need REINDEX. SQL Server / MySQL InnoDB have similar maintenance under heavy write loads. 4. Anticipate: "How do you decide which indexes to add?" — Profile actual slow queries with EXPLAIN, add the minimum number of composite indexes to cover the hottest patterns, monitor `pg_stat_user_indexes` for unused ones and drop them.

Edge cases to mention: - Bulk loads benefit from dropping indexes, loading, then recreating — much faster than maintaining indexes per row. - Indexes on UUID primary keys cause random insertion patterns and high page-split rates — consider time-ordered UUIDs (UUIDv7) or sequential IDs for write-heavy tables. - Unused indexes are pure overhead — Postgres `pg_stat_user_indexes` shows scan counts; drop anything with zero scans over a representative period.

What NOT to say: - Do not propose "index everything" — it's the most common junior mistake. - Do not ignore index maintenance jobs in monitoring; bloat sneaks up.

Common follow-up: "*How would you find unused indexes in production?*" — Postgres: `SELECT * FROM pg_stat_user_indexes WHERE idx_scan = 0`. SQL Server: `sys.dm_db_index_usage_stats`.

Q30 · Clustered vs Non-Clustered Index

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Wells Fargo, JP Morgan · Topic: Indexes

The question (interviewer's verbatim phrasing):

> Explain the difference between a clustered and a non-clustered index. How does MySQL InnoDB handle this versus Postgres?

The 30-second answer: A clustered index physically orders the table by the index key — the table itself is the index leaf. A non-clustered index is a separate structure pointing back to the row. MySQL InnoDB stores tables as a clustered B-tree on the primary key; secondary indexes store the PK, not a row pointer. Postgres has no clustered indexes — the heap is unordered (a "heap table"), and all indexes point to physical row addresses (CTIDs).

Step-by-step thought process: 1. Define both clearly. Clustered = table data IS the index leaf, sorted by index key. Non-clustered = separate B-tree pointing to row location. 2. MySQL InnoDB specifics. PRIMARY KEY is the clustered index — table is physically sorted by PK. Secondary index leaves contain the PK value, not a row pointer — so a secondary index lookup does two B-tree walks: secondary index → PK → row. This is why a short, monotonically increasing PK is critical for InnoDB. 3. Postgres specifics. All tables are heap-organized (unordered). All indexes are "secondary" and store the row's physical address (CTID). There's a `CLUSTER` command that physically reorders the heap by an index — but it's a one-time operation; new inserts don't maintain order. 4. SQL Server specifics: similar to InnoDB — every table can have one clustered index; non-clustered indexes store the clustering key.

Edge cases to mention: - InnoDB: choosing a random UUID as PK kills write performance because every insert lands at a random position in the clustered index, causing page splits. - Postgres CLUSTER takes an exclusive table lock — only viable for offline maintenance. - "Heap" in SQL Server means a table with no clustered index — generally avoided in production because non-clustered indexes then point to physical RIDs, which break on row movement.

What NOT to say: - Do not say "Postgres clustered indexes work like SQL Server's." They don't — Postgres lacks an always-on clustered storage model. - Do not assume the primary key is automatically clustered in every database — true for InnoDB and SQL Server default, but not for Postgres.

Common follow-up: "*Why does InnoDB recommend short integer primary keys?*" — Because the PK is embedded in every secondary index leaf; a long PK bloats every index.

Section D — Transactions & Isolation

Q31 · ACID Explained Practically

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay · Topic: Transactions

The question (interviewer's verbatim phrasing):

> Explain ACID. Don't recite the textbook — tell me what each letter actually means when you're writing payment code.

The 30-second answer: ACID = Atomicity (all-or-nothing), Consistency (DB ends in a valid state per its rules), Isolation (concurrent txns don't see each other's half-done work), Durability (once COMMIT returns, the data survives a crash).

Step-by-step thought process: 1. Atomicity — debit + credit in a fund transfer must both succeed or both roll back. If only the debit lands, money vanishes. 2. Consistency — DB-level invariants (FK, CHECK, NOT NULL, triggers) hold before and after the txn. App-level invariants (account balance >= 0) are your job, not the DB's. 3. Isolation — if two txns run concurrently, the outcome should look as if they ran one after the other. The isolation level controls how strict this is. 4. Durability — after COMMIT, even if the server is yanked from the rack, the change is on disk (WAL fsync'd).

Edge cases to mention: - Atomicity is per-transaction, not per-statement — a single INSERT that fails mid-way is auto-rolled back, but if you have 5 statements and statement 3 fails, you must explicitly ROLLBACK or the DB may leave 1 and 2 committed (depends on the driver / autocommit setting). - Durability has gradations — `synchronous_commit = off` in Postgres trades durability for throughput; a crash can lose the last few txns. - Consistency in CAP theorem is a different word — that's about replicas converging, not about DB invariants.

What NOT to say: - "Consistency means the data is correct" — too vague, the interviewer wants the invariant-preservation meaning. - "Isolation means transactions are independent" — they are not always; READ UNCOMMITTED makes them very dependent.

Common follow-up: *Which of A-C-I-D do you most often see relaxed for performance, and where?*

```
-- Atomicity in action: classic fund transfer
BEGIN;
UPDATE accounts SET balance = balance - 5000 WHERE id = 101;
UPDATE accounts SET balance = balance + 5000 WHERE id = 202;
-- If either UPDATE fails or app crashes, ROLLBACK leaves both untouched
COMMIT;
```

Q32 · Read Uncommitted Isolation Level

Tags: Difficulty: Easy · Asked at: Cognizant, Accenture, Wells Fargo India · Topic: Isolation

The question (interviewer's verbatim phrasing):

> What does READ UNCOMMITTED actually allow? Have you ever used it in production?

The 30-second answer: READ UNCOMMITTED allows "dirty reads" — a txn can read rows another txn has modified but not yet committed. You almost never want this. The legitimate use is approximate analytics queries on hot tables where you'd rather see slightly-wrong-but-fast data than block.

Step-by-step thought process: 1. Define it: lowest isolation level. No read locks, no waiting on writers. 2. The anomaly it permits — dirty read. Reader sees value A, writer rolls back, value A never existed. 3. Real use case — `SELECT COUNT(*) FROM orders WITH (NOLOCK)` in SQL Server for a dashboard tile where being off by 200 rows is fine. 4. Postgres doesn't really support it — request READ UNCOMMITTED and you get READ COMMITTED, because Postgres's MVCC never reads uncommitted versions anyway.

Edge cases to mention: - SQL Server `NOLOCK` hint is the same thing applied per-query; widely (over)used in legacy reporting code. - Beyond dirty reads, READ UNCOMMITTED can return the same row twice or skip rows entirely if a page split happens mid-scan in SQL Server. - MySQL InnoDB supports it but most teams stay on REPEATABLE READ (the default).

What NOT to say: - "It's faster so use it everywhere." Performance gain is small and the correctness loss is large. - "Postgres supports all four levels equally" — it does syntactically, but READ UNCOMMITTED silently behaves as READ COMMITTED.

Common follow-up: *Why is dirty read dangerous in a financial report?*

```
-- SQL Server pattern often seen in legacy dashboards
SELECT COUNT(*) FROM orders WITH (NOLOCK) WHERE status = 'PLACED';
-- Same effect:
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SELECT COUNT(*) FROM orders WHERE status = 'PLACED';
```

Q33 · Read Committed Isolation Level

Tags: Difficulty: Medium · Asked at: PhonePe, Flipkart, JP Morgan India · Topic: Isolation

The question (interviewer's verbatim phrasing):

> READ COMMITTED is the default in most databases. What does it prevent, and what does it still let through?

The 30-second answer: READ COMMITTED prevents dirty reads — you only see committed data. It still allows non-repeatable reads (same row read twice in one txn returns different values) and phantom reads (same range query returns different row counts).

Step-by-step thought process: 1. Default in Postgres, Oracle, SQL Server (MySQL is the odd one — defaults to REPEATABLE READ). 2. Mechanism — each statement sees a fresh snapshot in MVCC engines; in lock-based engines, short read locks released immediately. 3. What it does prevent: dirty read. 4. What still bites you: non-repeatable read. Run `SELECT balance FROM accounts WHERE id = 101` twice in one txn — value can change between reads because another txn committed in between. 5. Phantom reads also possible: `SELECT COUNT(*) FROM orders WHERE status = 'PLACED'` twice can return different counts.

Edge cases to mention: - In Postgres, READ COMMITTED takes a new snapshot per statement, not per transaction — this is what causes the non-repeatable behavior. - If you need stability within a txn, escalate to REPEATABLE READ or SNAPSHOT. - `SELECT ... FOR UPDATE` even in READ COMMITTED locks the row, so you can read-then-update without losing the update.

What NOT to say: - "READ COMMITTED prevents all anomalies." It only prevents one. - "It's slow because of locking." In MVCC engines (Postgres, Oracle), READ COMMITTED uses snapshots, not read locks.

Common follow-up: *Give an example where non-repeatable read causes a bug.*

```
-- Txn A reads twice, Txn B commits in between
-- Txn A:
BEGIN;
SELECT balance FROM accounts WHERE id = 101;  -- 5000
-- (Txn B updates to 8000 and commits)
SELECT balance FROM accounts WHERE id = 101;  -- 8000 — non-repeatable
COMMIT;
```

Q34 · Repeatable Read & InnoDB's Gap Lock

Tags: Difficulty: Medium · Asked at: Swiggy, Meesho, Walmart Labs · Topic: Isolation

The question (interviewer's verbatim phrasing):

> What does REPEATABLE READ add on top of READ COMMITTED? And is there anything special about MySQL's behaviour here?

The 30-second answer: REPEATABLE READ guarantees that if you read the same row twice in one txn, you get the same value (no non-repeatable reads). MySQL InnoDB goes further than the SQL standard — it also uses "gap locks" on range queries, which means InnoDB at REPEATABLE READ also prevents phantom reads (in most cases).

Step-by-step thought process: 1. The standard definition: REPEATABLE READ prevents dirty + non-repeatable reads, but phantom reads are still legal. 2. Postgres at REPEATABLE READ uses snapshot isolation — phantoms are also prevented in practice (any change after txn start is invisible). But you can get serialization errors on write-write conflicts. 3. MySQL InnoDB does it through gap locks on indexes — when you SELECT a range, it locks not just matching rows but the "gaps" between them, preventing inserts that would change the result set. 4. SQL Server at REPEATABLE READ holds shared locks until txn end — phantoms still possible.

Edge cases to mention: - InnoDB's gap locks are why you sometimes see deadlocks on inserts that look unrelated — two txns lock overlapping gaps. - Postgres's REPEATABLE READ is closer to SNAPSHOT isolation; can throw "could not serialize access due to concurrent update" forcing app retry. - Oracle doesn't expose REPEATABLE READ — it has READ COMMITTED and SERIALIZABLE (which is really snapshot).

What NOT to say: - "REPEATABLE READ always prevents phantoms" — only true on MySQL InnoDB and Postgres; SQL Server still allows them. - "It's just a stricter lock" — in MVCC engines it's about snapshots, not locks.

Common follow-up: *Why does MySQL default to REPEATABLE READ when most others use READ COMMITTED?*

```
-- MySQL InnoDB at REPEATABLE READ
BEGIN;
SELECT * FROM orders WHERE amount BETWEEN 100 AND 500;
-- Gap locks placed; another txn cannot INSERT a new row with amount = 200
-- until this txn commits
COMMIT;
```

Q35 · Serializable Isolation

Tags: Difficulty: Hard · Asked at: Goldman Sachs India, JP Morgan, Razorpay · Topic: Isolation

The question (interviewer's verbatim phrasing):

> What does SERIALIZABLE guarantee, and why don't we just always use it?

The 30-second answer: SERIALIZABLE guarantees the outcome is as if all txns ran one-after-another. It prevents all anomalies including phantoms and write skew. The cost — either heavy locking (SQL Server style) or frequent serialization-failure retries (Postgres SSI), which both kill throughput.

Step-by-step thought process: 1. Define the guarantee — serial-equivalent execution. The strongest level in the SQL standard. 2. Two implementation flavours: - **Lock-based** (SQL Server, DB2) — range locks, predicate locks; readers block writers and vice versa. - **MVCC + SSI (Serializable Snapshot Isolation)** — Postgres tracks read-write dependencies and aborts one txn if a cycle forms. Optimistic — no blocking, but retries needed. 3. The reason most prod code doesn't use it — throughput drops sharply under contention. 4. Where it does make sense — short, critical txns (financial settlement, ledger entries) where correctness trumps throughput.

Edge cases to mention: - Oracle's "SERIALIZABLE" is actually snapshot isolation — does not prevent write skew. Real serializable is not available in Oracle. - Postgres SERIALIZABLE will throw `could not serialize access` errors; app code must implement retry-on-40001. - Even at SERIALIZABLE, you still need to worry about replication lag if reads go to a replica.

What NOT to say: - "It's just slower" — the failure mode is different too (retries, not just latency). - "SERIALIZABLE means single-threaded" — it doesn't; concurrency is allowed, the *outcome* must be equivalent to some serial order.

Common follow-up: *What is write skew? Give an example.*

```
-- Postgres pattern with retry
DO $$
BEGIN
  FOR i IN 1..3 LOOP
    BEGIN
      SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
      -- ... txn body ...
      EXIT;
    EXCEPTION WHEN serialization_failure THEN
      -- retry
    END;
  END LOOP;
END $$;
```

Q36 · Dirty Read vs Non-Repeatable Read vs Phantom Read

Tags: Difficulty: Medium · Asked at: TCS, Cognizant, Target India · Topic: Isolation

The question (interviewer's verbatim phrasing):

> Walk me through dirty read, non-repeatable read, and phantom read with a concrete example of each.

The 30-second answer: Dirty read = you see uncommitted data. Non-repeatable read = you read the same row twice and get different values. Phantom read = you re-run the same range query and get a different number of rows. They are progressively more subtle.

Step-by-step thought process: 1. **Dirty read** — Txn A updates balance from 5000 to 8000 but hasn't committed. Txn B reads 8000, makes a decision, then Txn A rolls back. B used a value that never existed. 2. **Non-repeatable read** — Txn A reads order_amount = 500. Txn B updates it to 700 and commits. Txn A reads it again — 700. Same row, different value within one txn. 3. **Phantom read** — Txn A: `SELECT COUNT(*) FROM orders WHERE status = 'PLACED'` returns 50. Txn B inserts 3 new placed orders, commits. Txn A re-runs the same query — 53. 4. Map them to levels: READ UNCOMMITTED allows all three. READ COMMITTED kills dirty. REPEATABLE READ kills dirty + non-repeatable. SERIALIZABLE kills all three.

Edge cases to mention: - Phantom read is about row *count* or set membership, non-repeatable read is about row *value*. Subtle distinction interviewers love. - "Lost update" is a separate fourth anomaly not in the SQL standard list — two txns both read 100, both add 10, both write 110; one update is lost. - "Write skew" is a fifth anomaly that only SERIALIZABLE (real) prevents.

What NOT to say: - Mixing up phantom and non-repeatable read — common candidate slip. - "Dirty read is rare" — actually impossible at READ COMMITTED and above; not rare, just disallowed.

Common follow-up: *Which of these can happen at the default isolation level in your favourite DB?*

```
-- Phantom read scenario
-- Txn A:
BEGIN; SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT COUNT(*) FROM orders WHERE created_at::date = CURRENT_DATE;  -- 120
-- (Txn B inserts 5 new orders today, commits)
SELECT COUNT(*) FROM orders WHERE created_at::date = CURRENT_DATE;  -- 125 — phantom
COMMIT;
```

Q37 · Deadlocks — Detection & Retry

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, Infosys · Topic: Transactions

The question (interviewer's verbatim phrasing):

> What is a deadlock, how does the database handle it, and what should your application do about it?

The 30-second answer: A deadlock is when two txns each hold a lock the other wants — neither can proceed. The DB detects the cycle and kills one (the "victim"), returning an error. Your app must catch this error and retry the txn.

Step-by-step thought process: 1. Classic example: Txn A locks row 1 and asks for row 2; Txn B locks row 2 and asks for row 1. Cycle. 2. Detection — most DBs run a "wait-for graph" check every few hundred ms; on cycle, pick the victim (usually the one that has done less work) and roll it back. 3. The error code — Postgres: `40P01`, MySQL: `1213`, SQL Server: `1205`. App layer catches this specifically and retries with backoff. 4. Prevention beats handling — always acquire locks in a consistent order across txns (e.g., always lock the smaller id first). Keep txns short.

Edge cases to mention: - A pure "lock timeout" is *not* a deadlock — it's a long wait that hit `innodb_lock_wait_timeout` or `lock_timeout`. Different error code. - Distributed deadlocks across shards or microservices aren't detected by any single DB — you must build timeout + retry yourself. - InnoDB's gap locks create deadlocks that surprise developers — two inserts into the same range can deadlock.

What NOT to say: - "Deadlocks mean my schema is broken" — not necessarily; high-concurrency workloads can deadlock even with good schema. - "Just retry forever" — bounded retries with exponential backoff. Otherwise, you mask a real contention bug.

Common follow-up: *How would you reduce deadlock frequency without changing application logic?*

```
-- Always lock in the same order: smaller id first
BEGIN;
SELECT * FROM accounts WHERE id IN (101, 202) ORDER BY id FOR UPDATE;
-- now safe to UPDATE both in any order
UPDATE accounts SET balance = balance - 500 WHERE id = 101;
UPDATE accounts SET balance = balance + 500 WHERE id = 202;
COMMIT;
```

Q38 · Lock Escalation

Tags: Difficulty: Hard · Asked at: AmEx India, Goldman Sachs, Wipro · Topic: Transactions

The question (interviewer's verbatim phrasing):

> Tell me about lock escalation in SQL Server or Oracle — when does a row lock become a page or table lock?

The 30-second answer: Lock escalation is when the database decides that maintaining many fine-grained locks (row-level) is more expensive than holding one coarse lock (page or table). It "escalates" — releases the small locks, takes a big one. SQL Server kicks in around 5000 locks per statement; Oracle largely avoids row-lock escalation.

Step-by-step thought process: 1. Why locks aren't free — each lock takes memory (~96 bytes in SQL Server). Holding millions is expensive. 2. SQL Server's rule of thumb — at ~5000 locks on a single object in one statement, escalate row locks to a table lock (no intermediate page lock by default for HoBT escalation). 3. Effect on concurrency — once you have a table-level X lock, no other writer can touch any row of that table. 4. How to avoid — keep txns short, batch DELETE/UPDATE into smaller chunks (e.g., 1000 rows at a time), use partition-level locking via `ALTER TABLE ... SET (LOCK_ESCALATION = AUTO)`.

Edge cases to mention: - Oracle does not escalate row locks to table locks — it uses row-level locking with a per-block transaction list, so this question is largely SQL Server / DB2-specific. - MySQL InnoDB doesn't escalate either; uses row-level locking on indexed columns. - Postgres uses row-level + table-level locks but does not "escalate" in the SQL Server sense.

What NOT to say: - "All DBs escalate locks" — Oracle and Postgres really don't, in this sense. - "Escalation is always bad" — sometimes a table lock for a big batch is intentional and faster.

Common follow-up: *How would you do a bulk DELETE of 10 million rows without causing lock escalation?*

```
-- SQL Server: chunked delete to avoid escalation
WHILE 1 = 1
BEGIN
  DELETE TOP (1000) FROM orders WHERE status = 'EXPIRED' AND created_at < DATEADD(year, -3, GETDAT
  IF @@ROWCOUNT = 0 BREAK;
  WAITFOR DELAY '00:00:00.100';
END
```

Q39 · MVCC — How Snapshot Isolation Works

Tags: Difficulty: Hard · Asked at: Zerodha, PhonePe, Flipkart, JP Morgan · Topic: Transactions

The question (interviewer's verbatim phrasing):

> Explain MVCC. How does Postgres or Oracle give you snapshot isolation without making readers block writers?

The 30-second answer: MVCC = each row update creates a new version with a transaction-id stamp. Readers see the version visible at their snapshot's start; writers create new versions. Readers never block writers, writers never block readers — they look at different physical versions of the same logical row.

Step-by-step thought process: 1. The core idea — a row is not a single value; it's a chain of versions, each tagged with `xmin` (creator txn) and `xmax` (deleter txn). At read time, the engine picks the version visible to your snapshot. 2. Postgres mechanics — versions stored inline in the same table page (HOT chains when possible). Old versions cleaned up by VACUUM. 3. Oracle mechanics — old versions stored in UNDO segments, not in the table itself. Readers reconstruct old image from UNDO. This is why "snapshot too old" errors happen when UNDO is too small. 4. MySQL InnoDB — old versions in undo log; similar to Oracle but different on-disk layout. 5. Trade-off — bloat. Postgres in particular suffers if VACUUM falls behind on a heavy-write table.

Edge cases to mention: - MVCC does not eliminate locks — writers still take row locks to prevent two updates colliding. It eliminates *read* locks. - Long-running queries on a busy table can hold back VACUUM, causing table bloat. Common Postgres prod issue. - The "snapshot too old" Oracle error and Postgres's `xmin horizon` are the same family of problem — old version evicted before the reader was done.

What NOT to say: - "MVCC means no locks anywhere" — wrong; write locks still exist. - "MVCC and snapshot isolation are the same thing" — close but not identical. MVCC is the implementation; snapshot isolation is the guarantee.

Common follow-up: *What is `VACUUM` doing in Postgres, and what happens if it's too slow?*

```
-- See row visibility in Postgres
SELECT xmin, xmax, * FROM orders WHERE id = 101;
-- xmin = txn id that created this version
-- xmax = txn id that deleted/updated it (0 if still current)
```

Q40 · Optimistic vs Pessimistic Locking

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Meesho, TCS · Topic: Transactions

The question (interviewer's verbatim phrasing):

> When do you use `SELECT FOR UPDATE` versus a version-column check? What's the trade-off?

The 30-second answer: Pessimistic (`SELECT FOR UPDATE`) — lock the row up front, others wait. Use when conflict is likely and the txn is short. Optimistic (version column) — read freely, check on write that nobody changed it; if they did, retry. Use when conflict is rare and you don't want readers blocking each other.

Step-by-step thought process: 1. Pessimistic — `SELECT * FROM accounts WHERE id = 101 FOR UPDATE` puts a row lock; other writers block until commit. Safe and simple, but blocks. 2. Optimistic — add a `version` (or `updated_at`) column. On read, capture it. On UPDATE, set `version = version + 1` with a `WHERE version = :captured` . If 0 rows updated, somebody beat you to it — retry or fail. 3. Pessimistic is right for inventory decrements (likely contention), payment debits. 4. Optimistic is right for user-profile edits (rare contention), where the cost of a blocked reader is worse than the cost of occasional retry. 5. There's also "advisory locks" (Postgres `pg_advisory_lock`) for cross-row coordination that doesn't fit either pattern.

Edge cases to mention: - `SELECT FOR UPDATE NOWAIT` — fail instantly instead of waiting. Useful in API hot paths. - `SELECT FOR UPDATE SKIP LOCKED` — skip locked rows entirely; great for job-queue tables. - Optimistic needs careful handling of partial updates — if you UPDATE 5 columns conditionally, the version bump must cover all of them.

What NOT to say: - "Optimistic is always faster" — under high contention, retries thrash and it ends up slower than pessimistic. - "Pessimistic locks everything" — only the rows you SELECT FOR UPDATE; other readers without the clause still see them (in MVCC engines).

Common follow-up: *How would you implement a job-queue table so two workers don't pick the same job?*

```
-- Optimistic pattern with version column
-- Read:
SELECT id, balance, version FROM accounts WHERE id = 101;
-- captured version = 7

-- Write:
UPDATE accounts
   SET balance = balance - 500, version = version + 1
 WHERE id = 101 AND version = 7;
-- If 0 rows affected, retry
```

Section E — Modern SQL Hot Topics

Q41 · Recursive CTE

Tags: Difficulty: Medium · Asked at: TCS, Walmart Labs, JP Morgan India, Flipkart · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> Write a query that returns all employees under a given manager — direct reports, their reports, all the way down.

The 30-second answer: Use a recursive CTE — anchor member picks the seed (the manager), recursive member joins back to the CTE itself, walking the tree. Add a depth column if you need to know how far down each employee is.

Step-by-step thought process: 1. Recursive CTE has two parts joined by `UNION ALL` — the anchor (base case) and the recursive (steps). 2. Anchor: pick the starting manager(s). 3. Recursive: join the table to the CTE on `manager_id = parent.id`, returning the next layer. 4. Termination is automatic when the recursive step returns zero rows. 5. Use `depth + 1` to cap runaway recursion; add `LIMIT` defensively when testing.

Edge cases to mention: - Cycles (employee A reports to B reports to A) cause infinite loops. Track visited ids in an array (`PATH` column) and exclude in the recursive step. Postgres has `CYCLE` clause; SQL Server / MySQL don't. - `UNION ALL` (cheap, preserves dupes) vs `UNION` (forces distinct, much slower). Almost always `UNION ALL`. - MySQL added recursive CTEs in 8.0; before that you needed stored procs or self-joins.

What NOT to say: - "I'd write a stored procedure with a loop" — recursive CTE is the standard answer interviewers want. - "Recursive CTEs are O(1)" — they're O(rows in result), and bad anchors can explode them.

Common follow-up: *How would you find the shortest path between two users in a friend-graph table?*

```
WITH RECURSIVE reports AS (
  -- Anchor: the seed manager
  SELECT id, name, manager_id, 0 AS depth
    FROM employees WHERE id = 42
  UNION ALL
  -- Recursive: next layer down
  SELECT e.id, e.name, e.manager_id, r.depth + 1
    FROM employees e
    JOIN reports r ON e.manager_id = r.id
   WHERE r.depth < 10  -- safety cap
)
SELECT * FROM reports ORDER BY depth, name;
```

Q42 · UPSERT — MERGE / ON CONFLICT / ON DUPLICATE KEY

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, PhonePe, Infosys · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> I have a `users` table with `email` as unique. New signup comes in — if the email exists, update last_seen; if not, insert. How do you write this in one statement?

The 30-second answer: Postgres: `INSERT ... ON CONFLICT (email) DO UPDATE`. MySQL: `INSERT ... ON DUPLICATE KEY UPDATE`. SQL Server: `MERGE` (or workaround). Each has gotchas — MySQL touches the auto-increment counter on conflict, SQL Server MERGE has historical concurrency bugs.

Step-by-step thought process: 1. Postgres ON CONFLICT — specify the conflict target (unique index column), then DO UPDATE or DO NOTHING. Access the rejected row via `EXCLUDED.col`. 2. MySQL ON DUPLICATE KEY — checks all unique constraints, not a specific one. Use `VALUES(col)` or `NEW.col` in MySQL 8.0.20+. 3. SQL Server MERGE — verbose, but flexible. Has known race conditions if the unique index isn't there or if WHEN MATCHED conditions skip rows. 4. Vendor-portable workaround — INSERT, catch unique-violation, UPDATE. Two round-trips but no surprises.

Edge cases to mention: - MySQL `ON DUPLICATE KEY UPDATE` increments AUTO_INCREMENT *even when no insert happens* — your id sequence will have gaps. - Postgres `ON CONFLICT` requires a unique constraint or unique index — error if the conflict target doesn't have one. - SQL Server MERGE has been criticized — Aaron Bertrand's "Use Caution with MERGE" piece is the classic reference. Many shops use INSERT + UPDATE instead. - Returning the affected row — Postgres supports `RETURNING`; vanilla MySQL still does not (as of 8.4). MariaDB added `RETURNING` for INSERT/DELETE in 10.5, but Oracle MySQL has no equivalent — fall back to `LAST_INSERT_ID()` or a follow-up SELECT.

What NOT to say: - "SELECT first, then INSERT or UPDATE" — race condition; two concurrent calls both SELECT no row and both INSERT. - "MERGE is the standard ANSI way so use it everywhere" — performance and concurrency vary wildly by vendor.

Common follow-up: *What happens if two clients UPSERT the same email at the same instant?*

```
-- Postgres
INSERT INTO users (email, last_seen) VALUES ('a@b.com', NOW())
ON CONFLICT (email) DO UPDATE SET last_seen = EXCLUDED.last_seen;

-- MySQL
INSERT INTO users (email, last_seen) VALUES ('a@b.com', NOW())
ON DUPLICATE KEY UPDATE last_seen = VALUES(last_seen);
```

Q43 · GROUPING SETS / ROLLUP / CUBE

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target India, AmEx · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> A report needs total orders by city, by state, and overall — all in one result set. Without UNION ALL spaghetti. How?

The 30-second answer: Use `GROUPING SETS` to specify exactly which sub-totals you want. `ROLLUP` gives you a hierarchical chain; `CUBE` gives you every possible combination. All three reduce to one query plan and one scan of the source.

Step-by-step thought process: 1. `GROUP BY GROUPING SETS ((city), (state), ())` — three sub-totals: per city, per state, grand total. 2. `GROUP BY ROLLUP (state, city)` — hierarchical: (state, city), (state), (). Use when there's a natural parent-child. 3. `GROUP BY CUBE (a, b)` — every combination: (a, b), (a), (b), (). Use rarely; explodes with more columns. 4. `GROUPING(col)` function tells you whether a NULL in the output is a real NULL or a "this column was not grouped at this level" placeholder.

Edge cases to mention: - NULL ambiguity — a `state = NULL` row in the result can mean "state was rolled up" or "the underlying data had NULL state". Use `GROUPING()` to distinguish. - MySQL supports only `WITH ROLLUP` — GROUPING SETS and CUBE are still not implemented (true through 8.4). - BigQuery and Snowflake support all three; very useful in analytics dashboards.

What NOT to say: - "I'd write three queries with UNION ALL." Works, but the interviewer is specifically testing if you know these features. - "ROLLUP and CUBE are the same." They're not — ROLLUP is hierarchical, CUBE is exhaustive.

Common follow-up: *How do you distinguish a real NULL state from a rolled-up NULL state in the output?*

```
SELECT state, city, COUNT(*) AS orders_count,
       GROUPING(state) AS is_state_rollup,
       GROUPING(city)  AS is_city_rollup
  FROM orders
 GROUP BY GROUPING SETS ((state, city), (state), ());
```

Q44 · JSON in SQL — JSONB Operators & When to Use

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Meesho, Zerodha · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> Postgres has a `jsonb` column. Tell me about the operators and when you'd use a JSON column versus proper relational columns.

The 30-second answer: Postgres: `->` returns json, `->>` returns text, `@>` checks containment, `?` checks key existence. MySQL: `JSON_EXTRACT(col, '$.path')` or `col->'$.path'`. Use JSON columns for semi-structured payloads with sparse / evolving fields — never as a substitute for normal modelling of frequently-queried fields.

Step-by-step thought process: 1. Postgres operators: - `->` get json field, returns jsonb: `data->'address'` - `->>` get text field: `data->>'name'` - `@>` contains: `data @> '{"status":"active"}'` - `?` key exists: `data ? 'phone'` 2. Index them with GIN: `CREATE INDEX ON users USING gin(data jsonb_path_ops);` 3. When to use JSON column — variable user metadata, third-party webhook payloads, A/B test config blobs, audit log "context" fields. 4. When NOT to — anything you JOIN, ORDER BY, or filter on regularly. Make it a real column. 5. MySQL — similar set with `JSON_EXTRACT`, `JSON_CONTAINS`, but indexing requires generated columns.

Edge cases to mention: - Postgres `jsonb` is binary-stored and indexable; `json` is text-stored and only good for storage. Almost always pick `jsonb`. - Updates rewrite the whole document — heavy update workloads on JSON columns cause bloat. - BigQuery `JSON` type and Snowflake `VARIANT` are similar but with schema-on-read; even better for analytics.

What NOT to say: - "JSON columns make the schema flexible so always use them." That's how you end up with a slow, un-typed mess. - "JSON in SQL is for storage only." It's queryable with predicate pushdown if you index correctly.

Common follow-up: *How would you index `data->>'status' = 'active'` for fast lookup?*

```
-- Postgres
SELECT id FROM users WHERE data @> '{"city":"Bengaluru"}';
SELECT id FROM users WHERE data->>'plan' = 'pro';

-- Index for @> queries
CREATE INDEX idx_users_data ON users USING gin (data jsonb_path_ops);

-- Index for ->> equality
CREATE INDEX idx_users_plan ON users ((data->>'plan'));
```

Q45 · CTE vs Subquery vs Temp Table

Tags: Difficulty: Medium · Asked at: Flipkart, JP Morgan, Wipro, Cognizant · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> When would you reach for a CTE versus a subquery versus a temp table?

The 30-second answer: Subquery — quick, inline, one-time use. CTE — same query but more readable and reusable within the statement. Temp table — when you need to materialize once and hit it many times across statements, or when the optimizer is making a bad plan and you want to force materialization.

Step-by-step thought process: 1. **Subquery** — fine for simple inline use; can sometimes be flattened by the optimizer. 2. **CTE** — readability win. In Postgres <12, CTEs were always materialized (acting as an optimization fence). From Postgres 12 onwards, CTEs are inlined by default — use `WITH ... AS MATERIALIZED` to force the old behaviour. 3. **Temp table** — `CREATE TEMP TABLE tmp AS SELECT ...`. Persists for the session, can be indexed, statistics gathered. Best when reused across multiple queries. 4. Reusability rule of thumb — if you reference the result once, subquery or CTE; many times, temp table.

Edge cases to mention: - Postgres 12 inlining means your old-style "use a CTE to force materialization" trick no longer works. Add `AS MATERIALIZED` to keep it. - SQL Server temp tables (`#tmp`) get statistics; table variables (`@tmp`) don't and often plan badly. - BigQuery / Snowflake — temp tables are often slower than CTEs because of storage roundtrip; prefer CTEs there.

What NOT to say: - "CTEs are always faster than subqueries" — depends on version and engine. They're a readability tool first. - "Temp tables are slow" — often the fastest option when you're hitting the result repeatedly.

Common follow-up: *If a CTE is being inlined and the plan is bad, how do you force materialization?*

```
-- Postgres 12+: force materialization
WITH active_users AS MATERIALIZED (
  SELECT id, plan FROM users WHERE is_active = TRUE
)
SELECT a.plan, COUNT(*)
  FROM active_users a JOIN orders o ON o.user_id = a.id
 GROUP BY a.plan;
```

Q46 · Materialized View

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target India, Goldman, Razorpay · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> What is a materialized view and how is it different from a regular view? What refresh strategies have you used?

The 30-second answer: A regular view is just a saved query — re-runs every time. A materialized view stores the result on disk and serves it directly, much like a cached table. You refresh it on a schedule or trigger. Trade-off: fast reads, stale data, refresh cost.

Step-by-step thought process: 1. Use case — expensive aggregations (daily revenue per merchant per city) that downstream dashboards hit every 30 seconds. 2. Refresh strategies: - **Full refresh** — `REFRESH MATERIALIZED VIEW`. Simple, locks readers in plain form. - **Concurrent refresh** (Postgres) — `REFRESH MATERIALIZED VIEW CONCURRENTLY` — needs a unique index on the view; readers continue but refresh is slower. - **Incremental / delta refresh** — Oracle "fast refresh", Snowflake dynamic tables, BigQuery materialized views. Only re-computes changed rows. Best but has restrictions on the SQL. 3. Trigger-based refresh on small lookup-style MVs is also common. 4. The eternal tension — freshness vs cost. Daily? Hourly? Every 5 min? Match to the business question's tolerance.

Edge cases to mention: - Postgres has no auto-refresh; you schedule it (cron, pg_cron, app-side). - Snowflake auto-refreshes but charges credits for the refresh — review billing. - Materialized views can become invalid if the underlying tables change schema; check application deployment workflow.

What NOT to say: - "Materialized views are always faster" — only on read. Refresh cost can be brutal. - "Just use a cache like Redis" — sometimes right, but materialized views keep the data near the SQL engine which simplifies joins.

Common follow-up: *How do you avoid blocking readers during refresh?*

```
-- Postgres
CREATE MATERIALIZED VIEW mv_daily_revenue AS
  SELECT date_trunc('day', created_at) AS day,
         merchant_id, SUM(amount) AS revenue
    FROM orders WHERE status = 'PAID'
   GROUP BY 1, 2;

-- Required for CONCURRENTLY
CREATE UNIQUE INDEX ON mv_daily_revenue (day, merchant_id);

REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_revenue;
```

Q47 · Table Partitioning — Range, List, Hash

Tags: Difficulty: Hard · Asked at: Flipkart, PhonePe, Walmart Labs, JP Morgan · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> When does partitioning help and when does it hurt? Range, list, or hash — how do you pick?

The 30-second answer: Partitioning helps when queries naturally restrict to one partition (partition pruning) or when you need to drop old data fast (drop a partition vs a billion-row DELETE). It hurts when queries span partitions — partition overhead with no payoff. Range for time-series, list for fixed categories, hash to spread load evenly.

Step-by-step thought process: 1. **Range** — by date or numeric range. `PARTITION BY RANGE (created_at)`. Classic time-series; old partitions get archived/dropped. 2. **List** — by an enumerated set. `PARTITION BY LIST (region) (...)`. Useful when data has a natural categorical split (region, tenant). 3. **Hash** — `PARTITION BY HASH (user_id) PARTITIONS 16`. For load distribution when no natural range/list exists. 4. Wins: partition pruning skips entire partitions; DROP PARTITION is instant; per-partition statistics. 5. Losses: cross-partition queries pay overhead; foreign keys to partitioned tables have restrictions in older Postgres; partition key must be in WHERE for pruning to kick in.

Edge cases to mention: - Postgres declarative partitioning since 10; better in 11+ (default partition, hash partitions, FK support). - MySQL partitioning has limits — partition key must be part of every unique key. - Don't over-partition — 10000 partitions is too many; plan metadata overhead becomes significant. - Sub-partitioning (range + hash) is sometimes used in very large warehouses but adds complexity.

What NOT to say: - "Partitioning makes everything faster." Only specific access patterns benefit. - "Always range-partition by date." Only if your queries actually filter by date.

Common follow-up: *Why might a partitioned table be slower than a non-partitioned one for some queries?*

```
-- Postgres range partitioning by month
CREATE TABLE orders (
  id BIGSERIAL, created_at TIMESTAMP NOT NULL, amount NUMERIC
) PARTITION BY RANGE (created_at);

CREATE TABLE orders_2026_05 PARTITION OF orders
  FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');
-- Pruning works: WHERE created_at >= '2026-05-15' scans only May partition
```

Q48 · COALESCE / NULLIF / IS DISTINCT FROM

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> Two columns. How do you check if they're different, treating NULLs as just another value to compare?

The 30-second answer: `a = b` returns NULL when either is NULL — never TRUE, never FALSE. Use `IS DISTINCT FROM` (Postgres / SQL Server 2022 / Snowflake) for NULL-safe inequality. Use COALESCE to substitute a default; use NULLIF to convert a sentinel value to NULL.

Step-by-step thought process: 1. **The NULL trap** — `WHERE old_value <> new_value` will silently exclude rows where either side is NULL. Probably not what you want for change detection. 2. **`IS DISTINCT FROM`** — NULL-safe inequality: NULL vs 5 is "distinct", NULL vs NULL is "not distinct". Postgres has it; MySQL uses `<=>` (NULL-safe equals); SQL Server 2022 finally added it. 3. **`COALESCE(a, b, c)`** — returns the first non-NULL. Useful for defaults and fallbacks. 4. **`NULLIF(a, b)`** — returns NULL if a = b, else a. Common trick: `NULLIF(divisor, 0)` to avoid divide-by-zero.

Edge cases to mention: - MySQL's `<=>` is its NULL-safe equals; `a <=> b` returns 1 if both NULL or both equal, 0 otherwise. - `IS NOT DISTINCT FROM` is the equality version. - `COALESCE` is ANSI standard; `ISNULL()` is SQL Server-specific and behaves slightly differently with type precedence. - `COALESCE` short-circuits — later expressions not evaluated if an earlier one is non-NULL.

What NOT to say: - "NULL = NULL is true." It's NULL, which is treated as false in WHERE clauses. - "Use `IFNULL` everywhere." MySQL/SQL Server have it but `COALESCE` is portable.

Common follow-up: *Write a query that finds rows where the user changed their email — including NULL-to-value and value-to-NULL.*

```
-- NULL-safe change detection
SELECT id FROM user_changes
 WHERE old_email IS DISTINCT FROM new_email;  -- Postgres / SQL Server 2022

-- MySQL equivalent
SELECT id FROM user_changes WHERE NOT (old_email <=> new_email);

-- Divide-by-zero guard
SELECT total / NULLIF(quantity, 0) AS unit_price FROM orders;
```

Q49 · CTE in DML — WITH ... UPDATE / DELETE / INSERT

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> In Postgres, you can use a CTE with INSERT, UPDATE, DELETE. Show me a useful pattern.

The 30-second answer: Postgres lets you stack data-modifying CTEs in one statement using `WITH ... INSERT`, `WITH ... UPDATE`, `WITH ... DELETE`. Combined with `RETURNING`, this gives you "do A and feed its output into B" — atomic, single round-trip.

Step-by-step thought process: 1. The canonical example — archive then delete: select rows to archive, insert into archive, delete from main, all in one statement. 2. `RETURNING` from the first CTE feeds the second. 3. Important quirk — sibling CTEs see a *snapshot* of the source tables; the order of effect is "as if all CTEs ran on the original state". A `DELETE` in CTE 1 isn't visible to a `SELECT` in CTE 2 against the same table. 4. Other engines vary — SQL Server allows `WITH` with `INSERT/UPDATE/DELETE` to define source data, but doesn't allow chaining multiple data-modifying CTEs. MySQL added basic support; Oracle's syntax is different.

Edge cases to mention: - Snapshot semantics — be careful when one CTE deletes and another counts rows; the count will be the pre-delete count. - Auditing / logging table inserts driven off updates — clean pattern using RETURNING. - Performance — one network round-trip, one transaction, one plan. Generally faster than two statements.

What NOT to say: - "WITH is read-only." Not in Postgres. - "All DBs support this pattern." Mostly Postgres-specific.

Common follow-up: *If a CTE deletes rows from `orders`, will a sibling CTE that SELECTs from `orders` see those deletions?*

```
-- Archive-and-delete pattern (Postgres)
WITH moved AS (
  DELETE FROM orders
   WHERE status = 'CANCELLED' AND created_at < CURRENT_DATE - INTERVAL '90 days'
  RETURNING *
)
INSERT INTO orders_archive
SELECT * FROM moved;

-- Update-and-audit pattern
WITH updated AS (
  UPDATE accounts SET balance = balance - 500 WHERE id = 101
  RETURNING id, balance
)
INSERT INTO audit_log (account_id, new_balance, ts)
SELECT id, balance, NOW() FROM updated;
```

Q50 · Window Function vs Aggregate in Same Query

Tags: Difficulty: Medium · Asked at: Flipkart, Swiggy, JP Morgan India, Zerodha · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> Show me a query that returns each order's amount alongside the running total for that customer and the customer's grand total — all in one go.

The 30-second answer: Use a window function for the running total (it preserves the row) and either an aggregate window or a GROUP BY-based join for the grand total. The key idea: an aggregate function collapses rows, a window function keeps every row and computes alongside.

Step-by-step thought process: 1. The conceptual difference — `SUM(amount)` in `GROUP BY` returns one row per group. `SUM(amount) OVER (PARTITION BY customer_id)` returns the same value attached to every row of that group. 2. For a running total — `SUM(amount) OVER (PARTITION BY customer_id ORDER BY created_at)` — the `ORDER BY` inside `OVER` turns it from total to running. 3. For grand total per customer — `SUM(amount) OVER (PARTITION BY customer_id)` — no ORDER BY inside. 4. Mix them — three columns: amount (the row), running_total (with ORDER BY in OVER), customer_total (no ORDER BY).

Edge cases to mention: - `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` is the default frame when ORDER BY is present — that's why "running total" works. - Without ORDER BY in OVER, the frame is the whole partition, giving the partition total on every row. - Mixing `GROUP BY` and `OVER` is legal but tricky — the window function operates *after* GROUP BY. - Performance — one scan with window functions vs join-back to aggregate. Usually faster.

What NOT to say: - "Window functions are just a fancy GROUP BY." They preserve rows; GROUP BY collapses them. Different shape. - "Running totals need self-joins." Old pattern; window functions are the modern way.

Common follow-up: *What does the default window frame look like, and how would you change it to compute a 7-day moving average?*

```
SELECT
  id,
  customer_id,
  amount,
  SUM(amount) OVER (PARTITION BY customer_id ORDER BY created_at)
    AS running_total,
  SUM(amount) OVER (PARTITION BY customer_id)
    AS customer_grand_total,
  AVG(amount) OVER (PARTITION BY customer_id
                    ORDER BY created_at
                    RANGE BETWEEN INTERVAL '6 days' PRECEDING
                              AND CURRENT ROW)
    AS rolling_7d_avg
FROM orders
ORDER BY customer_id, created_at;
```

Tier A — Asked in 50-80% of SQL interviews (80 Qs)

These questions live in the bulk of mid-level interview rounds — past the headline Top 50 and into the meat of what gets asked in a typical 45-60 minute SQL round. Work through Tier A over a weekend; you don't need to memorize, but you should be able to sketch the answer within 20 seconds for any of them.

Q51 · LATERAL Join — Per-Row Subquery

Asked at: Razorpay, Goldman Sachs, JP Morgan India, Walmart Labs, Swiggy **Difficulty**: Medium · **Topic**: Advanced Joins

The question: "*For every merchant, pull their three most recent successful payments — write it without GROUP BY hacks.*"

The 30-second answer: Use a `LATERAL` join (Postgres) or `CROSS APPLY` (SQL Server) so the subquery on the right side can reference columns from the left row — that lets you run a per-row `ORDER BY ... LIMIT 3` cleanly.

Step-by-step thought process: 1. "*Without LATERAL I'd be stuck — a normal subquery in the FROM clause can't see the outer merchant_id.*" 2. "*With LATERAL, the right-side subquery is evaluated once per left row, so `LIMIT 3` is per merchant, not global.*" 3. "*Edge case — if a merchant has zero payments, INNER LATERAL drops them; use LEFT JOIN LATERAL ... ON true to keep them.*" 4. "*Follow-up will be 'why not use ROW_NUMBER() with PARTITION BY?' — LATERAL is often faster when an index on (merchant_id, created_at DESC) exists, because it can index-scan-and-stop after 3 rows.*"

Edge cases to mention: - Without `LEFT JOIN LATERAL`, merchants with no payments vanish — use `ON true` to retain them - The subquery sees only columns from tables listed *before* it in the FROM clause — order matters - MySQL added `LATERAL` in 8.0.14; older MySQL needs a correlated subquery or window function rewrite

What NOT to say: - "I'd use GROUP BY merchant_id and pick the top 3" — GROUP BY collapses rows, you can't pick 3 per group with it - "LATERAL is the same as a regular subquery" — regular FROM-clause subqueries can't reference outer columns, that's the whole point

Common follow-up: "*How does the planner execute this — is it a nested loop?*"

Q52 · Anti-Join — NOT EXISTS vs LEFT JOIN ... IS NULL

Asked at: Flipkart, Cred, Wells Fargo, TCS Digital, Cognizant **Difficulty**: Medium · **Topic**: Joins

The question: "*You need users who have never made a payment. Compare NOT EXISTS and LEFT JOIN ... WHERE IS NULL — when would you pick one over the other?*"

The 30-second answer: Both are anti-joins and modern planners usually generate the same plan; pick `NOT EXISTS` for clarity and NULL-safety, pick `LEFT JOIN ... IS NULL` only when you also need columns from the joined table for some reason.

Step-by-step thought process: 1. "*Semantically both filter out left rows that have any match on the right — that's an anti-join.*" 2. "*NOT EXISTS is the canonical form because the optimizer recognises the pattern instantly and it's NULL-safe by construction.*" 3. "*LEFT JOIN ... IS NULL materialises matches and then throws them away — if the right side has millions of matches per left row, that's wasted work.*" 4. "*Follow-up will be NOT IN — flag the NULL trap immediately so they know you've thought about it.*"

Edge cases to mention: - If the right-side join column is nullable, `LEFT JOIN ... WHERE rightcol IS NULL` can wrongly include rows where rightcol is NULL but a match did exist - `NOT EXISTS` short-circuits on first match; `LEFT JOIN` always materialises all matches before filtering - On Postgres, `EXPLAIN` will show "Anti Join" for both — confirms they're equivalent for the planner

What NOT to say: - "LEFT JOIN with IS NULL is always faster" — false, it does more work in the join phase - "NOT EXISTS is slow because it's correlated" — modern planners de-correlate it into a hash anti-join

Common follow-up: "*What about NOT IN? Show me what breaks.*"

Q53 · Correlated Subquery to JOIN Rewrite

Asked at: Infosys, Accenture, Capgemini, AmEx India, Target **Difficulty**: Medium · **Topic**: Query Rewriting

The question: "*This query has a correlated subquery in the SELECT clause that runs for every row. Rewrite it as a JOIN and explain the performance difference.*"

The 30-second answer: Pull the correlated subquery into a derived table that pre-aggregates by the join key, then LEFT JOIN it back — you turn N executions of the inner query into one scan plus a hash join.

Step-by-step thought process: 1. *"A correlated subquery in SELECT runs once per outer row — for 10M rows that's 10M inner executions."* 2. *"Rewrite as `LEFT JOIN (SELECT key, COUNT(*) FROM x GROUP BY key) sub ON ...` so the inner work is done once."* 3. *"LEFT JOIN keeps outer rows where the inner returned nothing; the count comes out as NULL and you wrap it in COALESCE if you need 0."* 4. *"Modern Postgres can sometimes de-correlate automatically — but don't rely on it, write the JOIN form yourself."*

Edge cases to mention: - If the correlated subquery uses `LIMIT 1` for "most recent", rewrite using a window function or `DISTINCT ON` instead - Watch the cardinality — if the derived table has duplicates on the join key, you'll inflate the outer row count - A correlated subquery in WHERE (not SELECT) can usually be rewritten as `EXISTS` or a semi-join, which is different

What NOT to say: - "Correlated subqueries are always slow" — for tiny outer tables they're fine - "Just add an index" — indexes help, but they don't fix N executions vs 1

Common follow-up: *"What if the correlated subquery returns multiple columns?"*

Q54 · Window Function Frame Clauses — ROWS vs RANGE vs GROUPS

Asked at: Goldman Sachs, JP Morgan India, Swiggy, Meesho, PhonePe **Difficulty**: Hard · **Topic**: Window Functions

The question: *"Walk me through ROWS, RANGE and GROUPS in a window frame. When does it actually change the answer?"*

The 30-second answer: `ROWS` counts physical rows, `RANGE` includes all rows with the same value as the current row in the ORDER BY column (logical), `GROUPS` extends in whole peer groups — the difference shows up when ORDER BY has ties.

Step-by-step thought process: 1. *"With no ties in ORDER BY, all three behave identically — the question only matters when the ordering column has duplicates."* 2. *"ROWS BETWEEN 1 PRECEDING AND CURRENT ROW gives exactly 2 rows; RANGE gives the current row plus every earlier row with the same value — could be 50 rows if you ordered by day and 50 transactions happened that day."* 3. *"The default frame for ranking and value functions is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` — this is the classic LAST_VALUE trap, it returns the wrong row when ties exist."* 4. *"GROUPS was added in SQL:2011 — Postgres 11+ supports it; lets you say 'two peer groups back' which is cleaner than ROWS for some windowed stats."*

Edge cases to mention: - `RANGE` with offset (e.g. `RANGE BETWEEN INTERVAL '7 days' PRECEDING AND CURRENT ROW`) requires the ORDER BY to be a single numeric or temporal column - Default frame differs between aggregate functions (RANGE) and offset functions like LAG/LEAD (which don't use a frame at all) - SQL Server didn't support `RANGE` with numeric offsets until 2022; before that, only `UNBOUNDED` and `CURRENT ROW`

What NOT to say: - "ROWS and RANGE are the same thing" — they diverge the moment you have ties - "I always use the default frame" — the default bites you with LAST_VALUE; be explicit

Common follow-up: *"Show me a query where ROWS and RANGE return different results."*

Q55 · LAG / LEAD with Default Value and Partition Edges

Asked at: Razorpay, Cred, Walmart Labs, Cognizant, Wipro Digital **Difficulty**: Medium · **Topic**: Window Functions

The question: *"Compute the gap in days between each user's consecutive logins. What does LAG return on the user's very first login?"*

The 30-second answer: `LAG(login_date) OVER (PARTITION BY user_id ORDER BY login_date)` returns NULL for the first row of each partition; pass a third argument as the default to replace NULL with something usable.

Step-by-step thought process: 1. *"PARTITION BY user_id resets the LAG at each user boundary — first login per user has no previous row, hence NULL."* 2. *"`LAG(login_date, 1, login_date)` returns the current date as the default — gap becomes 0 for first login, which is often what you want."* 3. *"Don't forget `ORDER BY` inside the OVER clause — LAG is meaningless without it."* 4. *"Common mistake: forgetting that LAG/LEAD are offset functions, not frame functions — they don't use ROWS/RANGE at all."*

Edge cases to mention: - The offset can be any positive integer — `LAG(x, 7)` looks back 7 rows, useful for week-over-week deltas - If ORDER BY has ties, LAG picks an arbitrary tied row — add a tiebreaker like `ORDER BY login_date, login_id` - LAG and LEAD respect partition boundaries — they never look across into another partition, even if rows are physically adjacent

What NOT to say: - "LAG returns the previous row in the table" — it returns the previous row in the ordered window - "I'd use COALESCE on LAG" — works but it's clumsier than the third argument

Common follow-up: *"Now compute the average gap per user, excluding the first login."*

Q56 · FIRST_VALUE / LAST_VALUE Default Frame Trap

Asked at: JP Morgan India, Goldman Sachs, Flipkart, AmEx, Target **Difficulty**: Hard · **Topic**: Window Functions

The question: *"I wrote `LAST_VALUE(price) OVER (PARTITION BY product_id ORDER BY date)` and it's returning the current row's price, not the last one. Why?"*

The 30-second answer: The default frame is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, so the "last value" is the last value *up to and including the current row* — which is the

current row itself. Specify `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` to get the actual last value of the partition.

Step-by-step thought process: 1. *"FIRST_VALUE works as expected with the default frame because the first row of `UNBOUNDED PRECEDING ... CURRENT ROW` is always the partition's first row."* 2. *"LAST_VALUE breaks because the last row of that frame is the current row — every row sees itself as the 'last'."* 3. *"Fix: explicit frame `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` — now the frame covers the whole partition."* 4. *"Alternative: use `FIRST_VALUE` with descending ORDER BY, or use a subquery — but the explicit frame is the cleanest fix."*

Edge cases to mention: - Switching to `ROWS` from default `RANGE` also dodges the ties issue - If you only want the partition-wide last value, `MAX(date) OVER (PARTITION BY product_id)` plus a self-lookup may be clearer - Some interviewers test this by asking you to spot the bug in their query — read the frame clause before answering

What NOT to say: - "LAST_VALUE is broken in Postgres" — it's not broken, it's behaving per SQL standard - "I'd just use MAX instead" — MAX gives you the value, not the row attribute; they're different

Common follow-up: *"Now what about NTH_VALUE — does it have the same trap?"*

Q57 · Recursive CTE for Hierarchy Traversal

Asked at: Infosys, TCS Digital, Cognizant, Accenture, Wells Fargo **Difficulty**: Hard · **Topic**: CTEs

The question: *"Employees table has id, name, manager_id. Print every employee with their full chain of managers up to the CEO."*

The 30-second answer: Use a recursive CTE — anchor on the employee row, then repeatedly join to the same table on `manager_id = parent.id`, accumulating the chain via string concatenation or an array.

Step-by-step thought process: 1. *"Anchor query selects the starting rows — every employee, depth 0, chain = name."* 2. *"Recursive part joins employees back to the CTE on parent_id = employee.id; depth + 1, chain = chain || ' -> ' || manager.name."* 3. *"Termination: when manager_id is NULL (CEO), the join finds nothing, recursion stops for that branch."* 4. *"Always cap the depth — `WHERE depth < 20` — to protect against cycles in dirty data."*

Edge cases to mention: - Cycles in the data will recurse forever — use a depth limit or track visited nodes in an array and check - The anchor can be a single employee (find one person's chain) or all employees (build the full tree); same recursive logic - Postgres allows `WITH RECURSIVE`; SQL Server uses plain `WITH` (recursion auto-detected); Oracle has `CONNECT BY` as an alternative

What NOT to say: - "Recursive CTEs are slow" — for shallow hierarchies (under 20 levels) they're typically fast enough - "I'd self-join 5 times" — works for fixed depth but doesn't scale to variable hierarchies

Common follow-up: "*How would you detect cycles in the data?*"

Q58 · CTE Materialization — Postgres 12+ Change

Asked at: Razorpay, Swiggy, Meesho, PhonePe, Walmart Labs **Difficulty**: Medium · **Topic**: Query Performance

The question: "*You wrote a CTE that you expect to be inlined. Why might Postgres still materialise it, and how do you force the behaviour you want?*"

The 30-second answer: Before Postgres 12, every CTE was an optimisation fence (materialised); from Postgres 12 onwards, CTEs are inlined like subqueries unless they're recursive, side-effecting, or referenced more than once. Use `WITH foo AS MATERIALIZED (...)` or `AS NOT MATERIALIZED (...)` to be explicit.

Step-by-step thought process: 1. "*Pre-12, the planner couldn't push filters through a CTE — sometimes a feature, often a footgun.*" 2. "*From 12, default behaviour matches subqueries — predicate push-down works, plan is usually faster.*" 3. "*Forcing MATERIALIZED is useful when the CTE is expensive and you reference it multiple times — compute once, reuse.*" 4. "*NOT MATERIALIZED forces inlining even when referenced multiple times — useful when the planner is overly cautious.*"

Edge cases to mention: - A CTE with `INSERT`, `UPDATE`, `DELETE`, or `RETURNING` is always materialised — side effects can't be inlined - Other databases (SQL Server, MySQL 8) inline CTEs by default and don't expose the materialise hint - Multiple references to an inlined CTE will re-execute the CTE each time — confusing if you assume one-shot behaviour

What NOT to say: - "CTEs are always slower than subqueries" — pre-12 sometimes, post-12 they're equivalent in plan - "MATERIALIZED makes it faster" — only when you reuse it; otherwise it's a wasted spool

Common follow-up: "*Show me a plan where MATERIALIZED actually wins.*"

Q59 · Covering Indexes with INCLUDE Columns

Asked at: Goldman Sachs, JP Morgan India, AmEx, Target, Walmart Labs **Difficulty**: Medium · **Topic**: Indexing

The question: "*My query filters on `merchant_id` and selects `amount` and `status`. How do I make the index cover the query without bloating the key?*"

The 30-second answer: Create the index on the filter column and add the selected columns via `INCLUDE` — they're stored at the leaf level but not in the B-tree key, so the index stays small and ordered while still serving an index-only scan.

Step-by-step thought process: 1. "*A normal index on (merchant_id) requires a heap fetch to grab amount and status — extra IO.*" 2. "`CREATE INDEX ON payments (merchant_id) INCLUDE (amount, status)` *keeps amount and status in the leaf page; the planner reads everything from the index.*" 3. "*INCLUDE columns are not indexed for searching — only for retrieval; they don't affect ordering or key uniqueness.*" 4. "*Trade-off: larger index pages, more memory in the buffer cache, slower writes — measure before adding to a hot table.*"

Edge cases to mention: - Postgres 11+ supports INCLUDE on B-tree; SQL Server has had it since 2005 - The visibility map must be up to date for index-only scans — `VACUUM` regularly on Postgres - MySQL InnoDB has no INCLUDE syntax — secondary indexes always store the PK and you build covering indexes by adding columns to the key

What NOT to say: - "Just add all the columns to the index key" — bloats the B-tree, slows everything down - "Covering indexes always win" — wide indexes hurt writes and cache; balance is everything

Common follow-up: "*How do you confirm it's actually an index-only scan?*"

Q60 · Partial Indexes for Hot Subsets

Asked at: Razorpay, Cred, Swiggy, Flipkart, PhonePe **Difficulty**: Medium · **Topic**: Indexing

The question: "*99% of your queries on the orders table filter status = 'pending'. The table has 100M rows but only 200K pending. How do you index this efficiently?*"

The 30-second answer: Build a partial index with a `WHERE status = 'pending'` predicate — Postgres only indexes the matching rows, so the index is tiny, fits in memory, and writes on non-pending rows don't touch it.

Step-by-step thought process: 1. "*A full index on status has 100M entries — most of them useless for the hot query.*" 2. "`CREATE INDEX ON orders (created_at) WHERE status = 'pending'` *indexes only 200K rows — index is maybe 10MB instead of 5GB.*" 3. "*Planner uses the partial index whenever the query has a matching predicate —* `WHERE status = 'pending' AND ...`." 4. "*Writes that don't match the predicate (status = 'paid' inserts) skip the index entirely — faster writes too.*"

Edge cases to mention: - Only Postgres and SQLite support partial indexes natively; SQL Server has filtered indexes with the same idea - The query's WHERE clause must include a condition that implies the index predicate — `WHERE status IN ('pending', 'paid')` won't use the partial index for pending - When the status of a row changes from pending to paid, the index entry is removed — same as a normal index update

What NOT to say: - "Partial indexes are obscure" — they're table-stakes for any table with skewed status columns - "I'd just filter in the application" — misses the IO and cache benefit of a smaller B-tree

Common follow-up: "*What if the predicate has multiple values like status IN ('pending', 'retrying')?*"

Q61 · Expression Indexes for UPPER / LOWER Searches

Asked at: Infosys, Cognizant, Accenture, Wells Fargo, TCS Digital **Difficulty**: Medium · **Topic**: Indexing

The question: "*Users search by email but the column has mixed case.* `WHERE LOWER(email) = 'a@b.com'` *isn't using the index. Fix it.*"

The 30-second answer: Create an expression index on `LOWER(email)` — the index stores the lowercased values, so the predicate `WHERE LOWER(email) = ...` becomes a direct index lookup.

Step-by-step thought process: 1. "*A normal index on email stores the exact case —* `LOWER(email)` *doesn't match the stored values, so the planner can't use it.*" 2. "`CREATE INDEX ON users (LOWER(email))` *builds a B-tree on the computed expression — now the predicate matches exactly.*" 3. "*The query must use the* same *expression —* `LOWER(email)`*, not* `email ILIKE '...'` *— for the index to be picked.*" 4. "*Alternative: store a normalised lower-case column at write time and index that — works in any database that lacks expression indexes.*"

Edge cases to mention: - Postgres and SQLite support expression indexes; MySQL added them in 8.0.13 as "functional indexes" - Expression index works for any deterministic function — `EXTRACT(year FROM date)`, `JSON ->> 'field'`, `(col1 + col2)` - The expression is computed on every row insert and update — slower writes if the function is expensive

What NOT to say: - "Just use ILIKE" — ILIKE without a trigram index is still a full scan - "Postgres should be smart enough to handle LOWER" — the planner can't rewrite `LOWER(col)` to `col` automatically

Common follow-up: "*What about LIKE 'abc%' on a case-insensitive search?*"

Q62 · Composite Index Column Ordering

Asked at: Goldman Sachs, JP Morgan India, Razorpay, Walmart Labs, Meesho **Difficulty**: Medium · **Topic**: Indexing

The question: "*You have queries that filter on (user_id, status, created_at) and (user_id, created_at). How do you order the composite index columns?*"

The 30-second answer: Put the most selective equality-filter column first, then other equality columns, then range columns last — for these queries, `(user_id, status, created_at)` works for both because user_id and status are equality filters and created_at is the range.

Step-by-step thought process: 1. "*B-tree index columns are evaluated left to right — the leftmost column must appear in the WHERE for the index to be considered.*" 2. "*Equality before range:*

`(user_id, status, created_at)` *lets the index seek to a narrow range; reversing would scan more.*" 3. "*For the second query* `(user_id, created_at)`*, the same composite index works — Postgres can skip-scan on status, or you just need user_id to be leftmost.*" 4. "*If the second query is hot and skip-scan is unavailable (MySQL), add a separate* `(user_id, created_at)` *index.*"

Edge cases to mention: - "Leftmost prefix" rule — the index serves any query that filters on a prefix of the key columns - Sort order matters for ORDER BY — declare the index DESC on columns where you sort descending - Postgres can skip-scan in some cases (Postgres 17+); MySQL and older Postgres need a separate index

What NOT to say: - "Put the lowest cardinality first" — backwards; high cardinality (selective) goes first for equality - "One column order works for all queries" — depends on which prefixes match your WHERE clauses

Common follow-up: "*What does the planner do when the WHERE skips the leftmost column?*"

Q63 · Index-Only Scan — When You Actually Get One

Asked at: Flipkart, PhonePe, Cred, Walmart Labs, Target **Difficulty**: Medium · **Topic**: Indexing

The question: "*My EXPLAIN says 'Index Scan' but I expected 'Index Only Scan'. What's missing?*"

The 30-second answer: All columns in SELECT, WHERE and ORDER BY must be in the index (key or INCLUDE), and on Postgres the visibility map must indicate the page is all-visible — otherwise the planner falls back to a heap fetch.

Step-by-step thought process: 1. "*Index-only scan reads tuples directly from the index, skipping the heap — much faster on wide tables.*" 2. "*Check the visibility map: a recently written page may not be marked all-visible, forcing a heap fetch even if the index covers the columns.*" 3. "`VACUUM` *updates the visibility map — on a write-heavy table, autovacuum lag prevents index-only scans.*" 4. "*SELECT * always defeats an index-only scan unless every column is in the index — list only what you need.*"

Edge cases to mention: - SQL Server doesn't have the visibility-map concept — covering index = index-only scan, no extra step - MySQL InnoDB always reads through the clustered index; "covering index" means the secondary index includes all needed columns - Even a tiny SELECT clause that misses one column kills the optimisation

What NOT to say: - "Index-only is always faster" — for tiny result sets the difference is negligible - "Just trust the planner" — index-only scans need explicit attention to column lists and vacuum

Common follow-up: "*How do you force the planner to choose an index-only scan in Postgres?*"

Q64 · B-tree vs Hash vs GIN — Choosing the Index Type

Asked at: Razorpay, Swiggy, Meesho, JP Morgan India, Goldman Sachs **Difficulty**: Medium · **Topic**: Indexing

The question: "*When would you pick a hash index, a GIN index or a GiST index over the default B-tree?*"

The 30-second answer: B-tree for ordered scans and equality on scalars; hash only for pure equality on a single column (rarely worth it); GIN for arrays, JSONB containment, and full-text search; GiST for ranges, geometry and exclusion constraints.

Step-by-step thought process: 1. "*B-tree is the default and right answer 90% of the time — it handles equality, range, ORDER BY, and prefix LIKE.*" 2. "*Hash supports* `=` *only — no range, no sorting — but is slightly faster for equality on large keys. Marginal win.*" 3. "*GIN inverts the data — one index entry per element — perfect for* `WHERE tags @> ARRAY['x']` *or* `jsonb @> '{...}'`." 4. "*GiST supports range overlap, KNN searches, and exclusion constraints — used for geometry and time ranges.*"

Edge cases to mention: - Postgres hash indexes were unlogged before version 10 — risky on crash recovery; fixed in 10+ - GIN indexes are slow to update — use `fastupdate=on` to batch writes if you do bulk loads - BRIN is a fifth type — block-range, tiny, great for append-only time-series tables

What NOT to say: - "Use hash for everything that's an equality query" — B-tree handles equality just as fast and supports more - "GIN is for text only" — also for arrays, JSONB, hstore, anything with composite values

Common follow-up: "*Walk me through indexing a JSONB column where queries filter on different keys.*"

Q65 · Read Committed vs Repeatable Read — Concrete Differences

Asked at: Goldman Sachs, JP Morgan India, AmEx, Wells Fargo, TCS Digital **Difficulty**: Hard · **Topic**: Transactions & Isolation

The question: "*In one transaction you run the same SELECT twice and get different results. Which isolation levels prevent that, and what's the trade-off?*"

The 30-second answer: Read Committed allows non-repeatable reads — a row visible at t=1 may show different data at t=2 if another txn committed in between. Repeatable Read takes a snapshot at the first read and reuses it, eliminating the inconsistency but raising serialisation failures on writes.

Step-by-step thought process: 1. "*Read Committed is the default in Postgres and Oracle — each statement sees a fresh snapshot, so two SELECTs in one txn can disagree.*" 2. "*Repeatable Read takes one snapshot for the entire transaction — both SELECTs see the same data, even if other txns commit in between.*" 3. "*In Postgres, Repeatable Read is actually Snapshot Isolation — it also prevents the phantom read for queries within the snapshot.*" 4. "*Trade-off: in RR, writes can fail with 'could not serialize access' if your txn updates a row that another txn modified after your snapshot — application must retry.*"

Edge cases to mention: - MySQL InnoDB defaults to Repeatable Read; Postgres, Oracle, SQL Server default to Read Committed - Postgres Repeatable Read prevents non-repeatable reads AND phantom reads (because of MVCC snapshots); standard SQL only requires it to prevent the former - Serialisable is one step further — also detects write skew and other anomalies

What NOT to say: - "Repeatable Read prevents phantoms in all databases" — true in Postgres, not in standard SQL or older systems - "Just use Serialisable always" — the retry overhead can dominate; pick the weakest level that's correct

Common follow-up: "*What's a write skew and which level catches it?*"

Q66 · MVCC Mechanics — Postgres Tuple Versions vs Oracle Undo

Asked at: Razorpay, JP Morgan India, Goldman Sachs, Meesho, Cred **Difficulty**: Hard · **Topic**: Transactions & Storage

The question: "*Explain how Postgres MVCC works under the hood and why it's different from Oracle's approach.*"

The 30-second answer: Postgres creates a new tuple version on every UPDATE and marks the old one with a deletion xmax — both versions live in the heap until VACUUM cleans up. Oracle keeps the current version in the heap and writes the old version to an undo segment that's discarded after the txn commits.

Step-by-step thought process: 1. "*Postgres: UPDATE never modifies in place — it inserts a new tuple, updates indexes, and marks the old tuple as superseded by xmax.*" 2. "*Old tuples accumulate as bloat until autovacuum reclaims the space — that's why VACUUM is critical on Postgres.*" 3. "*Oracle: UPDATE modifies in place and writes the old value to an undo log; readers see the old value by reading undo if their snapshot predates the write.*" 4. "*Trade-off: Postgres has cheap reads (no undo lookup) but expensive writes (index updates on every UPDATE); Oracle has cheap writes but pays read cost via undo.*"

Edge cases to mention: - Postgres HOT update — when no indexed column changes, the new tuple goes on the same page and indexes aren't updated - Long-running queries in Postgres prevent VACUUM from cleaning rows — bloat snowballs - Oracle can throw "ORA-01555 snapshot too old" if undo is overwritten before a long read finishes — Postgres equivalent is bloat-driven IO

What NOT to say: - "Postgres doesn't have undo" — correct, but don't stop there; explain the implications - "MVCC means no locking" — wrong, MVCC eliminates read-write locks but writes still take row locks

Common follow-up: "*What's HOT update and when does it kick in?*"

Q67 · Snapshot Isolation vs Serialisable — Write Skew

Asked at: Goldman Sachs, JP Morgan India, AmEx, Walmart Labs, Target **Difficulty**: Hard · **Topic**: Transactions

The question: "*Two doctors both check 'are there at least two on-call doctors' and both decide to go home, leaving zero. Which isolation level catches this, and why doesn't Snapshot?*"

The 30-second answer: Snapshot Isolation lets both transactions see the same starting state, both validate "two doctors on call", and both commit their UPDATE because they wrote different rows — that's write skew. Serialisable Snapshot Isolation (SSI, Postgres 9.1+) detects the conflicting read-write dependency and aborts one txn.

Step-by-step thought process: 1. "*Snapshot Isolation gives each txn a consistent point-in-time view — but it only checks for write-write conflicts on the same row, not read-write conflicts across rows.*" 2. "*Write skew: two txns read overlapping data, write to disjoint rows, and the combined effect violates an invariant — no row-level conflict, so SI commits both.*" 3. "*Postgres's SERIALIZABLE level uses SSI — it tracks read-write dependency graphs and aborts a txn that would create a cycle.*" 4. "*Application alternative: use SELECT FOR UPDATE on the rows you read, forcing serial execution at the application level — works under any isolation level.*"

Edge cases to mention: - Oracle's SERIALIZABLE is actually Snapshot Isolation — does NOT prevent write skew despite the name - SQL Server SERIALIZABLE uses range locks (pessimistic) — slower under contention but no retries - SSI retries can cascade — applications must be designed to retry serialisation failures

What NOT to say: - "Snapshot Isolation is the strongest level" — it doesn't catch write skew, SSI does - "Just lock everything" — kills concurrency; SSI is designed to avoid that

Common follow-up: "*How would you implement this safely with SELECT FOR UPDATE?*"

Q68 · Optimistic Locking via Version Column

Asked at: Razorpay, Cred, Flipkart, PhonePe, Swiggy **Difficulty**: Medium · **Topic**: Concurrency Patterns

The question: "*You don't want to hold a row lock across a user's edit session. How do you prevent lost updates without locking?*"

The 30-second answer: Add a `version` integer column; on read, capture the current version; on update, include `WHERE id = ? AND version = ?` and increment the version — if the row count returned is zero, another txn modified the row first and the application retries.

Step-by-step thought process: 1. "*Pessimistic locking holds a lock from read to write — fine for short txns, brutal for human-paced edits.*" 2. "*Optimistic locking assumes no conflict and detects it at write time — version column is the simplest implementation.*" 3. "*The update SQL: `UPDATE t SET col = ?, version = version + 1 WHERE id = ? AND version = ?` — atomic, no extra round*

trip." 4. *"Zero affected rows means a conflict — application either retries with new data or surfaces a 409 to the user."*

Edge cases to mention: - Works across web requests where the user might walk away mid-edit — no DB lock held - Alternative: use a `last_updated_at` timestamp — but watch for clock skew and same-second collisions - Combine with a version-bump trigger so every update increments regardless of application code

What NOT to say: - "Optimistic locking is always faster" — under high contention, retries can be worse than a short lock - "Just use the timestamp" — millisecond resolution + clock skew = silent data loss

Common follow-up: *"What if the same user opens two tabs and edits both — how do you handle that?"*

Q69 · SELECT FOR UPDATE — Pessimistic Locking Patterns

Asked at: Goldman Sachs, JP Morgan India, AmEx, Walmart Labs, Target **Difficulty**: Medium · **Topic**: Concurrency

The question: *"You need to read a row, compute something, and write back, all atomically. Show me SELECT FOR UPDATE."*

The 30-second answer: Inside a transaction, `SELECT ... FOR UPDATE` takes a row-level exclusive lock on every matched row; concurrent txns block on those rows until your txn commits or rolls back.

Step-by-step thought process: 1. *"`BEGIN; SELECT balance FROM accounts WHERE id = ? FOR UPDATE; ... UPDATE accounts SET balance = ? WHERE id = ?; COMMIT;` — classic read-modify-write."* 2. *"Other txns trying to SELECT FOR UPDATE on the same row block until your COMMIT — no lost updates possible."* 3. *"Plain SELECT (without FOR UPDATE) sees the pre-lock value via MVCC — readers don't block, only writers do."* 4. *"Variants: `FOR NO KEY UPDATE` (lighter, doesn't block FK checks), `FOR SHARE` (lets concurrent FOR SHARE through, blocks FOR UPDATE)."*

Edge cases to mention: - Always lock rows in a consistent order across your codebase — random order causes deadlocks - Postgres releases all locks at COMMIT or ROLLBACK — no early release without subtransactions - SQL Server uses `WITH (UPDLOCK, HOLDLOCK)` for the same intent; MySQL InnoDB uses `FOR UPDATE` natively

What NOT to say: - "FOR UPDATE locks the whole table" — it's row-level in modern databases - "I don't need FOR UPDATE because I'm in a transaction" — wrong, MVCC reads don't block writes, you'd still race

Common follow-up: *"What if you have many rows to process — how do you avoid lock contention?"*

Q70 · SELECT FOR UPDATE SKIP LOCKED — Queue Processing

Asked at: Razorpay, Swiggy, Meesho, Cred, PhonePe **Difficulty**: Hard · **Topic**: Concurrency Patterns

The question: "*Ten workers poll a jobs table — how do you make sure no two workers grab the same job, without blocking?*"

The 30-second answer: `SELECT ... WHERE status = 'pending' FOR UPDATE SKIP LOCKED LIMIT 1` — each worker locks the next available job and skips rows already locked by other workers, so contention turns into parallelism instead of blocking.

Step-by-step thought process: 1. "*Without SKIP LOCKED, workers all hit the same first-pending row and queue up on the lock — serial throughput.*" 2. "`FOR UPDATE SKIP LOCKED` *tells the planner: if a row is already locked, skip it and move to the next — workers naturally fan out.*" 3. "*Inside a txn: lock the row, update status to 'in_progress', commit — lock released, next pickup will skip it because status changed.*" 4. "*Combine with* `LIMIT 1` *for one job at a time or* `LIMIT N` *for batch processing — same lock semantics.*"

Edge cases to mention: - Available in Postgres 9.5+, Oracle 11g+, SQL Server 2014+ (via READPAST hint), MySQL 8.0.1+ - Use a partial index `WHERE status = 'pending'` so the SELECT scans only candidate rows - For very high contention, partition the queue by worker_id modulo to reduce hot-spotting

What NOT to say: - "Just use a message queue like Kafka" — sometimes the database queue is the right tool; know both - "FOR UPDATE SKIP LOCKED loses data" — it skips locked rows during the SELECT, but they'll be available again once the other txn commits

Common follow-up: "*What happens to jobs whose worker crashes mid-processing?*"

Q71 · INSERT ON CONFLICT DO UPDATE — Postgres Upsert

Asked at: Razorpay, Flipkart, Cred, Swiggy, Walmart Labs **Difficulty**: Medium · **Topic**: DML

The question: "*You're loading user records — if a user exists update their last_seen, otherwise insert. Write the upsert.*"

The 30-second answer:
`INSERT INTO users (id, last_seen) VALUES (?, ?) ON CONFLICT (id) DO UPDATE SET last_seen = EXCLUDED.last_seen` — atomic, no race, returns inserted/updated row.

Step-by-step thought process: 1. "*The ON CONFLICT target must be a unique constraint or unique index — here, the primary key on id.*" 2. "`EXCLUDED.column` *refers to the values from the proposed INSERT — use this to express 'use the new value'.*" 3. "*To skip the update entirely on conflict, use* `ON CONFLICT DO NOTHING` *— useful for idempotent inserts.*" 4. "*Combine with a* `WHERE` *clause on the DO UPDATE to skip pointless updates:* `WHERE users.last_seen < EXCLUDED.last_seen`."

Edge cases to mention: - Requires Postgres 9.5+ — older versions needed a CTE-based workaround - The conflict target must specify the unique index/constraint exactly — can't be "any unique constraint" - A row that triggers DO UPDATE consumes the same identity/sequence value as a successful INSERT — gaps appear

What NOT to say: - "Just try INSERT and catch the error" — race-prone, slow under contention - "Use MERGE" — was unavailable in Postgres until version 15, and even then ON CONFLICT is often clearer

Common follow-up: "*What's the difference between ON CONFLICT DO UPDATE and MERGE in Postgres 15?*"

Q72 · MERGE in Postgres 15

Asked at: Goldman Sachs, JP Morgan India, AmEx, Target, Wells Fargo **Difficulty**: Hard · **Topic**: DML

The question: "*Postgres 15 added MERGE. When would you pick it over ON CONFLICT, and what gotchas should you watch for?*"

The 30-second answer: MERGE handles INSERT, UPDATE and DELETE in one statement based on a source table join — use it when you need multi-action logic from a staging table. ON CONFLICT only does insert-or-update and only by unique constraint match.

Step-by-step thought process: 1. "`MERGE INTO target USING source ON target.id = source.id WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT ...` — *declarative bulk merge.*" 2. "*MERGE supports DELETE in the WHEN MATCHED clause — useful for sync-style workloads.*" 3. "*Gotcha: in Postgres 15, MERGE does not handle concurrent inserts gracefully — two MERGEs racing can both insert the same row; ON CONFLICT does not have this issue.*" 4. "*For concurrent-safe upsert with a single source row, ON CONFLICT remains the right tool; MERGE shines for batch processing from a staging table.*"

Edge cases to mention: - MERGE requires the source to be a query — can be `(VALUES (...)) AS source` for a single row - Postgres 16 added `RETURNING` support to MERGE; 15 did not have it - The order of WHEN clauses matters — first match wins, so structure conditions carefully

What NOT to say: - "MERGE is always better than ON CONFLICT" — concurrency semantics differ; ON CONFLICT is safer for high-concurrency single-row upserts - "MERGE is the SQL standard so it's universal" — every vendor implements it differently; portability is an illusion

Common follow-up: "*Show me the SQL Server MERGE concurrency bug.*"

Q73 · SQL Server MERGE Pitfalls

Asked at: TCS Digital, Cognizant, Accenture, Wells Fargo, AmEx **Difficulty**: Hard · **Topic**: DML

The question: "*SQL Server MERGE has known concurrency issues. What are they and how do you work around them?*"

The 30-second answer: SQL Server MERGE can violate primary-key uniqueness under concurrency because the WHEN NOT MATCHED check and the INSERT are not atomic at the row level — two concurrent MERGEs can both decide to INSERT the same key. Workaround: serialise with `HOLDLOCK` or use a separate INSERT/UPDATE with `MERGE`-equivalent logic.

Step-by-step thought process: 1. "*The MERGE statement reads the target, plans actions, then executes — between the read and the INSERT, another txn can sneak in.*" 2. "*Add `WITH (HOLDLOCK)` to the target table reference — promotes the read locks to range locks, blocks concurrent inserts on the same key range.*" 3. "*Microsoft's own KB and Aaron Bertrand have documented this for years — the fix is consistently 'add HOLDLOCK'.*" 4. "*Alternative: skip MERGE entirely, use an UPDATE-then-INSERT-where-not-exists pattern inside a serialisable txn.*"

Edge cases to mention: - The issue is documented in MS Connect items and persists in current SQL Server versions - Triggers fire differently under MERGE vs separate DML — read the docs before relying on them - MERGE in SQL Server also has a long history of bugs around OUTPUT and trigger timing — test thoroughly

What NOT to say: - "Just trust MERGE, it's a single statement so it's atomic" — atomic statement, non-atomic semantics for unique checks - "Use snapshot isolation, that fixes it" — snapshot isolation does not help with write-write conflicts on the same row

Common follow-up: "*How would you write a safe upsert in SQL Server without MERGE?*"

Q74 · GROUPING SETS / ROLLUP / CUBE

Asked at: Goldman Sachs, JP Morgan India, AmEx, Walmart Labs, Target **Difficulty**: Hard · **Topic**: Aggregation

The question: "*Sales by region, by product, by region+product, and a grand total — all in one query. How?*"

The 30-second answer: Use `GROUP BY GROUPING SETS ((region), (product), (region, product), ())` — Postgres aggregates each set independently and unions the results, returning one row per (region, product, grand-total) combination.

Step-by-step thought process: 1. "*GROUPING SETS lets you explicitly list which combinations of columns to group on — the empty set `()` produces the grand total row.*" 2. "*`ROLLUP(a, b, c)` is shorthand for `GROUPING SETS ((a,b,c), (a,b), (a), ())` — a hierarchy from most detailed to grand total.*" 3. "*`CUBE(a, b)` is every subset: `GROUPING SETS ((a,b), (a), (b), ())` — useful for pivot-style cross tabs.*" 4. "*Use `GROUPING(column)` function to identify which rows are subtotals vs detail — returns 1 if the column was rolled up, 0 if it's a real grouping value.*"

Edge cases to mention: - Detail rows and subtotal rows mix in the output — wrap NULL in `GROUPING` checks or `COALESCE(region, 'TOTAL')` for display - Cardinality explodes with CUBE on many columns — n columns give 2^n grouping sets - All major databases support GROUPING SETS / ROLLUP / CUBE since the early 2010s

What NOT to say: - "I'd UNION ALL multiple GROUP BY queries" — works but verbose, runs more scans, harder to maintain - "ROLLUP and CUBE are the same" — ROLLUP is a hierarchy (3 sets for 3 columns), CUBE is all subsets (8 sets for 3 columns)

Common follow-up: "*How do you distinguish a 'real NULL in the region column' from a 'subtotal NULL' in the output?*"

Q75 · FILTER Clause for Conditional Aggregation

Asked at: Razorpay, Swiggy, Meesho, Cred, PhonePe **Difficulty**: Medium · **Topic**: Aggregation

The question: "*Count successful payments and failed payments in one query, split into two columns. Show me the cleanest way.*"

The 30-second answer: `COUNT(*) FILTER (WHERE status = 'success') AS succ, COUNT(*) FILTER (WHERE status = 'failed') AS fail` — the FILTER clause limits the aggregation to matching rows, cleaner than `SUM(CASE WHEN ... THEN 1 END)`.

Step-by-step thought process: 1. "*`FILTER (WHERE ...)` is the SQL-standard way to apply a per-aggregate predicate.*" 2. "*Reads more clearly than the CASE-WHEN trick — interviewer sees you know modern SQL.*" 3. "*Works with any aggregate — SUM, AVG, COUNT, MAX, even string_agg.*" 4. "*Equivalent to `SUM(CASE WHEN status = 'success' THEN 1 ELSE 0 END)` but the planner can sometimes optimise FILTER better.*"

Edge cases to mention: - Postgres supports FILTER since 9.4; SQLite since 3.30; SQL Server still doesn't have it — must use CASE - FILTER vs WHERE — WHERE filters before grouping (all aggregates see the same filtered set); FILTER filters per aggregate - Combine with GROUP BY for per-group conditional counts

What NOT to say: - "FILTER is just syntactic sugar" — true, but it's much more readable and the standard preferred form - "Use it everywhere" — SQL Server doesn't support it; check your target dialect

Common follow-up: "*What's the difference between WHERE and FILTER in a query with both?*"

Q76 · PERCENTILE_CONT for Median

Asked at: Goldman Sachs, JP Morgan India, AmEx, Walmart Labs, Target **Difficulty**: Medium · **Topic**: Aggregation

The question: "*There's no MEDIAN function in standard SQL. How do you compute the median order amount?*"

The 30-second answer: `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount)` — continuous percentile interpolates between adjacent values when the median falls between two rows; `PERCENTILE_DISC` picks the actual value at the percentile.

Step-by-step thought process: 1. "*PERCENTILE_CONT returns a value between two rows if the percentile lands between them — gives a smoothed median.*" 2. "*PERCENTILE_DISC returns the closest actual row value — useful when interpolation makes no sense (categorical, integers you can't fractionalise).*" 3. "*Both are ordered-set aggregates — note the `WITHIN GROUP (ORDER BY ...)` syntax, not the regular `ORDER BY` inside the function.*" 4. "*For grouped medians: `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount) OVER (PARTITION BY merchant_id)` — Postgres supports this as a window function in some versions; otherwise wrap in a subquery.*"

Edge cases to mention: - The argument can be any value in [0, 1] — `PERCENTILE_CONT(0.95)` for the 95th percentile is the same syntax - Pass an array: `PERCENTILE_CONT(ARRAY[0.25, 0.5, 0.75]) WITHIN GROUP (ORDER BY amount)` returns quartiles in one call - MySQL doesn't have PERCENTILE_CONT — use window functions to compute it manually

What NOT to say: - "Just sort and pick the middle row" — fine for small datasets, fragile for grouped medians - "Use AVG of MIN and MAX" — that's the midrange, not the median

Common follow-up: "*How does PERCENTILE_CONT handle an even number of rows?*"

Q77 · PIVOT via CASE Expressions

Asked at: Infosys, TCS Digital, Cognizant, Accenture, Capgemini **Difficulty**: Medium · **Topic**: Aggregation

The question: "*You have transactions with `month` and `amount`. Produce a wide report with one row per merchant and one column per month — without using any vendor-specific PIVOT keyword.*"

The 30-second answer:
`SELECT merchant_id, SUM(CASE WHEN month = 'Jan' THEN amount END) AS jan, SUM(CASE WHEN month = 'Feb' THEN amount END) AS feb, ... FROM transactions GROUP BY merchant_id` — classic conditional aggregation pivot.

Step-by-step thought process: 1. "*The trick is `SUM(CASE WHEN ... THEN amount ELSE NULL END)` — non-matching rows contribute NULL, which SUM ignores.*" 2. "*One column per distinct value of the pivot column — works only when you know the values in advance (12 months, 4 quarters).*" 3. "*For unknown pivot values you need dynamic SQL — build the query string in application code or with a stored procedure.*" 4. "*FILTER clause makes this cleaner in Postgres: `SUM(amount) FILTER (WHERE month = 'Jan') AS jan` — same idea, modern syntax.*"

Edge cases to mention: - SQL Server and Oracle have a native `PIVOT` keyword; Postgres and MySQL do not — use CASE or FILTER - For high-cardinality pivots, JSON aggregation (`jsonb_object_agg(month, amount)`) is often more practical - `NULL` vs `0` for missing months — wrap in `COALESCE(SUM(...), 0)` if your downstream expects zeros

What NOT to say: - "Postgres doesn't support PIVOT so I can't do it" — CASE-based pivot works in every dialect - "Use the tablefunc extension's crosstab" — works but rarely seen in practice; CASE is more portable and clearer

Common follow-up: "*What if the months are not known in advance?*"

Q78 · The N+1 Query Problem

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Flipkart, Walmart Labs · Topic: Query patterns

The question (interviewer's verbatim phrasing):

> Your ORM-based service is taking 4 seconds to render an orders list page. Each row also shows the customer name. The DBA shows you 401 queries hitting the database for one page. What is happening and how do you fix it?

The 30-second answer: That is the classic N+1 problem — one query to fetch 400 orders, then 400 follow-up queries, one per order, to fetch each customer. Fix it with a single JOIN or an `IN (...)` batch, or in the ORM with eager-loading (`includes`, `JOIN FETCH`, `select_related`).

Step-by-step thought process: 1. Name the pattern: "1 parent query + N child queries" — every senior interviewer wants the term first. 2. Explain why ORMs do this: lazy-loading of associations. Accessing `order.customer.name` inside a loop triggers a new SELECT per iteration. 3. Show the fix at the SQL layer first — one JOIN returning `order_id, order_amount, customer_name` — then the ORM-specific fix (Rails `includes`, Hibernate `JOIN FETCH`, Django `select_related`, SQLAlchemy `joinedload`). 4. Mention detection: enable query logging in dev, or use tools like pghero, New Relic, or rack-mini-profiler. In Postgres, `pg_stat_statements` shows you the offending repeated query.

Edge cases to mention: - If you have nested associations (`order.customer.city.state`), naive eager-loading can JOIN four tables and balloon row count — use a two-step query (`IN (...)` batch) instead. - N+1 also hides in serializers — fixing the controller but leaving a stale serializer keeps the bug. - Cursor-paginated lists with eager-loaded associations sometimes blow up memory; eager-load only what the view actually renders.

What NOT to say: - Do not say "add an index" — the problem is round-trip count, not query cost per round. - Do not blame the ORM and rewrite everything in raw SQL; the ORM fix is usually one line.

Common follow-up: "*How would you detect N+1 in production without enabling full query logs?*" — Sample-based slow-log + `pg_stat_statements` with `calls` desc; any query called 400× per request is suspect.

```
-- Bad: triggered N times by the ORM
SELECT name FROM customers WHERE id = ?;

-- Good: one query
SELECT o.id, o.amount, c.name
FROM orders o
JOIN customers c ON c.id = o.customer_id
WHERE o.created_at >= CURRENT_DATE - INTERVAL '7 days';
```

Q79 · Reading EXPLAIN ANALYZE

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Walmart Labs, Cred, Zerodha · Topic: Query plans

The question (interviewer's verbatim phrasing):

> Run `EXPLAIN ANALYZE` on this query, paste the output, and tell me what is going wrong.

The 30-second answer: Read the plan bottom-up — leaf nodes are the scans, the top is the final operator. Watch three numbers per node: estimated rows vs actual rows (planner accuracy), loops (how often the inner side runs), and actual time. A 100× estimation error or a Seq Scan on a million-row table is usually the smell.

Step-by-step thought process: 1. Open with the bottom-up rule and the three-number rule (estimate, actual, loops). 2. Call out the common red flags: `Seq Scan` on a large table when an index exists, `Rows Removed by Filter` much higher than `Rows`, a nested loop with `loops=200000`, or `Hash Batches: 32` (spilled to disk). 3. Mention `BUFFERS` — `EXPLAIN (ANALYZE, BUFFERS)` shows shared-hit vs read; a query with 200k buffer reads is doing disk I/O. 4. Anticipate follow-up: "What is the difference between EXPLAIN and EXPLAIN ANALYZE?" — EXPLAIN shows the planner's intent, ANALYZE actually runs the query and reports real timing and row counts.

Edge cases to mention: - `EXPLAIN ANALYZE` on an UPDATE or DELETE actually performs the write — wrap it in `BEGIN; ... ROLLBACK;` on hot tables. - Plan output differs between MySQL (`EXPLAIN FORMAT=JSON`), Postgres, and SQL Server — vocabulary is similar but operator names are not. - A correct plan today can become wrong tomorrow if statistics drift; re-check after large data changes.

What NOT to say: - Do not focus on `cost=` — it is a planner heuristic, not seconds. Real time is in `actual time`. - Do not declare a plan "bad" just because it has a Seq Scan — on a 10k-row table, Seq Scan is often correct.

Common follow-up: "*What does it mean if estimated rows = 10 but actual rows = 100000?*" — Stale statistics or skewed distribution; run `ANALYZE` and consider increasing `default_statistics_target` for that column.

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT u.id, COUNT(o.id)
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE u.created_at >= '2026-01-01'
GROUP BY u.id;
```

Q80 · Hash Join vs Nested Loop vs Merge Join

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Wells Fargo, Walmart Labs · Topic: Query plans

The question (interviewer's verbatim phrasing):

> The optimizer can pick a nested loop, a hash join or a merge join. When does each one win, and why might the optimizer pick the wrong one?

The 30-second answer: Nested loop wins when one side is tiny and the other has an index on the join key — typical for OLTP lookups. Hash join wins for medium-to-large unindexed equi-joins because it builds an in-memory hash on the smaller side. Merge join wins when both sides are already sorted on the join key, often as a side-effect of an index range scan.

Step-by-step thought process: 1. State the picking rule: row count + index availability + sort order. 2. Walk through each: nested loop is `for each row on outer, lookup inner` — cost grows linearly with outer size, fine if inner lookup is O(log n) via index, terrible if inner needs a Seq Scan. 3. Hash join: build phase hashes the smaller side, probe phase scans the larger; requires `work_mem` ≥ build-side size, otherwise spills to disk in batches. 4. Merge join: both sides sorted, walk in lockstep; great for already-sorted inputs (range scans on a B-tree), bad if a sort step is added.

Edge cases to mention: - Optimizer picks badly when row estimates are off — a "5 rows estimated, 5 million actual" outer triggers a disastrous nested loop. - Hash joins only work for equi-joins; range joins (`a.id BETWEEN b.lo AND b.hi`) force nested loop. - MySQL InnoDB before 8.0.18 had no hash join at all; only nested loop with index lookup (often called "Block Nested Loop" without an index).

What NOT to say: - Do not say "merge join is always fastest" — only if both inputs are pre-sorted. - Do not call nested loop "primitive" — it is the right choice for indexed OLTP lookups.

Common follow-up: "*How do you force a plan change without hints?*" — Update statistics, add an index, or restructure the predicate. Postgres deliberately has no hints; you fix the inputs to the planner.

Q81 · Sort Aggregate vs Hash Aggregate

Tags: Difficulty: Medium · Asked at: Walmart Labs, Cred, Goldman, Zerodha · Topic: Query plans

The question (interviewer's verbatim phrasing):

> Explain the difference between a sort-based aggregate and a hash aggregate, and tell me when `work_mem` matters.

The 30-second answer: Hash aggregate builds an in-memory hash keyed by the GROUP BY columns and accumulates aggregates in place — fast, requires that the hash fits in `work_mem`. Sort aggregate sorts the input by group key then folds runs of equal keys — slower per row but constant memory, handles unlimited group counts.

Step-by-step thought process: 1. State the trade-off: hash is faster but memory-bound; sort is steadier but I/O-heavy on large inputs. 2. Postgres prefers hash aggregate when estimated group count fits in `work_mem`; otherwise it picks sort + group, or in Postgres 13+ a hash that spills to disk. 3. Mention the symptom: a query that ran in 2 seconds suddenly takes 60 seconds — often the hash spilled to disk because `work_mem` is too small or the planner under-estimated group count. 4. Anticipate: "What is `work_mem`?" — Per-operation working memory; a query with three sorts and a hash can consume `4 × work_mem` per backend.

Edge cases to mention: - `DISTINCT` uses the same code path as `GROUP BY` — same hash vs sort decision applies. - Setting `work_mem` too high globally causes OOM when many concurrent queries each grab their share — tune per-session for big analytical queries. - MySQL uses temporary tables (in-memory then on-disk) for GROUP BY; `tmp_table_size` and `max_heap_table_size` are the equivalent knobs.

What NOT to say: - Do not say "always raise `work_mem`" — it is per-operation per-connection, easy to OOM a 32GB box. - Do not confuse `work_mem` with `shared_buffers`; one is per-op, the other is the global cache.

Common follow-up: "*You see `Sort Method: external merge Disk: 850MB` — what does it mean?*" — The sort spilled to disk because the data exceeded `work_mem`. Raise `work_mem` for that session or add an index that returns rows pre-sorted.

Q82 · Stale Statistics and ANALYZE

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Goldman, Walmart Labs · Topic: Query plans

The question (interviewer's verbatim phrasing):

> A query that ran in 30 ms yesterday is now taking 8 seconds. Nothing about the schema changed. Where do you look first?

The 30-second answer: Statistics. The planner uses row counts and value distributions to pick a plan; if a bulk load just doubled the table or skewed a column, the old plan is now wrong. Run `ANALYZE table_name;` in Postgres or `UPDATE STATISTICS` in SQL Server / `ANALYZE TABLE` in MySQL, then re-EXPLAIN.

Step-by-step thought process: 1. State the cause: stale stats lead to bad plans, especially nested loops with under-estimated outer cardinality. 2. Explain when stats go stale: large bulk inserts, large deletes, sudden value-distribution shifts (a new country joining, a promo day), partition swap-ins. 3. Postgres has `autovacuum` which triggers `ANALYZE` based on dead-tuple thresholds; tune `autovacuum_analyze_scale_factor` for very large tables where the default 10% threshold means analyse fires too late. 4. Anticipate: "What does ANALYZE actually do?" — Samples rows from the table, computes histograms and most-common-values lists, stores them in `pg_statistic`.

Edge cases to mention: - Partitioned tables: stats live per-partition; an unanalyzed new partition gets default selectivity guesses and ruins plans. - `default_statistics_target` controls sample size — raise to 1000 for skewed columns; default 100 is too low for highly-skewed dimensions. - After a `pg_restore`, run `ANALYZE` before serving traffic — restore does not gather stats automatically.

What NOT to say: - Do not blame the query; the query is fine, the plan is wrong because the inputs are stale. - Do not raise `default_statistics_target` globally; it makes ANALYZE slow on every table.

Common follow-up: "*Why does Postgres autovacuum not analyse the table fast enough?*" — Threshold is 10% changes by default; on a 500M-row table that is 50M changes before ANALYZE fires. Lower `autovacuum_analyze_scale_factor` to 0.01 or 0.005 for hot large tables.

Q83 · Non-Sargable Predicates — Function on Indexed Column

Tags: Difficulty: Medium · Asked at: TCS GCC, Infosys, Wells Fargo, Cognizant, JP Morgan · Topic: Indexing

The question (interviewer's verbatim phrasing):

> Why does `WHERE YEAR(created_at) = 2026` not use the index on `created_at`?

The 30-second answer: Because you wrapped the column in a function. The B-tree index stores raw `created_at` values, not `YEAR(created_at)` values, so the planner can no longer do a range lookup. Rewrite as a range predicate — `created_at >= '2026-01-01' AND created_at < '2027-01-01'` — and the index is back in play.

Step-by-step thought process: 1. Name the concept: this is a "non-sargable" predicate — "search ARGument-able" means the optimizer can push it down to an index seek; functions on columns break that. 2. Walk the rewrite to a half-open range; explain why half-open beats `BETWEEN '2026-01-01' AND '2026-12-31'` (the latter misses the last second of the year for `timestamp` columns). 3. Mention the alternative: a functional index. Postgres allows `CREATE INDEX ON orders (YEAR(created_at))`; MySQL 8.0+ also supports functional indexes; SQL Server supports computed-column indexes. 4. Anticipate: same pattern shows up with `LOWER(email)`, `CAST(id AS TEXT)`, `DATE(timestamp_col)`.

Edge cases to mention: - Postgres `date_trunc('day', ts) = '2026-05-26'` is also non-sargable — rewrite as a range or add an expression index. - Case-insensitive search (`LOWER(email) = ?`) — either a functional index on `LOWER(email)` or use a case-insensitive collation (`citext` in Postgres). - Some optimizers (SQL Server) are smart enough to translate simple `YEAR()` to a range; do not rely on it.

What NOT to say: - Do not say "add an index on `YEAR(created_at)`" without explaining functional indexes — junior answer. - Do not claim the index "works but is slow"; it is not used at all.

Common follow-up: "*What about `WHERE col + 0 = 5` — is that sargable?*" — No. Arithmetic on the column side is non-sargable. Move the math to the literal: `WHERE col = 5`.

```
-- Bad
SELECT * FROM orders WHERE YEAR(created_at) = 2026;

-- Good
SELECT * FROM orders
WHERE created_at >= '2026-01-01' AND created_at < '2027-01-01';
```

Q84 · Implicit Type Cast Disabling Index

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Cred · Topic: Indexing

The question (interviewer's verbatim phrasing):

> You have an index on `mobile_number VARCHAR(10)` but `WHERE mobile_number = 9876543210` does a Seq Scan. Why?

The 30-second answer: Type mismatch — the literal is a bigint, the column is a varchar, so the planner injects an implicit cast on the column side: effectively `CAST(mobile_number AS BIGINT) = 9876543210`. That cast disables the index for the same reason any function on a column disables it. Pass the literal as a string: `'9876543210'`.

Step-by-step thought process: 1. State the cause: implicit cast on the column side, not the literal side, because of the type-promotion rules. 2. Demonstrate the fix — quote the literal so it is varchar-on-varchar and the index seek is back. 3. Mention the worse variant: storing Indian mobile numbers as bigint loses the leading zero on landlines and forces casts elsewhere — varchar is the right call. 4. Anticipate the follow-up about parameter binding: ORMs that bind `params[:mobile] = 9876543210` (integer) reproduce the bug; explicit `.to_s` in the app or column-type-aware binding fixes it.

Edge cases to mention: - The opposite case — `INT` column compared to a quoted string — Postgres usually casts the literal to int (safe), but MySQL may cast both sides to double (lossy). - Time-zone-naive vs aware timestamps cause the same problem — comparison forces a runtime cast on every row. - JSON columns: `WHERE data->>'amount' = 100` casts the text result to int per row, never indexed unless you build an expression index.

What NOT to say: - Do not say "MySQL is dumb" — the SQL standard requires implicit conversion; the engine is following the rules. - Do not just add an index on the cast expression; fix the binding instead.

Common follow-up: "*How would you catch this in code review?*" — Look at the plan in dev with `EXPLAIN`; look for any "filter:" line that mentions the column wrapped in a cast.

Q85 · Composite Index Leftmost-Prefix Rule

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Walmart Labs, Cred, JP Morgan · Topic: Indexing

The question (interviewer's verbatim phrasing):

> You have an index on `(state, city, pincode)`. Which of these queries can use it? `WHERE state = ?`, `WHERE city = ?`, `WHERE state = ? AND pincode = ?`, `WHERE state = ? AND city = ? AND pincode = ?`.

The 30-second answer: B-tree composite indexes follow the leftmost-prefix rule. `state` alone uses it. `city` alone does not. `state + pincode` uses the `state` part only (pincode becomes a filter). `state + city + pincode` uses the whole index.

Step-by-step thought process: 1. Explain the rule: a composite index is sorted by the first column, then within ties by the second, then within ties by the third — like a phone book sorted by surname, then first name, then middle name. 2. Walk each case using that mental model: looking up "all Mehtas" works; looking up "all Rakesh" (without surname) means scanning the whole book. 3. Show the fix for the `state + pincode` case: either add a `(state, pincode)` index, or accept the partial use plus filter. 4. Anticipate follow-up about index order: put the most selective column first if equality, the column you most often need for range last (range scans terminate the prefix).

Edge cases to mention: - Equality on a leftmost column allows range or sort on the next: `WHERE state = 'MH' ORDER BY city` can use the index for both filter and sort. - Skip-scan (Oracle, MySQL 8.0) can sometimes use a non-leftmost prefix; Postgres does not have skip-scan. - Index-only scan needs every selected column to be in the index — composite + INCLUDE columns (Postgres 11+) for that.

What NOT to say: - Do not say "the index is unused for `WHERE city = ?`" without offering the fix (a separate index on `city`). - Do not claim "more columns in the index is always better" — write cost grows per index.

Common follow-up: "*Should `(state, city)` be one index or two separate indexes on each column?*" — One composite if queries always filter both or filter by `state` alone. Two singles only if the access patterns are genuinely independent.

Q86 · ORDER BY + LIMIT Optimization

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Walmart Labs · Topic: Query plans

The question (interviewer's verbatim phrasing):

> `SELECT * FROM orders ORDER BY created_at DESC LIMIT 20` is doing a full sort of 50 million rows. How do you fix it?

The 30-second answer: Create an index on `created_at` (descending or just plain — B-tree can walk both directions). The plan flips from `Sort + Limit` to `Index Scan Backward + Limit`, which stops after reading 20 rows.

Step-by-step thought process: 1. State the rule: an index on the ORDER BY column lets the planner pull rows in pre-sorted order and stop early at LIMIT. 2. Mention the descending index nuance: Postgres B-tree handles forward/backward equally well; MySQL InnoDB added efficient descending indexes in 8.0. 3. Demonstrate the more interesting case — `WHERE status = 'paid' ORDER BY created_at DESC LIMIT 20` needs `(status, created_at DESC)` composite, otherwise it filters first then sorts. 4. Anticipate the follow-up about LIMIT + OFFSET being slow at high offsets — leads into keyset pagination.

Edge cases to mention: - If `created_at` is not unique, ties make LIMIT non-deterministic across runs — add a tiebreaker (`ORDER BY created_at DESC, id DESC`). - The `SELECT *` is wasteful if only a few columns are needed; an index-only scan is possible with a covering index. - Postgres can sometimes return rows in unexpected order from index scans on parallel plans; rely on `ORDER BY` , not implicit order.

What NOT to say: - Do not add `LIMIT 20` and assume it fixes performance; without the sort index it still sorts the whole table first. - Do not say "indexes do not help with ORDER BY" — they absolutely do when the order matches.

Common follow-up: "*What if the user wants the top 20 by* `amount` *within each* `status` *?*" — Top-N per group; window function or LATERAL with LIMIT.

Q87 · Top-N Per Group via Window Function

Tags: Difficulty: Medium · Asked at: Walmart Labs, Flipkart, Cred, Goldman, Razorpay · Topic: Window functions

The question (interviewer's verbatim phrasing):

> Give me the three most recent orders for each customer.

The 30-second answer: Use `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY created_at DESC)` in a subquery, then filter `WHERE rn <= 3` in the outer query.

Step-by-step thought process: 1. State the pattern — top-N per group is the textbook window-function use case. 2. Walk the SQL: window functions cannot appear in WHERE directly, so you nest them in a subquery or CTE. 3. Choose ROW_NUMBER vs RANK vs DENSE_RANK consciously: ROW_NUMBER gives exactly 3 even on ties; RANK can give more (ties share a rank); DENSE_RANK same but no rank gaps. 4. Anticipate the follow-up about performance: a `(customer_id, created_at DESC)` index lets the planner walk per partition in order — fast even at 100M rows.

Edge cases to mention: - Ties on `created_at` need a tiebreaker (`ORDER BY created_at DESC, id DESC`) for determinism. - For top-1 only, `DISTINCT ON` in Postgres or a correlated subquery can be faster than ROW_NUMBER. - For very high N (top 1000), window scan may not be faster than a sort + LIMIT per partition via LATERAL.

What NOT to say: - Do not use `GROUP BY customer_id` with `MAX(created_at)` — it gives one row, not three, and you cannot pull the rest of the columns cleanly. - Do not say "ROW_NUMBER and RANK are interchangeable" — they differ on ties.

Common follow-up: "*Now write it without a window function.*" — Self-join with a correlated count: `WHERE (SELECT COUNT(*) FROM orders o2 WHERE o2.customer_id = o.customer_id AND o2.created_at > o.created_at) < 3`. Slow but works on engines without window functions.

```
SELECT customer_id, id, amount, created_at
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY created_at DESC) AS rn
  FROM orders
) t
WHERE rn <= 3;
```

Q88 · Top-N Per Group via DISTINCT ON (Postgres)

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Swiggy · Topic: Postgres specifics

The question (interviewer's verbatim phrasing):

> You are on Postgres and you only need the single latest order per customer. Is there a shorter way than ROW_NUMBER?

The 30-second answer: `SELECT DISTINCT ON (customer_id) * FROM orders ORDER BY customer_id, created_at DESC` — Postgres returns the first row of each `customer_id` group as ordered. One pass, no window function.

Step-by-step thought process: 1. Explain DISTINCT ON as Postgres-specific shorthand: keeps the first row per group based on the ORDER BY. 2. Show the critical rule: the leading ORDER BY column must match the DISTINCT ON column, then the secondary ORDER BY decides which row "wins" per group. 3. Mention performance: with `(customer_id, created_at DESC)` index, this can do an index-skip-like scan, very fast. 4. Anticipate the portability question: DISTINCT ON is Postgres-only; for MySQL or SQL Server, fall back to ROW_NUMBER.

Edge cases to mention: - ORDER BY mismatch is a runtime error — leading columns must match. - Returns one row per group only; for top-3, use ROW_NUMBER. - Cannot use DISTINCT ON with UNION or in CTEs portably across engines.

What NOT to say: - Do not claim DISTINCT ON works on MySQL — it does not. - Do not use it for top-N where N > 1 — it does not generalize.

Common follow-up: "*How does the planner execute DISTINCT ON internally?*" — Usually a `Unique` node on top of a sorted scan; with the right index, the sort step disappears.

```
SELECT DISTINCT ON (customer_id) customer_id, id, amount, created_at
FROM orders
ORDER BY customer_id, created_at DESC;
```

Q89 · Keyset Pagination vs OFFSET

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Flipkart, Walmart Labs · Topic: Pagination

The question (interviewer's verbatim phrasing):

> Page 1 of your orders list takes 50ms. Page 1000 takes 4 seconds. Why, and how do you fix it?

The 30-second answer: OFFSET 19980 LIMIT 20 forces the database to fetch and discard the first 19,980 rows before returning the 20 you want — work grows linearly with page depth. Switch to keyset pagination: instead of OFFSET, remember the last row's sort key from the previous page and use `WHERE (created_at, id) < (?, ?) ORDER BY created_at DESC, id DESC LIMIT 20`.

Step-by-step thought process: 1. Explain the cost model: OFFSET N actually reads N+LIMIT rows; keyset uses an index seek to jump directly to the cursor. 2. Walk the keyset query — note the tuple comparison `(created_at, id) < (?, ?)` to handle ties correctly. 3. Trade-off: keyset cannot do "jump to page 1000" — only "next" and "previous". For most user-facing lists, that is fine (Twitter, Gmail, Instagram all use keyset). 4. Anticipate the follow-up about the cursor being stable: encode the cursor (base64 of last sort key) to prevent users from forging it and to keep URLs clean.

Edge cases to mention: - Need a strictly-monotonic tiebreaker (`id`) when the sort column has ties; otherwise the cursor skips or repeats rows. - Sorts on text columns with collation differences across nodes can break tuple comparison; pin collation. - Cursor-based "previous" requires storing the first key of the current page, not just the last.

What NOT to say: - Do not say "OFFSET is fine for small pages" without saying when it breaks — page 100 with 100/page is already 10k rows skipped. - Do not add a count(*) for "page X of Y" UI on a 500M-row table — explain why it is impossible at scale.

Common follow-up: "*How do you implement keyset when sorting by a non-unique column like `amount`?*" — Composite cursor: `(amount, id) < (?, ?)`.

```
-- Page 1
SELECT id, customer_id, amount, created_at FROM orders
ORDER BY created_at DESC, id DESC LIMIT 20;

-- Page N: pass last row's (created_at, id) as cursor
SELECT id, customer_id, amount, created_at FROM orders
WHERE (created_at, id) < (?, ?)
ORDER BY created_at DESC, id DESC LIMIT 20;
```

Q90 · EXISTS vs IN — Semantics and Performance

Tags: Difficulty: Medium · Asked at: Goldman, Walmart Labs, Cred, JP Morgan, Razorpay · Topic: Query patterns

The question (interviewer's verbatim phrasing):

> When would you prefer `EXISTS (SELECT 1 ...)` over `IN (SELECT ...)`?

The 30-second answer: EXISTS is a correlated check that stops at the first match — best when the inner table is large and the predicate is selective. IN with a subquery is fine for small static lists. Most modern planners (Postgres, SQL Server, MySQL 8) normalize both to the same semi-join, but EXISTS is clearer when the inner query references outer columns.

Step-by-step thought process: 1. State the cardinality rule: both return one row per outer match — neither duplicates rows, unlike a JOIN. 2. Performance trade-off: EXISTS short-circuits per outer row; IN materialises the subquery first in older engines. 3. Readability point: when the subquery is correlated (uses outer columns), EXISTS reads as English; IN reads awkwardly. 4. Anticipate the follow-up about IN with a literal list (`IN (1, 2, 3)`) — that is fine and indexable, the discussion is about IN with a subquery.

Edge cases to mention: - IN with a NULL in the value list is fine for matching, but `NOT IN` with NULL is broken (separate question). - Large IN lists (10,000+ literals) hit parser/plan-cache limits — switch to a temp table or VALUES. - MySQL 5.5 had infamous `IN (subquery)` slowness; fixed in 5.6+.

What NOT to say: - Do not claim EXISTS is always faster — modern optimizers usually pick the same plan. - Do not blindly rewrite IN to EXISTS for "performance" without measuring.

Common follow-up: "*What does the plan look like for both?*" — Both usually show as `Hash Semi Join` or `Nested Loop Semi Join` in Postgres.

Q91 · NOT IN vs NOT EXISTS — NULL Safety

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Walmart Labs, Wells Fargo · Topic: Query patterns

The question (interviewer's verbatim phrasing):

> What is the difference between `NOT IN (subquery)` and `NOT EXISTS (subquery)` when the subquery can return NULL?

The 30-second answer: `NOT IN` becomes the empty set the moment any NULL appears in the subquery — because `x <> NULL` is UNKNOWN, not TRUE, so no row qualifies. `NOT EXISTS` is unaffected by NULLs in the inner result.

Step-by-step thought process: 1. Walk the three-valued logic: `5 NOT IN (1, 2, NULL)` becomes `5 <> 1 AND 5 <> 2 AND 5 <> NULL` = TRUE AND TRUE AND UNKNOWN = UNKNOWN, filtered out. 2. Show why NOT EXISTS is safe: it asks "does any row exist satisfying the predicate?" — a NULL in the inner table simply does not satisfy `=` and is ignored. 3. Mention the silent-failure mode: query returns zero rows when it should return many; usually discovered when someone notices a dashboard went blank. 4. Anticipate the follow-up about how to make NOT IN safe — add `AND col IS NOT NULL` to the subquery, or just always use NOT EXISTS.

Edge cases to mention: - If the subquery column is declared `NOT NULL`, NOT IN is safe — but defending on schema constraints is brittle. - `NOT IN` with a literal list containing NULL (`NOT IN (1, 2, NULL)`) has the same problem. - Some ORMs auto-rewrite NOT IN to LEFT JOIN ... IS NULL when they detect nullable columns; do not rely on that.

What NOT to say: - Do not say "they are equivalent" — that is the trap. - Do not blame the planner; the behaviour is standard SQL three-valued logic.

Common follow-up: *"Show me an anti-join rewrite of NOT EXISTS."* — `LEFT JOIN ... ON ... WHERE other.id IS NULL` — also NULL-safe and often what the planner does internally.

```
-- Dangerous if blocked_user_id can be NULL
SELECT id FROM users WHERE id NOT IN (SELECT blocked_user_id FROM blocklist);

-- Always safe
SELECT id FROM users u
WHERE NOT EXISTS (SELECT 1 FROM blocklist b WHERE b.blocked_user_id = u.id);
```

Q92 · Self-Join with `a.id < b.id` Pattern

Tags: Difficulty: Medium · Asked at: Flipkart, Cred, Walmart Labs · Topic: Joins

The question (interviewer's verbatim phrasing):

> Find all pairs of orders placed by the same customer on the same day. Each pair should appear once.

The 30-second answer: Self-join `orders a` to `orders b` on `a.customer_id = b.customer_id AND DATE(a.created_at) = DATE(b.created_at)`, then add `AND a.id < b.id` so each pair shows up once instead of twice and a row never pairs with itself.

Step-by-step thought process: 1. State the cartesian-product problem: a self-equi-join without the `<` would return both (A, B) and (B, A) plus (A, A). 2. The `a.id < b.id` trick keeps the canonical ordering — pair appears exactly once, and `a.id = b.id` (self-pair) is excluded automatically. 3. Mention that this is the same pattern used in graph problems ("undirected edges from a directed table") and pairwise-overlap detection. 4. Anticipate the follow-up about performance: needs an index on `(customer_id, created_at)` — without it, this is O(N²) within each customer group.

Edge cases to mention: - If the table has no integer id, use a tuple compare on the natural key. - Date truncation is non-sargable — better to range-join with `b.created_at BETWEEN DATE(a.created_at) AND DATE(a.created_at) + 1`. - Self-joins on huge tables can be slow; for large customers (1000+ orders/day), the per-customer pairs explode quadratically.

What NOT to say: - Do not use `a.id != b.id` — that still returns both orderings of each pair. - Do not forget the self-pair issue when using DISTINCT to dedupe — DISTINCT is slower than the `<` trick.

Common follow-up: "*What if you wanted ordered pairs (A → B where A placed first)?*" — `AND a.created_at < b.created_at` instead of `a.id < b.id`.

Q93 · Multi-Table JOIN Cardinality Multiplication

Tags: Difficulty: Medium · Asked at: Walmart Labs, JP Morgan, Goldman, Razorpay · Topic: Joins

The question (interviewer's verbatim phrasing):

> You JOIN orders, order_items, and payments. The result has 10× the rows you expected. What happened?

The 30-second answer: You joined two child tables to a parent. Each order has, say, 4 items and 2 payment attempts — the join produces 4 × 2 = 8 rows per order. Either join the children separately and aggregate before joining, or use LATERAL/derived tables to keep each child join independent.

Step-by-step thought process: 1. State the multiplication law: a one-parent-to-many-children join multiplies, not adds, across child tables. 2. Show the fix using subqueries-with-aggregation: `JOIN (SELECT order_id, SUM(amount) FROM payments GROUP BY order_id) p ON ...` collapses each child to one row per parent before joining. 3. Mention the LATERAL alternative in Postgres / CROSS APPLY in SQL Server for picking the latest payment row per order. 4. Anticipate the follow-up about `SELECT DISTINCT` as a "fix" — it dedupes the result but the JOIN still does the heavy work, so it is wasteful.

Edge cases to mention: - When all child counts are 1 (one item per order, one payment per order), the bug hides until a parent gets a second child. - SUM over multiplied rows is wrong by the multiplication factor — a classic "double-counted revenue" bug. - COUNT(DISTINCT) can mask the issue at the cost of performance.

What NOT to say: - Do not call this "a JOIN bug" — it is correct SQL semantics; the bug is in your understanding. - Do not just slap DISTINCT on top — silently slow and may not even dedupe correctly with aggregates.

Common follow-up: "*Rewrite this to compute order total + total paid in one row per order.*" — Pre-aggregate each child in its own subquery, then JOIN those subqueries to orders.

Q94 · Bulk INSERT — COPY, LOAD DATA INFILE, BULK INSERT

Tags: Difficulty: Medium · Asked at: TCS GCC, Walmart Labs, Cred, Razorpay · Topic: Bulk operations

The question (interviewer's verbatim phrasing):

> You need to load 50 million rows from a CSV into Postgres. How do you do it without taking down the database?

The 30-second answer: Use `COPY orders FROM '/path/file.csv' CSV HEADER` — orders of magnitude faster than `INSERT` because it bypasses parser overhead and uses a streaming protocol. For MySQL it is `LOAD DATA INFILE`, for SQL Server it is `BULK INSERT` or `bcp`.

Step-by-step thought process: 1. Name the tool per engine and explain the win: single transaction, no per-row parse, no per-row WAL flush. 2. Mention pre-load hygiene: drop or disable non-essential indexes and foreign keys before the load, rebuild after — index maintenance per row is what slows naive INSERT. 3. Chunk huge files into batches of 1-10M rows to keep transactions short and avoid bloating the WAL. 4. Anticipate the follow-up about validation: COPY is all-or-nothing per transaction; use a staging table, validate, then INSERT INTO target.

Edge cases to mention: - COPY ignores triggers by default? No — it fires triggers, which can be a huge slowdown. Disable triggers explicitly if appropriate. - File path is server-side for `COPY FROM`; use

`\copy` from psql for client-side files. - Character-set mismatch (UTF-8 vs Latin1) silently corrupts data — explicitly set encoding.

What NOT to say: - Do not use a loop of `INSERT` from your app — 1000× slower at best. - Do not say "just disable WAL" — you cannot, and you should not want to.

Common follow-up: "*How would you do this with zero downtime on the target table?*" — Load into a staging table, then either INSERT ... SELECT in batches or swap via partition exchange.

Q95 · Bulk UPDATE Patterns

Tags: Difficulty: Medium · Asked at: Walmart Labs, Razorpay, Cred, JP Morgan · Topic: Bulk operations

The question (interviewer's verbatim phrasing):

> You need to update a `status` column on 20 million rows. A single `UPDATE` statement runs for 40 minutes and locks the table. What is your strategy?

The 30-second answer: Chunk the update: a loop of `UPDATE ... WHERE id BETWEEN ? AND ?` over 10k-100k row windows, with a brief COMMIT between chunks. This keeps transactions short, releases locks, and lets autovacuum clean up dead tuples between batches.

Step-by-step thought process: 1. State the problem: one huge UPDATE holds row locks, bloats undo/WAL, and blocks readers (in MySQL InnoDB) or writers. 2. Show the chunking pattern with `WHERE id BETWEEN ? AND ? AND status <> 'target_value'` so the loop is idempotent and re-runnable. 3. Mention pacing — a short `pg_sleep(0.05)` per batch lets replication catch up and reduces lag spikes. 4. Anticipate the follow-up about partitioned tables: for time-partitioned data, update partition-by-partition for natural batching.

Edge cases to mention: - Without batching, Postgres creates one new row version per updated row in a single transaction — table bloat plus huge autovacuum work after. - MySQL InnoDB row locks scale with rows touched; long UPDATE blocks all reads in REPEATABLE READ if uncommitted. - For an update derived from another table, `UPDATE ... FROM joined` (Postgres) or `UPDATE ... JOIN` (MySQL) is the right syntax — but still chunk it.

What NOT to say: - Do not run the unsplit UPDATE on production "because it will eventually finish." - Do not assume `LIMIT` works in UPDATE — Postgres does not allow it directly; use a subquery with `ctid` or `id IN (SELECT ... LIMIT 10000)`.

Common follow-up: "*Show me the loop in pseudocode.*" — `loop: UPDATE ... WHERE id > :last AND id <= :last + 10000; COMMIT; :last += 10000; until no rows affected.`

Q96 · TRUNCATE vs DELETE vs DROP

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Accenture, Wells Fargo · Topic: DDL

The question (interviewer's verbatim phrasing):

> What is the difference between TRUNCATE, DELETE and DROP?

The 30-second answer: DELETE removes rows one at a time, fully logged, fires triggers, can be rolled back, keeps the table structure. TRUNCATE removes all rows in a metadata operation — fast, minimal logging, resets identity, does not fire row triggers in most engines, also rollback-able in Postgres but not in MySQL. DROP removes the table itself plus all data and dependent objects.

Step-by-step thought process: 1. Lay out the three on the row-vs-table axis: DELETE = some/all rows, TRUNCATE = all rows fast, DROP = table itself. 2. Highlight transactional differences: TRUNCATE is transactional and rollback-able in Postgres and SQL Server; in MySQL it is a DDL that implicitly commits and cannot be rolled back. 3. Mention referential integrity: TRUNCATE fails if other tables have FK to it (unless CASCADE); DELETE respects ON DELETE rules. 4. Anticipate identity-column behaviour: TRUNCATE resets auto-increment in most engines; DELETE does not.

Edge cases to mention: - TRUNCATE bypasses row-level triggers; statement-level triggers may still fire. - DELETE on a 100M-row table writes 100M WAL records; TRUNCATE writes a single metadata change. - DROP is irreversible without backup; TRUNCATE inside a transaction in Postgres can be rolled back.

What NOT to say: - Do not say "TRUNCATE is just a fast DELETE" — they differ in logging, locking, and triggers. - Do not call DROP and TRUNCATE interchangeable — DROP destroys the schema object too.

Common follow-up: "*Is TRUNCATE always faster than DELETE?*" — For "all rows" yes; for "some rows" it does not apply.

Q97 · UPSERT Atomic Patterns

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Walmart Labs · Topic: Concurrency

The question (interviewer's verbatim phrasing):

> Write me an UPSERT — insert if not exists, update if it does — and tell me why a naive "SELECT then INSERT or UPDATE" is wrong.

The 30-second answer: Use the engine's atomic UPSERT: Postgres `INSERT ... ON CONFLICT (key) DO UPDATE`, MySQL `INSERT ... ON DUPLICATE KEY UPDATE`, SQL Server `MERGE` (with caveats). Naive "SELECT then INSERT or UPDATE" is a race: between the SELECT and the

INSERT, another transaction can insert the same key, and your INSERT fails with a unique-violation, or worse, you get duplicates.

Step-by-step thought process: 1. Walk through the race: T1 SELECT (no row) → T2 SELECT (no row) → T1 INSERT → T2 INSERT → unique violation, or both succeed if no unique constraint. 2. Show the atomic Postgres form: `INSERT ... VALUES (...) ON CONFLICT (user_id) DO UPDATE SET amount = EXCLUDED.amount`. 3. Note `EXCLUDED` is Postgres's name for the "would-have-been-inserted" row; lets the UPDATE clause reference incoming values cleanly. 4. Anticipate follow-up about `DO NOTHING` — useful for idempotent inserts where you do not care about updating existing rows.

Edge cases to mention: - Postgres ON CONFLICT requires a unique constraint or index to target — without it, the planner has no anchor. - MySQL ON DUPLICATE KEY consumes auto-increment values even on the UPDATE branch — gappy primary keys. - SQL Server MERGE has well-documented concurrency bugs; many teams write IF EXISTS UPDATE / ELSE INSERT inside a transaction with HOLDLOCK instead.

What NOT to say: - Do not write the naive SELECT+INSERT pattern even in single-user dev code — it conditions bad habits. - Do not say "UPSERT is a SQL standard" — it is not; each engine spells it differently.

Common follow-up: "*What happens on Postgres ON CONFLICT if two transactions race?*" — One wins the INSERT, the other sees the conflict and runs the UPDATE branch atomically. No application retry needed.

```
INSERT INTO user_wallets (user_id, balance)
VALUES (101, 500)
ON CONFLICT (user_id)
DO UPDATE SET balance = user_wallets.balance + EXCLUDED.balance;
```

Q98 · INSERT ... ON CONFLICT vs MERGE

Tags: Difficulty: Medium · Asked at: Goldman, Walmart Labs, JP Morgan, Wells Fargo · Topic: Concurrency

The question (interviewer's verbatim phrasing):

> Postgres 15 added MERGE. Should we replace ON CONFLICT with MERGE everywhere?

The 30-second answer: No. MERGE is for bulk row-by-row reconciliation (DW-style "merge source into target") with separate WHEN MATCHED / WHEN NOT MATCHED branches. ON CONFLICT is for single-row or VALUES-list UPSERTs against a unique constraint. They overlap but the idioms differ; for plain UPSERT, ON CONFLICT remains shorter and more performant.

Step-by-step thought process: 1. State the shapes: MERGE takes a source query and joins it to the target with USING; ON CONFLICT operates on the INSERT being attempted. 2. ON CONFLICT requires a unique constraint to target; MERGE just needs a join condition. 3. MERGE supports DELETE branches (`WHEN MATCHED THEN DELETE`); ON CONFLICT does not. 4. Anticipate the follow-up about MERGE concurrency — in some engines MERGE is not atomic against unique violations; ON CONFLICT in Postgres is.

Edge cases to mention: - MERGE in SQL Server has had several published concurrency anomalies; some teams ban it. - Postgres 15 MERGE does not (yet) honour ON CONFLICT semantics; concurrent inserts can still violate unique constraints. - For full-table sync from a staging table to a target, MERGE is cleaner than three separate statements.

What NOT to say: - Do not say "MERGE replaces all DML" — it is a niche statement, useful but not universal. - Do not assume MERGE is the SQL standard answer to UPSERT — it is the standard for merging, not for upserting concurrently.

Common follow-up: "*Show me a MERGE that syncs a daily report from staging to target.*" — Three branches: WHEN MATCHED THEN UPDATE, WHEN NOT MATCHED THEN INSERT, WHEN NOT MATCHED BY SOURCE THEN DELETE.

Q99 · ALTER TABLE Downtime Risks

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Swiggy, Walmart Labs, Flipkart · Topic: DDL

The question (interviewer's verbatim phrasing):

> You need to add a new column with a default value to a 200-million-row table on a live Postgres. Walk me through what could go wrong.

The 30-second answer: Pre-Postgres 11, `ALTER TABLE ... ADD COLUMN ... DEFAULT 'x'` rewrites every row and holds an ACCESS EXCLUSIVE lock — table is unreadable for hours. Postgres 11+ stores the default in metadata when it is a constant — instant operation. Type changes still rewrite the table. Always split: `ADD COLUMN` (instant), backfill in batches, then `SET DEFAULT` / `SET NOT NULL` separately.

Step-by-step thought process: 1. State the lock concern: ACCESS EXCLUSIVE blocks reads as well as writes. 2. Walk the safe pattern: (1) ADD COLUMN nullable with no default — instant in modern Postgres; (2) backfill in chunks; (3) add CHECK NOT VALID, then VALIDATE CONSTRAINT (also non-blocking); (4) optionally set the default for future inserts. 3. Mention type changes — `ALTER COLUMN TYPE int → bigint` rewrites the whole table; use the ADD-new-column + backfill + swap-and-drop pattern instead. 4. Anticipate the follow-up about MySQL — same pattern there but use `pt-online-schema-change` or `gh-ost`, since native ALTER blocks writes until 8.0 (and even then with caveats).

Edge cases to mention: - `SET NOT NULL` validates the whole table under ACCESS EXCLUSIVE in older Postgres; in 12+ a CHECK constraint NOT VALID then VALIDATE avoids the long lock. - Dropping a column is metadata-only (fast) but space is reclaimed only by VACUUM FULL or pg_repack. - Foreign-key ADD with NOT VALID + VALIDATE pattern avoids long locks too.

What NOT to say: - Do not run `ALTER TABLE ADD COLUMN ... DEFAULT '...'` blindly on a hot table without checking the Postgres version. - Do not assume MySQL is "safer" — older MySQL versions copy the table for many ALTERs.

Common follow-up: *"How would you change a column from INT to BIGINT online?"* — Add `id_new BIGINT`, backfill, dual-write via trigger, swap, drop old. Hours of work; needed for legacy 32-bit id exhaustion.

Q100 · CREATE INDEX CONCURRENTLY (Postgres)

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, Swiggy · Topic: DDL

The question (interviewer's verbatim phrasing):

> You need to add an index on a 50M-row Postgres table without blocking writes. How?

The 30-second answer: `CREATE INDEX CONCURRENTLY idx_x ON tbl (col)` — Postgres builds the index in two passes, holding only a brief lock at the start and end, allowing reads and writes throughout. Slower in wall-clock time but zero blocking.

Step-by-step thought process: 1. State the trade-off: CONCURRENTLY takes 2-3× longer because it must do two table scans and wait for in-flight transactions, but it does not block DML. 2. Cannot be run inside a transaction block, and on failure leaves an INVALID index — must DROP and retry. 3. Mention that primary keys cannot be created concurrently directly; create the unique index concurrently, then `ALTER TABLE ... ADD CONSTRAINT ... PRIMARY KEY USING INDEX`. 4. Anticipate the follow-up about REINDEX CONCURRENTLY (Postgres 12+) for rebuilding bloated indexes without downtime.

Edge cases to mention: - An INVALID index from a failed CONCURRENTLY build still consumes write overhead — drop it immediately. - CONCURRENTLY waits for old long-running transactions; one open transaction can stall the build for hours. - Replicas: physical replication carries the index build; logical replication needs the index built independently on each subscriber.

What NOT to say: - Do not run plain `CREATE INDEX` on a hot production table — it takes an ACCESS EXCLUSIVE lock. - Do not run CREATE INDEX CONCURRENTLY inside a migration tool's transactional block; most tools opt-out per-statement.

Common follow-up: "*What if CREATE INDEX CONCURRENTLY fails mid-build?*" — Check `pg_index.indisvalid`; drop the invalid index, fix the cause (long-running transaction, disk-full), retry.

Q101 · Online Schema Change in MySQL — pt-osc and gh-ost

Tags: Difficulty: Hard · Asked at: Flipkart, Swiggy, Walmart Labs, Razorpay · Topic: DDL

The question (interviewer's verbatim phrasing):

> You're on MySQL 5.7 and need to add a column on a 300M-row InnoDB table. Native ALTER will lock for 6 hours. What tools do you reach for?

The 30-second answer: `pt-online-schema-change` (Percona) or `gh-ost` (GitHub). Both create a shadow table with the new schema, copy rows in chunks, and keep the original in sync — pt-osc uses triggers, gh-ost reads the binlog. At the end they swap the tables atomically.

Step-by-step thought process: 1. Name both tools and their core trick: shadow table + sync mechanism + atomic swap via RENAME. 2. Walk the trade-offs — pt-osc uses INSERT/UPDATE/DELETE triggers on the original, doubling write cost during the copy; gh-ost is binlog-based and has less write amplification but needs binlog access. 3. Both throttle based on replication lag — they pause copying if replicas fall behind. 4. Anticipate the follow-up about foreign keys — pt-osc has limited FK support; gh-ost does not handle FKs at all, common reason FKs are avoided at scale.

Edge cases to mention: - Both need a unique key on the original table to chunk safely. - During the swap, there is a brief metadata lock; long-running queries can stall the swap and time it out. - Aurora/cloud-managed MySQL may have its own preferred tool (Aurora Fast DDL) — check the docs.

What NOT to say: - Do not say "just use ALGORITHM=INPLACE" — many alters still copy under the hood, and the lock behaviour varies by alter type. - Do not run pt-osc on a table without testing on a staging copy first.

Common follow-up: "*Why do startups avoid foreign keys at scale?*" — Online schema-change tools struggle with FKs; replication, partitioning, and sharding all get easier without FK enforcement.

Q102 · VACUUM in Postgres

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman · Topic: Postgres operations

The question (interviewer's verbatim phrasing):

> What does VACUUM do in Postgres, and why does autovacuum matter?

The 30-second answer: Postgres MVCC creates a new row version on every UPDATE/DELETE; the old versions stay until VACUUM marks them reusable. Without vacuum the table bloats — disk usage grows, scans get slower, indexes get worse. Autovacuum runs in the background based on dead-tuple thresholds.

Step-by-step thought process: 1. Explain MVCC briefly: each row has `xmin` and `xmax`; an UPDATE makes a new row visible to new transactions and marks the old one dead when no old transaction needs it. 2. VACUUM (plain) reclaims dead tuples within the file; does not shrink the file. VACUUM FULL rewrites the table and shrinks it — but takes ACCESS EXCLUSIVE lock. 3. Autovacuum triggers based on `autovacuum_vacuum_scale_factor` (default 0.2 = 20% dead tuples) plus `autovacuum_vacuum_threshold`. 4. Anticipate the follow-up about hot tables: large UPDATE-heavy tables need tuned thresholds (scale_factor 0.05 or lower) or autovacuum never catches up.

Edge cases to mention: - Long-running transactions block VACUUM from cleaning dead tuples — the dead tuples are still "potentially visible" to that snapshot. One forgotten `BEGIN` in a console session causes bloat. - Wraparound: transaction IDs are 32-bit; if VACUUM is far behind, Postgres goes into emergency anti-wraparound mode and may shut down to protect data. - `pg_repack` rebuilds tables online without VACUUM FULL's lock.

What NOT to say: - Do not say "disable autovacuum" — almost always wrong outside of bulk load windows. - Do not run `VACUUM FULL` on production unless you have a maintenance window — it locks the table.

Common follow-up: "*How would you detect a bloated table?*" — `pgstattuple` extension or queries against `pg_stat_user_tables` for `n_dead_tup` and `n_live_tup`.

Q103 · Connection Pooling with PgBouncer

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Walmart Labs · Topic: Operations

The question (interviewer's verbatim phrasing):

> Your Postgres has `max_connections = 200`. Your app cluster has 50 pods, each with a connection pool of 20. Math says you're going to break. How do you fix this?

The 30-second answer: 50 × 20 = 1000 desired connections vs 200 allowed — you need a connection pooler. Put PgBouncer in front of Postgres in transaction-pooling mode; PgBouncer multiplexes thousands of client connections onto a smaller pool of server connections.

Step-by-step thought process: 1. Explain the math problem: each Postgres connection is a backend process eating ~10MB RAM and a slot; 1000 connections needs 10GB RAM and a planner that scales poorly past a few hundred. 2. PgBouncer modes: session pooling (one server conn per client conn — no win), transaction pooling (server conn returned to pool at COMMIT — big win, but no session-scoped features like prepared statements pre-PG 14), statement pooling (rare). 3. Mention `proxysql` as the MySQL equivalent. 4. Anticipate the follow-up about prepared-statement breakage: in transaction mode, server-side prepared statements created on one connection may not exist on the next — apps need protocol-level prepared statements (Postgres 14+) or unnamed prepared statements.

Edge cases to mention: - LISTEN/NOTIFY, advisory locks, temporary tables — all session-scoped, broken under transaction pooling. - PgBouncer is single-threaded; on very high QPS you run multiple PgBouncer processes with SO_REUSEPORT. - Connection limits in Aurora and Cloud SQL — they bundle a pooler (RDS Proxy, Cloud SQL Auth Proxy) but you may still want PgBouncer for tighter control.

What NOT to say: - Do not raise `max_connections` to 2000 — Postgres performance falls off a cliff well before that. - Do not say "pooling fixes slow queries" — it does not; it fixes connection scarcity.

Common follow-up: "*How do you size the PgBouncer pool to Postgres?*" — Roughly: `(cores × 2) + spindles` for OLTP workload, scaled by query mix.

Q104 · Long-Running Transactions and Statement Timeouts

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, Walmart Labs · Topic: Operations

The question (interviewer's verbatim phrasing):

> What is the harm in a single long-running transaction holding open for an hour, and how do you defend against it?

The 30-second answer: A long open transaction holds its snapshot, which blocks VACUUM from cleaning dead tuples (causing bloat), holds row locks (blocking other writers), and in some engines holds undo segments (causing replica lag). Defend with `statement_timeout`, `idle_in_transaction_session_timeout`, and monitoring `pg_stat_activity` for old `xact_start`.

Step-by-step thought process: 1. List the three harms: bloat, lock contention, replication lag. 2. Show the Postgres knobs: `statement_timeout` per session, `idle_in_transaction_session_timeout` for the "I opened a transaction and forgot to commit" case, `lock_timeout` for individual statements waiting on locks. 3. Mention enforcement: set these at the role level for app users so any one bad query is auto-killed; analytics roles get longer timeouts. 4. Anticipate the follow-up about debugging — query `pg_stat_activity` for `state = 'idle in transaction'` and old `xact_start` to find the culprit.

Edge cases to mention: - Application connection pools with long-held checked-out connections can leak idle-in-transaction sessions if the app crashes between BEGIN and COMMIT. - Replicas with `hot_standby_feedback=on` propagate the snapshot back to the primary, so a long query on the replica also blocks VACUUM on the primary. - Killing a long transaction with `pg_terminate_backend` triggers rollback, which on a large UPDATE can itself take minutes.

What NOT to say: - Do not say "long transactions are always wrong" — analytical queries and bulk loads legitimately need them; the defence is segregating workloads, not banning them. - Do not rely on the application to "always commit" — defence in depth via timeouts is mandatory.

Common follow-up: "*What about MySQL — same concerns?*" — Same family of issues; `max_execution_time` hint and `innodb_lock_wait_timeout` are the equivalents.

```
-- Set defensive timeouts on the app role
ALTER ROLE app_user SET statement_timeout = '30s';
ALTER ROLE app_user SET idle_in_transaction_session_timeout = '5min';
ALTER ROLE app_user SET lock_timeout = '5s';
```

End of Tier A — Part 2. Next file: Tier B (Q105 onwards).

Q105 · Normalization — 1NF, 2NF, 3NF, BCNF Basics

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wipro, Accenture, Cognizant · Topic: Schema Design

The question (interviewer's verbatim phrasing):

> Walk me through 1NF, 2NF, 3NF and BCNF with one short example each. Where in real life do you actually stop?

The 30-second answer: 1NF removes repeating groups and forces atomic columns; 2NF removes partial dependencies on a composite key; 3NF removes transitive dependencies via non-key columns; BCNF is a stricter 3NF that handles the rare case where a non-key attribute determines a key attribute. Most OLTP shops stop at 3NF in production.

Step-by-step thought process: 1. Frame normalization as "one fact in one place" — the goal is to kill update anomalies, not to win a theory exam. 2. 1NF: a `customers` table storing `phone_numbers = '98xxx, 88xxx'` in one column violates 1NF. Split into a child `customer_phones` table. 3. 2NF: an `order_items` table with PK `(order_id, sku)` storing `product_name` — `product_name` depends only on `sku`, not the full PK. Move it to `products`. 4. 3NF: a `users` table with `pincode` and `city` — city depends on pincode, not the user. Move pincode-to-city into a `pincodes` table. 5. BCNF: rare in practice — comes up when a non-prime attribute functionally determines part of a candidate key. Mention it; don't dwell.

Edge cases to mention: - Lookup tables (states, currencies, GST rate slabs) sit at 3NF naturally — they're often the answer to "where do I put this dependent column." - Storing both `price_paise` and `price_rupees` in the same row violates 3NF as a derived column — keep one, compute the other. - Performance teams sometimes intentionally break 3NF in read-heavy reporting tables; that's denormalization, covered in the next question.

What NOT to say: - Do not claim BCNF is "always required" — most production schemas live at 3NF. - Do not list textbook definitions verbatim without a real-world example; interviewers immediately spot rote memorisation.

Common follow-up: *"Give me a column you'd deliberately leave in violation of 3NF."* — A snapshot column like `order_state_at_purchase` on the orders row, so the historical record doesn't change when the state table updates.

```
-- 3NF: pincode → city moved out of users
CREATE TABLE users (
  id BIGINT PRIMARY KEY,
  name TEXT,
  pincode CHAR(6) REFERENCES pincodes(pincode)
);
CREATE TABLE pincodes (
  pincode CHAR(6) PRIMARY KEY,
  city TEXT,
  state TEXT
);
```

Q106 · When to Denormalize on Purpose

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Cred, Walmart Labs · Topic: Schema Design

The question (interviewer's verbatim phrasing):

> Give me three situations where you would deliberately denormalize. What's the cost?

The 30-second answer: Denormalize when read latency outweighs the write/storage cost — typical cases are reporting marts, audit snapshots, and feed timelines where joining at read time blows your p99. The cost is duplicated data, write amplification, and the discipline of keeping copies in sync.

Step-by-step thought process: 1. Lead with the trade — every denormalization buys read speed and sells write simplicity. 2. Case 1: reporting marts. A `daily_seller_metrics` table pre-joins orders, payments, refunds and SKU info so analysts don't run a five-table join every refresh. 3. Case 2: audit snapshots. The orders table stores `customer_name_at_order` and `gst_rate_at_order` so a

2026 retro view of a 2024 order looks the same as it did then. 4. Case 3: high-traffic feeds. A `notifications_feed` table stores the rendered text instead of joining to template + user + event tables at every read. 5. Be ready to explain the sync mechanism — triggers, CDC, batch refresh, or application-level dual-write.

Edge cases to mention: - Denormalized columns drift; you need a reconciliation job that flags rows where the cached value differs from the source of truth. - Storage cost is not zero — duplicating a 200-byte address across 50 million order rows is 10 GB extra disk. - Denormalizing into a `JSONB` column to store the snapshot is often cleaner than ten frozen columns.

What NOT to say: - Do not say "denormalize for performance" without naming which workload — denormalizing hurts write throughput. - Do not denormalize speculatively; do it after you measure a real read bottleneck.

Common follow-up: "*How do you keep the denormalized copy in sync?*" — Either a trigger (cheap, immediate, hard to debug), a CDC stream feeding a worker (scalable, eventually consistent), or a nightly batch refresh (simple, lag-tolerant).

```
-- Snapshot pricing into orders so historical reports stay stable
CREATE TABLE orders (
  id BIGINT PRIMARY KEY,
  user_id BIGINT,
  sku TEXT,
  unit_price_paise INT NOT NULL,         -- snapshotted from products at purchase
  gst_rate_pct NUMERIC(4,2) NOT NULL,    -- snapshotted; products table may change
  placed_at TIMESTAMPTZ DEFAULT now()
);
```

Q107 · Soft Delete (`deleted_at`) vs Hard Delete

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Cred, Meesho, JP Morgan · Topic: Schema Design

The question (interviewer's verbatim phrasing):

> When would you soft-delete a row versus actually deleting it? What goes wrong with soft delete in production?

The 30-second answer: Soft delete sets a `deleted_at` timestamp and excludes the row in every read query; hard delete physically removes it. Use soft delete when you need recoverability, audit, or referenced rows from history. The gotcha is unique constraints, queries that forget to filter `deleted_at`, and storage bloat over time.

Step-by-step thought process: 1. Open with intent: soft delete preserves referential integrity for historical rows and lets you build "restore from trash" features. 2. Pattern: add `deleted_at TIMESTAMPTZ NULL` (NULL means live), wrap every read in `WHERE deleted_at IS NULL`, and never let app code forget that filter — use a view or ORM scope. 3. Unique constraint trap: a `UNIQUE(email)` on users blocks creating a new user with an email that belonged to a soft-deleted user. Fix with a partial index: `UNIQUE(email) WHERE deleted_at IS NULL` on Postgres. 4. For genuinely PII-sensitive data (DPDP Act, India's data protection law), soft delete is not enough — you need hard delete or column-level redaction. 5. Mention archival tables as the third option — move deleted rows to an `orders_archive` table to keep the live table small.

Edge cases to mention: - Foreign keys to soft-deleted rows still resolve; that's a feature for audit and a bug for the "is this user valid" check. - Indexes on soft-deleted rows still take space and slow down writes; partial indexes help. - Reports built on the live table silently drift if the deletion criteria change.

What NOT to say: - Do not say "soft delete is always better" — it complicates every query and inflates table size. - Do not soft-delete payment or KYC records when regulation requires hard purge after a defined retention window.

Common follow-up: "*How do you handle unique constraints across live and deleted rows?*" — Postgres partial unique index `WHERE deleted_at IS NULL`; in MySQL, generated columns or check at app layer; in SQL Server, filtered index `WHERE deleted_at IS NULL`.

```
ALTER TABLE users ADD COLUMN deleted_at TIMESTAMPTZ;
CREATE UNIQUE INDEX users_email_unique_live
  ON users(email) WHERE deleted_at IS NULL;  -- Postgres partial index

-- Read pattern
SELECT * FROM users WHERE deleted_at IS NULL AND id = $1;
```

Q108 · Audit Log Table Design

Tags: Difficulty: Medium · Asked at: JP Morgan, Wells Fargo, Goldman, Razorpay, AmEx · Topic: Schema Design

The question (interviewer's verbatim phrasing):

> Design an audit log for a payments app. Every change to an order row must be traceable to who changed what, when, and from where.

The 30-second answer: A separate `audit_log` table keyed by `(table_name, row_id, changed_at)` storing actor, action (INSERT/UPDATE/DELETE), old and new row snapshots (typically

as JSONB), and request context like IP and request ID. Write via trigger or via the app's repository layer — pick one and stick with it.

Step-by-step thought process: 1. State the goal: tamper-evident, append-only, queryable by row and by actor. 2. Schema: `id`, `actor_id`, `table_name`, `row_pk`, `action`, `old_row JSONB`, `new_row JSONB`, `changed_at`, `request_id`, `source_ip`. 3. Mechanism choice: triggers capture every write including admin scripts, but they hide logic from the codebase; app-layer audit captures business context like reason codes but misses out-of-band writes. Most teams use both, with triggers as the safety net. 4. Partitioning: audit grows fast — partition by month and detach old partitions to cold storage after 12-24 months. 5. Mention that for regulated payment systems, the audit table itself often has `INSERT`-only grants and no `UPDATE` or `DELETE` for the app role.

Edge cases to mention: - Storing full JSONB snapshots on every update bloats fast; store only the diff (changed columns) once the table is mature. - Triggers fire inside the same transaction — a failing audit insert rolls back the business change. That's usually correct, occasionally fatal. - Replication can lag the audit table — for "who changed this row in the last 10 seconds" reads, hit the primary, not the replica.

What NOT to say: - Do not say "we'll just log to a file" — files aren't queryable and don't join to user tables. - Do not put the audit columns inside the business table (like `last_modified_by`) and call it audit; that's only the latest state, not history.

Common follow-up: "*What if the actor is a background job, not a user?*" — Have a `system_actors` table with rows like `etl-payments-reconciliation`, and use those IDs so audit queries still group by actor cleanly.

```
CREATE TABLE audit_log (
  id BIGSERIAL PRIMARY KEY,
  table_name TEXT NOT NULL,
  row_pk TEXT NOT NULL,
  action CHAR(1) NOT NULL CHECK (action IN ('I','U','D')),
  actor_id BIGINT,
  request_id UUID,
  source_ip INET,
  old_row JSONB,
  new_row JSONB,
  changed_at TIMESTAMPTZ DEFAULT now()
) PARTITION BY RANGE (changed_at);
CREATE INDEX ON audit_log (table_name, row_pk, changed_at DESC);
```

Q109 · Multi-Tenant Schema — `tenant_id` Column vs Schema-per-Tenant

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Zoho, Freshworks, Postman · Topic: Schema Design

The question (interviewer's verbatim phrasing):

> You're building a B2B SaaS for 5000 Indian merchants. Do you put a `tenant_id` column on every table, run one schema per tenant, or run one database per tenant? Defend your pick.

The 30-second answer: For thousands of small-to-medium tenants, share a database with a `tenant_id` column on every table — cheap, easy to operate, simple migrations. Schema-per-tenant suits hundreds of larger isolated customers needing per-tenant tuning. Database-per-tenant is for enterprise where data residency or compliance forces full isolation.

Step-by-step thought process: 1. Start with the dimension you're optimising for: cost, isolation, blast radius, or migration speed. 2. Shared table + `tenant_id`: lowest cost, hardest isolation. Every query carries `WHERE tenant_id = $1`, every index is composite with `tenant_id` first, RLS (Row Level Security in Postgres) is the safety net. 3. Schema-per-tenant: one logical schema per merchant, identical DDL. Migration becomes "apply DDL across 500 schemas in a loop" — needs tooling. Connection pooling per schema is the operational headache. 4. Database-per-tenant: full isolation, restore-single-tenant is trivial, costs explode in connection count and ops overhead. Used by banks and healthcare. 5. Pick based on tenant count, average tenant size, and the strictness of the isolation requirement.

Edge cases to mention: - "Noisy neighbour" — one big tenant's heavy query blocks the shared-table pattern; mitigate with statement timeouts and per-tenant rate limits. - Backup and restore of a single tenant on shared-table requires `pg_dump` with a filter or logical export — non-trivial. - Cross-tenant analytics is trivial on shared-table, painful on schema/database-per-tenant.

What NOT to say: - Do not say "always isolate per database for security" — for a 5000-tenant SaaS that's a six-figure infra bill. - Do not skip the `tenant_id` filter at the application layer just because RLS is on; defense in depth.

Common follow-up: "*How would you enforce that no query ever forgets `tenant_id`?*" — Postgres Row Level Security (RLS) with a `current_setting('app.tenant_id')` policy, plus a code-review rule that every new table has an RLS policy.

```
-- Shared-table pattern with Postgres RLS
CREATE TABLE invoices (
  id BIGSERIAL PRIMARY KEY,
  tenant_id BIGINT NOT NULL,
  amount_paise BIGINT,
  ...
);
ALTER TABLE invoices ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON invoices
  USING (tenant_id = current_setting('app.tenant_id')::BIGINT);
```

Q110 · Surrogate vs Natural Primary Key

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro, Accenture · Topic: Schema Design

The question (interviewer's verbatim phrasing):

> Should you use a surrogate ID like `id BIGSERIAL` or a natural key like `gstin` or `pan_number` as the primary key?

The 30-second answer: Default to surrogate. Natural keys look elegant until the business rule changes and you discover the customer's PAN was wrong, or a GSTIN got cancelled and reissued. A surrogate is stable, narrow, and decouples your physical model from real-world identifiers.

Step-by-step thought process: 1. Open with the principle: a primary key should never change. Real-world identifiers do change. 2. Surrogate (`BIGSERIAL` , `UUID`): narrow, fast joins, immune to business-rule churn. Slight downside: meaningless on its own; you need a `UNIQUE(natural_key)` constraint alongside. 3. Natural key (`pan` , `gstin` , `aadhaar_hash`): saves a join sometimes, but every reference table now has 12-15 byte keys instead of 8-byte BIGINTs, and updates to the natural key cascade everywhere. 4. Composite natural key (`(order_id, sku)`): valid when the row truly is identified by the combination, but child tables now carry both columns as FK — wider indexes, wider joins. 5. Be ready for the curveball: "But PAN never changes." — Yes it does. Reissue happens. So does GSTIN cancellation and recreation.

Edge cases to mention: - Some teams use both: surrogate as PK, natural as UNIQUE — best of both worlds. - For lookup tables (country, currency, gst_rate_slab) a short natural code like `'INR'` as PK is fine. - Audit logs and analytics queries become harder when joining via long string natural keys — surrogate keys keep joins narrow.

What NOT to say: - Do not say "natural keys are obsolete" — they're correct for static reference tables. - Do not use Aadhaar or PAN as a primary key — privacy and regulatory risk; hash it if you must store it.

Common follow-up: "*What if I told you the natural key is genuinely immutable, like an internally-issued voucher code?*" — Then it's defensible, but I'd still add a surrogate for join width; the difference between BIGINT and VARCHAR(16) joins shows up in p99.

```
-- Surrogate PK + natural UNIQUE — the standard pattern
CREATE TABLE merchants (
  id BIGSERIAL PRIMARY KEY,
  gstin CHAR(15) UNIQUE NOT NULL,
  legal_name TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

Q111 · UUID vs Auto-Increment ID — Trade-offs

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, PhonePe, Meesho · Topic: Schema Design

The question (interviewer's verbatim phrasing):

> Which would you pick as a primary key: `BIGSERIAL`, `UUIDv4`, or something else? Why?

The 30-second answer: Auto-increment BIGINT is the fastest at writes and the narrowest at joins; UUIDv4 is global and shard-friendly but randomises index inserts and bloats B-trees. For most single-database apps, BIGINT wins; for distributed systems, microservices, or anywhere you generate IDs client-side, prefer a sortable UUID variant like UUIDv7 or ULID.

Step-by-step thought process: 1. Lead with the three dimensions: write performance, join width, and uniqueness scope. 2. `BIGSERIAL` / `BIGINT IDENTITY`: 8 bytes, sequential, perfect for B-tree append-only inserts. Cannot be generated outside the database. 3. `UUIDv4`: 16 bytes (or 36 if stored as text — never do that), random, can be generated anywhere, but B-tree index becomes a write hotspot of random pages. 4. Storage and cache: a random UUID PK doubles index size and slaughters page cache hit rate for hot tables. 5. Mention sortable UUIDs as the modern answer — covered in the next question.

Edge cases to mention: - Storing UUIDs as `VARCHAR(36)` instead of `UUID` (Postgres) or `BINARY(16)` (MySQL) inflates index size 4-5x. - Auto-increment leaks count information — "your invoice ID is 14823" tells your competitor your transaction volume. - In sharded systems, auto-increment requires central coordination or interleaved offsets, which is fragile.

What NOT to say: - Do not say "UUIDs are always better because they're global" — they cost real B-tree performance. - Do not store UUID as a string; use the native UUID type or binary representation.

Common follow-up: "*How bad is the UUIDv4 write penalty actually?*" — On hot append-only tables, 2-4x slower bulk inserts and roughly 30-50% more index pages in cache because random inserts hit cold pages. Switch to UUIDv7 and the penalty disappears.

```
-- Postgres: use native UUID type, not VARCHAR
CREATE TABLE events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  payload JSONB,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

Q112 · UUIDv7 / ULID / KSUID — Sortable IDs

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Swiggy, Hasura · Topic: Schema Design

The question (interviewer's verbatim phrasing):

> What's wrong with UUIDv4 for high-throughput tables, and what would you use instead?

The 30-second answer: UUIDv4 is fully random, which destroys B-tree locality on inserts. Sortable variants — UUIDv7, ULID, KSUID — encode a millisecond timestamp in the high bits so new IDs sort near each other and inserts stay sequential. You keep global uniqueness without the index hotspot.

Step-by-step thought process: 1. Frame the problem: random inserts in a B-tree mean every new row hits a different page; cache misses explode. 2. UUIDv7: first 48 bits are Unix timestamp in milliseconds, rest is random. Standardised in RFC 9562 (May 2024). 3. ULID: 48-bit timestamp + 80-bit randomness, base32 string representation (`01ARZ3NDEKTSV4RRFFQ69G5FAV`), human-readable. 4. KSUID (Segment): similar idea, 32-bit timestamp + 128-bit randomness, base62 string. 5. Practical pick today: UUIDv7 if your driver and database support it; ULID if you want the string form for URLs; KSUID for backward compatibility in older systems.

Edge cases to mention: - Sortable IDs leak creation order — an attacker can infer signup velocity. Add a small random delay or use opaque public IDs separately. - Postgres added `uuidv7()` natively in version 18 (Sep 2025); on older versions use the `pg_uuidv7` extension or generate in the app. - Sortable does not mean monotonic across machines — clock skew between API servers can produce slightly out-of-order IDs.

What NOT to say: - Do not say "ULID is faster than UUID" — they're the same size; ULID is just better-encoded for sorting. - Do not mix UUIDv4 and UUIDv7 in the same table; you lose the locality benefit.

Common follow-up: *"My ORM doesn't know about UUIDv7. What do I do?"* — Generate the UUID in the application using a library (`uuid-v7` in Node, `uuid6` in Python, `uuid` in Java 21+), pass it as a regular UUID; the database doesn't care about the version bits.

```
-- Postgres 18+ native support
CREATE TABLE events (
  id UUID PRIMARY KEY DEFAULT uuidv7(),
  payload JSONB
);

-- Older Postgres: pg_uuidv7 extension
CREATE EXTENSION pg_uuidv7;
ALTER TABLE events ALTER COLUMN id SET DEFAULT uuid_generate_v7();
```

Q113 · Foreign Key Constraints — ON DELETE CASCADE Pitfalls

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Cognizant, Razorpay, JP Morgan · Topic: Schema Design

The question (interviewer's verbatim phrasing):

> Walk me through the options on `ON DELETE` and `ON UPDATE` for a foreign key. When is CASCADE actually dangerous?

The 30-second answer: The four common options are `RESTRICT` (block the delete), `CASCADE` (delete children too), `SET NULL` (null out the child FK), and `SET DEFAULT` (assign a default value). CASCADE is dangerous because one DELETE on a parent can silently wipe millions of related rows, including ones you wanted to keep for audit or reporting.

Step-by-step thought process: 1. Start with the four options and the default — Postgres and MySQL both default to `NO ACTION` / `RESTRICT`. 2. CASCADE is correct for tightly-coupled aggregates: an `order_items` row has no meaning without its `order`. 3. CASCADE is wrong for loosely-coupled data: deleting a `user` should not auto-delete their `orders`, `invoices`, and `payments` — those are regulatory records. 4. SET NULL needs the FK column to be nullable; otherwise it errors out. 5. Always pair FK design with a soft-delete strategy decision (Q107) — they interact.

Edge cases to mention: - CASCADE on a deep tree (parent → child → grandchild) can hold long locks and time out under load. - CASCADE on a table that triggers further CDC or audit can produce surprise downstream effects — the deletion ripples to Kafka. - Some teams disable FKs entirely in OLTP for write throughput and validate at the application layer; defensible but risky.

What NOT to say: - Do not say "FKs are bad for performance, drop them" without measuring; modern engines handle them fine for most workloads. - Do not enable CASCADE on user-facing tables without a code review — junior engineers add CASCADE without realising the blast radius.

Common follow-up: "*What happens if you DELETE a user with millions of dependent orders and CASCADE is on?*" — Long-held row locks across all child tables, possible lock escalation, replication lag, and a furious DBA. Do it in batches with a soft-delete flag instead.

```
CREATE TABLE orders (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT REFERENCES users(id) ON DELETE RESTRICT,  -- keep history
  total_paise BIGINT
);
CREATE TABLE order_items (
  id BIGSERIAL PRIMARY KEY,
  order_id BIGINT REFERENCES orders(id) ON DELETE CASCADE, -- delete with parent
  sku TEXT,
  qty INT
);
```

Q114 · Composite Primary Key vs Surrogate ID

Tags: Difficulty: Medium · Asked at: Walmart Labs, JP Morgan, Wells Fargo, Flipkart, Meesho · Topic: Schema Design

The question (interviewer's verbatim phrasing):

> Junction tables like `cart_items(cart_id, sku)` — would you keep that composite PK or add a surrogate `id` ?

The 30-second answer: Keep the composite PK when nothing else references the row and you want the duplicate-prevention guarantee for free. Add a surrogate ID when child tables reference this row (the composite gets dragged into every FK column) or when an ORM struggles with composite keys.

Step-by-step thought process: 1. Restate the trade: composite PK gives you uniqueness enforcement and a natural cluster, surrogate gives you a narrow stable handle. 2. Pure many-to-many junction with no children (cart items, role-permission map): composite PK is correct. 3. Junction with attributes the rest of the system references (`order_items.id` referenced by `refund_items`): add surrogate, keep composite as UNIQUE. 4. ORM concerns: Rails, Django, Hibernate all handle composite keys but with friction; for productivity-first stacks, surrogate is the boring-correct default. 5. Indexing tip: with a composite PK, choose the column order matching your dominant query — `(cart_id, sku)` for "show me one cart's items", `(sku, cart_id)` for "show carts containing this SKU".

Edge cases to mention: - A composite PK is a clustered index in InnoDB and SQL Server — picks the physical row order; choose wisely. - Composite PK with a wide first column (TEXT) wastes B-tree space; numeric-first is preferable. - Replication and CDC tools sometimes assume a single-column PK; check before relying on composite.

What NOT to say: - Do not say "always use surrogate IDs" — composite PKs are perfectly valid and free. - Do not pile on extra columns in the PK "just in case" — every PK column widens every FK and every index.

Common follow-up: "*What's the rule of thumb you use in real projects?*" — Composite for pure junctions; surrogate the moment any child table needs to reference the row. Default to surrogate for anything growing past a million rows.

```
-- Composite PK on a pure junction
CREATE TABLE user_roles (
  user_id BIGINT REFERENCES users(id) ON DELETE CASCADE,
  role_id BIGINT REFERENCES roles(id) ON DELETE CASCADE,
  granted_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (user_id, role_id)
);
```

Q115 · Star Schema Basics

Tags: Difficulty: Medium · Asked at: Walmart Labs, Flipkart, JP Morgan, Target, AmEx · Topic: Data Modeling

The question (interviewer's verbatim phrasing):

> What is a star schema and when would you use one over a normalized OLTP schema?

The 30-second answer: A star schema is one wide fact table at the centre (each row = one event with measures) surrounded by short dimension tables (customer, product, date, store) joined by FK. It's the standard for analytical workloads because the joins are predictable and the aggregations are cheap.

Step-by-step thought process: 1. Open with the contrast: OLTP is normalised for write correctness, OLAP is denormalised for read speed. 2. Fact table: `fact_orders` with `order_date_key`, `customer_key`, `product_key`, `store_key`, plus measures (`qty`, `revenue_paise`, `discount_paise`). 3. Dimension tables: `dim_customer`, `dim_product`, `dim_store`, `dim_date` — short, wide, often denormalised within themselves. 4. Why analysts love it: every query is "select sum(measure) from fact join dim where dim.attribute = X group by dim.attribute" — the join graph is a star. 5. Grain matters: define the grain of the fact (one row per order line item? per order? per day per SKU?) before writing any DDL.

Edge cases to mention: - Fact tables get huge — partition by date_key, cluster by frequently filtered dimension keys. - "Junk dimensions" (small flag combinations like `is_first_order`, `is_promo`) get bundled into one dim to avoid 10 single-column dims. - Conformed dimensions — the same `dim_customer` used by `fact_orders` and `fact_support_tickets` — is the discipline that lets you join across marts.

What NOT to say: - Do not use a star schema for OLTP — every update would cascade dimension lookups. - Do not put measures in dimension tables; that breaks the star and confuses analysts.

Common follow-up: *"What's the grain you'd pick for an e-commerce sales fact?"* — One row per order line item: `(order_id, order_line_id)`. That's the lowest level analysts actually ask about; you can roll up to order or day from there.

```
CREATE TABLE fact_orders (
  order_date_key INT REFERENCES dim_date(date_key),
  customer_key BIGINT REFERENCES dim_customer(customer_key),
  product_key BIGINT REFERENCES dim_product(product_key),
  qty INT,
  revenue_paise BIGINT,
  discount_paise BIGINT
);
```

Q116 · Snowflake Schema vs Star — When Each

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target, Flipkart, JP Morgan, Goldman · Topic: Data Modeling

The question (interviewer's verbatim phrasing):

> Star vs snowflake schema — explain the difference and tell me which one you'd build today.

The 30-second answer: A star keeps each dimension as one flat table; a snowflake normalises dimensions into sub-tables (product → product_subcategory → product_category). Snowflake saves storage and keeps dimension hierarchies clean; star is simpler for analysts and faster to query. In 2026 most cloud warehouses default to star because storage is cheap and joins are expensive.

Step-by-step thought process: 1. Frame the trade: snowflake is normalised OLAP, star is denormalised OLAP. 2. Snowflake example: `dim_product` references `dim_subcategory` references `dim_category` references `dim_department`. Saves storage if attributes repeat thousands of times. 3. Star example: `dim_product` has `subcategory_name`, `category_name`, `department_name` as columns. Storage cost is the duplication; benefit is one join instead of four. 4. Modern recommendation: star on BigQuery, Snowflake (the warehouse), Redshift, Databricks — columnar storage compresses the repetition cheaply and join cost dominates. 5. Snowflake schema still wins when dimensions are massive (millions of products, deep hierarchy) and you genuinely query at different levels.

Edge cases to mention: - Slowly Changing Dimensions (Q117) interact awkwardly with snowflake — versioning a parent dimension cascades. - BI tools (Power BI, Looker, Metabase) handle star schemas more naturally; snowflake requires explicit join definitions per query. - Hybrid is common: most dimensions flat, one or two hierarchical ones snowflaked.

What NOT to say: - Do not say "snowflake schema is more normalized therefore better" — for analytical workloads denormalisation usually wins. - Do not confuse Snowflake the schema with Snowflake the data warehouse; clarify in the answer.

Common follow-up: "*How would you handle a 5-level product hierarchy in a star?*" — Flatten all five levels as columns in `dim_product` (`level_1_name`, `level_2_name`, ...), accept the redundancy; columnar compression handles it.

```
-- Star: dimension is flat
CREATE TABLE dim_product (
  product_key BIGINT PRIMARY KEY,
  sku TEXT,
  product_name TEXT,
  subcategory_name TEXT,  -- repeated
  category_name TEXT,     -- repeated
  department_name TEXT    -- repeated
);
```

Q117 · Slowly Changing Dimensions — SCD Type 1 vs Type 2

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target, JP Morgan, Flipkart, AmEx · Topic: Data Modeling

The question (interviewer's verbatim phrasing):

> A customer's city changes. In your dimension table, do you overwrite the city, or keep history? Walk me through SCD Type 1 and Type 2.

The 30-second answer: Type 1 overwrites the old value — you lose history, queries always reflect "current". Type 2 inserts a new row with `effective_from`, `effective_to`, and `is_current` flag, preserving every version of the dimension and letting fact rows point to the version that was valid at event time.

Step-by-step thought process: 1. State the choice: Type 1 = current-truth only; Type 2 = full history. 2. Type 1 mechanics: `UPDATE dim_customer SET city = 'Bengaluru' WHERE customer_key = 42`. Done. Cheap, simple, lossy. 3. Type 2 mechanics: close the existing row (`effective_to = now()`, `is_current = false`), insert a new row with a new `customer_key` (surrogate), `effective_from = now()`, `is_current = true`. Fact rows store the surrogate that was current at the time. 4. Hybrid Type 3: keep one or two prior values as extra columns (`previous_city`); rarely used. 5. Pick: most attributes are Type 1 (name, email); attributes that drive analytics by time (city, segment, plan tier) are Type 2.

Edge cases to mention: - Type 2 explodes row count for high-churn dimensions — use a separate "history" dimension if churn is daily. - Fact tables must reference the surrogate key, not the natural key, otherwise the Type 2 versioning is wasted. - Late-arriving fact rows (an event from yesterday arriving today) must look up the dimension version valid at event time, not the current one.

What NOT to say: - Do not use Type 2 for every column — your dimension becomes unmanageable. - Do not store both Type 1 and Type 2 columns in the same row without clear naming; analysts will misuse them.

Common follow-up: "*How do you implement Type 2 in dbt?*" — `dbt_utils` and `dbt_snapshot` provide a `dbt snapshot` block that handles `effective_from`, `effective_to`, and the surrogate generation automatically.

```
-- SCD Type 2: one row per version
CREATE TABLE dim_customer (
  customer_key BIGSERIAL PRIMARY KEY,
  customer_id BIGINT NOT NULL,       -- natural key, repeats across versions
  name TEXT,
  city TEXT,
  effective_from DATE,
  effective_to DATE,
  is_current BOOLEAN
);
```

Q118 · Postgres `jsonb` Basics — vs `json` Text

Tags: Difficulty: Medium · Asked at: Razorpay, Hasura, Postman, Cred, Swiggy · Topic: Dialect — Postgres

The question (interviewer's verbatim phrasing):

> What's the difference between `json` and `jsonb` in Postgres, and when would you choose `jsonb`?

The 30-second answer: `json` stores the exact text you sent (whitespace, key order preserved, duplicate keys allowed); `jsonb` parses, decomposes, and stores a binary tree. `jsonb` is slower to insert, much faster to read and query, supports GIN indexing, and is what 99% of new Postgres apps should use.

Step-by-step thought process: 1. Start with the storage difference: `json` is just text with a validity check; `jsonb` is a parsed binary form. 2. Read performance: `jsonb` accesses a field in O(log n); `json` re-parses the whole document. 3. Operators: `->`, `->>`, `#>`, `#>>`, `@>`, `?` all work on `jsonb` with indexing support; `json` has them too but no GIN. 4. Use `json` only when the original byte representation matters (signed JWT payload, raw audit dump). 5. Indexing: `CREATE INDEX ON tbl USING gin (jsonb_col)` for membership and containment queries; `CREATE INDEX ON tbl ((jsonb_col->>'key'))` for specific field equality.

Edge cases to mention: - `jsonb` strips whitespace and reorders keys — diff tools on raw rows will look noisy. - `jsonb` is large; if you're storing 50 KB documents, consider a real table instead. - `jsonb_path_ops` GIN index variant is smaller and faster for containment queries (`@>`) only.

What NOT to say: - Do not say "JSON is faster in Postgres than relational columns" — only for the right workload; flat structured queries are still faster on real columns. - Do not store everything in one giant `jsonb` blob to "be flexible" — you lose constraints, types, and indexing benefits.

Common follow-up: *"Show me a GIN index and a query that uses it."* — `CREATE INDEX idx_meta ON events USING gin (metadata jsonb_path_ops);` then `SELECT * FROM events WHERE metadata @> '{"channel":"upi"}';`

```
CREATE TABLE webhook_events (
  id BIGSERIAL PRIMARY KEY,
  payload JSONB NOT NULL,
  received_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_payload_gin ON webhook_events USING gin (payload jsonb_path_ops);

SELECT id, payload->>'event_type' AS event_type
FROM webhook_events
WHERE payload @> '{"merchant_id": 482, "status": "captured"}';
```

Q119 · MySQL JSON Type Basics

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Flipkart, Swiggy, Razorpay · Topic: Dialect — MySQL

The question (interviewer's verbatim phrasing):

> How does MySQL's `JSON` type compare to Postgres `jsonb`, and what's the indexing story?

The 30-second answer: MySQL `JSON` (added in 5.7) stores a binary tree similar to Postgres `jsonb`, supports operators like `->` and `->>` (alias `JSON_EXTRACT` and `JSON_UNQUOTE`), but cannot be indexed directly — you index a generated column or use a multi-valued index for array fields.

Step-by-step thought process: 1. State the parity: MySQL `JSON` is parsed and binary, like `jsonb`; one type, no `json` text equivalent in MySQL. 2. Querying: `SELECT data->>'$.merchant_id' FROM events;` or the explicit `JSON_EXTRACT(data, '$.merchant_id')`. 3. Indexing scalar fields: add a generated column and index it — `ALTER TABLE events ADD merchant_id INT AS (data->>'$.merchant_id') VIRTUAL, ADD INDEX (merchant_id);`. 4. Indexing array fields (MySQL 8.0.17+): multi-valued indexes — `CREATE INDEX idx_tags ON events ((CAST(data->'$.tags' AS CHAR(50) ARRAY)));`. 5. Validation: `CHECK (JSON_VALID(data))` is implicit on `JSON` columns; you can add stricter `CHECK (JSON_SCHEMA_VALID(...))` in MySQL 8.0.17+.

Edge cases to mention: - VIRTUAL generated columns are computed on read and indexed; STORED is materialised, larger, faster on read. - MySQL `JSON` does not preserve key order; do not rely on it for signed payload comparison. - Updating a single key inside a large JSON blob rewrites the whole document — write amplification.

What NOT to say: - Do not say "MySQL JSON is the same as Postgres jsonb" — indexing model differs significantly. - Do not skip `JSON_VALID` mention — interviewers love that you know it's implicit.

Common follow-up: "*How would you index a JSON array of tags for fast `WHERE 'gst-refund' IN tags` queries?*" — Multi-valued index on the array path; one of MySQL 8's most useful additions.

```
CREATE TABLE webhook_events (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  payload JSON NOT NULL,
  merchant_id INT AS (payload->>'$.merchant_id') VIRTUAL,
  INDEX idx_merchant (merchant_id)
);
```

Q120 · SQL Server JSON Support — `JSON_VALUE` / `OPENJSON`

Tags: Difficulty: Medium · Asked at: Cognizant, Accenture, TCS, Wells Fargo, AmEx · Topic: Dialect — SQL Server

The question (interviewer's verbatim phrasing):

> SQL Server doesn't have a JSON type. How do you query JSON in it?

The 30-second answer: Until SQL Server 2025, JSON is stored as `NVARCHAR` and queried with built-in functions: `JSON_VALUE` for scalars, `JSON_QUERY` for sub-objects, `OPENJSON` for shredding to rows, and `ISJSON` to validate. Index a computed `PERSISTED` column to query JSON fast.

Step-by-step thought process: 1. Open with the storage reality: NVARCHAR(MAX) with `CHECK (ISJSON(col) = 1)` to enforce validity. 2. `JSON_VALUE(col, '$.key')` returns a scalar; `JSON_QUERY` returns a sub-object or array; `OPENJSON` returns a relational rowset. 3. Indexing: `ALTER TABLE events ADD merchant_id AS JSON_VALUE(payload, '$.merchant_id') PERSISTED; CREATE INDEX idx_merchant ON events(merchant_id);`. 4. SQL Server 2025 (in preview) adds a native `JSON` type — call it out to show you're current. 5. `OPENJSON` with `WITH` clause is the pattern for shredding webhook arrays into rows for joins.

Edge cases to mention: - Without a JSON type, validation is opt-in via `ISJSON` — easy to forget on legacy tables. - `JSON_VALUE` truncates results longer than 4000 characters silently; use

`JSON_QUERY` or cast carefully. - `OPENJSON` is set-based and parallelisable; iterating with cursors over JSON arrays is anti-pattern.

What NOT to say: - Do not say "SQL Server can't handle JSON" — it's been functional since 2016, well-indexed via computed columns. - Do not use string-parsing or LIKE on JSON columns; use the dedicated functions.

Common follow-up: *"Shred a JSON array of items into rows."* — `OPENJSON(payload, '$.items') WITH (sku NVARCHAR(50) '$.sku', qty INT '$.qty')` — pure relational result.

```
ALTER TABLE webhook_events
  ADD merchant_id AS JSON_VALUE(payload, '$.merchant_id') PERSISTED;
CREATE INDEX idx_merchant ON webhook_events(merchant_id);

SELECT e.id, i.sku, i.qty
FROM webhook_events e
CROSS APPLY OPENJSON(e.payload, '$.items')
  WITH (sku NVARCHAR(50) '$.sku', qty INT '$.qty') i;
```

Q121 · Snowflake VARIANT Type

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target, JP Morgan, Goldman, Razorpay · Topic: Dialect — Snowflake

The question (interviewer's verbatim phrasing):

> Snowflake has a `VARIANT` type. What is it and when do you use it?

The 30-second answer: `VARIANT` is Snowflake's semi-structured column type — it holds JSON, Avro, Parquet, or XML payloads in a columnar format that Snowflake auto-shreds and statistically optimises. You query it with dot/bracket notation and it's the default landing zone for semi-structured event data.

Step-by-step thought process: 1. Start with the use case: loading JSON event streams from Kafka, webhooks, or API logs into Snowflake without defining a schema upfront. 2. Access: `payload:event_type::STRING`, `payload:items[0].sku::STRING` — colon for fields, bracket for arrays, double-colon for cast. 3. Snowflake auto-detects repeating sub-fields and stores them as separate columns under the hood — query performance approaches that of explicit columns. 4. Flatten with `LATERAL FLATTEN(input => payload:items)` to shred arrays into rows. 5. Mention `OBJECT` and `ARRAY` as the typed variants of VARIANT when you know the structure.

Edge cases to mention: - VARIANT cells are capped at 16 MB compressed; huge documents must be split or stored elsewhere. - `:` access returns VARIANT; cast (`::STRING`, `::NUMBER`) before

comparing to scalars, otherwise you get unexpected ordering. - Schema evolution is free — new keys appear automatically without DDL.

What NOT to say: - Do not say "VARIANT is just JSONB" — Snowflake's storage and pruning behaviour is fundamentally different (micro-partitions, columnar). - Do not load 10 MB documents per row and expect fast scans; the columnar pruning works best on smaller, repeated structures.

Common follow-up: "*How would you shred a JSON array of order items into rows?*" — `SELECT order_id, f.value:sku::STRING AS sku FROM orders, LATERAL FLATTEN(input => payload:items) f;`

```
-- Snowflake
CREATE TABLE webhook_events (
  id BIGINT,
  payload VARIANT,
  received_at TIMESTAMP_NTZ
);

SELECT
  id,
  payload:event_type::STRING AS event_type,
  payload:merchant_id::NUMBER AS merchant_id
FROM webhook_events
WHERE payload:status::STRING = 'captured';
```

Q122 · BigQuery `STRUCT` and `ARRAY` Columns

Tags: Difficulty: Medium · Asked at: Flipkart, Swiggy, Walmart Labs, Target, JP Morgan · Topic: Dialect — BigQuery

The question (interviewer's verbatim phrasing):

> BigQuery has nested and repeated columns — STRUCT and ARRAY. How would you use them in a real schema?

The 30-second answer: A STRUCT is a record of named fields; an ARRAY is an ordered list of values (often of STRUCTs). You use them to keep one-to-many relationships within a single row — `orders` with an `items ARRAY<STRUCT<sku STRING, qty INT64>>` column eliminates the order-items join and exploits BigQuery's columnar storage for cheap scans.

Step-by-step thought process: 1. Reframe BigQuery's model: it's columnar but supports nested structures natively — you don't have to normalise to flat tables. 2. STRUCT: `address STRUCT<line1 STRING, city STRING, pincode STRING>` — access as `address.city`. 3. ARRAY: `tags ARRAY<STRING>` — query with `UNNEST(tags)` for set membership; correlated

`EXISTS (SELECT 1 FROM UNNEST(tags) t WHERE t = 'gst')`. 4. ARRAY of STRUCT: the canonical pattern for child rows — `items ARRAY<STRUCT<sku STRING, qty INT64, price_paise INT64>>`. 5. Query pattern: `SELECT order_id, item.sku FROM orders, UNNEST(items) AS item`.

Edge cases to mention: - Nested data is cheaper to scan than a join, but updates to a single item still rewrite the whole row in BigQuery (storage is immutable, partition rewrites are coarse). - ARRAYs cannot contain NULL elements directly (BigQuery rule); wrap in a STRUCT if you need optionality per element. - BI tools sometimes don't render nested columns well; flatten in a view for analyst consumption.

What NOT to say: - Do not say "BigQuery is just SQL" — nested types are a first-class concept and ignoring them gives you slow, over-normalised tables. - Do not over-nest — three levels deep starts hurting readability and BI compatibility.

Common follow-up: "*How do you query the third item in every order?*" — `SELECT order_id, items[OFFSET(2)].sku FROM orders` (zero-indexed) — or `items[SAFE_OFFSET(2)].sku` to return NULL when fewer than 3 items.

```
-- BigQuery
CREATE TABLE orders (
  order_id INT64,
  customer_id INT64,
  items ARRAY<STRUCT<sku STRING, qty INT64, price_paise INT64>>,
  shipping_address STRUCT<line1 STRING, city STRING, pincode STRING>,
  placed_at TIMESTAMP
);

SELECT
  o.order_id,
  item.sku,
  item.qty * item.price_paise AS line_total_paise
FROM orders o, UNNEST(items) AS item;
```

Q123 · Time-Series Schema Design

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, PhonePe, Cred, Walmart Labs · Topic: Schema Design

The question (interviewer's verbatim phrasing):

> Design a table for storing minute-level metrics from 50,000 merchants for the last 90 days.

The 30-second answer: Wide, partitioned-by-time, narrow rows: `(merchant_id, metric_ts, metric_name, value)` partitioned by day or week, with a covering index `(merchant_id,`

`metric_ts)`. For higher cardinality consider TimescaleDB hypertables (Postgres) or BigQuery's `TIMESTAMP` partitioning + clustering by `merchant_id`.

Step-by-step thought process: 1. Estimate volume: 50k merchants × 1440 minutes × 90 days × 10 metrics ≈ 65 billion rows. That dictates partitioning from day one. 2. Schema choice: narrow `(entity, ts, metric, value)` for flexibility, or wide `(entity, ts, metric_a, metric_b, ...)` for query speed. 3. Partition by `metric_ts` daily — drop old partitions in O(1) instead of `DELETE WHERE ts < cutoff`. 4. Index: `(merchant_id, metric_ts DESC)` for the most common "last 24h for this merchant" query. 5. Mention TimescaleDB or InfluxDB / ClickHouse for very high write rates — purpose-built engines outperform generic Postgres at this scale.

Edge cases to mention: - Pre-aggregate to hourly and daily rollups via materialized views or continuous aggregates; minute-level for hot data, rolled up for historic. - Use `BIGINT` for value if it's a counter (cumulative bytes, requests); use `NUMERIC` for monetary amounts; never `FLOAT` for money. - Retention: combine partition drop with archival to S3/GCS as Parquet for cold storage.

What NOT to say: - Do not store metrics as one row per `(entity, ts)` with hundreds of nullable columns; that breaks down as metrics evolve. - Do not skip partitioning and try to "DELETE WHERE old" on a 65B row table; you'll deadlock the cluster.

Common follow-up: "*What would you change for Snowflake or BigQuery?*" — Drop the index, partition the table by `metric_ts`, cluster by `merchant_id`. Both warehouses prune partitions and use clustering for sub-partition pruning automatically.

```
-- Postgres with native RANGE partitioning
CREATE TABLE merchant_metrics (
  merchant_id BIGINT NOT NULL,
  metric_ts TIMESTAMPTZ NOT NULL,
  metric_name TEXT NOT NULL,
  value NUMERIC NOT NULL
) PARTITION BY RANGE (metric_ts);

CREATE TABLE merchant_metrics_2026_05
  PARTITION OF merchant_metrics
  FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');

CREATE INDEX ON merchant_metrics (merchant_id, metric_ts DESC);
```

Q124 · Hierarchical Data — Adjacency List vs Nested Set vs Closure Table

Tags: Difficulty: Medium · Asked at: Flipkart, Meesho, Swiggy, Walmart Labs, JP Morgan · Topic: Schema Design

The question (interviewer's verbatim phrasing):

> A product category tree — Electronics → Mobiles → Smartphones → 5G. How would you store it for fast subtree queries?

The 30-second answer: Three patterns: adjacency list (`parent_id` column — simplest, requires recursive CTE for subtree); nested set (`lft` , `rgt` integers — fast reads, painful writes); closure table (separate table with one row per ancestor-descendant pair — best balance for read-heavy hierarchies). Pick closure table for most modern apps.

Step-by-step thought process: 1. Adjacency list: `categories(id, parent_id, name)` . Read parent → trivial. Read entire subtree → recursive CTE. Write → easy. 2. Nested set: `categories(id, name, lft, rgt)` . Read subtree → `WHERE lft BETWEEN parent.lft AND parent.rgt` . Insert → renumber the entire tree. 3. Closure table: `category_paths(ancestor_id, descendant_id, depth)` . Read subtree → join, simple. Write → insert one row per ancestor. 4. Path enumeration: `path TEXT` like `'1/4/12/47'` . Decent for shallow trees; messy for deep ones. 5. Modern recommendation: adjacency + recursive CTE for shallow trees (≤5 levels), closure table for deep or read-heavy ones.

Edge cases to mention: - Recursive CTEs need depth limits to avoid runaway on accidental cycles. - Closure table grows O(n × avg_depth) — a 5-level tree with 100k leaves is half a million path rows. - Path enumeration breaks when nodes move; closure table handles moves cleanly with delete + reinsert.

What NOT to say: - Do not say "nested set is the academic answer" without acknowledging that writes are O(n). - Do not jump to a graph database without justification; relational handles trees fine for most product needs.

Common follow-up: "*How would you query 'all descendants of Electronics' in each pattern?*" — Adjacency: recursive CTE walking down `parent_id` . Nested set: `WHERE lft BETWEEN ? AND ?` . Closure: `JOIN category_paths cp ON cp.ancestor_id = (electronics_id)` .

```
-- Closure table pattern
CREATE TABLE categories (
  id BIGSERIAL PRIMARY KEY,
  name TEXT
);
CREATE TABLE category_paths (
  ancestor_id BIGINT REFERENCES categories(id),
  descendant_id BIGINT REFERENCES categories(id),
  depth INT,
  PRIMARY KEY (ancestor_id, descendant_id)
);

-- All descendants of Electronics (assume id = 1)
SELECT c.* FROM categories c
JOIN category_paths cp ON cp.descendant_id = c.id
WHERE cp.ancestor_id = 1 AND cp.depth > 0;
```

Q125 · Many-to-Many Junction Table

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro, Razorpay · Topic: Schema Design

The question (interviewer's verbatim phrasing):

> A user can have many roles; a role belongs to many users. Design the schema.

The 30-second answer: Three tables: `users`, `roles`, and a junction `user_roles(user_id, role_id, granted_at)` with composite PK `(user_id, role_id)` and two FKs. Add an index on `role_id` so "who has this role" queries are fast.

Step-by-step thought process: 1. State the standard pattern: M:N is always a third table; there's no other clean way in relational SQL. 2. Composite PK `(user_id, role_id)` guarantees a user can have a role only once. 3. The PK index serves `WHERE user_id = ?` queries (left-prefix); add a separate index `(role_id, user_id)` for `WHERE role_id = ?` queries. 4. Add attributes to the junction when the relationship has facts: `granted_at`, `granted_by`, `expires_at`. 5. Be ready to discuss soft-delete on the junction — usually unnecessary, but `revoked_at` is a common variant.

Edge cases to mention: - Junctions with attributes that themselves need children (an `enrollment` row with grades) deserve a surrogate ID — Q114. - Don't denormalize roles into a `roles_csv TEXT` column on users; you'll regret it the first time you query "who has role admin". - `ON DELETE CASCADE` on both FKs is usually correct here — deleting a user purges their role assignments.

What NOT to say: - Do not say "MN can be modelled with a JSON array" — possible, but you lose joins, constraints, and indexability. - Do not skip the composite PK; you'll allow duplicate (user, role) pairs and your auth check breaks.

Common follow-up: "*Query: list every user with their roles concatenated.*" — `SELECT u.name, STRING_AGG(r.name, ', ') FROM users u LEFT JOIN user_roles ur ON ur.user_id = u.id LEFT JOIN roles r ON r.id = ur.role_id GROUP BY u.id, u.name;` — Postgres syntax; MySQL uses `GROUP_CONCAT`.

```
CREATE TABLE user_roles (
  user_id BIGINT REFERENCES users(id) ON DELETE CASCADE,
  role_id BIGINT REFERENCES roles(id) ON DELETE CASCADE,
  granted_at TIMESTAMPTZ DEFAULT now(),
  granted_by BIGINT REFERENCES users(id),
  PRIMARY KEY (user_id, role_id)
);
CREATE INDEX ON user_roles (role_id, user_id);
```

Q126 · Polymorphic Association — The Anti-Pattern

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Hasura, Postman, Swiggy · Topic: Schema Design

The question (interviewer's verbatim phrasing):

> You have a `comments` table that should attach to either an `order`, a `support_ticket`, or an `invoice`. Some teams use `(commentable_type, commentable_id)` — what's wrong with that?

The 30-second answer: The polymorphic association `(commentable_type, commentable_id)` cannot be enforced by a foreign key — the database doesn't know which table `commentable_id` references. You lose referential integrity, cascading deletes, and the optimiser can't help with the join. Use one nullable FK column per target type, or a join table per target, instead.

Step-by-step thought process: 1. Lead with the integrity loss: an FK to a "type-and-id" pair is impossible in standard SQL; orphan rows pile up. 2. Alternative 1 — exclusive FKs: `order_id BIGINT NULL`, `support_ticket_id BIGINT NULL`, `invoice_id BIGINT NULL` with a CHECK constraint that exactly one is set. 3. Alternative 2 — separate junction tables: `order_comments`, `ticket_comments`, `invoice_comments`. More tables, but each is properly FK'd. 4. Alternative 3 — shared `commentables` table: every commentable entity also inserts into `commentables(id)`, and `comments.commentable_id` FKs to `commentables.id`. Most elegant, more setup. 5. ORM convenience (Rails `belongs_to :commentable, polymorphic: true`) is the reason this anti-pattern spreads — productivity, not correctness.

Edge cases to mention: - Polymorphic associations break easy joins — you can't `JOIN ON commentable_id` because the target table varies. - Indexes get awkward: a composite `(commentable_type, commentable_id)` works but is wider than needed. - Renaming a target table requires a data migration to update the `commentable_type` string.

What NOT to say: - Do not defend polymorphic association by saying "Rails does it" — Rails does many things, not all of them defensible at scale. - Do not introduce polymorphic association in a regulated domain (payments, compliance) — auditors hate it.

Common follow-up: "*Show me the CHECK constraint for the exclusive-FK pattern.*" — `CHECK ((order_id IS NOT NULL)::INT + (ticket_id IS NOT NULL)::INT + (invoice_id IS NOT NULL)::INT = 1)` — ensures exactly one parent.

```
CREATE TABLE comments (
  id BIGSERIAL PRIMARY KEY,
  body TEXT,
  order_id BIGINT REFERENCES orders(id),
  ticket_id BIGINT REFERENCES support_tickets(id),
  invoice_id BIGINT REFERENCES invoices(id),
  created_at TIMESTAMPTZ DEFAULT now(),
  CHECK (
    (order_id IS NOT NULL)::INT +
    (ticket_id IS NOT NULL)::INT +
    (invoice_id IS NOT NULL)::INT = 1
  )
);
```

Q127 · ETL Idempotency Basics

Tags: Difficulty: Medium · Asked at: Flipkart, Walmart Labs, JP Morgan, Razorpay, Swiggy · Topic: Data Engineering

The question (interviewer's verbatim phrasing):

> What does "idempotent ETL" mean? Why does Airflow re-running yesterday's task matter?

The 30-second answer: An idempotent ETL job produces the same output when re-run with the same input — no duplicates, no double counts, no leftover rows. You achieve it via deterministic destination keys, MERGE/UPSERT instead of INSERT, partition replace (overwrite-by-partition), and externalising any non-idempotent side effect.

Step-by-step thought process: 1. Frame the why: backfills, retries, late-arriving data, and accidental double triggers are constant — non-idempotent jobs corrupt the warehouse. 2. Pattern 1 — UPSERT on a deterministic business key: `INSERT ... ON CONFLICT (order_id) DO UPDATE` instead of plain INSERT. 3. Pattern 2 — partition replace: process today's partition into a staging table, then `DELETE FROM target WHERE date_key = today; INSERT INTO target SELECT * FROM staging;` in one transaction. 4. Pattern 3 — full snapshot: rebuild the entire mart table from sources every run; trivially idempotent, expensive. 5. Non-idempotent traps: sending emails, calling external APIs, generating sequential numbers. Externalise them or guard with a "processed" flag.

Edge cases to mention: - Truly idempotent requires a stable input — if source data mutates between runs, the output changes; that's a different problem. - Late-arriving data and idempotency interact: yesterday's job re-run today must include rows that arrived in between, not just the original set. - Transactions and idempotency: the partition-replace pattern only works if DELETE + INSERT are in a single transaction.

What NOT to say: - Do not say "idempotent means transactions" — those overlap but they're not the same. - Do not skip the "external side effects" point; that's where idempotency breaks in real systems.

Common follow-up: "*How does dbt handle idempotency?*" — `dbt run` rebuilds models from sources every run; for incremental models it uses a `unique_key` and merges; for snapshot models it uses SCD Type 2 logic — all idempotent by design.

```
-- Idempotent UPSERT pattern (Postgres ON CONFLICT)
INSERT INTO daily_merchant_revenue (date_key, merchant_id, revenue_paise)
SELECT date_key, merchant_id, SUM(amount_paise)
FROM staging_orders
GROUP BY date_key, merchant_id
ON CONFLICT (date_key, merchant_id) DO UPDATE
SET revenue_paise = EXCLUDED.revenue_paise;
```

Q128 · dbt Staging Layer Pattern — Source / Staging / Mart

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Flipkart, Walmart Labs, Meesho · Topic: Data Engineering

The question (interviewer's verbatim phrasing):

> Explain the source → staging → intermediate → mart layering in dbt. Why not just write one big query?

The 30-second answer: Each layer has one job — sources are raw landed data, staging cleans and renames per source, intermediate joins and reshapes, marts deliver business-ready tables. The layering keeps each model small, testable, and reusable; downstream models depend on contracts (column names and types) instead of raw source quirks.

Step-by-step thought process: 1. Source layer: declared in `sources.yml`, no transformations — just the raw tables loaded by Fivetran or your ingestion tool. `source('raw_payments', 'transactions')`. 2. Staging (`stg_payments__transactions`): one model per source table; renames columns to project standards (`txn_id` → `transaction_id`), casts types, no joins. 3. Intermediate (`int_payments_with_orders`): joins across staging models, applies business logic that's not yet final. 4. Mart (`fct_revenue_daily`, `dim_merchant`): final business tables consumed by BI and analysts. 5. Why: each layer is independently testable, swappable, and renames the upstream chaos exactly once. Refactoring a source affects only its staging file, not the entire DAG.

Edge cases to mention: - Some teams skip the intermediate layer for simple projects — acceptable until cross-source joins multiply. - Staging should be materialized as `view` (cheap, always fresh); marts

often `table` or `incremental` (expensive, refreshed on schedule). - Tests live at every layer — schema tests on staging (unique, not_null), data tests on marts (revenue > 0).

What NOT to say: - Do not say "dbt is just SQL with extras" — the layering and dependency DAG is the actual value. - Do not skip naming conventions; `stg_`, `int_`, `fct_`, `dim_` prefixes are the project's load-bearing convention.

Common follow-up: "*What goes in `dbt_project.yml` to materialise staging as view and marts as table?*" — Under `models:`, configure each folder: `staging: +materialized: view`, `marts: +materialized: table`. dbt applies it across all models in that folder.

```
-- stg_payments__transactions.sql
WITH source AS (SELECT * FROM {{ source('raw_payments', 'transactions') }})
SELECT
  txn_id       AS transaction_id,
  merchant_id::BIGINT AS merchant_id,
  amount_paise::BIGINT AS amount_paise,
  status,
  created_at::TIMESTAMPTZ AS created_at
FROM source;
```

Q129 · Materialized View Refresh Strategies

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, JP Morgan, Goldman, AmEx · Topic: Performance

The question (interviewer's verbatim phrasing):

> A dashboard runs a 20-second aggregation. You decide to materialize it. Walk me through refresh strategies — full vs incremental, concurrent vs blocking.

The 30-second answer: Three axes: scope (full rebuild vs incremental), concurrency (blocking vs concurrent / online), and trigger (scheduled vs on-demand vs CDC-driven). Pick incremental + concurrent + scheduled for most BI dashboards; full + blocking + nightly for small marts; CDC-driven for near-real-time tiles.

Step-by-step thought process: 1. Full refresh: re-run the underlying query, drop and replace. Simple, slow on big tables, locks readers unless you use a concurrent variant. 2. Incremental: only re-aggregate rows that changed since the last refresh (tracked via watermark column or CDC log). Fast, complex, must be idempotent. 3. Postgres: `REFRESH MATERIALIZED VIEW CONCURRENTLY` — requires a unique index on the view; uses MERGE-like swap; readers stay unblocked. 4. Snowflake: `MATERIALIZED VIEW` auto-refreshes when source changes; cost is metered. BigQuery has

`MATERIALIZED VIEW` with smart auto-refresh on aggregate queries. 5. dbt's incremental models implement this pattern explicitly: `unique_key` + `is_incremental()` filter on changed rows.

Edge cases to mention: - Postgres concurrent refresh requires a unique index on the matview; without it you only get the blocking form. - Incremental refresh on a join query is hard — you must determine which output rows are affected by each source change. - Staleness contract matters — analysts ask "how fresh is this dashboard"; document the refresh schedule and expose `refreshed_at` in the view.

What NOT to say: - Do not say "materialized views are free" — they cost storage, refresh CPU, and freshness lag. - Do not refresh a 100 GB matview every 5 minutes; it'll dominate cluster load.

Common follow-up: "*What if your matview is downstream of another matview?*" — Refresh order matters; use a dependency-aware orchestrator (dbt, Airflow, Dagster) instead of cron-chaining REFRESH commands.

```
-- Postgres concurrent refresh
CREATE MATERIALIZED VIEW mv_daily_revenue AS
SELECT date_trunc('day', placed_at) AS day, merchant_id, SUM(amount_paise) AS revenue_pa
FROM orders
GROUP BY 1, 2;

CREATE UNIQUE INDEX ON mv_daily_revenue (day, merchant_id);  -- required for CONCURRENTL
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_revenue;
```

Q130 · BigQuery Partition + Cluster Combo (and Postgres Partitioning Recap)

Tags: Difficulty: Medium · Asked at: Flipkart, Walmart Labs, Swiggy, JP Morgan, Target · Topic: Performance

The question (interviewer's verbatim phrasing):

> In BigQuery, when should you partition and cluster a table? In Postgres, how do you decide between RANGE, LIST and HASH partitioning, and what is partition pruning?

The 30-second answer: BigQuery: partition by the column most queries filter on (usually a date), cluster by 1-4 high-cardinality filter columns (merchant_id, country) — the planner skips partitions and uses clustering for sub-partition pruning. Postgres: RANGE for ordered keys (dates, IDs), LIST for discrete labels (country, tenant), HASH for even distribution when there's no natural range. Partition pruning is the optimiser eliminating partitions the query can't touch.

Step-by-step thought process: 1. BigQuery partitioning: one column per table, type DATE / TIMESTAMP / INTEGER. Filter with constants in WHERE so the planner can prune; correlated subqueries break pruning. 2. BigQuery clustering: up to 4 columns; rows within a partition are ordered by cluster

keys; queries filtering on cluster keys read fewer blocks. 3. Postgres RANGE: `PARTITION BY RANGE (created_at)` with monthly child partitions — drop a partition to expire data in O(1). 4. Postgres LIST: `PARTITION BY LIST (country_code)` — one partition per major country, plus DEFAULT for the rest. 5. Postgres HASH: `PARTITION BY HASH (user_id)` with 16 partitions — when you want even write distribution but no natural range. 6. Partition pruning: `EXPLAIN` shows `Partitions removed by Filter: 11 of 12` — proves the planner used the partition key.

Edge cases to mention: - Postgres partition pruning works only when the WHERE clause uses the partition key directly with constants or stable functions, not opaque expressions. - BigQuery requires partition filter on every query if `require_partition_filter` is set — a great safety against runaway scans. - Clustering quality decays over time as new rows arrive; BigQuery auto-recluster runs in the background but lags on heavy churn.

What NOT to say: - Do not partition tables under ~10 million rows — overhead exceeds benefit. - Do not over-cluster on too many columns; the leading column dominates pruning.

Common follow-up: "*Give me a BigQuery DDL that partitions and clusters.*" — Date partition + cluster on the next-most-common filters — see code below.

```
-- BigQuery: partition + cluster
CREATE TABLE analytics.orders (
  order_id INT64,
  merchant_id INT64,
  country_code STRING,
  amount_paise INT64,
  placed_at TIMESTAMP
)
PARTITION BY DATE(placed_at)
CLUSTER BY merchant_id, country_code
OPTIONS (require_partition_filter = TRUE);

-- Postgres: RANGE partition with pruning
CREATE TABLE orders (
  id BIGSERIAL,
  placed_at DATE NOT NULL,
  amount_paise BIGINT
) PARTITION BY RANGE (placed_at);
CREATE TABLE orders_2026_05 PARTITION OF orders
  FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');

EXPLAIN SELECT SUM(amount_paise) FROM orders
WHERE placed_at >= '2026-05-01' AND placed_at < '2026-05-15';
-- Look for: Partitions removed by Filter
```

Tier B — Asked in 25-50% of SQL interviews (70 Qs)

These are the depth-layer questions — they don't show up in every interview, but when they do, getting them right is the difference between an "okay" round and a "this candidate is senior" signal. Skim Tier B the week of your interview; focus harder if you're targeting senior or staff-level roles, or if the JD lists a specific area (Snowflake / Postgres internals / locking depth) in detail.

Q131 · EXPLAIN ANALYZE BUFFERS in Postgres

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, JP Morgan India, Goldman · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> A query is slow in production. You run `EXPLAIN (ANALYZE, BUFFERS)` and see shared hit and shared read numbers. Walk me through how you read that output to figure out whether the bottleneck is IO or compute.

The 30-second answer: `shared hit` is pages already in the Postgres buffer cache (RAM — cheap). `shared read` is pages fetched from the OS / disk (expensive). If `shared read` dominates and the per-row actual time is high, you are IO-bound; if `shared hit` is huge but the timing is still bad, it is compute (CPU spent on filters, joins, sorts).

Step-by-step thought process: 1. Lead with the IO-versus-compute split — that is exactly what the interviewer is fishing for. 2. Explain the units: BUFFERS counts 8 KB pages by default. Multiply by 8 KB to get bytes touched. 3. Walk through a typical Seq Scan node — high `shared read` plus low rows returned means a missing index or a filter that is too late. 4. Mention `shared dirtied` and `shared written` — these matter when the query also has to evict dirty pages (write amplification under memory pressure). 5. Be ready for the follow-up: "How do you isolate which node is the culprit?" — read bottom-up, find the node where actual time jumps disproportionately versus its children.

Edge cases to mention: - `EXPLAIN ANALYZE` actually executes the query — never run it on a destructive statement without wrapping in a rolled-back transaction. - The first run will show high `shared read`; the second run on the same data warms the cache and looks artificially fast. Always run twice and report the warm number. - Parallel workers report their own buffer numbers — the leader aggregates, but per-worker counts in `VERBOSE` mode help spot skew.

What NOT to say: - Do not equate `cost` from plain EXPLAIN with milliseconds — cost is an arbitrary planner unit. - Do not say "high buffers always means slow query" — a hash join that hits buffers cheaply can still be the right plan.

Common follow-up: "*What additional option helps when you suspect sort or hash spilling?*" — Add `SETTINGS` and turn on `track_io_timing` plus `log_temp_files`; temp files in the output prove a spill.

```
EXPLAIN (ANALYZE, BUFFERS, VERBOSE, SETTINGS)
SELECT u.id, SUM(o.amount)
FROM users u JOIN orders o ON o.user_id = u.id
WHERE u.city = 'Pune' GROUP BY u.id;
```

Q132 · Diagnosing a Plan Regression

Tags: Difficulty: Hard · Asked at: Walmart Labs, AmEx, Wells Fargo, Flipkart · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> A report query was fast yesterday and slow today. Same data volume, same schema. Walk me through how you confirm and fix a plan regression on SQL Server, then on Postgres.

The 30-second answer: On SQL Server, open Query Store, find the query, compare plans across the two windows, and either force the good plan or update statistics. On Postgres, use `pg_stat_statements` to confirm `mean_exec_time` jumped, then re-run `EXPLAIN` to see which node changed — usually a flipped join order or a switch from index scan to seq scan.

Step-by-step thought process: 1. Frame it as a three-step loop: confirm regression, identify the changed node, fix the root cause. 2. SQL Server: Query Store is the first stop — it stores plan history and runtime stats. The "Top Resource Consuming Queries" view shows the regression in one click. 3. Postgres: `pg_stat_statements` gives you the timing delta; you then re-run with `EXPLAIN` to compare against a captured plan. 4. The fix is rarely "force the plan" — that is a band-aid. Usually it is stale statistics, a parameter sniffing miss, or a sudden cardinality shift in a key column. 5. Be ready for: "When is plan forcing actually correct?" — when the optimizer is genuinely picking the worse of two known plans and you have no time to fix stats before the demo.

Edge cases to mention: - Autostats on SQL Server fires at a threshold; bulk loads can leave a table looking under-sampled. Manual `UPDATE STATISTICS` with `FULLSCAN` after big loads. - On Postgres, `ANALYZE` after a bulk insert is the equivalent — autovacuum may not fire fast enough. - Plan cache flushes (server restart, memory pressure) can recompile to a different plan if statistics changed in between.

What NOT to say: - Do not jump to "add an index" without first confirming statistics are current. - Do not blame the optimizer without producing two captured plans side by side.

Common follow-up: "*How do you capture a baseline plan you can diff against later?*" — SQL Server: Query Store auto-captures. Postgres: use `auto_explain` to log slow plans, or snapshot `EXPLAIN (FORMAT JSON)` output into a table.

```
-- Postgres: find queries whose mean time recently spiked
SELECT query, calls, mean_exec_time, stddev_exec_time
FROM pg_stat_statements
ORDER BY mean_exec_time DESC LIMIT 20;
```

Q133 · Extended Statistics for Correlated Columns

Tags: Difficulty: Hard · Asked at: Cred, Razorpay, Goldman, Target · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> You have a Postgres table with `city` and `pincode`. Filtering on both gives terrible row estimates and the planner picks a bad join order. Why and how do you fix it?

The 30-second answer: The planner assumes columns are independent, so it multiplies selectivities — but `city` and `pincode` are functionally dependent, so the real row count is much higher than the estimate. Create extended statistics with `CREATE STATISTICS ... (dependencies, ndistinct) ON city, pincode FROM addresses;` and run `ANALYZE` — the planner then uses joint selectivity.

Step-by-step thought process: 1. Start with the independence assumption — that is the root cause of almost every correlated-column estimate miss. 2. Explain the three kinds of extended stats: `dependencies` (functional dependencies — pincode determines city), `ndistinct` (joint distinct count for GROUP BY estimates), and `mcv` (multi-column most common values). 3. Show the create-and-analyze pattern, then re-run EXPLAIN — the row estimate should snap back to reality. 4. Mention this only exists from Postgres 10 onward; before that you used hints (`pg_hint_plan`) or rewrote the query. 5. Anticipate: "Does it have a maintenance cost?" — yes, ANALYZE has to gather joint stats, but it is amortised across many queries.

Edge cases to mention: - Extended stats are not used for join estimates yet (only single-table predicates) — a known limitation. - They are dropped if you drop the table or any covered column — schema changes invalidate them. - For very wide tables, listing 5+ columns in one statistics object slows ANALYZE noticeably; keep groups tight.

What NOT to say: - Do not say "the optimizer is broken." It is doing the textbook thing — you have to tell it about the correlation. - Do not confuse extended statistics with regular `ANALYZE` — they are an opt-in object you create explicitly.

Common follow-up: "*What is the SQL Server / MySQL equivalent?*" — SQL Server has multi-column statistics on indexes and you can create explicit `CREATE STATISTICS` objects. MySQL has no equivalent — you rewrite the query or use optimizer hints.

```
CREATE STATISTICS addr_stats (dependencies, ndistinct)
  ON city, pincode, state FROM addresses;
ANALYZE addresses;
```

Q134 · Histograms in MySQL 8.0+

Tags: Difficulty: Medium · Asked at: Swiggy, Meesho, Flipkart, Zerodha · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> MySQL 8 introduced histograms. When does the planner actually use them and when does it ignore them?

The 30-second answer: The planner uses histograms for range and equality predicates on non-indexed columns where index dives are unavailable. They are ignored if the column already has a usable index — index dives give a better estimate. They also help on skewed data where the default uniform-distribution assumption is wildly wrong.

Step-by-step thought process: 1. Lead with the use case: a non-indexed column with skewed values where the planner would otherwise assume uniform distribution. 2. Mention the two histogram types: `SINGLETON` (one bucket per value, when distinct count is small) and `EQUI-HEIGHT` (each bucket holds roughly the same row count, for high-cardinality columns). 3. Create with `ANALYZE TABLE ... UPDATE HISTOGRAM ON col WITH N BUCKETS;` — max 1024 buckets. 4. Explain the trade-off: histograms are not auto-maintained, so after large data changes you must refresh them manually. 5. Be ready for: "Why not just add an index?" — indexes have write cost; histograms are read-only metadata that costs nothing on inserts.

Edge cases to mention: - Histograms are not used for join predicates — only single-table filters. - They cannot be created on encrypted columns or generated columns with `JSON` extraction expressions in some 8.0 minor versions. - If you add or drop the column, the histogram is automatically removed; if data shifts, it goes stale silently.

What NOT to say: - Do not claim "histograms replace indexes." They do not — they only improve cardinality estimates. - Do not create 1024-bucket histograms on tiny tables; the metadata cost outweighs the gain.

Common follow-up: "*How would you know a histogram is being used in the plan?*" — `EXPLAIN ANALYZE` shows the estimated rows; toggle the histogram on and off and re-run to see the estimate change.

```
ANALYZE TABLE orders UPDATE HISTOGRAM ON status WITH 16 BUCKETS;
SELECT histogram FROM information_schema.column_statistics
  WHERE table_name='orders' AND column_name='status';
```

Q135 · Cardinality Estimation Failures

Tags: Difficulty: Hard · Asked at: JP Morgan India, AmEx, Goldman, Wells Fargo · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> What are the classic situations where cardinality estimation goes badly wrong, and what is the downstream consequence for the chosen plan?

The 30-second answer: Top failures: correlated predicates assumed independent, parameter values out of histogram range, expressions wrapping the column (`WHERE LOWER(city) = ...`), large LIKE patterns, and JOIN predicates over many-to-many relationships. Downstream the planner picks nested loop where it should hash-join, or vice versa — and runtime explodes by 100x.

Step-by-step thought process: 1. Group the failures into three families: independence assumption, opaque expressions, and out-of-histogram values. 2. Independence — covered by extended statistics (Q133). Opaque expressions — covered by expression indexes (Q153) or function-based statistics. Out-of-histogram — fixed by more buckets or by reshaping the parameter. 3. Walk the consequence path: bad estimate → wrong join algorithm → either nested-loop over millions of rows or hash that spills to disk. 4. Mention the "leading 1 estimate" trap — when the optimizer cannot estimate, it assumes 1 row, which makes nested-loop look free. 5. Be ready for: "How do you spot this in EXPLAIN?" — look for `actual rows` orders of magnitude larger than `estimated rows` .

Edge cases to mention: - Recursive CTEs and table-valued functions often have no useful stats — the planner assumes 1000 rows by default (Postgres), 100 (SQL Server before TVF interleaved exec). - Encrypted columns or columns with column-level encryption hide distributions from the planner. - Bind-peeking / parameter sniffing can lock in a bad estimate for an unusual first value (Q137).

What NOT to say: - Do not say "always use hints." Hints freeze the plan and break when data shifts. - Do not assume the planner has access to data outside the histogram — it usually does not.

Common follow-up: "*What is the cheapest way to validate the estimate is bad?*" — Run `EXPLAIN ANALYZE` and compare `estimated rows` against `actual rows` on each node. A 10x mismatch on a join input is the smoking gun.

```
EXPLAIN ANALYZE
SELECT * FROM orders WHERE LOWER(city) = 'pune' AND status = 'paid';
-- estimated 100, actual 240000 → opaque function on city kills the estimate
```

Q136 · Parameter Sniffing in SQL Server

Tags: Difficulty: Hard · Asked at: AmEx, JP Morgan India, Wells Fargo, Walmart Labs · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> A stored procedure is fast for one user and painfully slow for another, with the same code. Explain parameter sniffing and how you would fix it.

The 30-second answer: SQL Server compiles the plan using the first parameter values it sees ("sniffs") and caches it. If those values are atypical — a customer with 10 million rows when most have 100 — the cached plan is wrong for everyone else. Fix with `OPTION (RECOMPILE)`, `OPTION (OPTIMIZE FOR ...)`, local-variable trick, or Query Store plan forcing.

Step-by-step thought process: 1. Start with the mechanism: the optimizer needs concrete values to estimate, so it uses the first call's values. That plan stays in cache until evicted. 2. List the four fixes in increasing surgical-ness: - `OPTION (RECOMPILE)` — compile every call (CPU cost, but always-right plan). - `OPTION (OPTIMIZE FOR @p = 'typical')` — compile once for a hand-picked typical value. - Local variable assignment inside the proc — the optimizer cannot peek into local variables, so it uses the average density estimate instead. - Query Store plan forcing — pin the known-good plan. 3. Explain when each is appropriate: RECOMPILE for low-volume procs, OPTIMIZE FOR when you know the typical value, local-variable for legacy procs you cannot rewrite, plan forcing as a last resort. 4. Be ready for: "What is the Postgres equivalent?" — Postgres has plan-cache modes (`plan_cache_mode`) and generic vs custom plans for prepared statements.

Edge cases to mention: - Adaptive query processing (SQL Server 2017+) helps with skew via interleaved execution and memory grant feedback — partially mitigates the problem. - `OPTIMIZE FOR UNKNOWN` is now generally discouraged because it uses the density vector, which can be worse than sniffing for skewed data. - The bad plan can persist for hours unless the cache is cleared or stats change.

What NOT to say: - Do not say "always use RECOMPILE." On hot procs, compilation cost dwarfs execution cost. - Do not blame the user — the engine designed for the cached-plan model, not malicious input.

Common follow-up: "*How would you reproduce the bad plan in a lower environment?*" — Use `DBCC FREEPROCCACHE` then call the proc with the atypical parameter first; the plan caches under that value.

```
CREATE PROC sp_get_orders @cust_id INT AS
SELECT * FROM orders WHERE customer_id = @cust_id
OPTION (OPTIMIZE FOR (@cust_id = 12345));
```

Q137 · Plan Cache Pollution

Tags: Difficulty: Medium · Asked at: Walmart Labs, Cred, PhonePe, Razorpay · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> What is plan cache pollution and how does an ORM with bad parameterisation cause it?

The 30-second answer: Plan cache pollution is when thousands of nearly identical queries each get their own cached plan because they were sent as literal SQL instead of parameterised. Memory fills with one-off plans, useful plans get evicted, and recompilations spike. The classic culprit is an ORM concatenating filter values into the query string.

Step-by-step thought process: 1. Define it crisply: one query template, many cache entries because values are inlined. 2. Show the symptom — on SQL Server, `sys.dm_exec_cached_plans` shows thousands of `Adhoc` plans with `usecounts = 1`. 3. Explain the cost: cache memory pressure, more recompilations, worse cache hit rate for queries you actually care about. 4. The fix: enable `forced parameterisation` (SQL Server) or use `PREPARE`/parameterised statements consistently from the app side. 5. Postgres flavour: `pg_stat_statements` deduplicates by query template, but plan cache itself is per-session for prepared statements — pollution is less of a memory issue, more of a planning-time issue.

Edge cases to mention: - SQL Server's "optimize for ad hoc workloads" stores only a stub on first execution; on second execution it stores the full plan — cheap insurance against pollution. - Forced parameterisation can cause its own problems on queries that depend on literal values for partition elimination. - ORM IN-list expansion (`IN (1)`, `IN (1,2)`, `IN (1,2,3)`...) creates a new template per list length — use table-valued parameters or array binding to flatten.

What NOT to say: - Do not call all ad-hoc plans pollution — a few are fine. - Do not blanket-enable `forced parameterisation` without testing partition-elimination queries.

Common follow-up: "*How do you measure the actual impact?*" — Sum plan cache size in MB by `objtype = 'Adhoc'` versus `'Prepared'`. If Adhoc dominates, you have pollution.

```
SELECT cacheobjtype, objtype, COUNT(*), SUM(size_in_bytes)/1024/1024 AS mb
FROM sys.dm_exec_cached_plans
GROUP BY cacheobjtype, objtype;
```

Q138 · When the Planner Goes Parallel

Tags: Difficulty: Medium · Asked at: Goldman, Target, JP Morgan India, AmEx · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> What does the optimizer look at to decide whether to use parallel query, and what are the common reasons it refuses to parallelise something you expect it to?

The 30-second answer: The planner parallelises when the estimated cost crosses a threshold (`parallel_setup_cost` + `parallel_tuple_cost` on Postgres; `cost threshold for parallelism` on SQL Server). It refuses when the query uses non-parallel-safe functions, when the table is too small, when MAXDOP is 1, or when the operation is inherently serial (e.g., scalar UDFs without inlining).

Step-by-step thought process: 1. Lead with the cost threshold — that is the first knob. 2. List the disqualifiers: non-parallel-safe functions (Postgres `PARALLEL UNSAFE`), `SERIALIZABLE` isolation on Postgres, scalar UDFs on SQL Server before 2019, queries with CTEs that the optimizer cannot push, temp tables in some cases. 3. Mention worker count: `max_parallel_workers_per_gather` (Postgres), `MAXDOP` (SQL Server). Going wider than the row count is wasted setup. 4. Explain Gather node — workers produce tuples, leader merges. If Gather Merge is needed (to preserve order), there is extra coordination cost. 5. Be ready for: "How do I force parallelism for a test?" — Postgres: `SET force_parallel_mode = on` . SQL Server: `OPTION (USE HINT('ENABLE_PARALLEL_PLAN_PREFERENCE'))` .

Edge cases to mention: - Parallel hash join requires `work_mem` per worker — easy to OOM if you crank `max_parallel_workers_per_gather` without raising memory limits. - Index scans can be parallelised, but only on sufficiently large indexes (`min_parallel_index_scan_size`). - CTAS / SELECT INTO parallelism is version-dependent on Postgres (parallel CREATE TABLE AS landed in 11).

What NOT to say: - Do not say "more workers = always faster." Past a point, coordination cost dominates. - Do not assume parallelism is always desirable — OLTP traffic suffers when one analytical query grabs all the workers.

Common follow-up: "*How do you cap parallelism for one specific query without changing the global setting?*" — Postgres: per-query `SET LOCAL max_parallel_workers_per_gather = 0`. SQL Server: query hint `OPTION (MAXDOP 1)`.

```
-- Postgres: see why a plan went serial
EXPLAIN (VERBOSE) SELECT ...;
-- look for "Workers Planned: 0" and the surrounding nodes
```

Q139 · Hash Spilling and work_mem

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Goldman, Cred · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> A hash join in Postgres is creating gigabytes of temp files. What is happening and how do you decide whether to raise `work_mem` or restructure the query?

The 30-second answer: The hash table for the build side did not fit in `work_mem`, so Postgres batched it to disk and is now doing multi-pass hashing. Raise `work_mem` per-query (`SET LOCAL`) if memory is available; otherwise restructure to reduce the build-side row count via a pre-aggregation or a more selective filter.

Step-by-step thought process: 1. Define the mechanism: the build side is hashed into memory; if it overflows `work_mem`, Postgres switches to batched mode and writes batches to temp files. 2. Diagnose via `EXPLAIN (ANALYZE, BUFFERS)` — look for `Batches: N`, `Memory Usage: X kB`, and temp file writes. 3. Decision tree: if the build side is genuinely small but estimate was wrong, fix statistics first. If build side is genuinely large, either bump `work_mem` per session or change algorithm (merge join with pre-sorted inputs). 4. Mention the parallel-hash twist: each worker gets its own `work_mem`, so total memory is `work_mem * workers` — easy to OOM. 5. Be ready for: "Why not just set `work_mem` globally to 1 GB?" — because it is per-operation, per-connection. 100 connections × 3 hashes × 1 GB = 300 GB.

Edge cases to mention: - Postgres 13+ has hash agg spilling (before 13, it just OOM'd silently — a famous footgun for upgrades). - `temp_file_limit` caps how much temp space one session can use; hitting it aborts the query. - Even with enough `work_mem`, a build side bigger than RAM forces disk regardless.

What NOT to say: - Do not say "set `work_mem` to RAM size." It is per-operation, not per-server. - Do not ignore parallel multipliers when estimating memory budget.

Common follow-up: "*What is the SQL Server equivalent of the symptom?*" — Look for `HASH_WARNING` in extended events or the `Hash Match` operator with a yellow exclamation in SSMS, indicating spill to tempdb.

```
SET LOCAL work_mem = '256MB';
EXPLAIN (ANALYZE, BUFFERS) SELECT ... FROM large_a JOIN large_b USING(id);
-- look for: Batches: 1  (no spill) vs Batches: 16  (spilled)
```

Q140 · Sort Spilling and External Merge Sort

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Walmart Labs, Wells Fargo · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> An `ORDER BY` in Postgres logs `external merge Disk: 2.3 GB`. What does that mean and what are the levers to fix it?

The 30-second answer: The sort did not fit in `work_mem`, so Postgres switched to external merge sort — chunks are sorted in memory, written to temp files, then merged. Levers: raise `work_mem`, add an index that returns rows already sorted, or reduce the row count before the sort with an early filter or LIMIT pushdown.

Step-by-step thought process: 1. Explain quicksort versus external merge: in-memory quicksort if it fits, multi-pass merge from disk if not. 2. Diagnose via `EXPLAIN ANALYZE` — `Sort Method: external merge Disk: N kB` is the giveaway. If it says `quicksort Memory: N kB`, you are fine. 3. Best fix is usually an index that produces the sort order for free — the planner shows `Index Scan` and skips the sort node entirely. 4. Second-best is `top-N heap sort` — if you have `ORDER BY ... LIMIT k`, the planner keeps only k rows in memory. 5. Be ready for: "Why does sort spilling matter even if disk is fast?" — write amplification, page cache eviction of useful data, longer wall-clock time.

Edge cases to mention: - `work_mem` is per-sort, per-connection — a query with two sorts uses 2x. - On Windows, `temp_tablespaces` defaults to the same volume as data — separate it for IO isolation. - Parallel sort (Gather Merge) divides the work but each worker still gets one `work_mem`.

What NOT to say: - Do not assume an index will always eliminate the sort — only if the sort columns match the index leading columns in the same direction. - Do not equate slow sort with bad disk; the first fix is almost always to avoid the sort.

Common follow-up: "*What if I cannot raise* `work_mem` *and cannot add an index?*" — Try `ORDER BY ... LIMIT k` if you only need the top rows, or pre-aggregate to shrink the row count before sorting.

```
-- Avoid the sort by giving the planner an index in the right order
CREATE INDEX ON orders (created_at DESC, status);
EXPLAIN ANALYZE
SELECT * FROM orders WHERE status='paid' ORDER BY created_at DESC LIMIT 100;
```

Q141 · REFRESH MATERIALIZED VIEW CONCURRENTLY

Tags: Difficulty: Medium · Asked at: Swiggy, Meesho, Zerodha, Razorpay · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> What is the difference between `REFRESH MATERIALIZED VIEW` and `REFRESH MATERIALIZED VIEW CONCURRENTLY` in Postgres, and what is the requirement to use the concurrent form?

The 30-second answer: Plain REFRESH takes an `ACCESS EXCLUSIVE` lock — readers are blocked for the entire refresh. CONCURRENTLY computes the new state in the background and applies the delta, so readers stay unblocked. Requirement: the matview must have at least one unique index so Postgres can identify which rows changed.

Step-by-step thought process: 1. Lead with the lock difference — that is the operational reason concurrent matters. 2. Explain the mechanism: CONCURRENTLY does a full re-compute into a temp matview, then `MERGE`-style applies inserts/updates/deletes against the live matview using the unique index. 3. State the requirement explicitly — without a unique index you get an error, not a fallback. 4. Mention the cost: CONCURRENTLY is slower than plain REFRESH because of the diff step; it is a trade for online availability. 5. Be ready for: "What if the matview has billions of rows?" — concurrent refresh on a giant matview can be slower than plain refresh during a maintenance window; pick the right tool per use case.

Edge cases to mention: - During concurrent refresh, you can still read the old data; you cannot run another concurrent refresh of the same matview. - The unique index must be on a column or set of columns that genuinely identifies a row — a composite is fine. - Matviews do not auto-refresh; you schedule via pg_cron or external scheduler.

What NOT to say: - Do not call matviews "always faster" than the underlying query — they trade staleness for speed. - Do not skip the unique index and hope it falls back — it errors out.

Common follow-up: "*How would you implement near-real-time matviews?*" — Either schedule frequent concurrent refreshes (every 1-5 min) or use triggers / logical replication to incrementally update a regular table acting as a materialised cache.

```
CREATE MATERIALIZED VIEW order_daily AS
SELECT date_trunc('day', created_at) AS day, COUNT(*) FROM orders GROUP BY 1;

CREATE UNIQUE INDEX ON order_daily (day);
REFRESH MATERIALIZED VIEW CONCURRENTLY order_daily;
```

Q142 · Indexed Views in SQL Server

Tags: Difficulty: Hard · Asked at: AmEx, JP Morgan India, Walmart Labs, Wells Fargo · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> SQL Server lets you create an indexed view. What are the requirements and what is the catch on writes?

The 30-second answer: The view must be created `WITH SCHEMABINDING`, must use deterministic, schema-bound, two-part-named expressions, no outer joins, no subqueries, no `SELECT *`, and the first index must be a unique clustered index. The catch: every write to the base tables must update the indexed view synchronously, so write throughput drops.

Step-by-step thought process: 1. List the rules in order interviewers care about: SCHEMABINDING, deterministic expressions, no non-deterministic functions (no GETDATE in the view), no LEFT/RIGHT/FULL OUTER, no subqueries / CTEs, no DISTINCT, GROUP BY must include COUNT_BIG(*). 2. Explain the first index — unique clustered — that is what physically materialises the view. 3. Spell out the write cost: an insert into the base table now touches the base + the indexed view's clustered index + any non-clustered indexes on the view. Writes can slow 2-5x. 4. Mention the silver lining: SQL Server Enterprise can use the indexed view to satisfy queries against the base tables automatically (query rewrite); Standard edition requires `WITH (NOEXPAND)` hint. 5. Be ready for: "When would you actually use one?" — aggregated reporting columns on a write-light, read-heavy table where the matview pattern fits but you want synchronous freshness.

Edge cases to mention: - `COUNT_BIG(*)` is mandatory in any GROUP BY indexed view — not `COUNT(*)`. - Indexed views over partitioned tables have constraints on partition alignment. - Schema changes on base tables blocked while the indexed view references them — SCHEMABINDING does what it says.

What NOT to say: - Do not equate "indexed view" with Postgres "materialised view" — Postgres matviews are async; SQL Server indexed views are sync. - Do not promise the optimizer always picks the indexed view; in Standard edition it does not without `NOEXPAND`.

Common follow-up: "*What is the Oracle equivalent and how does it differ?*" — Oracle materialised views with `REFRESH FAST ON COMMIT` are similar; they need materialised view logs on the base tables to compute deltas.

```
CREATE VIEW dbo.vw_OrderTotals WITH SCHEMABINDING AS
SELECT customer_id, COUNT_BIG(*) AS cnt, SUM(amount) AS total
FROM dbo.orders GROUP BY customer_id;
CREATE UNIQUE CLUSTERED INDEX ix_vot ON dbo.vw_OrderTotals (customer_id);
```

Q143 · Index Bloat and When to REINDEX

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, PhonePe, Swiggy · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> What is index bloat in Postgres and how do you decide when to REINDEX?

The 30-second answer: Bloat is dead-but-not-yet-reclaimed space in index pages, caused by updates and deletes that leave tombstones until VACUUM cleans them. When the bloat ratio exceeds ~30-50%, scans touch more pages than necessary. Use `pgstattuple` or the `pg_class.reltuples` heuristic to measure, and run `REINDEX CONCURRENTLY` to rebuild without blocking writes.

Step-by-step thought process: 1. Define bloat as the gap between logical row count and physical page count caused by MVCC garbage. 2. Measurement: `pgstattuple_approx(indexname)` gives a fast estimate; `pgstattuple(indexname)` is exact but expensive. 3. Threshold: 20% bloat is normal, 50%+ degrades cache hit rate noticeably. Above that, rebuild. 4. `REINDEX CONCURRENTLY` (Postgres 12+) builds a fresh index alongside, then swaps — no long lock. Before 12, you used `CREATE INDEX CONCURRENTLY ... ; DROP INDEX CONCURRENTLY ...` manually. 5. Be ready for: "What causes bloat to accumulate faster?" — long-running transactions block VACUUM from reclaiming dead tuples (Q145).

Edge cases to mention: - B-tree indexes bloat from random key inserts followed by deletes — append-only IDs bloat much less. - GIN indexes bloat from updates to the indexed array/jsonb columns. - `REINDEX CONCURRENTLY` doubles disk usage during the operation — plan capacity.

What NOT to say: - Do not run `REINDEX` (non-concurrent) on a production table — it takes an exclusive lock. - Do not assume autovacuum keeps bloat at zero; it controls heap bloat better than index bloat.

Common follow-up: "*How do you script bloat measurement across all indexes?*" — Query `pg_stat_user_indexes` joined with `pgstattuple_approx` for each index name; rank by bloat ratio × index size.

```
CREATE EXTENSION IF NOT EXISTS pgstattuple;
SELECT * FROM pgstattuple_approx('orders_user_id_idx');
REINDEX INDEX CONCURRENTLY orders_user_id_idx;
```

Q144 · Postgres VACUUM Internals

Tags: Difficulty: Hard · Asked at: Goldman, Razorpay, Cred, Target · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> Walk me through what VACUUM actually does in Postgres — the dead tuple cleanup, the freezing, and the visibility map updates.

The 30-second answer: VACUUM does three things: it marks dead tuples (rows past their `xmax` and invisible to all transactions) as reusable space within the page, it freezes very old tuples to prevent transaction-ID wraparound, and it updates the visibility map to mark pages as all-visible (enabling index-only scans). It does not return space to the OS — that is `VACUUM FULL`.

Step-by-step thought process: 1. Open with the three jobs — that is what separates a candidate who has read the docs from one who has not. 2. Dead tuple cleanup: MVCC means UPDATE writes a new tuple version and leaves the old; VACUUM identifies dead versions by `xmin` / `xmax` versus the oldest live snapshot, and the freed slots are reused on subsequent inserts. 3. Freezing: transaction IDs are 32-bit and wrap around at ~2 billion. VACUUM freezes rows older than `vacuum_freeze_min_age` so wraparound cannot lose data. Failure to freeze triggers anti-wraparound autovacuum (aggressive, cannot be cancelled). 4. Visibility map: a per-page bit that says "every tuple here is visible to every snapshot." Index-only scans depend on this — without it, the engine has to visit the heap. 5. Be ready for: "Why doesn't VACUUM return space?" — heap files only shrink at the tail; only `VACUUM FULL` or `pg_repack` rewrites the heap into a smaller file.

Edge cases to mention: - `VACUUM FULL` takes an `ACCESS EXCLUSIVE` lock — unacceptable on hot tables. - Long-running transactions hold an `xmin` horizon that prevents VACUUM from cleaning anything newer — common production bloat cause (Q145). - `vacuum_freeze_table_age` triggers aggressive freezing on full table scan; tune for very-large append-only tables.

What NOT to say: - Do not call VACUUM "garbage collection" without the qualifier that it does not release OS space. - Do not skip the freezing story — that is the part that catches mid-level candidates.

Common follow-up: "*How do you check autovacuum's progress on a specific table?*" — `pg_stat_progress_vacuum` gives live progress; `pg_stat_user_tables` shows last autovacuum timestamp and dead tuple count.

```
SELECT relname, n_dead_tup, last_autovacuum, last_vacuum
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC LIMIT 10;
```

Q145 · Long-Running Transactions and Bloat Risk

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, JP Morgan India, Flipkart · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> A junior engineer left a transaction open for two hours. What is the operational risk in Postgres and how would you have caught it earlier?

The 30-second answer: The open transaction holds an `xmin` horizon, meaning VACUUM cannot reclaim any dead tuple created during those two hours — across every table in the database. Bloat balloons, autovacuum runs but achieves nothing, and after the transaction commits you face hours of catch-up work. Catch it via `pg_stat_activity` alerting on `state = 'idle in transaction'` longer than a few minutes.

Step-by-step thought process: 1. Lead with the cross-table impact — that is the part most candidates miss. One open txn affects every table, not just the ones it touched. 2. Mechanism: VACUUM uses the oldest snapshot's `xmin` as the "do not delete newer than this" boundary. Open transaction keeps that boundary frozen. 3. Symptom progression: dead tuple count rises, autovacuum logs show "removable 0 / non-removable N," disk usage grows, query latency degrades as bloat eats cache. 4. Detection: query `pg_stat_activity` for `xact_start` older than threshold, or `state = 'idle in transaction'` — alert via Datadog / Prometheus. 5. Prevention: `idle_in_transaction_session_timeout` (Postgres 9.6+) auto-kills idle-in-txn sessions; `statement_timeout` for runaway statements.

Edge cases to mention: - Replication slots (logical and physical) act the same way — an inactive slot pins WAL and the xmin horizon. - Hot standby with `hot_standby_feedback = on` can cause primary bloat — the standby's snapshot pins the primary's horizon. - Prepared transactions (two-phase commit) that never commit are an even worse version of the same problem.

What NOT to say: - Do not say "kill the connection and it is fine." Bloat has already accumulated; you still need a maintenance pass. - Do not blame VACUUM for not cleaning up — VACUUM is doing exactly what it must.

Common follow-up: "*What is the recovery procedure after the long transaction finally commits?*" — Let autovacuum catch up, monitor `n_dead_tup`, and consider `pg_repack` on the worst-bloated tables if they need to shrink physically.

```
SELECT pid, usename, state, xact_start, query
FROM pg_stat_activity
WHERE xact_start < now() - interval '10 minutes' AND state <> 'idle';
```

Q146 · HOT Updates in Postgres

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, Walmart Labs · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> What is a HOT update in Postgres and what conditions are required for it to happen?

The 30-second answer: A Heap-Only Tuple update writes the new tuple version on the same page as the old, and crucially does not update any index. Conditions: no indexed column is being changed, and the same page has free space for the new tuple. HOT updates avoid index bloat and are significantly cheaper.

Step-by-step thought process: 1. Define HOT in one sentence: same-page update with no index update. 2. Two conditions both required: indexed columns unchanged, free space available on the page. 3. Mechanism: the old tuple's `ctid` points forward to the new tuple, forming a HOT chain that index lookups follow at read time. Eventually pruned during vacuum. 4. Impact: drastically reduces index write amplification and index bloat. A table with 20 indexes and an unindexed `last_seen_at` column updated frequently saves enormous IO. 5. Be ready for: "How do you increase HOT update frequency?" — lower `fillfactor` (e.g., 80) to leave room on each page, and avoid indexing columns that change frequently.

Edge cases to mention: - `pg_stat_user_tables.n_tup_hot_upd` tracks how many updates went HOT — ratio versus `n_tup_upd` is the key metric. - A single page-spilling update breaks the HOT chain — the new tuple goes to another page and all indexes are updated. - BRIN and GIN indexes participate slightly differently; the conditions are still the same.

What NOT to say: - Do not say "HOT means no work" — the heap page still gets a new tuple version; it is the index write that is skipped. - Do not confuse `fillfactor` with `autovacuum_vacuum_scale_factor` — completely different knobs.

Common follow-up: "*What fillfactor would you set on a heavily updated table?*" — 70-85 is typical; lower if you can spare the disk and update frequency is high.

```
ALTER TABLE user_sessions SET (fillfactor = 80);
SELECT n_tup_upd, n_tup_hot_upd,
  100.0*n_tup_hot_upd/NULLIF(n_tup_upd,0) AS hot_pct
FROM pg_stat_user_tables WHERE relname='user_sessions';
```

Q147 · Visibility Map and Index-Only Scans

Tags: Difficulty: Medium · Asked at: Goldman, Target, AmEx, Razorpay · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> What does index-only scan actually mean in Postgres and what is the role of the visibility map?

The 30-second answer: Index-only scan returns data from the index without visiting the heap. It works only when the index covers every column in the query and the page is marked all-visible in the visibility map — meaning every tuple on it is visible to every snapshot. If the bit is unset, the engine has to fall back to a heap fetch to check visibility.

Step-by-step thought process: 1. Lead with the two conditions: index covers all queried columns, and visibility map says the page is all-visible. 2. Mechanism: Postgres tuples carry MVCC visibility info in the heap, not the index. Without the visibility map, an index entry alone cannot prove the tuple is visible to your snapshot. 3. VACUUM is what sets the all-visible bit. A freshly inserted or updated page has the bit unset until the next VACUUM. 4. `EXPLAIN ANALYZE` shows `Heap Fetches: N` — when N is high, the index-only scan is not actually skipping the heap. Run VACUUM and re-check. 5. Be ready for: "How do you build a covering index?" — use the `INCLUDE` clause (Postgres 11+) so non-key columns are stored without affecting key ordering.

Edge cases to mention: - HOT updates that stay on the same page can keep the all-visible bit accurate longer. - High write tables rarely benefit from index-only scans because the visibility map is constantly invalidated. - The visibility map is one bit per page — tiny, lives in `_vm` file alongside the heap.

What NOT to say: - Do not call it "index-only" without the asterisk that visibility info still needs to be resolved. - Do not assume `INCLUDE` columns are searchable — they are storage-only, not part of the B-tree key.

Common follow-up: "*Run me through how INCLUDE differs from adding extra key columns.*" — INCLUDE columns are not used for ordering or filtering; they are only fetched. Smaller key = better B-tree fan-out.

```
CREATE INDEX ON orders (customer_id) INCLUDE (status, amount);
EXPLAIN ANALYZE SELECT status, amount FROM orders WHERE customer_id = 42;
-- look for: Index Only Scan ...  Heap Fetches: 0
```

Q148 · pg_repack and pt-online-schema-change

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Swiggy, Meesho · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> You need to reorganise a 500 GB Postgres table to reclaim bloat, but you cannot take downtime. Compare `pg_repack` with what you would use on MySQL.

The 30-second answer: `pg_repack` (Postgres) and `pt-online-schema-change` from Percona Toolkit (MySQL) both create a shadow table, copy data in chunks, capture concurrent writes via triggers, then atomically swap. Net effect: a defragmented table without an exclusive lock. Both need at least 2x disk during the operation.

Step-by-step thought process: 1. Frame both as the same pattern: copy + capture + swap. 2. `pg_repack` mechanism: creates a new empty table, copies rows via a snapshot, a trigger on the original captures concurrent changes into a log table, the log is replayed, then a brief `ACCESS EXCLUSIVE` lock swaps the names. Indexes are rebuilt as part of the copy. 3. `pt-osc` mechanism: same idea — `_table_new`, triggers on the original, chunked copy, atomic rename. Has knobs for chunk size, replica lag throttling, and aborting on locks. 4. Disk requirement: 2x the table size temporarily — verify space before starting. 5. Be ready for: "What can go wrong?" — `pg_repack` aborts if it cannot acquire the final swap lock; `pt-osc` can fall behind on write-heavy tables and never converge.

Edge cases to mention: - `pg_repack` requires a primary key or unique non-partial non-deferrable index to track row identity. - `pt-osc` does not work with triggers on the target table (cannot add a second set of triggers in older MySQL versions). - Both tools generate significant replica lag — use replica-lag throttling.

What NOT to say: - Do not call `VACUUM FULL` an alternative — it takes an exclusive lock for hours. - Do not run either tool on a table with foreign keys pointing in without checking — FK handling has caveats.

Common follow-up: "*What is the newer alternative on MySQL 8?*" — `gh-ost` from GitHub uses binlog instead of triggers, avoiding trigger overhead and giving better pausability.

```
# Postgres
pg_repack -h db.example -d shop -t orders --no-order

# MySQL
pt-online-schema-change --alter "ENGINE=InnoDB" D=shop,t=orders --execute
```

Q149 · Autovacuum Tuning

Tags: Difficulty: Hard · Asked at: Goldman, Razorpay, JP Morgan India, Walmart Labs · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> Default autovacuum settings are too conservative for a high-write table. Which knobs do you tune and what is the trade-off?

The 30-second answer: The two main knobs are `autovacuum_vacuum_scale_factor` (default 0.2 — vacuum when 20% of rows are dead) and `autovacuum_naptime` (default 1 min — how often the launcher wakes up). For high-write tables, set per-table overrides with a smaller scale factor (e.g., 0.05) and a lower threshold so vacuum runs more often. Trade-off is more IO from vacuum itself.

Step-by-step thought process: 1. Start with the formula: vacuum fires when `n_dead_tup > vacuum_threshold + scale_factor * n_live_tup`. On a billion-row table, default 0.2 means waiting for 200M dead rows before vacuuming. 2. Per-table override (do not change globals for one table): `ALTER TABLE orders SET (autovacuum_vacuum_scale_factor = 0.02, autovacuum_vacuum_threshold = 10000);`. 3. Also tune `autovacuum_vacuum_cost_limit` higher (or `_cost_delay` to 0 on Postgres 12+) so vacuum is not throttled by the default cost-based delay. 4. `autovacuum_max_workers` controls parallelism across tables — bump if many large tables compete. 5. Be ready for: "How do you know your tuning worked?" — `pg_stat_user_tables` shows `last_autovacuum` timestamp and `n_dead_tup` should stay roughly flat instead of climbing.

Edge cases to mention: - Aggressive autovacuum competes with foreground IO — measure read latency before and after. - Anti-wraparound autovacuum (when txn age exceeds `autovacuum_freeze_max_age`) cannot be cancelled — plan for it. - Insert-only tables benefit from `autovacuum_vacuum_insert_scale_factor` (Postgres 13+) which triggers vacuum on inserts to maintain visibility map.

What NOT to say: - Do not turn autovacuum off — that is the way to disaster. - Do not tune globals when per-table overrides cover the problem cleanly.

Common follow-up: "*What is the difference between `autovacuum_naptime` and `vacuum_cost_delay`?*" — Naptime is how often the launcher checks tables; cost delay is how aggressively each individual vacuum runs.

```
ALTER TABLE orders SET (
  autovacuum_vacuum_scale_factor = 0.02,
  autovacuum_vacuum_threshold = 10000,
  autovacuum_analyze_scale_factor = 0.02
);
```

Q150 · TOAST in Postgres

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Zerodha, PhonePe · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> What is TOAST in Postgres and when does a query pay the TOAST cost?

The 30-second answer: TOAST (The Oversized Attribute Storage Technique) handles values too big for an 8 KB heap page by either compressing them in place or storing them out-of-line in a side table. Queries pay the cost only when they actually project a TOASTed column — a `SELECT count(*)` does not pay; a `SELECT body` does.

Step-by-step thought process: 1. Define the trigger: row would exceed ~2 KB, so the engine compresses large columns or moves them to the `pg_toast.pg_toast_xxx` side table. 2. Four strategies per column: `PLAIN` (no toasting), `EXTENDED` (compress then move, default for text/jsonb), `EXTERNAL` (move without compress, good for random access), `MAIN` (compress, keep inline if possible). 3. Cost surfaces: extra page reads from the toast table, decompression CPU. The main table looks small in `pg_class.relpages` while the toast table is huge. 4. `pg_total_relation_size` includes TOAST; `pg_relation_size` does not — easy way to miss real storage cost. 5. Be ready for: "How do I avoid TOAST overhead for a hot column?" — choose `EXTERNAL` storage so reads do not have to decompress.

Edge cases to mention: - JSONB compressed under EXTENDED is the most common TOAST source on modern Postgres apps. - Indexes on TOASTed columns work; the index entry stores the value if it fits, otherwise an inline TOAST pointer. - Postgres 14 added `lz4` as a TOAST compression algorithm — faster than `pglz` default.

What NOT to say: - Do not call TOAST "blob storage" — it is transparent column-level compression and external storage. - Do not assume `SELECT *` is the same cost as `SELECT id` — projection matters.

Common follow-up: "*How do you switch the compression algorithm?*" — Postgres 14+: `ALTER TABLE t ALTER COLUMN body SET COMPRESSION lz4;` Old rows keep the old algorithm until rewritten.

```
SELECT relname, pg_size_pretty(pg_relation_size(oid)) AS main,
       pg_size_pretty(pg_total_relation_size(oid)) AS total
FROM pg_class WHERE relname='articles';
```

Q151 · Bitmap Heap Scan

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Walmart Labs, Goldman · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> Explain bitmap heap scan in Postgres. When is it the right plan choice?

The 30-second answer: Bitmap heap scan reads matching tuple-IDs from one or more indexes into an in-memory bitmap, sorts them by heap page order, then reads each heap page exactly once. It is the right plan when the predicate matches more rows than an index scan should handle row-by-row but fewer than a seq scan justifies.

Step-by-step thought process: 1. Mechanism: two-stage — bitmap index scan builds the bitmap, bitmap heap scan reads pages in physical order. 2. Why it wins: by sorting page accesses, it converts random IO into largely sequential IO. Indexed columns with medium selectivity (1-20% of rows) benefit most. 3. It can combine bitmaps from multiple indexes with `BitmapAnd` or `BitmapOr` — this is why two separate indexes can sometimes outperform one composite index for ad-hoc filter combinations. 4. Lossy mode: if the bitmap exceeds `work_mem`, it drops per-row granularity and remembers only "this page might match" — heap then has to recheck the predicate. 5. Be ready for: "When is plain index scan better?" — when you only need a handful of rows (high selectivity) or when ordered output is needed (bitmap loses order).

Edge cases to mention: - Bitmap heap scan output is not ordered — adding `ORDER BY` forces an extra sort. - Lossy bitmap is visible in EXPLAIN as `Recheck Cond:` doing work. - For columns with very high cardinality but low predicate selectivity, B-tree + bitmap is gold.

What NOT to say: - Do not equate bitmap heap scan with parallel scan; they are orthogonal concepts. - Do not assume the planner always picks the cheapest — bad cardinality estimates can pick seq scan over bitmap.

Common follow-up: "*How do BitmapAnd / BitmapOr help with ad-hoc filtering?*" — They let you index columns individually and let the planner combine them at query time, instead of guessing every multi-column combination as a composite index.

```
EXPLAIN ANALYZE
SELECT * FROM orders WHERE city='Pune' AND status='paid';
-- look for: BitmapAnd  →  Bitmap Heap Scan on orders
```

Q152 · BRIN Indexes for Append-Only Tables

Tags: Difficulty: Medium · Asked at: Goldman, Target, JP Morgan India, Razorpay · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> When is a BRIN index the right choice in Postgres? Compare it to B-tree on the same column.

The 30-second answer: BRIN (Block Range Index) stores summary information (min, max) for ranges of heap pages instead of indexing every row. Right choice when the column is physically correlated with insert order — typically `created_at` on append-only logs. Index is tiny (KB versus GB) but only useful for range scans on correlated data.

Step-by-step thought process: 1. Mechanism: each BRIN entry covers `pages_per_range` heap pages (default 128) and stores min/max for the indexed column. 2. Use case: append-only tables (event logs, IoT, audit) where new rows have monotonically increasing values — `created_at`, sequential IDs. 3. Size advantage: B-tree on a 1 TB log table can be 50-100 GB; BRIN on the same column is single-digit megabytes. 4. Trade-off: range scans cost a few bitmap heap scans of the matching ranges, then heap recheck. Point lookups are useless — they degrade to scanning every range. 5. Be ready for: "What if data is not physically clustered?" — BRIN becomes useless. Either CLUSTER the table by that column once, or stick with B-tree.

Edge cases to mention: - `CREATE INDEX ... USING BRIN (col) WITH (pages_per_range = 32)` — smaller range = more precise but bigger index. - BRIN works with `bloom`-like operator classes for equality on multiple columns (extension). - VACUUM updates the BRIN summary; manual `brin_summarize_new_values` refreshes on demand.

What NOT to say: - Do not use BRIN for high-cardinality unique IDs that you do point lookups on — B-tree is the right tool. - Do not use BRIN when physical order is random — the summaries become useless.

Common follow-up: "*How would you confirm the column is correlated with physical order?*" — `pg_stats.correlation` close to 1 (or -1) means strong correlation; close to 0 means random.

```
CREATE INDEX ON events USING BRIN (created_at) WITH (pages_per_range = 64);
SELECT correlation FROM pg_stats
  WHERE tablename='events' AND attname='created_at';
```

Q153 · Functional and Expression Indexes

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Flipkart · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> A query filters on `LOWER(email) = ?` and is doing a sequential scan. Walk me through the fix.

The 30-second answer: A regular B-tree index on `email` cannot be used because the planner sees a function wrapping the column. Create an expression index: `CREATE INDEX ON users (LOWER(email));`. The planner then matches the predicate to the indexed expression directly.

Step-by-step thought process: 1. Diagnose the cause: any function or expression on the column blocks index use unless an index exists on that exact expression. 2. Fix: `CREATE INDEX users_lower_email_idx ON users (LOWER(email));` — Postgres and SQLite syntax. SQL Server uses computed columns indexed; Oracle and MySQL 8 use function-based / functional indexes. 3. Equivalent gotcha for unique constraints: case-insensitive uniqueness needs a unique index on the expression, not on the column. 4. Cardinality stats: Postgres maintains separate stats for the expression once the index exists — no `WHERE LOWER(email)` stats before that. 5. Be ready for: "What is the alternative without an expression index?" — store a generated column `email_lower` and index that; useful when the same expression appears in many queries.

Edge cases to mention: - The expression must be `IMMUTABLE` — functions like `NOW()` or `RANDOM()` cannot be indexed. - JSONB extraction is the most common modern use: `CREATE INDEX ON events ((payload->>'user_id'));`. - Expression indexes participate in HOT updates only if the underlying column also did not change.

What NOT to say: - Do not say "rewrite the app to avoid LOWER." Sometimes you do not own the app; index the expression. - Do not assume Postgres can "see through" the function — the predicate must match the indexed expression literally.

Common follow-up: "*What is the difference between a generated column index and an expression index?*" — Generated column stores the value (extra disk per row) and is queryable directly. Expression index computes only the index entry and saves no value in the heap.

```
CREATE INDEX users_lower_email_idx ON users (LOWER(email));
EXPLAIN ANALYZE
SELECT * FROM users WHERE LOWER(email) = 'priya@example.in';
```

Q154 · Partial Unique Indexes

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, Swiggy · Topic: Performance / Query Plans

The question (interviewer's verbatim phrasing):

> You need each `users` row to have at most one active referral code, but a user can have many revoked ones. How do you enforce this in Postgres?

The 30-second answer: Create a partial unique index with a WHERE predicate: `CREATE UNIQUE INDEX ON referrals (user_id) WHERE status = 'active';`. The unique constraint applies only to rows matching the predicate, so revoked rows can coexist freely.

Step-by-step thought process: 1. Define the need precisely: uniqueness only conditional on a status, not unconditional. 2. Solution: a partial index, which both enforces the conditional uniqueness and provides an efficient lookup for active rows. 3. The same pattern handles "one default address per user" — `WHERE is_default = true` — or "one open invoice per project." 4. Smaller index footprint: only the qualifying rows are stored, so the index is tiny if the predicate is selective. 5. Be ready for: "What is the SQL Server equivalent?" — filtered indexes, same idea, syntax `CREATE UNIQUE INDEX ... WHERE status='active';`. MySQL has no native equivalent — emulate with a computed column.

Edge cases to mention: - The predicate must be `IMMUTABLE` — no `NOW()` filters. - `ON CONFLICT (user_id) WHERE status='active' DO UPDATE` uses the partial index for upsert. - The planner uses the partial index for queries whose `WHERE` clause logically implies the index predicate.

What NOT to say: - Do not use a regular unique constraint and then "handle the conflict in app code" — the database is the right place for this invariant. - Do not assume the partial index covers queries that do not match the predicate; it does not.

Common follow-up: *"How does ON CONFLICT work with a partial unique index?"* — You must specify the same WHERE clause in the `ON CONFLICT` target so Postgres can match the right index.

```
CREATE UNIQUE INDEX one_active_referral
  ON referrals (user_id) WHERE status = 'active';

INSERT INTO referrals (user_id, code, status) VALUES (42, 'NEWCODE', 'active')
ON CONFLICT (user_id) WHERE status = 'active'
DO UPDATE SET code = EXCLUDED.code;
```

Q155 · Gap Locks in MySQL InnoDB

Tags: Difficulty: Hard · Asked at: Razorpay, PhonePe, Cred, JP Morgan India · Topic: Locking

The question (interviewer's verbatim phrasing):

> Explain what a gap lock is in InnoDB. Why does MySQL's Repeatable Read level prevent phantom reads even though the ANSI standard says Repeatable Read shouldn't?

The 30-second answer: A gap lock is a lock on the *empty space* between two index records — not on the rows themselves. InnoDB acquires gap locks under Repeatable Read so that another transaction cannot INSERT a row that would fall into the same range you've already scanned, which is exactly how MySQL prevents phantom reads at RR (a level where the ANSI standard does permit phantoms).

Step-by-step thought process: 1. State the core mechanic — `SELECT ... WHERE user_id BETWEEN 100 AND 200 FOR UPDATE` locks not just existing rows in that range, but the *gaps* before, between and after them. A concurrent INSERT of `user_id = 150` will block. 2. Mention that gap locks are purely *blocking* — they do not protect any data themselves, they just prevent insertion. Two transactions can hold the *same* gap lock at the same time, which is unusual. 3. Tie it back to phantoms — without gap locks, between your two reads of the same range, someone could insert a new matching row and you'd see a "phantom" on the second read. Gap locks plug that hole. 4. Be ready for the follow-up about disabling them — `innodb_locks_unsafe_for_binlog = ON` (deprecated) or running at Read Committed turns gap locking off, at the cost of phantom reads.

Edge cases to mention: - Gap locks only exist on indexed columns — if your WHERE clause hits no index, InnoDB locks the whole table or all rows scanned. - A gap lock on a unique index lookup that *finds* the row is downgraded to a record lock — the gap doesn't need protecting because uniqueness already does. - Postgres has no concept of gap locks; it prevents phantoms at Serializable via Serializable Snapshot Isolation (SSI) instead.

What NOT to say: - "Gap locks lock the row." They lock the gap, not the row — the record lock is a separate animal. - "Phantoms are impossible in Repeatable Read." Only in InnoDB's version of RR. In ANSI-spec RR (which Postgres implements at the Repeatable Read level), phantoms can still happen.

Common follow-up: "*What happens to gap locks under Read Committed in InnoDB?*" — They're disabled. You can insert into ranges another txn has read. This is why high-throughput OLTP shops often run InnoDB at RC.

```
-- Session 1
START TRANSACTION;
SELECT * FROM orders WHERE user_id BETWEEN 100 AND 200 FOR UPDATE;
-- Session 2 (will BLOCK on the gap lock)
INSERT INTO orders (user_id, amount) VALUES (150, 999);
```

Q156 · Next-Key Locks in InnoDB

Tags: Difficulty: Hard · Asked at: Flipkart, Swiggy, Goldman, Walmart Labs · Topic: Locking

The question (interviewer's verbatim phrasing):

> What's a next-key lock in InnoDB and how is it different from a gap lock?

The 30-second answer: A next-key lock is the *combination* of a record lock on an index row and a gap lock on the gap *just before* that row. It's the default locking unit for InnoDB at Repeatable Read — every range scan ends up acquiring a chain of next-key locks, which together protect both the rows you read and the spaces around them.

Step-by-step thought process: 1. Build it up: record lock = lock on a single index entry; gap lock = lock on the empty range before an entry; next-key lock = both, fused into one acquisition. 2. Walk through a range scan — `WHERE id > 10 AND id < 20`, with rows at 11, 15, 19. InnoDB takes a next-key lock on each row plus the gap leading up to it, so 11 with its gap (10, 11], 15 with (11, 15], 19 with (15, 19], plus the trailing gap (19, 20). 3. Note that the "next" in next-key refers to the index next-pointer — the lock protects the record and the half-open interval ending at it. 4. Anticipate the optimisation question — if InnoDB can prove no inserts would matter (Read Committed, or a unique index match), it falls back to a plain record lock.

Edge cases to mention: - Under Read Committed, InnoDB strips gap locks from next-key locks, leaving only the record half — much higher concurrency, but phantoms return. - Next-key locks on a secondary index also implicitly hold a record lock on the corresponding primary-key row — so secondary-index range scans can lock more than you expect. - The lock the supremum pseudo-record (the "after last row" sentinel) means a long-running range scan can block all inserts past the current max value.

What NOT to say: - "Next-key is just a gap lock." It is gap + record fused; saying "just" loses the credit. - "InnoDB only takes row locks." False — under RR, the default is next-key locks.

Common follow-up: "*Why might switching from RR to RC dramatically improve insert throughput?*" — Because next-key locks become plain record locks, freeing up gaps for concurrent inserts.

```
-- Under RR, this acquires next-key locks across the whole range
SELECT * FROM payments WHERE created_at >= '2026-05-01' FOR UPDATE;
-- Inserts of any new payment after May 1 will block until you commit
```

Q157 · Predicate Locks and Serializable Snapshot Isolation in Postgres

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Zerodha, JP Morgan India · Topic: Isolation

The question (interviewer's verbatim phrasing):

> How does Postgres implement Serializable isolation without using heavy locks like SQL Server does?

The 30-second answer: Postgres uses Serializable Snapshot Isolation (SSI) — every Serializable transaction runs on its own MVCC snapshot, but Postgres also tracks *predicate locks* (lightweight records of which rows and ranges each txn read) and detects unsafe read-write cycles between concurrent transactions. When such a cycle would produce a non-serializable outcome, Postgres aborts one transaction with the famous SQLSTATE 40001 error rather than blocking.

Step-by-step thought process: 1. Contrast against strict 2PL (what SQL Server does at SERIALIZABLE) — SQL Server holds long range locks. Postgres holds nothing extra; it just remembers. 2. Define the predicate lock — a SIREAD lock recorded on the page or tuple a Serializable txn reads. Multiple txns can hold SIREADs on the same data; they don't block writers, they just create an audit trail. 3. Detection — Postgres watches for rw-antidependencies: txn A read what txn B wrote (or vice versa). When two such edges form a cycle, one txn must die. 4. Cost — the optimistic style means writers never block readers, but you must be ready to retry on serialization failure. Wrap every SERIALIZABLE txn in a retry loop.

Edge cases to mention: - Predicate locks are kept in shared memory and can run out — `max_pred_locks_per_transaction` controls the ceiling; exceeding it escalates to coarser page-level tracking, leading to more false-positive aborts. - Read-only Serializable txns can be marked `READ ONLY DEFERRABLE` so they wait for a safe snapshot and never abort. - SSI's overhead is proportional to *write* activity in your dataset, not read — heavy-read, light-write workloads pay little.

What NOT to say: - "Postgres takes row locks at Serializable." It takes SIREAD predicate locks, which are different — they detect, they don't block. - "Serializable means slow." Postgres SSI is often faster than SQL Server's lock-heavy Serializable because writers stay non-blocking.

Common follow-up: "*What's the recommended retry strategy for 40001?*" — Catch the SQLSTATE, sleep with jitter, retry up to 3 times. Beyond that, escalate to the user.

```
-- Set Serializable for a critical txn
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT SUM(amount) FROM payments WHERE merchant_id = 7;
INSERT INTO settlement_log VALUES (7, NOW(), 'computed');
COMMIT; -- may fail with 40001; retry the whole BEGIN..COMMIT
```

Q158 · Postgres Advisory Locks

Tags: Difficulty: Medium-Hard · Asked at: Razorpay, Swiggy, PhonePe, Cred · Topic: Locking

The question (interviewer's verbatim phrasing):

> What's a `pg_advisory_lock` and when would you reach for it over a regular row lock?

The 30-second answer: An advisory lock is an application-defined lock attached to an arbitrary 64-bit integer key, not to any row or table. It exists entirely for *your* code's convenience — Postgres doesn't enforce anything semantically; it just guarantees mutual exclusion on the key. Use it when you need cross-process serialisation that doesn't fit naturally onto a row, like a cron job that must run on exactly one app server at a time.

Step-by-step thought process: 1. Set the scene — you have 6 app pods, and at midnight one of them should run an end-of-day reconciliation. You don't want a `cron-elected-leader` row, you want a 5-line guard. 2. The pattern: `SELECT pg_try_advisory_lock(99001)` — if it returns true, you got the lock and you do the work; if false, another pod has it and you skip. 3. Two flavours — session-level (held until the connection ends or you call `pg_advisory_unlock`) and transaction-level (auto-released on COMMIT/ROLLBACK). The transaction-level form is safer for queue workers. 4. Compare against alternatives — Redis SETNX is similar but Postgres lets you keep your single source of truth and avoid a second moving part.

Edge cases to mention: - The key namespace is global per database; collisions across unrelated services are real. Use a hash of `'service:job-name'` to derive the key and document it. - Session-level locks survive a query failure but die with the connection. Connection-pool-recycled connections can leak them. Prefer the txn-level variant where possible. - `pg_try_advisory_lock` never waits; `pg_advisory_lock` blocks. Don't use the blocking form inside an HTTP request — it can pin a connection forever.

What NOT to say: - "It locks a row." It does not — there's no row involved, only a number you make up. - "It's enforced by the schema." It's not; only code that calls `pg_try_advisory_lock` participates.

Common follow-up: "*How would you make sure exactly one worker processes a given user's payment retry?*" — `SELECT pg_try_advisory_xact_lock(user_id)` at the top of the txn; if false, skip.

```
-- Idempotent cron — only one pod runs the job at midnight
BEGIN;
SELECT pg_try_advisory_xact_lock(99001) AS got_lock;
-- if got_lock then run the reconciliation
COMMIT; -- lock auto-releases
```

Q159 · SKIP LOCKED for Queue Processing

Tags: Difficulty: Medium-Hard · Asked at: Razorpay, Swiggy, Flipkart, Cred · Topic: Locking

The question (interviewer's verbatim phrasing):

> You want to build a job queue inside Postgres or MySQL. How does `SKIP LOCKED` let you scale workers without them stepping on each other?

The 30-second answer: `SELECT ... FOR UPDATE SKIP LOCKED` lets a worker grab the next available rows while *skipping past* any rows another worker has already locked, instead of blocking. Twenty workers polling the same queue table will each pick up a disjoint set of jobs in a single statement — no coordinator, no advisory lock, no Kafka.

Step-by-step thought process: 1. Frame the alternative — without SKIP LOCKED, worker B trying to claim the next job would wait for worker A to commit, serialising everyone behind A. With SKIP LOCKED, B jumps over A's locked row to the next free one. 2. Show the canonical pattern — `SELECT id FROM jobs WHERE status='pending' ORDER BY created_at LIMIT 5 FOR UPDATE SKIP LOCKED`, then `UPDATE jobs SET status='running' WHERE id IN (...)`, then commit. 3. Mention the supported engines — Postgres since 9.5, MySQL 8.0+, Oracle has had it for years. SQL Server uses `READPAST` hint for the same effect. 4. Pair it with a heartbeat or visibility timeout — if a worker dies holding rows in `running`, you need a reaper that flips them back to `pending` after N minutes.

Edge cases to mention: - The `ORDER BY` is critical for fairness; without it, the scan order is undefined and old jobs may starve. - SKIP LOCKED does not skip rows that are *deleted* by uncommitted txns — only locked-but-still-visible rows. - For very high throughput, the queue table becomes a write hotspot; partition by hash of `id` to spread the load.

What NOT to say: - "It uses an in-memory queue." It uses the same row-level locks as any other `FOR UPDATE`; the only difference is the skip behaviour. - "It guarantees ordering." It guarantees disjoint claims; ordering depends on your `ORDER BY` and the planner's choice.

Common follow-up: "*What if a worker crashes after the claim but before completing the job?*" — Job stays in `running` forever. You need a reaper that resets stuck jobs based on `last_heartbeat_at`.

```
-- Postgres / MySQL 8 — pick up to 5 jobs, skip the rest
BEGIN;
SELECT id FROM jobs
WHERE status = 'pending'
ORDER BY created_at
LIMIT 5
FOR UPDATE SKIP LOCKED;
-- mark them running, commit, then process outside the txn
```

Q160 · Lock Escalation in SQL Server

Tags: Difficulty: Medium-Hard · Asked at: Wells Fargo India, Goldman, JP Morgan India, AmEx · Topic: Locking

The question (interviewer's verbatim phrasing):

> Explain lock escalation in SQL Server. What's the threshold and how do you stop it from happening?

The 30-second answer: SQL Server starts with row-level locks, but when a single statement acquires roughly 5,000 locks on one object, it tries to escalate to a single table-level lock to save memory. Escalation can murder concurrency because suddenly nobody else can touch that table. You prevent it by batching updates, using `WHERE` clauses that hit indexes, or setting `ALTER TABLE ... SET (LOCK_ESCALATION = DISABLE)`.

Step-by-step thought process: 1. The mechanic — SQL Server reserves around 2,500 to 5,000 lock acquisitions before triggering escalation. It is per-statement, per-object. 2. The reason — each lock takes about 96 bytes in the lock manager; a million row locks would chew through ~96 MB of memory just for tracking. 3. The trade-off — escalation saves memory but turns a fine-grained `UPDATE` into a table lock, blocking every other writer and most readers (depending on isolation level). 4. Mitigation — batch your DML (`DELETE TOP 1000` in a loop), pre-grab a `TABLOCKX` if you actually want the whole table, or disable escalation on critical tables. Partitioned tables can escalate to partition-level via `SET (LOCK_ESCALATION = AUTO)`.

Edge cases to mention: - Escalation goes row → table; SQL Server *cannot* escalate to page first. It is row-or-nothing-then-table. - Trace flag 1224 disables escalation server-wide if you hit a memory threshold; trace flag 1211 disables it always (dangerous). - Other engines don't escalate the same way — Postgres uses MVCC and doesn't track individual row locks in the same fashion; InnoDB has internal lock objects but no clean escalation semantics.

What NOT to say: - "Lock escalation happens in every database." It is mostly a SQL Server concept in this exact form. - "Just disable escalation everywhere." You may run out of lock memory and get error 1204.

Common follow-up: "*How do you tell from a wait stat that escalation hurt you?*" — Look for `LCK_M_X` or `LCK_M_S` waits at the table grain alongside Lock Memory spikes in `sys.dm_os_memory_clerks`.

```
-- SQL Server — batched delete to dodge escalation
DECLARE @batch INT = 1;
WHILE @batch > 0
BEGIN
  DELETE TOP (1000) FROM events WHERE created_at < DATEADD(month, -6, GETDATE());
  SET @batch = @@ROWCOUNT;
END
```

Q161 · How Deadlock Detection Actually Works

Tags: Difficulty: Hard · Asked at: PhonePe, Razorpay, Goldman, Walmart Labs · Topic: Concurrency

The question (interviewer's verbatim phrasing):

> Walk me through how the database detects a deadlock. Is it instant?

The 30-second answer: The engine maintains a wait-for graph — nodes are transactions, edges are "txn A is waiting on a lock held by txn B." A background process periodically searches the graph for a cycle. When a cycle is found, one transaction is picked as victim and killed with a deadlock error, releasing its locks. Detection is *not* instant — Postgres runs the check every `deadlock_timeout` (default 1 s), InnoDB checks on every lock wait (so faster but more CPU), SQL Server runs a deadlock monitor thread every 5 s, dropping to every 100 ms when activity is high.

Step-by-step thought process: 1. The graph — nodes are active txns, directed edges point from "I am waiting" to "the holder I am waiting on." A cycle is a deadlock. 2. Victim selection — Postgres picks the txn that requested the lock causing the cycle; InnoDB picks the one with the smallest amount of work to undo; SQL Server uses `DEADLOCK_PRIORITY` setting plus rollback cost. 3. The error — Postgres throws SQLSTATE 40P01, InnoDB returns error 1213, SQL Server raises error 1205. Your app must catch and retry. 4. Latency — there is always a window between "deadlock formed" and "deadlock detected" where everyone involved is just sitting on locks. Tune `deadlock_timeout` carefully; too low burns CPU, too high stalls workloads.

Edge cases to mention: - Distributed deadlocks involving more than one database are *not* detected by either engine — each engine sees half the cycle and waits forever (see Q163). - A long-running txn

holding many locks is a juicy target — detection cost is proportional to graph size. - InnoDB shows the last detected deadlock via `SHOW ENGINE INNODB STATUS` — invaluable for postmortems.

What NOT to say: - "Deadlocks are prevented by the engine." They are *detected and resolved*, not prevented. - "Detection is free." It is not — it's why InnoDB's setting is per-wait but most others poll.

Common follow-up: "*How would you log every deadlock for analysis?*" — Enable `log_lock_waits = on` plus extended deadlock logging (`deadlock_timeout` + `log_min_duration_statement`) in Postgres; in SQL Server, enable trace flag 1222 or Extended Events `xml_deadlock_report` .

```
-- Postgres — bump the detection window if your workload tolerates it
SET deadlock_timeout = '500ms';
-- Lower means faster detection, higher CPU
```

Q162 · Deadlock Prevention via Consistent Lock Ordering

Tags: Difficulty: Medium-Hard · Asked at: TCS, Infosys, Razorpay, Cred · Topic: Concurrency

The question (interviewer's verbatim phrasing):

> Two transactions are deadlocking on a fund transfer. How do you fix it without changing the isolation level?

The 30-second answer: Acquire locks in a globally consistent order across every code path. If txn A locks account 101 then 202, and txn B locks 202 then 101, a deadlock is inevitable under load. Sort the IDs and lock the smaller one first — both txns now reach for 101 first, the cycle breaks.

Step-by-step thought process: 1. Reproduce the failure — fund transfer from 101 to 202 takes locks in user-supplied order; a concurrent transfer from 202 to 101 takes them in the opposite order. The two txns form a cycle. 2. The fix — `ORDER BY id` before `SELECT ... FOR UPDATE` . Sort the accounts numerically (or by any total ordering) and lock smallest-first. Both transfers now grab 101 first; the second one waits but does not deadlock. 3. State the broader rule — every transaction that touches multiple objects of the same kind must lock them in the same canonical order. Apply this to inventory adjustments, ledger postings, anything multi-row. 4. Be ready for the inevitable "what if you have many resource types?" — define a total order across resource *classes* too (e.g. always lock `users` rows before `wallets` rows).

Edge cases to mention: - Sorting by primary key is convenient but breaks if you re-key the table later. Sorting by a stable business key is safer. - ORM-generated SQL often does not respect ordering — `entityManager.lock(a); entityManager.lock(b);` follows your Java call order, not the DB's. Audit your data-access layer. - Even with perfect ordering, you can still hit lock-wait timeouts if a holder runs slow. Add retries with backoff.

What NOT to say: - "Switch to Read Committed and the deadlock goes away." It usually doesn't — deadlocks are about lock ordering, not isolation level. - "Use NOLOCK." It avoids the lock but breaks correctness for writes.

Common follow-up: "*What if you can't sort — say the resources are user-supplied URLs?*" — Hash them and sort by the hash; you give up readability but get a stable order.

```
-- Lock accounts in canonical order (smaller id first) to avoid deadlock
BEGIN;
SELECT id, balance FROM accounts
WHERE id IN (101, 202)
ORDER BY id
FOR UPDATE;
-- ... apply debit and credit ...
COMMIT;
```

Q163 · Distributed Deadlock Across Databases

Tags: Difficulty: Hard · Asked at: PhonePe, Goldman, JP Morgan India, Wells Fargo India · Topic: Distributed

The question (interviewer's verbatim phrasing):

> Can a deadlock span two different databases? How do you even find it, let alone resolve it?

The 30-second answer: Yes — if app server A holds a lock in DB1 while waiting for a lock in DB2, and app server B holds a lock in DB2 while waiting in DB1, you have a cross-database cycle. Neither DB's deadlock detector sees the full graph; both threads just wait for `lock_timeout`. You resolve it the same way you do single-DB deadlocks — consistent ordering across services, plus aggressive lock timeouts so at least someone gets killed and retries.

Step-by-step thought process: 1. Sketch the topology — a payments service writes to DB1 (orders) and an inventory service writes to DB2 (stock). Each takes a local row lock, then RPCs the other and waits for *its* row lock. 2. Why detection fails — each engine only sees its own wait-for graph. From DB1's view, A holds lock-1 and is "idle"; from DB2's view, B holds lock-2 and is idle. No cycle visible locally. 3. The standard treatment — set short `lock_timeout` / `innodb_lock_wait_timeout` (e.g. 5 s) so at least one txn dies and rolls back, breaking the cycle even though it was never formally detected. 4. The architectural fix — collapse the cross-DB pattern. Use a saga (Q165) or the outbox pattern (Q166) so each service touches only its own DB inside any single txn.

Edge cases to mention: - Distributed transactions via XA (Q164) extend a transaction manager's reach across DBs, but most app teams avoid XA precisely because of its operational pain. - Cross-DB deadlocks frequently masquerade as "slow API" alerts in observability; trace the wait chain across

services to spot them. - Service-mesh / circuit-breaker timeouts further shorten the window and help break cycles.

What NOT to say: - "Postgres can detect deadlocks across databases." It cannot; the detection is per-cluster. - "Just retry forever." Without a timeout, you have a permanent hang.

Common follow-up: "*How do you tell from observability data that you've hit a distributed deadlock?*" — Two services each showing high `lock_wait_time` against rows the *other* service touches, around the same timestamp.

```
-- Set tight lock timeouts so at least one side gives up
-- Postgres
SET lock_timeout = '5s';
-- MySQL
SET SESSION innodb_lock_wait_timeout = 5;
```

Q164 · Two-Phase Commit and XA Transactions

Tags: Difficulty: Hard · Asked at: JP Morgan India, Goldman, Wells Fargo India, AmEx · Topic: Distributed

The question (interviewer's verbatim phrasing):

> Explain two-phase commit. Why is XA the standard implementation, and why do most modern systems avoid it?

The 30-second answer: Two-phase commit (2PC) is a protocol for atomic commit across multiple resource managers — typically multiple DBs or a DB plus a message broker. Phase 1 (prepare): each participant promises it *can* commit and persists undo/redo. Phase 2 (commit): the coordinator tells everyone to commit, or to roll back if anyone said no. XA is the X/Open standard API that exposes prepare/commit/rollback hooks; Postgres has `PREPARE TRANSACTION`, MySQL InnoDB supports XA, JTA in Java wraps it. Most modern systems avoid it because the coordinator becomes a single point of failure: if it crashes between phases, prepared txns are stuck holding locks forever.

Step-by-step thought process: 1. Phase 1 (prepare) — each RM does the work, flushes WAL with intent, and answers PREPARED. From this point on, the RM holds all its locks but cannot unilaterally finish. 2. Phase 2 (commit/abort) — coordinator collects votes. All YES → COMMIT message to all. Any NO → ABORT message to all. RMs finalize and release locks. 3. The failure modes — coordinator dies after PREPARE but before sending phase 2; prepared txns hold locks indefinitely. Operators must resolve manually (`COMMIT PREPARED` / `ROLLBACK PREPARED`) using info from logs. 4. Why it fell out of favour — operational complexity, latency cost (two network round-trips per resource), inability to scale across many participants, and the cleaner saga pattern as an alternative for most business workflows.

Edge cases to mention: - Postgres requires `max_prepared_transactions > 0` in `postgresql.conf` — defaults to 0, which means 2PC is silently disabled. - A heuristic decision (an RM unilaterally committing or rolling back during the window) breaks atomicity and is rare but possible. - XA does not handle message brokers consistently — Kafka has no XA at all; classic JMS providers do but few teams enable it.

What NOT to say: - "2PC is fully fault-tolerant." It is not — the coordinator is a single point of failure unless you also run 3PC or Paxos-style commit, which almost nobody does. - "Microservices should use XA." That is the opposite of common practice; sagas are preferred.

Common follow-up: "*What's the difference between 2PC and consensus protocols like Raft?*" — 2PC is atomic commit (all-or-none for a fixed set of participants); Raft is replicated log agreement, which is a different problem and tolerates participant failures gracefully.

```
-- Postgres 2PC sketch
BEGIN;
INSERT INTO ledger VALUES (1, 100);
PREPARE TRANSACTION 'txn-abc-123';
-- Coordinator now talks to other RMs; eventually:
COMMIT PREPARED 'txn-abc-123';
-- or ROLLBACK PREPARED 'txn-abc-123';
```

Q165 · Saga Pattern as an Alternative to 2PC

Tags: Difficulty: Medium-Hard · Asked at: Razorpay, Swiggy, Flipkart, Cred · Topic: Distributed

The question (interviewer's verbatim phrasing):

> Instead of 2PC across order and payment services, your architect suggests a saga. What does that mean?

The 30-second answer: A saga is a sequence of local transactions, each in one service's own database, connected by *compensating* transactions that semantically undo earlier steps if a later one fails. There's no global commit and no distributed locks — you trade atomicity for eventual consistency plus explicit business-level rollbacks (refund the payment if the order can't ship).

Step-by-step thought process: 1. Concretise — Step 1: reserve inventory (commit). Step 2: charge wallet (commit). Step 3: create order (commit). If step 3 fails, run compensation: refund wallet, release inventory. 2. Two flavours — *choreography* (services emit events and listen to each other, no central brain) vs *orchestration* (a saga orchestrator service explicitly drives the sequence). Orchestration is easier to reason about; choreography is more loosely coupled. 3. The hard part — compensations are not technical rollbacks, they are business semantics. "Refund" is a separate transaction visible to the user,

not a hidden rollback. 4. Pair sagas with idempotency keys (Q167) and the outbox pattern (Q166) so retries don't double-charge or double-ship.

Edge cases to mention: - Some steps are *not* compensable — sending an SMS, charging a third-party API with no refund endpoint. Plan the saga so unsavoury steps go last. - Network partitions can leave a saga half-done for minutes; expose a "saga state" dashboard so ops can intervene. - Strong reads after a saga step are not guaranteed — design UI to show "processing" until the full saga completes.

What NOT to say: - "A saga is just 2PC without the prepare phase." It is fundamentally different — no global atomicity, only step-local atomicity. - "Compensations always succeed." They can fail too; you need a fallback (manual intervention queue).

Common follow-up: *"How do you know a saga is stuck halfway?"* — Persist saga state with timestamps, alert on step durations exceeding SLA, expose a retry/abort console.

```
-- Saga state table — one row per saga instance
CREATE TABLE saga_orders (
  saga_id        UUID PRIMARY KEY,
  current_step   VARCHAR(40),
  status         VARCHAR(20),     -- 'running' / 'completed' / 'compensating' / 'failed'
  step_history   JSONB,
  updated_at     TIMESTAMPTZ DEFAULT NOW()
);
```

Q166 · The Outbox Pattern for Reliable Event Publishing

Tags: Difficulty: Medium-Hard · Asked at: Razorpay, Swiggy, PhonePe, Flipkart · Topic: Distributed

The question (interviewer's verbatim phrasing):

> You need to update your DB and publish a Kafka event in the same step. How do you make sure both happen, without 2PC?

The 30-second answer: Use the outbox pattern. Inside the same DB transaction, INSERT into a local `outbox` table the event you would have published. A separate poller (or Debezium-style CDC) reads from `outbox` and publishes to Kafka, marking rows as sent on success. The DB commit and the outbox row are atomic; the eventual publish is at-least-once and idempotent.

Step-by-step thought process: 1. The problem — without outbox, you have two writes (`UPDATE orders` and `kafka.send`) that can independently succeed or fail; you can ship an order without notifying or notify without shipping. 2. The mechanic — both writes become one DB transaction by routing the event through the DB. The poller is decoupled; it only needs at-least-once delivery. 3. Two implementations — naive polling (`SELECT FROM outbox WHERE sent_at IS NULL LIMIT 100`

`FOR UPDATE SKIP LOCKED`) or CDC (Debezium reads the WAL/binlog and streams). CDC scales further; polling is simpler. 4. Pair with consumer-side idempotency — Kafka delivers at-least-once, so the consumer must use idempotency keys (Q167) to dedupe.

Edge cases to mention: - Outbox tables grow unbounded if you forget the cleanup job; partition or TRUNCATE old shipped rows on a schedule. - Strict event ordering across multiple aggregates is hard; the outbox preserves per-aggregate order if you key by aggregate id. - For very high write loads, the outbox table becomes a hotspot; partition by hash, or shard the events table per service.

What NOT to say: - "Just use Kafka transactions." Kafka transactions don't span Kafka and a Postgres DB; they only cover Kafka topics. - "Outbox guarantees exactly-once." It guarantees at-least-once; combine with consumer idempotency for the effect of exactly-once.

Common follow-up: "*How do you handle a poll that processes the outbox row but crashes before marking it sent?*" — On next poll it republishes; consumers must dedupe by event id.

```
-- Inside the order-creation transaction
BEGIN;
INSERT INTO orders (id, user_id, amount) VALUES (uuid_generate_v4(), 42, 999);
INSERT INTO outbox (event_id, topic, payload, created_at)
VALUES (uuid_generate_v4(), 'order.created', '{"order_id": "..."}', NOW());
COMMIT;
-- Separate worker:
SELECT event_id, topic, payload FROM outbox
WHERE sent_at IS NULL ORDER BY created_at LIMIT 100
FOR UPDATE SKIP LOCKED;
```

Q167 · Idempotency Keys and At-Least-Once Delivery

Tags: Difficulty: Medium-Hard · Asked at: Razorpay, Cred, PhonePe, Swiggy · Topic: Distributed

The question (interviewer's verbatim phrasing):

> Why does every payments API require an idempotency key? What happens if you ignore it?

The 30-second answer: Idempotency keys let the API safely handle a client retry without performing the same business operation twice. The client picks a unique key per logical request (typically a UUID). The server stores `(key → result)` in a dedupe table; if the same key arrives again, the server returns the cached result without re-charging. Without idempotency keys, network retries can charge a customer twice for the same purchase.

Step-by-step thought process: 1. The premise — at-least-once delivery is the default for any network call. Timeouts make retries inevitable; the server has no way to know whether a request really failed or

just the response was lost. 2. The dedupe table — `idempotency_keys(key PRIMARY KEY, response_body, status_code, created_at)`. UNIQUE on `key` guarantees you can never insert the same key twice. 3. Atomic check-and-insert — `INSERT ... ON CONFLICT DO NOTHING` (Postgres) or `INSERT IGNORE` (MySQL); if no row was inserted, you already processed this key, so return the cached response. 4. Lifetime — keys are typically kept for 24 hours; long enough for client retries, short enough to keep the table small.

Edge cases to mention: - The key must cover the *full request semantics*, not just the URL — two different POST bodies with the same key should produce an error, not a silent reuse. - Idempotency only helps if the side-effecting operation lives inside the same DB transaction as the dedupe row; otherwise you can dedupe and still double-charge. - Concurrent in-flight retries are the trickiest case; lock the key row (`SELECT ... FOR UPDATE`) while the first request is processing.

What NOT to say: - "Idempotency means the operation is safe to retry." Strictly, it means repeated execution has the same effect — the implementation needs the dedupe table. - "GET requests don't need idempotency keys." Generally true for pure reads, but if a GET has side effects (logging, counters), it does.

Common follow-up: "*What if the client uses the same idempotency key for two genuinely different operations?*" — You return 422 with a "key reused with different payload" error; never silently overwrite.

```
-- Dedupe table for payments
CREATE TABLE idempotency_keys (
  key            UUID PRIMARY KEY,
  response_body  JSONB,
  status_code    SMALLINT,
  created_at     TIMESTAMPTZ DEFAULT NOW()
);

-- Atomic claim
INSERT INTO idempotency_keys (key) VALUES ($1) ON CONFLICT DO NOTHING;
-- If 0 rows inserted, fetch the stored response and return it
```

Q168 · Eventual Consistency vs Strong Consistency

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Walmart Labs, Cred · Topic: Distributed

The question (interviewer's verbatim phrasing):

> When would you accept eventual consistency? Where is strong consistency non-negotiable?

The 30-second answer: Strong consistency is non-negotiable wherever an incorrect read causes real-money loss or compliance breach — wallet balance, inventory countdowns, order placement, KYC.

Eventual consistency is fine where the user can tolerate seeing slightly stale data for a few seconds — search results, recommendation lists, "people you may know," counters on a dashboard. The trade-off is latency and availability: strong consistency typically blocks during partitions, eventual keeps serving.

Step-by-step thought process: 1. Define both precisely — strong consistency = every read reflects the latest write; eventual consistency = replicas converge to the same value given enough time and no new writes. 2. Pick the example — payments wallet must be strong (deduct only if balance allows). A like counter on a post can be eventual (off by 3 for 10 seconds is invisible). 3. Tie to CAP — strong + partition tolerance forces unavailability during a network split (CP system); eventual + partition tolerance keeps serving (AP system). 4. In SQL terms — strong consistency = read on the primary or use `quorum reads`; eventual = read replica or async-replicated cluster.

Edge cases to mention: - "Read your writes" consistency is a useful middle ground — guarantees a user sees their own update even if other users see stale data. - Monotonic reads — once you've seen version N, you never see version N-1; useful for paginating feeds without items shuffling. - Causal consistency — if event A causes event B, every observer sees A before B; weaker than strong but stronger than eventual.

What NOT to say: - "Eventual consistency is weaker so it's worse." It is a deliberate engineering trade for availability and latency; saying "worse" misses the point. - "All NoSQL is eventually consistent." Many are tunable; some default to strong (DynamoDB strongly-consistent reads, MongoDB write-concern majority).

Common follow-up: *"How would you give the user a 'strong read after their write' even on a replica architecture?"* — Sticky-route the user's next read to the primary, or to a replica that has confirmed catch-up via the WAL position the writer returned.

```
-- Postgres pattern — read the WAL position after write, route subsequent reads
-- to a replica that has caught up
SELECT pg_current_wal_lsn(); -- store this; check on replica before reading
```

Q169 · Async Commit in Postgres

Tags: Difficulty: Medium-Hard · Asked at: Swiggy, Flipkart, Cred, Walmart Labs · Topic: Durability

The question (interviewer's verbatim phrasing):

> What does `synchronous_commit = off` do in Postgres? Why would you ever turn off durability?

The 30-second answer: With `synchronous_commit = off`, `COMMIT` returns to the client as soon as the WAL is written to the in-memory buffer; the fsync to disk happens asynchronously in the

background. You get 2-5x higher commit throughput at the cost of losing a small window (typically <600 ms) of recently-committed transactions if the server crashes. Use it for high-write workloads where occasional loss is acceptable — analytics ingest, metrics, low-value telemetry. Never for payments.

Step-by-step thought process: 1. The mechanic — normal `synchronous_commit = on` blocks the commit until the WAL is fsynced to disk. With `off`, the commit returns earlier and a background WAL writer flushes within `wal_writer_delay` (200 ms default). 2. Failure window — a power loss can lose up to ~3x `wal_writer_delay` of commits. The DB itself stays consistent (no torn pages); you only lose the most recent transactions. 3. Per-session control — you can SET `synchronous_commit = off` for a specific session or transaction, letting analytics jobs run async while payments stay sync. 4. Compare with `fsync = off` — that's much more dangerous; it disables fsync entirely and a crash can corrupt the database, not just lose recent commits.

Edge cases to mention: - On replicated clusters, `synchronous_commit = remote_apply` waits for replicas to apply the change — strongest durability but highest latency. - The `on/off` setting interacts with WAL archive lag — if archiving falls behind, you can't restore to the latest committed point regardless of fsync settings. - MySQL's equivalent is `innodb_flush_log_at_trx_commit` — values 0/1/2 trade off similar way (1 is fully durable, 2 fsyncs once per second, 0 is purely buffered).

What NOT to say: - "Async commit corrupts the database." It does not; it only loses recent commits, with consistency intact. - "It's free performance." It's a durability trade — make sure the workload owner accepts it.

Common follow-up: "*For a payments service, where could you safely use async commit?*" — Audit-trail inserts that already have a separate event log; never for the ledger itself.

```
-- Per-session async commit for an analytics ingest job
SET LOCAL synchronous_commit = off;
INSERT INTO event_log SELECT * FROM staging_events;
-- Crash in the next 200 ms can lose these rows; acceptable for telemetry
```

Q170 · Connection Pool Transaction Scope Leakage

Tags: Difficulty: Hard · Asked at: PhonePe, Razorpay, Cred, JP Morgan India · Topic: Transactions

The question (interviewer's verbatim phrasing):

> What is connection pool transaction scope leakage and why is it terrifying?

The 30-second answer: A pooled connection is reused across requests. If one request begins a transaction, fails to commit or rollback (because of an unhandled exception, a `return` before commit, or the framework forgot), the next request that grabs that connection inherits an open transaction. Its

writes are now part of an unintended txn, locks are held across requests, and rollback by the second request can roll back the first's work too. It looks like random data corruption.

Step-by-step thought process: 1. Picture the lifecycle — pool returns conn C to request A. A does `BEGIN`, does work, throws, and returns C to the pool without committing or rolling back. B gets C, does `INSERT`, commits — but commits both its own and A's pending writes (or worse, rolls back). 2. The framework's job — Spring's `@Transactional`, Django's `atomic()`, Rails' `transaction do` are supposed to wrap and clean up. Bare `Connection.setAutoCommit(false)` without a finally clause is the classic leak source. 3. Detection — enable `log_min_duration_statement = 0` plus connection-id logging; look for `BEGIN` from one request id followed by statements from a different request id. 4. Defensive defaults — pool libraries (HikariCP, pgbouncer in transaction mode) can RESET or ROLLBACK on connection release; configure `connectionTestQuery` or `idleTransactionTimeout` to catch leaks.

Edge cases to mention: - In pgbouncer "transaction" pooling mode, prepared statements and session-level state (SET, advisory locks) leak similarly — pgbouncer doesn't isolate them. - ORM auto-flush can hide an open txn — Hibernate's `flush()` doesn't commit but does send writes; if the txn never commits, the writes vanish at next checkout. - Long-running async tasks holding a pooled conn through `await` boundaries can monopolise the conn and effectively starve the pool.

What NOT to say: - "The pool handles cleanup automatically." Only some pools do, only with the right config. - "It's the database's fault." It is your framework's responsibility to demarcate transactions.

Common follow-up: "*How would you write a unit test for this leak?*" — Spy on the pool's checkout/return; assert no connection is returned with `inTransaction() == true`.

```
-- Postgres: detect connections sitting in idle-in-transaction
SELECT pid, state, query_start, NOW() - query_start AS idle_for, query
FROM pg_stat_activity
WHERE state = 'idle in transaction'
ORDER BY idle_for DESC;
-- Set a server-side guard
ALTER SYSTEM SET idle_in_transaction_session_timeout = '30s';
```

Q171 · Read Replica Lag Visibility

Tags: Difficulty: Medium-Hard · Asked at: Swiggy, Flipkart, PhonePe, Walmart Labs · Topic: Replication

The question (interviewer's verbatim phrasing):

> Your dashboard reads from a replica. A user reports their freshly-placed order isn't showing up. Walk me through the diagnosis.

The 30-second answer: Almost certainly replica lag — the write went to the primary, the replica hasn't applied the change yet, and the dashboard is reading the stale replica. Diagnose with `pg_last_wal_replay_lsn()` vs `pg_current_wal_lsn()` (Postgres), `SHOW SLAVE STATUS\G` looking at `Seconds_Behind_Master` (MySQL), or `sys.dm_hadr_database_replica_states` (SQL Server). Fix by routing "read after write" to the primary, or wait for the replica to confirm catch-up using the WAL LSN the writer returned.

Step-by-step thought process: 1. Identify the symptom — write succeeded, immediate read on a replica returns the old data. Common when read traffic is load-balanced across replicas without consistency hints. 2. Measure the lag — in Postgres bytes (`pg_wal_lsn_diff(pg_current_wal_lsn(), pg_last_wal_replay_lsn())`); in MySQL seconds (`Seconds_Behind_Master` though it has known accuracy issues, prefer GTID-based monitoring); in SQL Server, AG dashboard shows `redo_queue_size`. 3. Causes of lag — long single-statement on the replica that blocks apply (vacuum, conflicting lock), network saturation, replica disk slower than primary, large transaction on primary serialised through one replay process. 4. Mitigations — route writes-then-reads to primary using sticky session, or send the WAL LSN with the write and have the client poll until the replica reports `pg_last_wal_replay_lsn() >= writer_lsn`.

Edge cases to mention: - MySQL's `Seconds_Behind_Master` is computed from binlog timestamps and goes to 0 incorrectly when the replica is idle; prefer `pt-heartbeat` or GTID gap monitoring. - Synchronous replication trades latency for consistency — Postgres `synchronous_standby_names` makes the primary block for replica ack; great for finance, slower for everything else. - Read replicas in cloud RDS/Aurora have their own lag metrics in CloudWatch (`ReplicaLag`); set alerts.

What NOT to say: - "Replicas are always behind by a few milliseconds." Sometimes minutes or hours under load — don't blanket-assume. - "Just always read from primary." Defeats the purpose of replicas for scale.

Common follow-up: "*What's read-your-writes consistency and how do you implement it across replicas?*" — Capture the writer's LSN; wait/poll on the replica until applied, then read.

```
-- Postgres lag check
SELECT NOW() - pg_last_xact_replay_timestamp() AS replication_lag_seconds,
       pg_wal_lsn_diff(pg_current_wal_lsn(), pg_last_wal_replay_lsn()) AS bytes_behind;
```

Q172 · Write-Ahead Log Basics

Tags: Difficulty: Medium-Hard · Asked at: Razorpay, Swiggy, Goldman, AmEx · Topic: Durability

The question (interviewer's verbatim phrasing):

> Explain WAL. Why is "write to log first, then the data page" the standard?

The 30-second answer: A Write-Ahead Log is an append-only sequence of every change about to be made to the data pages, fsynced to disk *before* the actual page modification is allowed to be considered durable. If the server crashes after WAL fsync but before the data pages were flushed, recovery replays the WAL forward to reconstruct the state. Append-only sequential writes are faster than random page writes, so WAL is both a durability and a performance device.

Step-by-step thought process: 1. The invariant — for every change C, the WAL record describing C must reach durable storage strictly before the data page holding C is allowed to be considered the canonical version. Engines enforce this with the "WAL rule." 2. The recovery story — on restart, find the last checkpoint, replay WAL from there forward, rolling forward committed txns and rolling back uncommitted ones (ARIES-style). 3. Why it works — random writes to scattered data pages are slow; sequential writes to a single log are fast. Group commit further batches multiple txns' WAL records into one fsync. 4. Common settings — Postgres `wal_buffers`, `wal_writer_delay`, `checkpoint_timeout`. MySQL's redo log is the equivalent (`innodb_log_file_size`, `innodb_flush_log_at_trx_commit`).

Edge cases to mention: - WAL is also the foundation for replication — replicas consume WAL records and apply them; logical decoding reads WAL to emit per-row events. - A WAL disk that fills up halts all writes; monitor `pg_wal` size and the `pg_stat_archiver` lag. - Checkpoints flush dirty pages so old WAL can be recycled; tuning `max_wal_size` and `checkpoint_completion_target` controls the rhythm.

What NOT to say: - "WAL is optional." It is not — every relational engine needs it for crash recovery. - "WAL is the same as the binlog." Binlog in MySQL is for replication; the redo log is for crash recovery. They're distinct.

Common follow-up: "*What's the difference between Postgres WAL and MySQL binlog?*" — WAL is physical (page-level redo); binlog is logical (row events) and used mainly for replication and PITR.

```
-- Postgres: see current WAL position and recent activity
SELECT pg_current_wal_lsn(), pg_walfile_name(pg_current_wal_lsn());
SELECT * FROM pg_stat_wal;
```

Q173 · Logical Replication in Postgres

Tags: Difficulty: Medium-Hard · Asked at: PhonePe, Cred, Razorpay, Swiggy · Topic: Replication

The question (interviewer's verbatim phrasing):

> Compare physical vs logical replication in Postgres. When would you pick logical?

The 30-second answer: Physical replication ships byte-level WAL records — replica is an exact binary copy, same major version, all-or-nothing. Logical replication ships *decoded* row changes (INSERT/UPDATE/DELETE with column values) via the publication-subscription model — you choose which tables and even which rows replicate, can replicate across major versions, and can transform along the way. Pick logical when you need selective replication, zero-downtime major-version upgrades, or to feed a different system (Kafka via wal2json/pgoutput).

Step-by-step thought process: 1. Physical — `pg_basebackup` plus streaming WAL; the replica is bit-identical, including indexes and bloat. Cannot write on the replica. Used for HA and read scaling. 2. Logical — uses logical decoding plugins (`pgoutput` is built-in since v10); a publication on the primary, a subscription on the downstream. Replays row events through normal SQL, so the downstream can have different indexes, even different tables. 3. Limitations of logical — DDL is not replicated automatically (you must run schema changes manually on both sides), sequences can drift, large transactions stream slowly (improved in 14+ with `streaming = on`). 4. Use cases — cross-version upgrades (replicate v13 → v15, then cut over), feeding analytics warehouses, selective region replication.

Edge cases to mention: - A replication slot prevents WAL cleanup until the subscriber catches up — if the subscriber disconnects for days, the primary's disk fills up. Monitor `pg_replication_slots` . - Logical replication requires `wal_level = logical` and a restart; the higher WAL volume costs disk. - Conflict resolution on the subscriber is manual — primary key conflict pauses the subscription until you intervene.

What NOT to say: - "Logical replication is faster." Usually slower per row than physical due to decode overhead. - "It replicates DDL." It does not, except for the basics in Postgres 17+ with experimental support.

Common follow-up: "*How would you do a zero-downtime Postgres major-version upgrade using logical replication?*" — Stand up the new version as a subscriber, let it catch up, briefly pause writes, switch the app over, drop the subscription.

```
-- On the publisher (v13)
ALTER SYSTEM SET wal_level = logical; -- requires restart
CREATE PUBLICATION pub_payments FOR TABLE payments, refunds;

-- On the subscriber (v15)
CREATE SUBSCRIPTION sub_payments
  CONNECTION 'host=primary dbname=app user=replica'
  PUBLICATION pub_payments;
```

Q174 · MySQL 8 Group Replication

Tags: Difficulty: Medium-Hard · Asked at: Flipkart, Walmart Labs, Wells Fargo India, AmEx · Topic: Replication

The question (interviewer's verbatim phrasing):

> What is MySQL Group Replication and how does it differ from traditional async replication?

The 30-second answer: Group Replication is MySQL's built-in synchronous multi-primary (or single-primary) replication using a Paxos-style consensus protocol. Every transaction is certified by a majority of group members before commit — guaranteeing each transaction is either visible everywhere or rolled back, with no split-brain. It contrasts with classic async replication, where the primary commits locally and ships binlog events to replicas without waiting for ack.

Step-by-step thought process: 1. The architecture — 3 or 5 nodes (odd number for quorum), each running mysqld with the group_replication plugin loaded. Single-primary mode (one writer) or multi-primary (all writers) — single-primary is the recommended default. 2. The commit path — txn commits locally, sends write-set to all members, waits for certification (majority ack), then applies. No two members can commit conflicting txns; the loser gets a conflict error. 3. Failure handling — if a member becomes unreachable, the remaining majority continues. A member returning after a partition catches up via distributed recovery from the binlogs. 4. Use cases — replaces classic Master-Slave + manual failover with auto-failover and stronger consistency; the basis for InnoDB Cluster.

Edge cases to mention: - Network latency between members directly impacts commit latency; co-locate the group in one region or one AZ for OLTP. - Multi-primary mode is operationally tricky — schema changes, foreign keys, and gap-locking semantics restrict what you can do; most teams pick single-primary. - Certification database (in-memory) grows with txn rate; flow control kicks in to throttle writers if a slow member can't keep up.

What NOT to say: - "Group Replication is the same as Galera." Galera (used in MariaDB / Percona XtraDB Cluster) is conceptually similar but a separate product with different operational quirks. - "It's async." It is synchronous certification, not async streaming.

Common follow-up: "*What's the difference between InnoDB Cluster and Group Replication?*" — InnoDB Cluster bundles Group Replication + MySQL Router + MySQL Shell into a turnkey HA package.

```
-- Inspect group state on any member
SELECT MEMBER_HOST, MEMBER_STATE, MEMBER_ROLE
FROM performance_schema.replication_group_members;
```

Q175 · SQL Server Always On Availability Groups

Tags: Difficulty: Medium-Hard · Asked at: Wells Fargo India, JP Morgan India, AmEx, Goldman · Topic: Replication

The question (interviewer's verbatim phrasing):

> Explain SQL Server Always On Availability Groups in one paragraph. What are sync vs async commit modes?

The 30-second answer: Always On Availability Groups (AG) is SQL Server's HA + DR feature where a primary replica ships log records to one or more secondary replicas. In *synchronous commit* mode, the primary waits for a secondary to harden the log to disk before acknowledging commit — zero data loss, auto-failover capable. In *asynchronous commit*, the primary commits immediately and ships logs in the background — minimal latency, manual failover only, possible data loss on failure.

Step-by-step thought process: 1. Topology — one primary, up to 8 secondaries (since 2016). Secondaries can be readable (for reporting / backup) or non-readable. 2. Sync vs async — sync = bounded latency to secondary + auto-failover (with a witness), no data loss; async = best for DR across regions where latency would kill OLTP. 3. Listener — a virtual network name that clients connect to; on failover, the listener resolves to the new primary, so app strings don't change. 4. Built on Windows Server Failover Clustering (WSFC) historically; since 2017, cluster-less AGs are supported on Linux too via Pacemaker.

Edge cases to mention: - Readable secondaries use snapshot isolation internally to avoid blocking the log-apply thread — so even your default `READ COMMITTED` reads behave like snapshots there. - Distributed AGs link two AGs together for cross-region DR — useful for geo-redundancy. - Backup can be offloaded to a secondary (preferred-replica setting), reducing primary load.

What NOT to say: - "AG is the same as database mirroring." Mirroring was the predecessor; AG superseded it and is much more capable. - "All replicas are writable." Only the primary is writable; secondaries are read-only.

Common follow-up: "*How would you choose between sync and async commit for a tier-1 payments app?*" — Sync for the within-region secondary (zero loss + auto-failover), async for the cross-region DR replica (latency-tolerant).

```
-- Inspect AG health on the primary
SELECT replica_server_name, synchronization_health_desc,
       synchronization_state_desc, last_commit_time
FROM sys.dm_hadr_availability_replica_states ars
JOIN sys.availability_replicas ar ON ars.replica_id = ar.replica_id;
```

Q176 · Snapshot Isolation Engine Comparison

Tags: Difficulty: Hard · Asked at: Razorpay, PhonePe, Goldman, JP Morgan India · Topic: Isolation

The question (interviewer's verbatim phrasing):

> Snapshot isolation exists in Postgres, SQL Server, Oracle and InnoDB but they implement it differently. Compare them.

The 30-second answer: All four use MVCC — each transaction sees a consistent snapshot of committed data as of its start. Postgres stores every version inline with the heap, marked by `xmin` / `xmax`, cleaned by VACUUM. Oracle uses undo segments — current row is "live," prior versions reconstructed from undo. SQL Server uses tempdb-based version store, populated when you opt in via `ALLOW_SNAPSHOT_ISOLATION`. InnoDB uses rollback segments much like Oracle. The differences manifest in bloat behaviour, long-txn cost, and operational ceiling.

Step-by-step thought process: 1. Postgres — version visibility tracked via `xmin` / `xmax` on every row; dead tuples accumulate until autovacuum removes them; long-running snapshots block vacuum and cause "bloat." 2. Oracle — undo segments hold prior versions; ORA-01555 "snapshot too old" fires when undo is recycled before a long read finishes. Operationally, set `UNDO_RETENTION` appropriately. 3. SQL Server — versions live in tempdb; you must explicitly enable `ALLOW_SNAPSHOT_ISOLATION` and/or `READ_COMMITTED_SNAPSHOT` (RCSI). Long snapshots make tempdb grow; sizing tempdb matters. 4. InnoDB — rollback segments in the system tablespace (now in undo tablespaces in 8.0). Similar long-txn cost; `purge` thread cleans up.

Edge cases to mention: - Postgres' SSI (Q157) is an upgrade *on top of* snapshot isolation — only Serializable adds the conflict detection layer. - SQL Server's RCSI changes Read Committed behaviour for the *whole database*; turning it on retroactively can shift query results in subtle ways. - Oracle and SQL Server can be tuned to avoid version-store overflow; Postgres' answer is to keep transactions short and vacuum aggressive.

What NOT to say: - "MySQL doesn't have snapshot isolation." InnoDB does — its default Repeatable Read is essentially snapshot-based MVCC. - "All engines bloat the same way." They don't — Postgres in-heap dead tuples vs Oracle/SQL Server centralised version store have very different operational profiles.

Common follow-up: "*Why does a long-running SELECT in Postgres cause table bloat even if it only reads?*" — Because vacuum cannot remove dead tuples that might still be visible to that snapshot; the table grows.

```
-- Postgres: find oldest active snapshot blocking vacuum
SELECT pid, NOW() - xact_start AS txn_age, state, query
FROM pg_stat_activity
WHERE state <> 'idle' AND xact_start IS NOT NULL
ORDER BY xact_start
LIMIT 5;
```

Q177 · The 40001 Serialization Failure and Retry Pattern

Tags: Difficulty: Medium-Hard · Asked at: Razorpay, PhonePe, Cred, Swiggy, JP Morgan India · Topic: Isolation

The question (interviewer's verbatim phrasing):

> Your Postgres app is throwing SQLSTATE 40001 in production. What does it mean and how do you handle it?

The 30-second answer: 40001 is "serialization failure" — Postgres aborted your transaction because committing it would have produced a non-serializable schedule with another concurrent txn (under Serializable isolation), or because of a snapshot conflict (under Repeatable Read). It's not a bug; it's the engine telling you to retry. The standard handler is: catch the SQLSTATE, back off with jitter, retry the whole transaction (not just the failing statement) up to a small number of times.

Step-by-step thought process: 1. Decode the error — class 40 is "Transaction Rollback"; 40001 specifically is `serialization_failure`. Postgres also has 40P01 (`deadlock_detected`); some retry policies handle both. 2. Why retrying mid-transaction does not work — the snapshot is invalid; you must `BEGIN` again from scratch to get a fresh snapshot. 3. The pattern — try/catch around the whole `BEGIN...COMMIT`. On 40001, sleep `random(0..base * 2^attempt)` (exponential with jitter), retry up to 3-5 times, then surface to the user as "please try again." 4. Causes — at Serializable, it's a true read-write cycle (Q157). At Repeatable Read, it's an UPDATE losing the snapshot race to a concurrent UPDATE on the same row.

Edge cases to mention: - Make the retry idempotent — your business logic must not double-charge or double-ship if it runs twice. Pair with idempotency keys (Q167). - Some ORMs swallow 40001 as a generic exception; check that your retry handler actually sees the SQLSTATE, not just the message. - Excessive 40001s indicate contention hotspots; analyse the queries involved and consider Read Committed + advisory locks if Serializable's overhead is too high.

What NOT to say: - "Just lower the isolation to Read Committed." That trades retries for incorrect results — only valid if you've reasoned about each transaction's invariants. - "Retry forever." Always cap the attempts; runaway retries amplify load during incidents.

Common follow-up: "*How do you tell whether 40001 is from SSI or from a UPDATE conflict?*" — Look at the `pg_stat_database` `xact_rollback` rate together with the message text; SSI errors say "could not serialize access due to read/write dependencies," update conflicts say "concurrent update."

```
-- Application-side pattern (pseudo-SQL/Python)
-- for attempt in range(MAX_RETRIES):
--     try:
--         BEGIN ISOLATION LEVEL SERIALIZABLE;
--         ... do work ...
--         COMMIT;
--         break
--     except SerializationFailure (40001):
--         ROLLBACK;
--         time.sleep(random.uniform(0, 0.1 * (2 ** attempt)))
-- else: raise to user
```

Q178 · Snowflake Clustering Keys vs Auto-Clustering

Tags: Difficulty: Hard · Asked at: Walmart Labs, Target, Razorpay data team, EXL · Topic: Snowflake

The question (interviewer's verbatim phrasing):

> What's the difference between defining a clustering key on a Snowflake table and letting Snowflake handle clustering automatically? When would you bother defining one?

The 30-second answer: Defining a clustering key tells Snowflake which columns to co-locate within micro-partitions; auto-clustering is the background service that maintains that ordering as you insert and delete. You only define a clustering key on multi-terabyte tables where queries consistently filter on the same one or two columns and where the cost of clustering credits is less than the cost of full-partition scans.

Step-by-step thought process: 1. Open with the basics — Snowflake stores data in immutable 50-500 MB compressed micro-partitions and prunes them at query time using per-partition min/max metadata. 2. Without a clustering key, partition order is whatever load order gave you. For a small table or a table queried on the load-order column (usually a timestamp), that's fine. 3. A clustering key is a hint — `CLUSTER BY (region, order_date)` tells Snowflake to keep rows with the same region and date close together so the pruner can skip more partitions. 4. Auto-clustering is the credit-billed background process that re-clusters partitions when clustering depth degrades. You do not run it manually; you turn it on with `ALTER TABLE ... RESUME RECLUSTER`. 5. Anticipate the follow-up: "How do you know clustering is working?" Use `SELECT SYSTEM$CLUSTERING_INFORMATION('orders')` — it returns `average_depth`, `partition_count`, and clustering ratio.

Edge cases to mention: - Clustering keys add ongoing credit cost — high-churn tables can spend more on reclustering than they save on scans. - Cardinality matters — clustering on a column with 5 distinct values does nothing; clustering on a unique key is wasteful. Pick columns with thousands to millions of

distinct values and frequent range filters. - If the table is already naturally clustered by load order (e.g. append-only event log loaded by date), do not add a clustering key — you're paying for nothing.

What NOT to say: - Do not say "clustering keys are like indexes." Snowflake has no traditional indexes — clustering is about physical co-location, not a separate B-tree. - Do not cluster on more than 3-4 columns — past that, Snowflake's micro-partition metadata stops helping.

Common follow-up: "*How is clustering depth computed and what's a good number?*" — Average depth is the average number of partitions that overlap on the clustering key range. Below 4 is great, above 100 means re-clustering is overdue or the key is wrong.

```
-- Cluster a large orders table by region + month
ALTER TABLE orders CLUSTER BY (region, DATE_TRUNC('MONTH', order_ts));

-- Check clustering health
SELECT SYSTEM$CLUSTERING_INFORMATION('orders', '(region, order_ts)');
```

Q179 · Snowflake Micro-Partitions — The Storage Unit

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target, Genpact, Mu Sigma · Topic: Snowflake

The question (interviewer's verbatim phrasing):

> Explain Snowflake micro-partitions in plain language. How do they make scans fast?

The 30-second answer: Micro-partitions are immutable, columnar, compressed files of about 50-500 MB uncompressed (16 MB compressed) that Snowflake creates as you load data. Every micro-partition carries metadata — min/max per column, distinct counts, null counts — and the query planner uses that metadata to skip entire partitions without reading them. That's pruning, and it's the reason a well-filtered query on a billion-row table can read 0.1 percent of the data.

Step-by-step thought process: 1. Lead with the physical layout — columnar storage inside the partition, so reads only fetch the columns the query needs. 2. Explain the pruning chain: parse query, extract WHERE predicates, compare against per-partition min/max metadata in the cloud services layer, return only the partition list to the worker. 3. Mention that micro-partitions are immutable — UPDATE and DELETE in Snowflake actually create new partitions and mark the old ones as deleted (kept around for time travel). 4. Be ready for the follow-up: "Why does SELECT * defeat pruning?" Answer: pruning skips partitions, not columns — but SELECT * forces every column file in every surviving partition to be read.

Edge cases to mention: - Pruning only works on predicates the optimizer recognizes — wrapping a column in a function (`WHERE TO_DATE(order_ts) = '2026-01-01'`) usually defeats it. Use range predicates. - Loading data out of order (random INSERT) creates partitions with wide min/max ranges

that prune poorly — bulk-load sorted data when possible. - Very small loads create many tiny partitions and hurt prune efficiency — use `COPY INTO` with reasonable batch sizes.

What NOT to say: - Do not call micro-partitions "blocks" or "pages" — that's RDBMS vocabulary and Snowflake folks will correct you. - Do not say "Snowflake has no indexes so it scans everything." Pruning is the index equivalent.

Common follow-up: "*How would you measure pruning effectiveness for a query?*" — Run the query, then check the query profile — the `Partitions scanned` vs `Partitions total` ratio is your pruning rate.

```
-- This prunes well — date range on a clustered column
SELECT SUM(amount) FROM payments
WHERE payment_ts BETWEEN '2026-01-01' AND '2026-01-31';

-- This prunes poorly — function wraps the column
SELECT SUM(amount) FROM payments
WHERE MONTH(payment_ts) = 1;
```

Q180 · Snowflake Time Travel and Zero-Copy Cloning

Tags: Difficulty: Medium · Asked at: Walmart Labs, JP Morgan India, EXL · Topic: Snowflake

The question (interviewer's verbatim phrasing):

> Walk me through Snowflake time travel and zero-copy cloning. How do they work and where would you use each?

The 30-second answer: Time travel lets you query a table as it existed at any point within the retention window (1 day for standard, up to 90 days for enterprise editions) using `AT (TIMESTAMP => ...)` or `BEFORE (STATEMENT => ...)`. Zero-copy cloning creates a metadata-only copy of a table, schema, or database — both clone and source share the same underlying micro-partitions until one of them writes. Cloning is instant and free; the storage cost only grows as the two diverge.

Step-by-step thought process: 1. Start with what time travel solves — "I dropped the wrong rows, can I recover them?" Answer: `INSERT INTO orders SELECT * FROM orders AT (TIMESTAMP => '2026-01-15 14:00')` — done. 2. Mention the retention knob — `DATA_RETENTION_TIME_IN_DAYS` at account, database, schema, or table level. After retention expires, data falls into Fail-safe (Snowflake-managed, 7 days, not user-queryable). 3. Cloning is the other half — `CREATE TABLE orders_clone CLONE orders` creates a new logical table sharing partitions. Useful for cheap dev environments, branch-style data testing, and pre-prod validation. 4.

Anticipate: "Are clones writable?" Yes — and writes are copy-on-write at the micro-partition level, so a clone you barely modify costs almost nothing.

Edge cases to mention: - Time travel costs storage — every retained version is paid for. Keeping 90-day retention on a churn-heavy table can quadruple your storage bill. - Cloning a table does not clone load history or pipes — Snowpipe ingestion against a clone is a separate setup. - Transient and temporary tables have shorter retention and no Fail-safe — do not rely on time travel for them.

What NOT to say: - Do not claim cloning is a backup. It shares storage; if you `DROP` the source and pass retention, you lose both. - Do not say time travel is free. The data has to be retained somewhere.

Common follow-up: "*How would you stand up a pre-production environment from prod in under a minute?*" — Clone the database. Schemas, tables, views all clone in seconds. Then point the dev warehouse at the cloned database.

```
-- Recover rows deleted by a bad statement
INSERT INTO orders
SELECT * FROM orders BEFORE (STATEMENT => '0193-abc-def-...')
WHERE order_id IN (...);

-- Snapshot prod for staging in one statement
CREATE DATABASE prod_clone_20260526 CLONE prod;
```

Q181 · BigQuery Partition Pruning and Clustering

Tags: Difficulty: Medium · Asked at: Flipkart data team, Swiggy, PhonePe, Walmart Labs · Topic: BigQuery

The question (interviewer's verbatim phrasing):

> When I write a query against a partitioned and clustered BigQuery table, how does the engine decide what to read?

The 30-second answer: BigQuery first prunes by partition — for a time-partitioned table, only partitions matching the `WHERE partition_column = ...` predicate are touched. Within each surviving partition, clustering further sorts rows by the clustering columns and stores block-level min/max metadata, so the engine can skip blocks that cannot contain matching rows. The combination — partition prune, then cluster skip — is what makes a 50 TB table cheap to query for a single day's worth of data.

Step-by-step thought process: 1. Lead with partitioning — only DATE, TIMESTAMP, DATETIME, or integer-range partitioning is allowed. Maximum 4,000 partitions per table. 2. Clustering — up to 4 columns, applied in order, stored as block-level sort within each partition. No separate index structure;

the sort and metadata is the index. 3. The pruning ladder: filter by partition column first (in WHERE), then by leading clustering column. Skipping the partition column in WHERE makes BigQuery scan the whole table. 4. Be ready for: "Why did adding a filter not reduce bytes scanned?" Common cause: predicate references a non-partition column, or wraps the partition column in a function (`WHERE DATE(event_ts) = '2026-01-01'` may or may not prune depending on the partition column type — TIMESTAMP partitioning needs raw timestamp comparison).

Edge cases to mention: - Require partition filter on the table (`require_partition_filter = TRUE`) — guarantees no one accidentally scans the whole table. Common safety guardrail for big tables. - Clustering only helps filtering and aggregation on the leading clustering columns. Putting `user_id` first then filtering only on `event_type` skips no blocks. - Re-clustering is automatic and free, but only happens when BigQuery decides cluster depth has degraded. For fresh tables, query speed may not be optimal until a few hours after load.

What NOT to say: - Do not confuse BigQuery partitioning with Hive-style external partitioning. BigQuery partitions are managed metadata, not directory structure (unless you're querying an external table). - Do not say "clustering is like indexing." There's no separate B-tree; clustering is row reordering plus block metadata.

Common follow-up: "*How would you find out whether a query pruned partitions?*" — Look at the query plan in the BigQuery UI, or run `SELECT * FROM ... WHERE ...` with the `dry_run` option to see `totalBytesProcessed` before actually running.

```
-- Create a partitioned + clustered table
CREATE TABLE analytics.events (
  event_ts TIMESTAMP, user_id STRING, event_type STRING, payload JSON
)
PARTITION BY DATE(event_ts)
CLUSTER BY user_id, event_type
OPTIONS(require_partition_filter = TRUE);

-- Prunes one partition, skips most blocks
SELECT COUNT(*) FROM analytics.events
WHERE DATE(event_ts) = '2026-01-15' AND user_id = 'u_8742';
```

Q182 · BigQuery Slots vs Reservations

Tags: Difficulty: Hard · Asked at: Flipkart, PhonePe data platform, Walmart Labs, Target · Topic: BigQuery

The question (interviewer's verbatim phrasing):

> Explain BigQuery slots and reservations. How does on-demand pricing differ from a reservation, and when would you switch?

The 30-second answer: A slot is a unit of compute — roughly one virtual CPU plus memory — that executes a stage of a query. On-demand pricing gives you a shared pool with a soft cap of 2,000 slots per project, billed per byte scanned. A reservation pre-purchases a fixed number of slots (with monthly or annual commitments) and bills flat-rate, regardless of bytes scanned. You switch to reservations when monthly on-demand spend exceeds the equivalent reservation cost, when you need predictable performance, or when one heavy team is starving everyone else.

Step-by-step thought process: 1. Start with the billing models — on-demand bills `$X per TB scanned`, reservations bill `$Y per slot-hour committed`. 2. A slot processes one stage of one query at a time. Big queries fan out across hundreds of slots; small queries use a handful. 3. Reservations allow assignments — you create a reservation, assign projects or folders to it, and queries from those projects consume that reservation's slots. Unassigned projects fall back to on-demand. 4. Mention autoscaling editions (Standard, Enterprise, Enterprise Plus) — these let slot count flex between a baseline and max, billed per slot-second used. 5. Be ready for: "How do you size a reservation?" Run on-demand for a month, look at the `INFORMATION_SCHEMA.JOBS_BY_PROJECT` view, sum `total_slot_ms`, divide by the time window — that's your average slot-second demand.

Edge cases to mention: - A reservation can starve a query if every slot is busy — queries queue. On-demand has dynamic burst capacity that reservations do not, unless you enable autoscaling. - Idle slots in one reservation can be shared with other reservations (idle slot sharing) — useful when one team's queries spike outside business hours. - Flat-rate is not always cheaper for low-volume teams; on-demand wins when you only scan a few TB per month.

What NOT to say: - Do not claim reservations are always faster. If the reservation is undersized for the workload, queries queue and feel slower than on-demand. - Do not confuse slot count with concurrency limits — they are independent. Concurrency caps how many queries run; slots cap the parallelism inside each query.

Common follow-up: "*A team is paying $50,000 per month on-demand. Would a reservation help?*" — Probably yes, but quantify it. 50K / (current slot-second consumption) tells you the per-slot cost; compare against the listed price per slot-month for Enterprise edition.

```
-- See what slots queries actually used last week
SELECT user_email, SUM(total_slot_ms) / 1000 / 60 / 60 AS slot_hours
FROM `region-us`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
WHERE creation_time >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)
GROUP BY user_email
ORDER BY slot_hours DESC;
```

Q183 · BigQuery Bytes-Scanned Billing Optimization

Tags: Difficulty: Medium · Asked at: Flipkart, Swiggy data, Walmart Labs, PhonePe · Topic: BigQuery

The question (interviewer's verbatim phrasing):

> Our BigQuery bill jumped 4x this month and the team is panicking. How would you find and fix the expensive queries?

The 30-second answer: Bytes scanned drives on-demand cost. Find the offenders in `INFORMATION_SCHEMA.JOBS_BY_PROJECT` ordered by `total_bytes_billed`, then rewrite — narrow column list (kill SELECT star), add partition filters, push predicates before joins, materialize repeated subqueries into smaller intermediate tables. Also check for missing partition filters and accidental cross-region scans, both of which can 10x cost overnight.

Step-by-step thought process: 1. Open with the diagnostic — `INFORMATION_SCHEMA.JOBS_BY_PROJECT` over the past week, sorted by `total_bytes_billed` and grouped by user and SQL hash, surfaces the top spenders fast. 2. Quick wins: replace `SELECT *` with the actual column list — BigQuery is columnar, only requested columns are read. On a 200-column wide table this alone can cut cost by 95 percent. 3. Partition filters — make sure every query against a partitioned table has a predicate on the partition column. Consider `require_partition_filter = TRUE` to enforce. 4. Materialization — if the same expensive subquery appears in 10 dashboards, materialize it as a daily-refreshed table. 5. Reservations — if optimization gets you to a stable monthly spend, that's also when reservation pricing usually beats on-demand.

Edge cases to mention: - A WITH clause in BigQuery is re-evaluated each time it's referenced — if a CTE is used 4 times in the same query, you pay for it 4 times. Materialize it explicitly. - Wildcard table queries (`FROM` project.dataset.events_* `` `) scan every matching table — apply `` _TABLE_SUFFIX` filters or migrate to a partitioned table. - Cross-region queries pay egress on top of bytes scanned. Always check the dataset region.

What NOT to say: - Do not say "use LIMIT to save money." LIMIT does not reduce bytes scanned — it only reduces bytes returned to the client. - Do not blame the analysts without showing them the cost — surface the data with a `SELECT * FROM JOBS_BY_PROJECT ORDER BY bytes_billed DESC LIMIT 50` and let the conversation happen.

Common follow-up: "*How do you stop a specific user from running expensive queries?*" — Quota policies — `maximum_bytes_billed` at the query level, or daily user quotas at the project level via IAM.

```
-- Top 20 most expensive queries last 7 days
SELECT user_email, query, total_bytes_billed / POW(1024, 4) AS tb_billed
FROM `region-us`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
WHERE creation_time >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)
  AND statement_type = 'SELECT'
ORDER BY total_bytes_billed DESC
LIMIT 20;
```

Q184 · Postgres Logical vs Streaming Replication

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Zerodha, Goldman · Topic: Postgres

The question (interviewer's verbatim phrasing):

> When would you pick logical replication over streaming replication in Postgres?

The 30-second answer: Streaming (physical) replication ships byte-for-byte WAL to a standby — exact replica, all or nothing, same Postgres major version. Logical replication decodes WAL into row-level change events and ships them per-publication — selective tables, cross-version, even cross-platform. Use streaming for HA and read replicas; use logical for selective subscription, blue-green major-version upgrades, and feeding downstream systems like a data warehouse.

Step-by-step thought process: 1. Start with the unit of replication — physical replicates WAL records byte-by-byte; logical replicates row-level INSERT/UPDATE/DELETE events through publications and subscriptions. 2. Physical replicas are read-only, exact copies, and the standby must be the same Postgres major version with the same operating system page format. Failover is fast. 3. Logical lets you publish only `public.orders` and subscribe selectively. The subscriber is fully writable, can be a different major version, and can have a different schema for the subscribed tables (as long as types are compatible). 4. Anticipate: "Why is logical more expensive?" Because decoding WAL into logical change events is CPU-intensive, and the publisher must keep a replication slot that retains WAL until the subscriber acknowledges — runaway lag can fill the disk.

Edge cases to mention: - Logical replication does not replicate DDL — schema changes must be applied separately to publisher and subscriber. - Tables under logical replication must have a `REPLICA IDENTITY` — primary key by default, or `FULL` for tables without one. Without it, UPDATE and DELETE cannot be replicated. - Sequences, large objects, and TRUNCATE handling have caveats — TRUNCATE became replicable in Postgres 11; sequences need explicit handling.

What NOT to say: - Do not say "logical replication is better because it's newer." Streaming is rock-solid for HA; pick the tool that fits. - Do not forget that logical slots can fill the disk — always monitor `pg_replication_slots.confirmed_flush_lsn` and set `max_slot_wal_keep_size`.

Common follow-up: "*How would you do a major-version upgrade with near-zero downtime?*" — Set up logical replication from old primary to a new-version standby, let it catch up, fail over.

```
-- Publisher side
CREATE PUBLICATION ledger_pub FOR TABLE payments, refunds;

-- Subscriber side (on a Postgres 16 instance subscribing to a PG 14 primary)
CREATE SUBSCRIPTION ledger_sub
  CONNECTION 'host=primary.internal dbname=app user=replicator'
  PUBLICATION ledger_pub;
```

Q185 · Postgres 16 Logical Replication Improvements

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postgres-heavy GCCs · Topic: Postgres

The question (interviewer's verbatim phrasing):

> What changed in Postgres 16 around logical replication, and why does it matter for production?

The 30-second answer: Postgres 16 added logical replication from physical standbys (so a standby can be a publisher), parallel apply of large in-progress transactions on the subscriber, and the ability to use any column as a replica identity, not just the primary key. Together these reduce primary load, cut subscriber lag for big transactions, and let you replicate tables that don't have a natural PK.

Step-by-step thought process: 1. Lead with the standby-publisher feature — before Postgres 16, only the primary could publish. Now you can offload logical decoding to a hot standby, removing CPU pressure from the primary. 2. Parallel apply — large transactions (above `logical_decoding_work_mem`) used to stream to the subscriber serially. Postgres 16 can apply them in parallel using multiple worker processes, dramatically cutting lag during bulk loads. 3. Custom replica identity — `REPLICA IDENTITY USING INDEX my_unique_idx` now works on more cases, letting tables with composite uniqueness replicate cleanly. 4. Be ready for: "Is there a catch?" Yes — parallel apply needs `streaming = parallel` on the subscription, and the subscriber must have enough `max_logical_replication_workers`.

Edge cases to mention: - Standby publishers still require the primary to keep WAL — replication slots cannot be created on a standby that is too far behind. - Parallel apply has overhead — small transactions may actually run slower. The win is concentrated on bulk updates. - Migrating an existing subscription to `streaming = parallel` needs a DROP and CREATE of the subscription, not just ALTER.

What NOT to say: - Do not say "logical replication is now as fast as streaming." Streaming is still the throughput champion. Logical is more flexible, not faster. - Do not confuse Postgres 16 changes with Postgres 14's binary mode or PG 15's row filters and column lists — different features.

Common follow-up: *"What's still missing in Postgres logical replication that you wish was there?"* — Built-in DDL replication. As of 16, schema changes are still a manual coordination problem; teams reach for tools like pglogical or Bucardo to close that gap.

```
-- Postgres 16+ subscription using parallel apply
CREATE SUBSCRIPTION payments_sub
  CONNECTION '...'
  PUBLICATION payments_pub
  WITH (streaming = parallel);
```

Q186 · MERGE — Postgres 15+ Syntax

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Cred, Goldman · Topic: Postgres

The question (interviewer's verbatim phrasing):

> Postgres 15 brought MERGE as a first-class statement. How does it differ from INSERT ON CONFLICT, and when would you reach for each?

The 30-second answer: MERGE is the SQL standard upsert — it joins a source against a target and lets you specify WHEN MATCHED, WHEN NOT MATCHED, and WHEN NOT MATCHED BY SOURCE actions per row. INSERT ON CONFLICT is Postgres-specific and only handles the conflict-on-unique-constraint case. Use MERGE when you need to mix INSERT, UPDATE, and DELETE in one statement against a complex source; use INSERT ON CONFLICT for simple per-row upserts where you only care about uniqueness.

Step-by-step thought process: 1. Start with the semantic difference — INSERT ON CONFLICT runs row-by-row and only triggers on a unique constraint violation. MERGE is a set-based operation matched via a join condition. 2. MERGE can do three things in one statement — insert new rows, update matched rows, delete rows that no longer exist in the source. That's the classic SCD-style merge pattern. 3. Postgres 17 added WHEN NOT MATCHED BY SOURCE support (Postgres 15 lacked it, requiring a separate DELETE for the target-only rows). 4. Anticipate: "Is MERGE atomic?" Each statement is one transaction step, so yes — either the whole MERGE commits or it rolls back. But MERGE is not safe under high concurrency unless you SERIALIZABLE or SELECT FOR UPDATE the source.

Edge cases to mention: - MERGE in Postgres does not raise on duplicate matches the way some dialects do — by default, multiple matches on the same target row are an error to prevent silent data loss. - INSERT ON CONFLICT can use `DO NOTHING` for soft-fail upserts; MERGE cannot silently skip — you write the equivalent `WHEN MATCHED THEN UPDATE` clause that no-ops. - RETURNING clause is supported on INSERT ON CONFLICT but only added to MERGE in Postgres 17 — keep that in mind if you need affected rows back.

What NOT to say: - Do not claim MERGE replaces INSERT ON CONFLICT in all cases. Both have their niche. - Do not write MERGE without thinking about concurrency — under READ COMMITTED, two MERGEs can race and produce duplicates.

Common follow-up: "*Show me a MERGE that synchronizes a daily customer dump into a master table.*" — Practice this; it's a real load pattern.

```
MERGE INTO customers c
USING customer_dump_today d ON c.customer_id = d.customer_id
WHEN MATCHED AND d.updated_ts > c.updated_ts THEN
  UPDATE SET name = d.name, email = d.email, updated_ts = d.updated_ts
WHEN NOT MATCHED THEN
  INSERT (customer_id, name, email, updated_ts)
  VALUES (d.customer_id, d.name, d.email, d.updated_ts);
```

Q187 · Postgres SQL/JSON Path Queries — @? and @@

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, GCCs working on event stores · Topic: Postgres

The question (interviewer's verbatim phrasing):

> Walk me through the SQL/JSON path operators @? and @@ in Postgres. When do they help and what's the difference between them?

The 30-second answer: Both operators evaluate a JSON path expression against a jsonb value. `@?` returns TRUE if the path matches any item — it's an existence check. `@@` returns TRUE only when the path predicate evaluates to a boolean TRUE. You use `@?` to ask "does this JSON contain anything at this path?", and `@@` to ask "does this JSON contain a value at this path that satisfies a condition?"

Step-by-step thought process: 1. Lead with the SQL/JSON path language — `$` is the root, `.field` and `[*]` traverse, `?(...)` filters. Postgres adopted this from the SQL:2016 standard. 2. `@?` returns true if at least one item exists at the path — perfect for "rows where this attribute is present at all". 3. `@@` returns true if the predicate evaluates to a boolean — perfect for "rows where the value at this path is greater than 1000" or similar. 4. Both can use a GIN index on jsonb with the `jsonb_path_ops` operator class for fast lookups. 5. Be ready for: "How is this different from the older `->` and `?` operators?" Answer: `->` extracts; `?` checks top-level key existence only. SQL/JSON path can traverse deeply with one expression.

Edge cases to mention: - SQL/JSON path is strict by default — missing keys raise errors. Use the lax mode (default in Postgres path functions) which silently returns empty. - GIN indexes on jsonb with default ops support `@>`, `?`, `?&`, `?|`. For `@?` and `@@`, use the `jsonb_path_ops` opclass and an index expression that wraps the path. - Path queries do not use a B-tree index even if you index a specific JSON field — you'd use a functional B-tree index for that, not a GIN.

What NOT to say: - Do not claim `@?` and `@@` are interchangeable. The first returns existence; the second evaluates a boolean. - Do not say "SQL/JSON path is just JSONPath." It's a SQL-standard variant — similar syntax, different evaluation semantics.

Common follow-up: "*Index a jsonb column so deep path queries are fast.*" — Build a GIN with `jsonb_path_ops`, then add a functional index on the specific path if a single attribute is queried 100x more than others.

```
-- Rows where the payload has any item with amount > 1000
SELECT * FROM events
WHERE payload @? '$.items[*] ? (@.amount > 1000)';

-- Rows where payment.method is exactly 'upi'
SELECT * FROM events
WHERE payload @@ '$.payment.method == "upi"';
```

Q188 · BRIN vs B-tree on Append-Only Big Tables

Tags: Difficulty: Hard · Asked at: Postgres-heavy GCCs, Razorpay infra, Cred · Topic: Postgres

The question (interviewer's verbatim phrasing):

> You have a billion-row event table that's append-only and queried mostly by date range. Would you index it with B-tree or BRIN, and why?

The 30-second answer: BRIN — Block Range Index — stores min/max values per page range (default 128 pages) and is ideal when data is naturally clustered on disk by the indexed column. For an append-only event table where rows arrive ordered by timestamp, BRIN gives you a tiny index (kilobytes vs gigabytes), fast date-range pruning, and almost zero write overhead. B-tree is the right choice when you need lookups for individual rows or the column isn't physically clustered.

Step-by-step thought process: 1. Open with the structural difference — B-tree maps every row to a leaf; BRIN maps page ranges to per-range min/max. 2. BRIN size is proportional to table-pages-divided-by-pages-per-range, not row count. A 1 TB table with default settings might have a 1 MB BRIN. 3. The catch: BRIN only helps if data is physically ordered (or close to it). On a hot table where inserts go to the end and the indexed column grows monotonically (like a timestamp), BRIN is brilliant. On a randomly inserted column, it's useless. 4. Be ready for: "What about lookups by primary key?" BRIN does not help random point lookups — you'd still keep a B-tree on the PK.

Edge cases to mention: - The `pages_per_range` parameter is tunable — smaller ranges give finer pruning at the cost of larger index. Tune based on query selectivity. - VACUUM updates BRIN ranges; without it, deleted rows may not be reflected in the min/max and pruning stays correct but less aggressive. - BRIN cannot enforce uniqueness — for unique constraints, you still need a B-tree.

What NOT to say: - Do not say "BRIN is faster than B-tree." It's smaller and cheaper to maintain on the right workload, not faster on lookups. - Do not put BRIN on every column — it adds maintenance cost during heap updates and is only useful where data is physically clustered.

Common follow-up: "*On a multi-tenant SaaS where rows are clustered by tenant_id, would BRIN work on tenant_id?*" — Only if you physically CLUSTER the table on tenant_id; otherwise inserts will fragment and BRIN ranges will become useless.

```
-- BRIN on an append-only event log
CREATE INDEX events_brin ON events
  USING BRIN (event_ts) WITH (pages_per_range = 64);

-- Will prune to the page ranges containing 2026-05-26
SELECT COUNT(*) FROM events
WHERE event_ts BETWEEN '2026-05-26' AND '2026-05-27';
```

Q189 · pgvector — Vector Similarity Search in Postgres

Tags: Difficulty: Hard · Asked at: Indian AI startups, Razorpay ML, Swiggy search · Topic: Postgres

The question (interviewer's verbatim phrasing):

> We need semantic search over product descriptions and want to stay in Postgres. What does pgvector give us, and what are the limits?

The 30-second answer: pgvector is a Postgres extension that adds a `vector` data type plus distance operators (`<->` Euclidean, `<#>` inner product, `<=>` cosine) and two approximate-nearest-neighbour indexes — IVFFlat and HNSW. You store embeddings as vectors, build an HNSW index, and run `ORDER BY embedding <=> query_vector LIMIT 10`. It scales well into the tens of millions of vectors per node; beyond that you start looking at dedicated vector DBs.

Step-by-step thought process: 1. Lead with the value — keeping vectors in Postgres alongside your relational data means joins and filters work naturally. No second system to sync. 2. The two index types — IVFFlat (clusters vectors into lists, picks nearest list at query time) is older, smaller, faster to build. HNSW (hierarchical graph) is newer in pgvector 0.5+, larger, slower to build, but typically higher recall at the same speed. 3. Dimensions and types — modern embedding models are 768 or 1536 dims; pgvector supports up to 16,000. `halfvec` (16-bit floats) cuts storage in half with little accuracy loss. 4. Be ready for: "What's the limit?" HNSW index build memory grows with vector count and dimension — building over 50M rows can exhaust RAM. Beyond that point, look at sharding or move to Pinecone/Weaviate/ Milvus.

Edge cases to mention: - Filters before the ANN search defeat the index — Postgres has post-filter and pre-filter options; if you need both attribute filters and similarity ranking, partial indexes per category often beat a single global index. - Cosine similarity needs L2-normalized vectors; otherwise `<=>` gives you the cosine distance of unnormalized vectors which is not what you want. - Recall is tunable — `ef_search` and `lists` are the knobs. Default values trade recall for speed; raise them when accuracy matters.

What NOT to say: - Do not say "pgvector is the same as a dedicated vector DB." For sub-100M vectors with relational filters it's often better; past that, dedicated systems win on memory and throughput. - Do not store full-precision 1536-dim vectors on a small Postgres instance without thinking about disk — 1M vectors at 1536 dims is roughly 6 GB.

Common follow-up: "*Build a hybrid search — BM25 keyword plus vector — in Postgres.*" — Combine `ts_rank_cd` on a `tsvector` with `1 - (embedding <=> query)` and blend with a weighted sum.

```
CREATE EXTENSION vector;

CREATE TABLE products (
  id BIGSERIAL PRIMARY KEY,
  name TEXT,
  embedding vector(768)
);

CREATE INDEX ON products USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

-- Top 5 most similar products
SELECT name FROM products
ORDER BY embedding <=> $1 LIMIT 5;
```

Q190 · CREATE INDEX CONCURRENTLY and Parallel Index Builds

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Zerodha, GCCs · Topic: Postgres

The question (interviewer's verbatim phrasing):

> Production table, 500 GB, you need a new index without locking out writes. Walk me through CREATE INDEX CONCURRENTLY and the gotchas.

The 30-second answer: `CREATE INDEX CONCURRENTLY` builds the index in two scans of the table while only taking a SHARE UPDATE EXCLUSIVE lock — DML continues normally. The trade-off is it takes 2-3x as long, can't run inside a transaction, and if it fails midway you're left with an `INVALID` index

that has to be dropped. Postgres 11+ also supports parallel index builds using `max_parallel_maintenance_workers` to use multiple CPUs for the sort phase.

Step-by-step thought process: 1. Start with the lock difference — normal CREATE INDEX takes an ACCESS EXCLUSIVE lock and blocks all writes. CONCURRENTLY trades speed and atomicity for lock-friendliness. 2. The two-scan dance — first scan builds the initial index, second scan picks up any rows changed during the first scan, and a brief lock at the end validates the index. 3. Failure mode — if anything goes wrong (deadlock, cancellation, server restart), the index is marked `INVALID` and isn't used by the planner. You have to `DROP INDEX CONCURRENTLY` and start over. 4. Parallel maintenance — for in-memory sortable indexes, raise `max_parallel_maintenance_workers` to 4-8 and the SORT phase parallelizes. Helps a lot for B-tree on a single large column. 5. Be ready for: "Why can't I CREATE INDEX CONCURRENTLY inside a transaction?" Because the two-phase build manages its own snapshots; wrapping it in a transaction would defeat the design.

Edge cases to mention: - Long-running transactions on the table block the second phase from completing — kill or wait for them. - CONCURRENTLY does not work for primary key creation if you're trying to add a PK to an existing table without locking — you have to create a unique index concurrently first, then `ALTER TABLE ... ADD CONSTRAINT ... USING INDEX`. - Replicas need to apply the index too — logical replication subscribers do not get DDL, so you build separately on each.

What NOT to say: - Do not run `CREATE INDEX CONCURRENTLY` and not check `pg_index.indisvalid` afterward. A silent INVALID index is worse than no index. - Do not skip CONCURRENTLY on a production table because "it's only 50 GB" — locking writes for 5 minutes during the day is enough to take down a payment flow.

Common follow-up: "*You started CONCURRENTLY and it failed. What's the cleanup?*" — `DROP INDEX CONCURRENTLY idx_name`, fix the underlying issue (often a long-running transaction blocking the validation phase), and retry.

```
-- Build without blocking writes; parallel sort with 4 workers
SET max_parallel_maintenance_workers = 4;
CREATE INDEX CONCURRENTLY idx_orders_user_created
  ON orders (user_id, created_at);

-- Always verify validity afterward
SELECT indexname, indexdef
FROM pg_indexes WHERE indexname = 'idx_orders_user_created';

SELECT relname, indisvalid
FROM pg_class c JOIN pg_index i ON c.oid = i.indexrelid
WHERE relname = 'idx_orders_user_created';
```

Q191 · Hot Row Contention — Sharded Counter Pattern

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Flipkart, Cred · Topic: Concurrency

The question (interviewer's verbatim phrasing):

> We have a global counter row that every payment increments. Lock contention is killing throughput. How do you fix it without changing the database?

The 30-second answer: Shard the counter into N rows (say 64), let each writer pick a random shard and increment that, and compute the total with a SUM aggregate when needed. Contention drops linearly with shard count because writers no longer fight over a single row. Reads are slightly more expensive but stay sub-millisecond up to thousands of shards.

Step-by-step thought process: 1. Lead with the root cause — every UPDATE on the same row takes a row-level lock; concurrent transactions queue, and at high QPS this becomes the bottleneck even if the DB is otherwise idle. 2. Sharded counter — instead of one row `counter`, have rows `counter_shard_00` through `counter_shard_63`, and writers pick one at random or by hash of session ID. 3. Reads sum across shards — `SELECT SUM(value) FROM counter_shards` — cheap as long as the table has few rows. 4. Be ready for: "How do you pick the shard count?" Rule of thumb: 2x the concurrent writer count for headroom; powers of 2 make modulo cheap. 5. Mention the variant — for sequence-like ID generation, the same pattern works with a per-shard sequence and a composite ID `(shard_id, local_seq)`.

Edge cases to mention: - Read consistency — summing across shards in READ COMMITTED can briefly under-count if a transaction is in flight on another shard; usually acceptable for counters, not for billing totals. - Skew — if your shard picker is biased (e.g. hash of a user ID that's concentrated), some shards still get hammered. Random pick avoids this. - Garbage collection — sharded counters bloat with MVCC tuple versions; aggressive autovacuum settings on the counter table are essential.

What NOT to say: - Do not say "just use a sequence" — sequences avoid the contention but only generate IDs, not accumulating sums. - Do not propose Redis as the first answer unless the interviewer asked about cross-system options. Solve in-DB first.

Common follow-up: "*What if you need strong consistency on the total?*" — Use a SUM in a SERIALIZABLE transaction, or accept that the counter is eventually consistent and reconcile in a background job.

```
-- Schema
CREATE TABLE payment_counter_shards (
  shard_id  INT PRIMARY KEY,
  cnt       BIGINT NOT NULL DEFAULT 0
);
INSERT INTO payment_counter_shards (shard_id, cnt)
SELECT g, 0 FROM generate_series(0, 63) g;

-- Writer
UPDATE payment_counter_shards SET cnt = cnt + 1
WHERE shard_id = floor(random() * 64);

-- Reader
SELECT SUM(cnt) AS total FROM payment_counter_shards;
```

Q192 · B-tree Page Splits — Internals

Tags: Difficulty: Hard · Asked at: Senior panels at Razorpay, Cred, Goldman, JP Morgan · Topic: Internals

The question (interviewer's verbatim phrasing):

> Walk me through what happens at the page level when a B-tree index leaf fills up. Why do random inserts hurt and monotonic inserts help?

The 30-second answer: A B-tree leaf page has fixed size (8 KB in Postgres, 16 KB in InnoDB). When an insert can't fit, the page splits — roughly half the entries move to a new sibling page, the parent gets a new pointer, and if the parent is full the split cascades upward. Monotonic inserts (sequential IDs or timestamps) avoid mid-page splits because new keys always go to the rightmost leaf, which fills and rolls over cleanly. Random inserts spray keys across the tree, causing splits everywhere and leaving pages 50-70 percent full.

Step-by-step thought process: 1. Set the scene — a B-tree is balanced top-down; every split that propagates is extra writes and extra journal/WAL. 2. Sequential insertion pattern — keys always land at the rightmost leaf, fill it, split once, and move on. Page fill stays high. 3. Random insertion — keys hit random pages, each leading to a 50/50 split; over time the index occupies twice the space it should and cache hit rate drops. 4. Mention the optimization — Postgres has the "rightmost leaf" fastpath that recognizes monotonic inserts and avoids re-checking the full tree. InnoDB does similar. 5. Be ready for: "Why do UUID v4 keys hurt and UUID v7 (or ULIDs) help?" UUID v4 is random; v7 is timestamp-prefixed, so inserts cluster at the right edge.

Edge cases to mention: - After a split, the half-empty pages can be reclaimed by `VACUUM` (Postgres) or merged (InnoDB) only under specific conditions; in practice bloat accumulates. - Wide index keys waste more page space — a 200-byte key in an 8 KB page caps you at 40 entries per leaf. Keep indexed

columns narrow. - The fillfactor parameter (`WITH (fillfactor = 70)` in Postgres) leaves headroom for HOT updates; on insert-heavy tables, default 90 is usually fine.

What NOT to say: - Do not claim "B-trees are always balanced." They are balanced in height but not in fill — bloat is real. - Do not say "use a hash index for randomly inserted keys" — hash indexes lose range scans, and Postgres hash indexes weren't even crash-safe until version 10.

Common follow-up: "*Show me a query to measure index bloat.*" — Postgres exposes this via `pgstattuple` extension or queries against `pg_class.relpages` vs theoretical row count.

```
-- Approximate index bloat in Postgres
CREATE EXTENSION IF NOT EXISTS pgstattuple;
SELECT * FROM pgstattuple('idx_orders_user_id');
-- Look at dead_tuple_percent and free_percent
```

Q193 · WAL fsync vs Async Durability Trade-off

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, JP Morgan, Cred infra · Topic: Internals

The question (interviewer's verbatim phrasing):

> What's the trade-off between `synchronous_commit = on` and `off` in Postgres? When would you turn it off?

The 30-second answer: With `synchronous_commit = on`, a COMMIT does not return until the WAL is flushed to disk via fsync — durable on power loss. With `off`, COMMIT returns as soon as the WAL is written to the OS page cache; the in-flight transactions can be lost on crash but the database stays internally consistent. You turn it off only when the workload can tolerate losing the last few hundred milliseconds of transactions — staging, bulk loads, analytics queues — in exchange for 2-5x higher commit throughput.

Step-by-step thought process: 1. Start with the durability ladder — `on` (default) fsyncs WAL before commit returns; `local` fsyncs locally but doesn't wait for replicas; `remote_write`, `remote_apply` are stricter modes for sync replication; `off` waits for nothing. 2. The cost of fsync — every commit triggers a disk flush. On spinning disks that's 5-10 ms; on SSDs it's sub-millisecond but still a serialization point. 3. With `off`, group commit batches more aggressively and throughput goes up. The window of loss is roughly `wal_writer_delay` (default 200 ms). 4. Be ready for: "What about `commit_delay`?" That's a different knob — it adds a brief wait to encourage commit batching. Useful at very high commit rates. 5. Mention durability is not the same as consistency — even with `off`, the database recovers to a self-consistent state; only the most recent transactions vanish.

Edge cases to mention: - Replication interacts — if you have a synchronous standby, the COMMIT wait extends to ack from the standby, not just local disk. That's `synchronous_commit = remote_apply`. - Set per session — you can run a bulk loader with `SET synchronous_commit = off` for the session without changing the global default. Useful for ETL. - Cloud storage often has hidden buffering — fsync on AWS EBS is usually fast, but on some block stores it's slower than you expect; benchmark before promising SLAs.

What NOT to say: - Do not say "synchronous_commit off is fine for production" without qualifying which production. For payments, never. For analytics ingest, often yes. - Do not confuse `synchronous_commit` with `fsync` (the bigger hammer that disables all fsync calls — dangerous and almost never the right answer).

Common follow-up: "*You enabled async commit on the payments DB and lost 10 minutes of transactions during a power blip. What was the architectural mistake?*" — Async commit on a system of record. The job was durability, and they traded it away for throughput on the wrong workload.

```
-- Per-session async commit for a bulk load
SET LOCAL synchronous_commit = off;
INSERT INTO event_log SELECT * FROM staging_events;
-- Reverts at end of transaction
```

Q194 · Indexes on JSON — GIN on jsonb

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, GCCs with event stores · Topic: Postgres

The question (interviewer's verbatim phrasing):

> You have a jsonb column with semi-structured data. Queries are slow. How do you index it, and what are the trade-offs?

The 30-second answer: For containment queries (`@>`) and key/path lookups, build a GIN index on the jsonb column. Default opclass supports `@>`, `?`, `?&`, `?|`; the `jsonb_path_ops` opclass is smaller and faster but only supports `@>`. For a single hot key, a functional B-tree index on `(payload->>'key')` beats GIN. The trade-off: GIN indexes are large and slow to build, and INSERT/UPDATE costs go up because every key in every row is indexed.

Step-by-step thought process: 1. Lead with the two index families — GIN for the whole jsonb (broad query support); B-tree on an expression for one specific path (small, fast, narrow). 2. GIN with default ops — indexes every key and value; supports `?` (key exists), `@>` (contains), `?|` (any key), `?&` (all keys). 3. `jsonb_path_ops` — only indexes paths-and-values, much smaller and faster, but only supports `@>`. Use it when your queries are all containment. 4. Functional B-tree — `CREATE INDEX ON events ((payload->>'tenant_id'))` gives you a tiny index on one path.

Perfect when one attribute is queried millions of times and others rarely. 5. Be ready for: "Why is my GIN index 3x the table size?" Because every distinct key-value pair becomes an index entry. Wide jsonb with high cardinality blows up GIN.

Edge cases to mention: - GIN updates are slow — Postgres uses a pending list (`fastupdate = on`) to batch GIN updates, flushed during VACUUM. Tune `gin_pending_list_limit` for write-heavy workloads. - Containment is left-to-right — `'{a: 1, b: 2}' @> '{a: 1}'` is true; `'{a: 1}' @> '{a: 1, b: 2}'` is false. - For SQL/JSON path queries (`@?` , `@@`), use the same GIN with `jsonb_path_ops` or add a separate functional index.

What NOT to say: - Do not say "just index the jsonb column" without specifying the opclass — the opclass determines what queries can use the index. - Do not propose extracting jsonb fields into separate columns "for indexing" as the only solution — for high-cardinality semi-structured data, GIN is what you want; column extraction is for stable, queried-always-together fields.

Common follow-up: "*How do you know GIN is actually being used?*" — `EXPLAIN ANALYZE` your query and look for `Bitmap Index Scan on idx_events_gin` . If it's a Seq Scan, the operator probably isn't supported by your opclass.

```
-- GIN with jsonb_path_ops (smaller, supports @>)
CREATE INDEX events_payload_gin ON events
  USING GIN (payload jsonb_path_ops);

-- Functional B-tree on a hot single key
CREATE INDEX events_tenant_id ON events ((payload->>'tenant_id'));

-- Query that uses the GIN
SELECT * FROM events WHERE payload @> '{"status": "failed"}';
```

Q195 · ANY_VALUE Aggregate — Postgres 16

Tags: Difficulty: Easy · Asked at: Cred, Razorpay analytics, GCCs · Topic: Postgres

The question (interviewer's verbatim phrasing):

> Postgres 16 added an `ANY_VALUE` aggregate. What problem does it solve?

The 30-second answer: `ANY_VALUE` returns an arbitrary value from a group, signalling to the engine and to the reader that you don't care which row's value is returned — only that some valid value comes back. Before Postgres 16 you had to use `MIN` or `MAX` or wrap the column in an aggregate just to satisfy `GROUP BY` , even when any value would do. `ANY_VALUE` is faster (no comparison work) and more semantically honest.

Step-by-step thought process: 1. Set the context — SQL standard requires every column in SELECT to either be in GROUP BY or be aggregated. Sometimes you genuinely don't care which value comes back. 2. Old pattern — `MAX(customer_name)` to pick "any name" for a customer when grouping by `customer_id`. Wasteful — MAX has to compare every value. 3. `ANY_VALUE(customer_name)` — returns any one value from the group. The planner is free to pick the cheapest (often the first one it encounters). 4. Be ready for: "Why didn't people just turn off the strict GROUP BY mode?" MySQL allowed this implicitly for years and it caused subtle correctness bugs because the unaggregated value was non-deterministic.

Edge cases to mention: - Not actually random — the value picked is implementation-defined but stable within a single execution. Don't rely on which row it picks. - Available in Postgres 16+, also SQL Server 2022+, also part of SQL:2023 standard. - Replaces `(array_agg(col))[1]` and `MIN(col)` tricks that were common workarounds.

What NOT to say: - Do not say "ANY_VALUE is random." It's unspecified, not random — and unspecified might still always pick the same value for the same data. - Do not use it when you actually need a specific row's value (e.g., latest). For that, use window functions or DISTINCT ON.

Common follow-up: "*Show a query where ANY_VALUE saves you a window function.*" — Joining customers and orders and grouping by customer_id, where you want one of the customer's emails alongside the order count, ANY_VALUE(email) does it in one aggregate.

```
-- Postgres 16+
SELECT
  customer_id,
  ANY_VALUE(name)  AS customer_name,
  COUNT(*)         AS order_count,
  SUM(amount)      AS total_spend
FROM orders
GROUP BY customer_id;
```

Q196 · IS DISTINCT FROM — NULL-Safe Equality

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, GCCs · Topic: NULL handling

The question (interviewer's verbatim phrasing):

> Compare `a = b`, `a IS NOT DISTINCT FROM b`, and `COALESCE(a, '') = COALESCE(b, '')`. Which one would you use to compare nullable columns for true equality?

The 30-second answer: `a = b` returns NULL when either side is NULL — useless for equality. `IS NOT DISTINCT FROM` is the SQL standard NULL-safe equality — it treats NULLs as equal to each other and returns TRUE or FALSE, never NULL. The COALESCE trick works but is fragile (the sentinel

value has to be one that can never appear in real data) and harder to read. Prefer `IS NOT DISTINCT FROM` in standard SQL; MySQL's `<=>` operator is the equivalent.

Step-by-step thought process: 1. Open with the three-valued logic rule — any comparison involving NULL returns NULL, which is then filtered out by WHERE. 2. `IS DISTINCT FROM` is the inverse — returns TRUE if the two values differ OR exactly one is NULL. `IS NOT DISTINCT FROM` is the equality version. 3. The COALESCE trick — substitutes a sentinel for NULL on both sides. Works but you have to pick a sentinel that's impossible in your data; brittle on real columns. 4. Be ready for: "Why does Postgres still default `=` to three-valued logic?" Because SQL standard. NULL means "unknown," and unknown compared to unknown is also unknown, which is the correct semantic.

Edge cases to mention: - Indexes — most databases will not use a B-tree index for `IS NOT DISTINCT FROM` because the operator is not a standard equality. Postgres 12+ added planner support for some cases; check `EXPLAIN`. - MySQL's `<=>` is identical in meaning. SQL Server doesn't have a built-in operator; use `EXISTS (SELECT a INTERSECT SELECT b)` as a workaround or just CASE. - In a JOIN ON clause, `IS NOT DISTINCT FROM` lets you match rows where both join keys are NULL — useful when NULL is a meaningful "default group" value.

What NOT to say: - Do not write `WHERE col1 = col2 OR (col1 IS NULL AND col2 IS NULL)` — it works but is verbose and obscures intent. - Do not say NULL = NULL is true. In SQL it's NULL — the most common interview trap of all.

Common follow-up: *"Show me an UPDATE that only fires when a value actually changed, accounting for NULLs."* — `WHERE col IS DISTINCT FROM new_value`. Common in CDC pipelines and trigger-based audit logs.

```
-- "Where the email changed, including transitions to/from NULL"
SELECT id FROM users_snapshot s
JOIN users u ON s.id = u.id
WHERE s.email IS DISTINCT FROM u.email;
```

Q197 · Common SQL Anti-Pattern Catalog

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wipro, all panels · Topic: Anti-patterns

The question (interviewer's verbatim phrasing):

> List the worst SQL anti-patterns you've seen in production and how you'd rewrite each one.

The 30-second answer: SELECT star in OLTP code (locks you into schema changes, defeats covering indexes), scalar UDFs in WHERE clauses (force row-by-row execution and kill the optimizer), OR

conditions on different indexed columns (planner often picks a seq scan — rewrite as UNION ALL), correlated subqueries that could be joins, implicit type casts on indexed columns, GROUP BY in subqueries that aren't used, and the classic `SELECT COUNT(*) WHERE col IS NOT NULL` instead of `COUNT(col)`.

Step-by-step thought process: 1. Start with the most expensive — SELECT star. Production code should always name columns: smaller plan, smaller payload, surfaces schema drift earlier. 2. Scalar UDFs in WHERE — in SQL Server pre-2019, scalar UDFs force the optimizer to abandon set-based plans. Postgres treats them as opaque too. Inline them or use CASE expressions. 3. OR across different columns — `WHERE a = 1 OR b = 2` on indexes on (a) and (b) — most planners fall back to a seq scan. Rewrite as `UNION ALL` of two indexed selects. 4. Implicit casts — `WHERE id = '123'` on a bigint id column casts the column, not the literal, in some dialects, killing the index. Always match types. 5. Be ready to list 3-5 more: `DISTINCT` masking a bad join, `ORDER BY` in subqueries, expensive subqueries in SELECT, no LIMIT on pagination, no statistics gathering after bulk load.

Edge cases to mention: - Some anti-patterns are dialect-specific — `SELECT *` is less harmful in Snowflake where columnar storage makes it cheap, but still bad practice. - Functions on indexed columns — `WHERE LOWER(email) = ?` — needs a functional index `(LOWER(email))` or it skips the email index entirely. - The "N+1 in SQL" — application code that loops calling a single-row query is the same pathology as N+1 in ORMs; collapse it into one set-based query.

What NOT to say: - Do not say "SELECT * is always wrong." It's fine in ad-hoc analytics and in CTEs that get column-pruned. The sin is in app code. - Do not list anti-patterns without the rewrite — interviewers want to hear you fix them too.

Common follow-up: "*Show me how you'd rewrite a query with three OR conditions on different indexed columns.*" — UNION ALL of three indexed queries with a final DISTINCT if needed.

```
-- Anti-pattern
SELECT * FROM events
WHERE user_id = 42 OR session_id = 'abc' OR device_id = 'd-99';

-- Rewrite (each branch uses its own index)
  SELECT * FROM events WHERE user_id = 42
UNION
  SELECT * FROM events WHERE session_id = 'abc'
UNION
  SELECT * FROM events WHERE device_id = 'd-99';
```

Q198 · Functional Indexes — Indexing Expressions

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, JP Morgan · Topic: Indexes

The question (interviewer's verbatim phrasing):

> A query filters by `LOWER(email) = ?` and is slow. The email column is indexed. What's wrong and how do you fix it?

The 30-second answer: The B-tree index is on `email`, not on `LOWER(email)`, so the planner can't use it once the column is wrapped in a function. Create a functional index: `CREATE INDEX idx_users_lower_email ON users (LOWER(email))`. The query plan switches from Seq Scan to Index Scan instantly.

Step-by-step thought process: 1. Lead with the principle — an index records values exactly as stored. Wrapping the column in any function transforms the value, so the index doesn't apply. 2. Functional indexes solve it — Postgres, MySQL 8.0+, and SQL Server (computed columns) all support indexing the expression itself. 3. Common cases — case-insensitive search (`LOWER`), JSON path extraction (`payload->>'tenant_id'`), date truncation (`DATE_TRUNC('day', ts)`), and any custom transformation. 4. Be ready for: "Why not use `CITEXT` for case-insensitive?" CITEXT in Postgres is one approach — it stores text but compares case-insensitively. Slower for ordering, faster to query without function wraps. Trade-offs both ways.

Edge cases to mention: - The function used in the index must be IMMUTABLE — Postgres rejects volatile or stable functions. `LOWER` is immutable; `NOW()` is not. - Storage cost — a functional index stores the function's output, so wide JSON extractions cost more space than the underlying column. - The query must use the exact same expression — `LOWER(email)` matches but `LOWER(trim(email))` does not; index has to be on the same combination.

What NOT to say: - Do not say "rewrite the query to not use LOWER." Sometimes you can't — case-insensitive matching is a real product requirement. - Do not index every expression — each index slows writes and consumes RAM. Pick the ones that hot queries use.

Common follow-up: "*Index a jsonb field that's queried by `payload->>'order_id'` millions of times a day.*" — Same pattern: `CREATE INDEX ON events ((payload->>'order_id'))`.

```
-- Before: Seq Scan
EXPLAIN SELECT * FROM users WHERE LOWER(email) = 'a@b.com';

-- Build the functional index
CREATE INDEX idx_users_lower_email ON users (LOWER(email));

-- After: Index Scan using idx_users_lower_email
EXPLAIN SELECT * FROM users WHERE LOWER(email) = 'a@b.com';
```

Q199 · Stable Pagination — Keyset / Seek Method

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Cred, Flipkart, Walmart Labs · Topic: Query patterns

The question (interviewer's verbatim phrasing):

> Pagination using OFFSET 50000 LIMIT 50 is killing the database. What's the fix?

The 30-second answer: Switch to keyset pagination (a.k.a. the seek method) — instead of `ORDER BY created_at, id OFFSET 50000 LIMIT 50`, use `WHERE (created_at, id) > (last_seen_ts, last_seen_id) ORDER BY created_at, id LIMIT 50`. The database uses the index to seek directly to the start of the next page; no rows are scanned and discarded. Result: constant-time pagination regardless of how deep you are.

Step-by-step thought process: 1. Explain why OFFSET is slow — to skip 50,000 rows, the engine has to find, read, and discard 50,000 rows before returning the next 50. Linear in the offset. 2. Keyset pagination remembers the last row's sort keys and asks for "rows after these keys." The B-tree index supports this lookup in O(log n). 3. Composite key tie-breaking — `created_at` alone is not unique, so two rows could share a timestamp. Add `id` as the tie-breaker: ORDER BY `(created_at, id)`, WHERE `(created_at, id) > (?, ?)`. 4. Anticipate: "What if the user jumps to page 999?" Keyset doesn't support arbitrary page numbers — only next/previous from a known cursor. For page numbers, you either pre-compute page boundaries or accept linear cost.

Edge cases to mention: - Mixed sort directions — Postgres supports row-comparison `(a, b) > (?, ?)` only for matching ASC/DESC; for mixed, you write explicit OR predicates. - New rows inserted between page fetches with offset pagination cause duplicates or skips; keyset is also affected unless the sort keys are monotonic. - For infinite-scroll UIs, keyset is the only sane approach beyond a few thousand rows.

What NOT to say: - Do not say "add an index on created_at and OFFSET will be fast." OFFSET is row-skipping work that the index cannot avoid. - Do not claim keyset is harder to implement — for next/prev cursors it's actually simpler than maintaining an OFFSET state.

Common follow-up: "*How do you encode the cursor for the client?*" — Base64-encode the JSON of the sort tuple, often signed to prevent tampering.

```
-- Slow at depth
SELECT * FROM orders ORDER BY created_at, id OFFSET 50000 LIMIT 50;

-- Keyset — constant time
SELECT * FROM orders
WHERE (created_at, id) > ('2026-05-26 12:00:00', 938472)
ORDER BY created_at, id
LIMIT 50;
```

Q200 · Capstone — Design a Payments Ledger Schema

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, PhonePe, Zerodha, Goldman, JP Morgan · Topic: System design

The question (interviewer's verbatim phrasing):

> Walk me through how you'd design a payments ledger schema with an audit log and sharding. Assume Indian payments scale — 10M transactions a day, regulatory audit requirements, and a 7-year retention horizon.

The 30-second answer: Two-tier schema. The ledger itself is double-entry: a `journal_entries` table where every monetary movement is a balanced pair of credit/debit rows referencing accounts, transactions, and a transaction id. An append-only `audit_log` table captures every state transition with actor, before/after, and timestamp. Shard the journal by account_id hash (modulo 64) so high-volume merchant accounts spread evenly. Partition each shard by month for retention pruning. The audit log lives in its own append-only table partitioned by week, with cold partitions archived to object storage after 90 days.

Step-by-step thought process:

1. **Core domain — accounts and journal.** Every monetary movement in the system is recorded as a balanced journal entry — for every credit there is an equal and opposite debit, and the sum of all entries against an account is its balance. Two tables: `accounts (id, owner_id, account_type, currency, created_at)` and `journal_entries (id, transaction_id, account_id, direction CHAR(1), amount_paise BIGINT, posted_at, idempotency_key)`. The `direction` is 'C' for credit or 'D' for debit; `amount_paise` is integer paise to avoid floating point. Every successful transaction has exactly two rows: one credit, one debit, sharing the same `transaction_id`. Balance is `SUM(CASE direction WHEN 'C' THEN amount_paise ELSE -amount_paise END)`.

2. **Idempotency.** Every write originates from an external source (the payment gateway callback, internal transfer engine, refund processor). Each carries an `idempotency_key` — a unique string from the caller. UNIQUE constraint on (`transaction_id`, `account_id`, `direction`) makes duplicate posts impossible. Retries are safe; reconciliation tools can confirm by key.

3. **Transactions table.** A higher-level `transactions (id, kind, status, amount_paise, source_account_id, destination_account_id, created_at, settled_at)` row exists per logical transfer. It's the application-facing entity; the journal entries are its accounting representation. Status transitions (`pending` → `authorized` → `captured` → `settled`, or `failed`, or `refunded`) are themselves auditable events.

4. **Audit log — append-only, immutable.** Every state change on `transactions`, every journal entry, every account update writes a row to `audit_log (id, actor_type, actor_id,`

`entity_type, entity_id, action, before_jsonb, after_jsonb, created_at, request_id)`. `actor_type` is 'system', 'user', 'service'. `request_id` ties an audit row back to the originating API call or job. No UPDATE or DELETE on this table — enforced by revoking those privileges from the application role.

5. **Sharding the journal.** At 10M transactions per day, each producing two journal rows, you're at 20M inserts daily, ~7 billion rows per year. A single Postgres table can handle this with partitioning, but write throughput on a single node bottlenecks around 30k-50k inserts/sec. Shard by `hash(account_id) % 64`, route writes via a thin shard-aware service. Reads for a single account stay on one shard; cross-account reports go through a separate analytics path (CDC into BigQuery / Snowflake).

6. **Partitioning within each shard.** Use Postgres declarative partitioning on `posted_at` by month: `PARTITION BY RANGE (posted_at)`. New month, new partition. Old partitions (older than 13 months) can be detached and archived to S3 as Parquet for cheap cold-storage compliance.

7. **Indexes that matter.** Per shard: B-tree on `(account_id, posted_at)` for balance lookups; B-tree on `(transaction_id)` for transaction joins; UNIQUE B-tree on `(idempotency_key)` to enforce idempotency. Avoid covering indexes on `amount_paise` — adds write cost without enough read benefit at this scale.

8. **Consistency model.** Within a shard, all writes for a single transaction (both journal entries plus the `transactions` row update plus the audit log row) happen in one Postgres transaction at READ COMMITTED. Cross-shard transfers are coordinated via a two-phase pattern: write a `pending_transfer` row, then atomically post on both shards via outbox + worker; if either fails, the transfer is marked `failed` and reversed.

9. **Retention and archival.** Regulatory requirement is 7 years. Active partitions stay in Postgres for 13 months. Older partitions are exported to Parquet on S3 with the same schema and queryable via Athena / Trino. Audit log follows the same path but with weekly partitions and a 90-day hot window.

10. **Operational concerns.** Backups: per-shard logical dumps weekly, WAL streaming for PITR. Reconciliation: a daily job that sums journal entries per account and compares to a computed `account_balances_daily` snapshot. Monitoring: alert on per-shard write lag, partition-creation success, and any audit_log row missing a `request_id`.

11. **What I would deliberately not do.** Store balance as a column on the accounts table updated in-place — that's a contention nightmare and removes the audit trail. Use floats for money. Use UUIDs without time prefix on the journal — random inserts shred the B-tree. Allow updates on audit_log rows — destroys forensics.

Edge cases to mention: - Money rounding: paise integers eliminate FP error, but division for splits (e.g. interchange fees) requires deterministic rounding rules — typically round-half-up with the remainder going to a residual ledger account. - Negative balances: depending on account type (settlement vs prepaid), the balance can go negative briefly. Constraints should be at the account-type level, not the

column level. - Timezone: `posted_at` in UTC always; display in IST at the UI layer. Mixing timezones in a ledger is how reconciliations fail.

What NOT to say: - Do not propose storing balance as a column on the account row and updating it in place. That's the lock-contention nightmare from Q191 at 10x the consequences. - Do not say "we'll use Postgres for everything" without addressing the shard layer; at 7 billion rows per year, a single instance is the wrong answer. - Do not skip idempotency. Every panel listening to this answer is waiting for it.

Common follow-up: "*Now walk me through what happens when the payment gateway double-posts a webhook callback at 11:47 PM on the 31st of the month, and the second post arrives just after the partition rolls over.*" — Idempotency key catches it before the row is inserted; the second post is a no-op. The partition rollover is irrelevant because the idempotency check is per-shard and the key is unique across all partitions on that shard.

```
-- Accounts
CREATE TABLE accounts (
  id            BIGSERIAL PRIMARY KEY,
  owner_id      BIGINT NOT NULL,
  account_type  TEXT NOT NULL,
  currency      CHAR(3) NOT NULL DEFAULT 'INR',
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Journal (one shard, partitioned by month)
CREATE TABLE journal_entries (
  id               BIGSERIAL,
  transaction_id   BIGINT NOT NULL,
  account_id       BIGINT NOT NULL,
  direction        CHAR(1) NOT NULL CHECK (direction IN ('C','D')),
  amount_paise     BIGINT NOT NULL CHECK (amount_paise > 0),
  posted_at        TIMESTAMPTZ NOT NULL,
  idempotency_key  TEXT NOT NULL,
  PRIMARY KEY (id, posted_at),
  UNIQUE (idempotency_key, posted_at)
) PARTITION BY RANGE (posted_at);

CREATE TABLE journal_entries_2026_05
  PARTITION OF journal_entries
  FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');

CREATE INDEX ON journal_entries_2026_05 (account_id, posted_at);
CREATE INDEX ON journal_entries_2026_05 (transaction_id);

-- Audit log (append-only)
CREATE TABLE audit_log (
  id            BIGSERIAL,
  actor_type    TEXT NOT NULL,
  actor_id      TEXT NOT NULL,
  entity_type   TEXT NOT NULL,
  entity_id     BIGINT NOT NULL,
  action        TEXT NOT NULL,
  before_jsonb  JSONB,
  after_jsonb   JSONB,
  request_id    TEXT NOT NULL,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (id, created_at)
) PARTITION BY RANGE (created_at);

-- Revoke UPDATE/DELETE from app role on audit_log
REVOKE UPDATE, DELETE ON audit_log FROM app_user;

-- Compute balance for an account
SELECT SUM(
  CASE direction WHEN 'C' THEN amount_paise
                 WHEN 'D' THEN -amount_paise END
) AS balance_paise
FROM journal_entries
```

```
  WHERE account_id = $1
    AND posted_at < $2;  -- as-of timestamp
```

That's the closing batch. If you walked through Q1 to Q200 and could answer each one in a minute without scrolling back, you are ready — not just for the screen, but for the offer-stage system design round where they hand you a whiteboard and ask "design us a ledger." The Q200 answer above is exactly what you would draw, with the same vocabulary and the same trade-offs called out the same way. Take it to the next interview. Good luck.